

Literature Survey: Efficient LLM Inference Serving Systems

A Comparative Analysis of ORCA, FlashInfer, and Helix

Covering Core Novelty, Experimental Setup, Benchmarks, GitHub Repositories,
Integration with Modern Systems, and Implications for System Design

Literature Survey Report

May 11, 2026

Abstract

The rapid growth of large language models (LLMs) has created an urgent need for efficient inference serving infrastructure. This survey analyzes three seminal systems—**ORCA** (OSDI '22), **FlashInfer** (MLSys '25), and **Helix** (ASPLOS '25)—that collectively represent major advances in LLM serving across three distinct dimensions: *scheduling granularity*, *attention kernel efficiency*, and *heterogeneous cluster utilization*. For each paper, we detail the core problem statement, key technical innovations, experimental methodology, benchmarks used, GitHub repositories, and integration with production serving stacks (vLLM, SGLang, MLC-Engine, FasterTransformer). We further compare the systems on design philosophy, performance claims, and applicability to modern deployment scenarios.

Contents

1	Introduction and Motivation	3
2	Paper 1: ORCA — A Distributed Serving System for Transformer-Based Generative Models	3
2.1	Bibliographic Information	3
2.2	Problem Statement	3
2.3	Core Technical Contributions	4
2.3.1	Contribution 1: Iteration-Level Scheduling	4
2.3.2	Contribution 2: Selective Batching	4
2.3.3	Contribution 3: Attention K/V Manager	5
2.3.4	Contribution 4: Distributed Architecture with Control-Data Plane Separation	5
2.3.5	Contribution 5: Pipeline Parallelism Without Microbatching	5
2.4	Scheduling Algorithm Details	5
2.5	Experimental Setup	6
2.6	Benchmarks and Evaluation Scenarios	6
2.7	Key Results	6
2.8	GitHub Repository and Availability	6
2.9	Integration with Modern Systems	7

3	Paper 2: FlashInfer — Efficient and Customizable Attention Engine for LLM Inference Serving	7
3.1	Bibliographic Information	7
3.2	Problem Statement	7
3.3	Core Technical Contributions	8
3.3.1	Contribution 1: Block-Sparse Format as Unified KV-Cache Abstraction	8
3.3.2	Contribution 2: Composable Formats for Memory Efficiency	8
3.3.3	Contribution 3: JIT Compiler for Attention Variants	8
3.3.4	Contribution 4: Load-Balanced Dynamic Scheduling	9
3.3.5	Contribution 5: CUDAGraph Compatibility	9
3.4	Microkernel Design	9
3.5	Experimental Setup	10
3.6	Benchmarks and Evaluation Scenarios	10
3.7	Key Results	11
3.8	GitHub Repository	11
3.9	Integration with Modern Systems	11
4	Paper 3: Helix — Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow	12
4.1	Bibliographic Information	12
4.2	Problem Statement	12
4.3	Core Technical Contributions	12
4.3.1	Contribution 1: Max-Flow Formulation of LLM Serving	12
4.3.2	Contribution 2: MILP-Based Optimal Model Placement	12
4.3.3	MILP Acceleration Techniques	13
4.3.4	Contribution 3: Per-Request Pipelines	13
4.3.5	Contribution 4: KV-Cache Estimation for Heterogeneous Nodes	13
4.4	Experimental Setup	14
4.5	Benchmarks and Evaluation Scenarios	14
4.6	Key Results	15
4.7	GitHub Repository	15
4.8	Integration with Modern Systems	15
5	Comparative Analysis	16
5.1	Problem Space Coverage	16
5.2	Design Philosophy Comparison	16
5.3	Benchmark Datasets Used	17
5.4	Performance Claims Summary	17
5.5	Limitations and Open Problems	17
5.5.1	ORCA	17
5.5.2	FlashInfer	18
5.5.3	Helix	18
6	Literature Survey Context and Related Work	18
6.1	Foundational Works Referenced	18
6.2	Chronological Research Trajectory	19
6.3	Key Research Gaps for Future Work	20
7	Summary and Recommendations for Literature Survey	20
7.1	Must-Read Companion Papers	20
7.2	Experimental Reproducibility Summary	21
7.3	Takeaways	21

1 Introduction and Motivation

Modern LLM serving presents challenges that differ fundamentally from classical DNN inference. Three characteristics make LLMs uniquely hard to serve at scale:

1. **Autoregressive generation:** Output is produced one token at a time, requiring many sequential model iterations per request.
2. **Variable-length outputs:** Different requests generate different numbers of tokens, creating heterogeneous completion times within a batch.
3. **Massive memory footprint:** Models with hundreds of billions of parameters combined with growing KV caches stress GPU memory systems.

The three papers surveyed address these challenges at different abstraction layers, as summarized in Table 1.

Table 1: High-level comparison of the three surveyed systems.

Aspect	ORCA (OSDI '22)	FlashInfer (MLSys '25)	Helix (ASPLOS '25)
Layer	Scheduler / Engine	Attention Kernel	Cluster Scheduler
Core Problem	Request-level scheduling stalls	Inflexible attention kernels	Heterogeneous GPU underutilization
Key Technique	Iteration-level scheduling + selective batching	Block-sparse attention + JIT compilation + load-balanced scheduling	Max-flow MILP model placement + per-request pipelines
Primary Benefit	36.9× throughput vs. FasterTransformer	29–69% ITL reduction vs. Triton	3.3× throughput vs. baselines
Venue	OSDI 2022	MLSys 2025	ASPLOS 2025
Code	Proprietary (FriendliAI)	github.com/flashinfer-ai/flashinfer	github.com/Thesyslab/Helix-ASPLOS25

2 Paper 1: ORCA — A Distributed Serving System for Transformer-Based Generative Models

2.1 Bibliographic Information

Citation

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, Byung-Gon Chun. *ORCA: A Distributed Serving System for Transformer-Based Generative Models*. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22), Carlsbad, CA, July 2022, pp. 521–538.

Affiliation: Seoul National University and FriendliAI.

2.2 Problem Statement

Prior to ORCA, inference serving systems (e.g., NVIDIA Triton + FasterTransformer) scheduled model execution at the *request granularity*. When a batch of requests was dispatched to the execution engine, the engine processed all requests together through multiple model iterations until *every* request in the batch had finished generating. This leads to two critical inefficiencies:

1. **Wasted computation on finished requests:** A request generating 2 tokens is forced to remain in the batch until the last request generating 128 tokens completes. The execution engine performs empty/wasted passes on already-finished requests.
2. **High queuing delay for late-arriving requests:** New requests that arrive after a batch has been dispatched must wait until the *entire* batch finishes before they can be scheduled, even if most of the batch’s work is done.

2.3 Core Technical Contributions

2.3.1 Contribution 1: Iteration-Level Scheduling

ORCA proposes scheduling at the **iteration granularity** rather than the request granularity. The scheduler calls the execution engine to run *exactly one* forward pass (one token generation step) for the current batch. After each iteration:

- The scheduler inspects which requests have generated an `<EOS>` token and removes them from the pool immediately.
- The scheduler can admit newly arrived requests into the *next* iteration’s batch, reducing queuing delay to at most one iteration’s latency.

ORCA’s Core Insight

By treating a single token-generation step as the atomic unit of scheduling, ORCA achieves continuous batching: the batch composition can change at every iteration, enabling early returns and late joins simultaneously.

2.3.2 Contribution 2: Selective Batching

A fundamental challenge with iteration-level scheduling is that requests in the same batch may be at different positions in their generation sequence. This means their KV cache tensors have different shapes, making standard batching inapplicable to the Attention operation.

ORCA introduces **selective batching**: instead of applying batching uniformly to all operations, batching is applied only to operations that can handle variable-shaped tensors.

Observation. The Attention operation requires per-request separation because each request’s attention must only attend to its own preceding tokens. However, operations like **LayerNorm**, **Linear** (MLP), **GeLU**, and **Add** do not require the notion of a request boundary—they operate element-wise or row-wise.

Mechanism. ORCA flattens the batch dimension for non-Attention operations by concatenating all token representations into a single $[\sum L_i, H]$ tensor (where L_i is the number of tokens for request i and H is the hidden dimension). This allows non-Attention operations to be batched across all tokens simultaneously. Before the Attention operation, a **Split** decomposes the tensor by request, runs attention per-request, then a **Merge** reassembles the output.

Efficiency argument. The Attention operation has no learnable parameters. When batching is applied to GPU operations involving parameters (e.g., Linear layers), the primary benefit is *parameter reuse*—loaded weight tensors are shared across all requests in the batch. Since Attention has no weights, not batching it incurs no parameter reuse penalty, making the overhead negligible.

2.3.3 Contribution 3: Attention K/V Manager

ORCA maintains an **Attention K/V Manager** that persists the key and value tensors for each active request across iterations. This enables incremental decoding (fairseq-style): only the new token’s query, key, and value are computed at each increment-phase iteration, while previous keys and values are retrieved from the manager. The manager allocates buffers using each request’s `max_tokens` attribute to guarantee no out-of-memory during generation.

2.3.4 Contribution 4: Distributed Architecture with Control-Data Plane Separation

For serving large models (e.g., GPT-3 175B) that do not fit on a single GPU, ORCA uses both intra-layer (tensor) parallelism and inter-layer (pipeline) parallelism. A key design decision is to **separate the control plane from the data plane**:

- **Data plane (NCCL):** Intermediate tensor activations between workers travel over GPU-to-GPU high-bandwidth interconnects via NCCL.
- **Control plane (gRPC):** Control messages (batch IDs, token indices, phase information) travel via gRPC between CPU controllers, avoiding GPU-to-GPU NCCL roundtrips for metadata.

Existing systems (FasterTransformer, Megatron-LM) exchange control messages over NCCL, incurring GPU synchronization overhead at every iteration. ORCA’s separation eliminates this bottleneck, providing up to 47% execution time improvement on the 175B model.

2.3.5 Contribution 5: Pipeline Parallelism Without Microbatching

ORCA’s iteration-level scheduling naturally enables efficient pipeline parallelism. The scheduler maintains n_{workers} in-flight batches simultaneously without needing to split a batch into microbatches. FasterTransformer, constrained by request-level scheduling, must split each batch into microbatches to keep the pipeline full, trading batching efficiency for pipeline efficiency. ORCA avoids this tradeoff entirely.

2.4 Scheduling Algorithm Details

Algorithm 1 in the paper implements iteration-level FCFS (First-Come-First-Served):

1. Select at most `max_bs` requests from the pool, sorted by arrival time.
2. When admitting a new request (initiation phase), reserve `max_tokens` slots in the Attention K/V manager’s pre-allocated memory region. If reservation fails, stop admitting.
3. Schedule the selected batch for one iteration.
4. Repeat at the next iteration.

The two tunable parameters are: (a) `max_bs` (max batch size, governs the latency-throughput tradeoff) and (b) `n_slots` (K/V memory region size, set to maximum allowed by available GPU memory).

2.5 Experimental Setup

Table 2: ORCA experimental configuration.

Component	Details
Hardware	Azure ND96asr A100 v4 VMs; 8× NVIDIA A100 40GB SXM per VM; NVLink intra-node; 8× Mellanox 200Gbps HDR InfiniBand (1.6 Tb/s inter-node); up to 4 VMs
Models	GPT-3 13B (1 GPU), GPT-3 101B (8 GPUs), GPT-3 175B (16 GPUs), GPT-3 341B (32 GPUs)
Precision	fp16 parameters and activations
Baseline	NVIDIA FasterTransformer with NVIDIA Triton Inference Server
Workload	Synthesized Poisson-arrival traces; input tokens $\sim U(32, 512)$; output tokens $\sim U(1, 128)$
Max sequence length	2048 tokens
Metrics	Latency (ms/token, normalized), throughput (req/s), execution time (ms)

2.6 Benchmarks and Evaluation Scenarios

Scenario 1 — Engine Microbenchmark: Evaluate the ORCA execution engine in isolation (without the scheduler). All requests in a batch have identical input lengths (32 or 128 tokens) and generate exactly 32 tokens. This isolates the engine’s raw efficiency from scheduling effects. Metrics: total batch execution time (ms).

Scenario 2 — End-to-End Performance: Evaluate the full ORCA stack (scheduler + engine) under synthesized request traces with heterogeneous input/output lengths and Poisson arrivals. Metrics: median normalized latency (ms/token) and throughput (req/s) across varying arrival rates.

Scenario 3 — Homogeneous traces: All requests have identical input/output lengths (e.g., 32/32 or 256/256 tokens). This tests ORCA in the best case for FasterTransformer and verifies ORCA still maintains competitive or superior performance.

2.7 Key Results

Table 3: ORCA vs. FasterTransformer key results (175B model, 16 GPUs).

Metric	FasterTransformer	ORCA
Peak throughput (at 190 ms/token latency)	0.185 req/s	6.81 req/s
Throughput improvement		36.9×
Engine vs. FT (175B, no scheduling)	Baseline	Up to 47% faster (due to control-data separation)
Memory headroom	Limited (fixed pre-allocation per request based on max sequence length)	Higher (per-request allocation based on <code>max_tokens</code>)

2.8 GitHub Repository and Availability

ORCA was developed as proprietary technology by FriendliAI (formerly Seoul National University spin-off). The original ORCA codebase is not publicly available as open source. However:

- FriendliAI has commercialized the technology under their **Friendli Engine** product (<https://friendli.ai>).

- The ORCA paper’s key ideas (continuous batching, iteration-level scheduling) were independently re-implemented and open-sourced by the vLLM team (<https://github.com/vllm-project/vllm>) and are now standard in all major LLM serving frameworks.
- The ORCA paper is cited as the foundational work for continuous batching in FlashInfer, Helix, vLLM, SGLang, TensorRT-LLM, and others.

2.9 Integration with Modern Systems

- **vLLM:** Implements ORCA-style continuous batching as its core scheduling algorithm, combined with PagedAttention for KV cache management.
- **TensorRT-LLM:** NVIDIA’s production serving library implements iteration-level scheduling inspired by ORCA.
- **SGLang:** Uses continuous batching as its default scheduling strategy.
- **Text Generation Inference (TGI):** HuggingFace’s serving library adopted continuous batching from ORCA.
- **FriendliAI Friendli Engine:** The direct commercial descendant of ORCA.

3 Paper 2: FlashInfer — Efficient and Customizable Attention Engine for LLM Inference Serving

3.1 Bibliographic Information

Citation

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, Luis Ceze. *FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving*. 8th Conference on Machine Learning and Systems (MLSys ’25), Santa Clara, CA, 2025.

Affiliation: University of Washington, NVIDIA, Perplexity AI, Carnegie Mellon University.

3.2 Problem Statement

Modern LLM serving attention kernels face three interrelated challenges that existing implementations (FlashAttention2/3, Triton-based kernels) fail to address simultaneously:

1. **KV-cache storage heterogeneity:** Different systems use different KV-cache storage layouts—contiguous tensors, paged attention (page tables), radix trees (RadixAttention), tree attention for speculative decoding, and importance-mask-based sparse caches. Each layout requires a different custom kernel, leading to high maintenance overhead.
2. **Workload dynamism and load imbalance:** Real LLM serving batches contain requests with widely varying KV cache lengths. Naive attention kernel implementations assign equal workload per CTA (Cooperative Thread Array), causing long-tail requests to create SM (Streaming Multiprocessor) idle time.
3. **Attention variant proliferation:** Modern LLMs increasingly use non-standard attention computations—RoPE fused into attention, logits soft-cap (Gemma-2), sliding window attention (Longformer), sigmoid attention (FlashSigmoid), Multi-head Latent Attention (MLA, DeepSeek-V2). Supporting each variant requires a separate kernel specialization, which is unsustainable.

3.3 Core Technical Contributions

3.3.1 Contribution 1: Block-Sparse Format as Unified KV-Cache Abstraction

FlashInfer represents all KV-cache storage layouts under a single **Block Compressed Sparse Row (BSR)** format with configurable block dimensions (B_r, B_c) :

- **Page Table (PagedAttention):** Block size $B_c = \text{page_size}$; indices array stores physical page pointers.
- **Radix Tree (RadixAttention):** Block size $B_c = \text{node_size}$; indices reflect tree node pointers.
- **Sparse mask (Quest-style importance pruning):** Block size $B_c = 1$; non-zero entries indicate selected tokens.
- **Tree Attention (speculative decoding):** Tree-shaped attention patterns are mapped to BSR with appropriate block sizes.

The query and output tensors are stored as **ragged tensors** (jagged arrays) without padding. This eliminates wasted computation on padding tokens in variable-length batches.

3.3.2 Contribution 2: Composable Formats for Memory Efficiency

Single block-sparse formats are constrained to a fixed block size. FlashInfer introduces **composable formats**: the KV cache sparse matrix is decomposed into *multiple* sparse matrices with different block sizes, each processed by a dedicated attention wrapper.

The motivating scenario is prefix sharing (RadixAttention): requests that share a common prefix should access the shared prefix’s KV cache with a larger block size (e.g., $B_r = 3$ for 3 requests sharing a prefix), allowing the shared KV data to be loaded once into shared memory and reused. Unique KV cache entries are stored with block size $(1, 1)$ and accessed per-request. The final attention output is composed via the **Attention State composition operator** $\oplus$ (derived from the log-sum-exp property of softmax):

$$\begin{pmatrix} O(I \cup J) \\ \text{LSE}(I \cup J) \end{pmatrix} = \begin{pmatrix} O(I) \\ \text{LSE}(I) \end{pmatrix} \oplus \begin{pmatrix} O(J) \\ \text{LSE}(J) \end{pmatrix}$$

This operator is associative and commutative, enabling any aggregation order.

3.3.3 Contribution 3: JIT Compiler for Attention Variants

FlashInfer provides a **customizable CUDA/CUTLASS template** that separates attention variant logic from the kernel skeleton. Users specify a variant class with typed functor members:

- **QueryTransform, KeyTransform, ValueTransform:** Per-element transformations before the attention dot products (e.g., fused RoPE, projection).
- **LogitsTransform:** Custom logits post-processing (e.g., logits soft-cap: $\sigma(x/\tau) \cdot \tau$).
- **LogitsMask:** Attention masking (e.g., sliding window, causal, tree attention).
- **OutputTransform:** Post-processing of attention output.

At Python-level specification time, FlashInfer’s **JIT compiler** generates fully specialized CUDA code by template substitution, compiles it with PyTorch’s JIT extension API, and registers it as a custom CUDA operator. This design enables:

- Fusing RoPE + attention into a single kernel (eliminating an extra memory pass).

- Supporting FlashSigmoid, MLA, and other custom variants with only ~ 20 lines of additional code.
- Using CUDA CUTLASS templates (FA2 for Ampere/Ada, FA3 for Hopper) rather than Triton, enabling GPU-architecture-specific features (warp specialization on Hopper, TMA instructions).

3.3.4 Contribution 4: Load-Balanced Dynamic Scheduling

LLM serving batches have variable KV lengths, causing naive GPU kernel schedulers to suffer from **wave quantization**: some SMs are idle while others process long sequences. FlashInfer implements a **load-balanced scheduler** (Algorithm 1 in the paper) inspired by Stream-K:

1. Compute a per-tile cost: $\text{cost}(l_q, l_{kv}) = \alpha l_q + \beta l_{kv}$.
2. Split long KV sequences into chunks of bounded size L_{kv} .
3. Assign chunks to CTAs using a greedy priority queue (longest chunk first) to equalize total cost per CTA.
4. The scheduler generates deterministic aggregation order (no atomic operations), ensuring reproducible outputs.

The scheduler runs on CPU per generation step and copies plan information to GPU asynchronously. Plan information is reused across all Transformer layers in a generation step.

3.3.5 Contribution 5: CUDAGraph Compatibility

Modern serving frameworks use CUDAGraph to capture entire generation steps as compiled graphs for low CPU launch overhead. CUDAGraph requires *static* kernel argument pointers. FlashInfer separates kernel execution into:

- **plan()**: CPU-side function that computes scheduling metadata and copies it to a fixed-offset GPU workspace buffer. This function runs outside CUDAGraph.
- **run()**: GPU kernel execution with fixed pointer arguments (pointing to the workspace buffer). This function is captured by CUDAGraph.

This plan/run split (inspired by the Inspector-Executor model) is the key design that makes FlashInfer compatible with CUDAGraph while still supporting dynamic sequence lengths.

3.4 Microkernel Design

FlashInfer implements the FA2 algorithm across tile sizes $(1, 16, 32, 64, 128) \times (32, 64, 128)$ and selects the optimal tile at compile time based on:

1. Average query length per batch (choose minimum tile size exceeding this).
2. Register and shared memory constraints (maximize SM occupancy).

For GQA (Grouped-Query Attention), FlashInfer uses **head-group fusion**: fuses the head-group dimension with the query length dimension in the threadblock mapping, so a single shared-memory KV load serves all query heads in the group.

For FA3 (Hopper), FlashInfer uses warp specialization and WGMMA instructions with 64-row multiples. TMA (Tensor Memory Accelerator) is used for dense KV-cache loading but falls back to Ampere-style async copies for sparse addressing.

3.5 Experimental Setup

Table 4: FlashInfer experimental configuration.

Component	Details
Hardware	NVIDIA A100 40GB SXM and H100 80GB SXM GPUs
CUDA / PyTorch	CUDA 12.4, PyTorch 2.4.0
Precision	fp16 (storage and computation); also fp8 KV-cache with fp16 queries/outputs
Models	Llama 3.1 8B (1×H100), Llama 3.1 70B (4×H100), Vicuna-13B
Serving Framework	SGLang v0.3.4, MLC-Engine, vLLM
Baseline	SGLang + Triton v3.0 backend; FlashAttention2/3 open-source library; PyTorch SDPA; FlexAttention

3.6 Benchmarks and Evaluation Scenarios

Scenario 1 — End-to-End LLM Serving (SGLang): Fixed request rate maintaining P99 TTFT ≤ 200 ms. Datasets: ShareGPT (real-world conversation lengths) and Variable (uniform $U(512, 2048)$ token lengths). Metrics: Median Inter-Token Latency (ITL, ms) and Median Time-to-First-Token (TTFT, ms).

Scenario 2 — Kernel-Level Decode/Prefill Performance: Batch size 16; sequence lengths: constant (1024), uniform (512–1024), Zipf-skewed (average 1024). MHA, GQA-4, GQA-8 variants. Metrics: bandwidth utilization (%) and FLOPs utilization (%).

Scenario 3 — Long-Context Inference (Streaming-LLM): Vicuna-13B on MT-Bench dataset. FlashInfer fused RoPE+attention kernel vs. unfused FlashAttention. Metrics: ITL (ms) at varying recent window sizes (1000, 2000, 4000 tokens).

Scenario 4 — Parallel Generation (MLC-Engine): ShareGPT dataset; parallel degree $n \in \{1, 2, 4, 8, 16, 32, 64\}$; Llama 3.1 8B and 70B. FlashInfer composable formats vs. single format. Metrics: ITL and TTFT.

Scenario 5 — Ablation: Load-Balancing Scheduler: SGLang + FlashInfer with/without load balancing, vs. Triton baseline. Three datasets: ShareGPT, $U(512, 2048)$, $U(4096, 16384)$. Metrics: ITL and TTFT.

Scenario 6 — Fine-Grained Block Sparsity (Quest algorithm): Batch decoding on H100 SXM5; block size 16; sequence lengths 4096–32768; page budgets 64–512. Metrics: latency (μ s) vs. PyTorch SDPA and FlexAttention.

Scenario 7 — FlexAttention Comparison: 4 attention variants (causal, logits soft-cap, ALiBi, sliding window) on H100 SXM5. Batch 16, heads 16, head dim 128. Metrics: TFLOPs/s at sequence lengths 512–16384.

Scenario 8 — Shared-Prefix Attention: Suffix length 128; shared prefix 1024, 8192, 32768 tokens; batch sizes 16 and 64. Composable format vs. single format. Metrics: latency (μ s).

3.7 Key Results

Table 5: FlashInfer key quantitative results.

Evaluation	Result
End-to-end ITL (SGLang, Triton vs. FlashInfer)	29–69% ITL reduction
Long-context (Streaming-LLM)	28–30% latency reduction
Parallel generation (composable formats)	13–17% speedup (peak at $n = 4$)
Decode bandwidth utilization	FlashInfer consistently > FlashAttention in non-constant sequence length distributions
FlexAttention comparison (causal, 16384 tokens)	FlashInfer: 612 TFLOPs/s vs. FlexAttention: 453 TFLOPs/s (35% faster)
Fine-grained sparsity (Quest, 32768 tokens)	FlashInfer: $\sim 22 \mu\text{s}$ vs. PyTorch SDPA: $\sim 1712 \mu\text{s}$ ($\sim 78\times$ faster)

3.8 GitHub Repository

Repository

<https://github.com/flashinfer-ai/flashinfer>

Language: Python (bindings), CUDA/C++ (kernels), CUTLASS templates.

Key components:

- **flashinfer/**: Python API wrappers, JIT compilation infrastructure.
- **include/flashinfer/**: CUDA/CUTLASS attention kernel templates (FA2 and FA3).
- **csrc/**: C++ binding code.
- **tests/**: Unit and performance tests.
- **python/**: AttentionWrapper, BatchPrefillWrapper, BatchDecodeWrapper APIs.

Deployment: Integrated into vLLM, SGLang, MLC-Engine as an optional attention backend. Installation: `pip install flashinfer`.

3.9 Integration with Modern Systems

- **vLLM**: FlashInfer is an optional attention backend that replaces FlashAttention, providing improved decode ITL especially for fp8 KV-cache workloads.
- **SGLang**: FlashInfer is the default attention backend since v0.3; SGLang uses FlashInfer’s composable formats for RadixAttention prefix sharing.
- **MLC-Engine**: FlashInfer composable formats are used for parallel generation with prefix caching.
- **TensorRT-LLM**: Inspiration for XQA kernel design (cited in FlashInfer).
- **FlexAttention**: FlashInfer can serve as a higher-performance backend for FlexAttention’s forward pass.

4 Paper 3: Helix — Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow

4.1 Bibliographic Information

Citation

Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, Rashmi Vinayak. *Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow*. 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25), Rotterdam, Netherlands, March 2025. **Affiliation:** Carnegie Mellon University.

4.2 Problem Statement

Modern cloud platforms (Google Compute Engine, AWS, Azure) consist of heterogeneous GPU instances—H100, A100, L4, T4, V100—with varying compute capacity, memory, and inter-GPU network bandwidth. These instances are spread across geographic regions with diverse interconnect quality (54–204 Mbps cross-region).

Existing LLM serving systems (ORCA, vLLM) target *homogeneous* GPU clusters and fail to efficiently serve large models across heterogeneous clusters because:

1. **Sub-optimal model placement:** Uniform layer partitioning wastes high-capability GPUs by pacing them to the slowest device.
2. **Inflexible request scheduling:** Fixed pipeline assignment (round-robin) does not account for heterogeneous GPU speeds or network bottlenecks.
3. **GPU scarcity:** Serving models like LLaMA-3 405B requires 68 L4s or 21 H100s—often unavailable in a single region. Geo-distributed serving across regions becomes necessary.

4.3 Core Technical Contributions

4.3.1 Contribution 1: Max-Flow Formulation of LLM Serving

Helix models a heterogeneous cluster as a **directed weighted graph** where:

- Each compute node c_i is represented by two vertices ($c_i^{\text{in}}, c_i^{\text{out}}$) connected by an edge with capacity equal to the node’s *token processing throughput* (tokens/second).
- Each network connection between nodes is represented by a directed edge with capacity equal to bandwidth divided by activation tensor size (for inter-layer connections) or token size (for coordinator-to-worker connections).
- Source and sink vertices represent the coordinator node.

Under a given model placement, the **maximum flow** from source to sink equals the maximum achievable serving throughput of the cluster. This elegant reduction allows Helix to use standard max-flow algorithms to evaluate any model placement.

4.3.2 Contribution 2: MILP-Based Optimal Model Placement

Given the max-flow framework, Helix formulates model placement as a **Mixed Integer Linear Program (MILP)** that finds the model partition maximizing max-flow:

Variables:

- $s_i \in \mathbb{Z}$: First layer held by compute node c_i .
- $b_i^j \in \{0, 1\}$: Whether c_i holds exactly j layers.
- $f_{i,j} \in \mathbb{R}_{\geq 0}$: Flow on connection (c_i, c_j) .
- $d_{i,j} \in \{0, 1\}$: Whether connection (c_i, c_j) is valid under the current placement.

Constraints:

- C1. Model placement validity: each node holds exactly one contiguous layer range.
- C2. Flow conservation at each node.
- C3. Inference throughput: flow through a node $\leq$ its processing capacity.
- C4. Connection validity: linearized via auxiliary binary variables $\text{cond}_{i,j}^1, \text{cond}_{i,j}^2$.
- C5. Transmission throughput: flow on an edge $\leq$ edge capacity.

The MILP has $O(|C| + |E|)$ variables and constraints (linear in cluster size), making it tractable for clusters of 24–42 nodes.

4.3.3 MILP Acceleration Techniques

For large clusters, Helix applies three optimizations:

- **Cluster pruning:** Remove slow network connections, keeping average node degree ≤ 12 . Reduces problem size by 36–72% without significant throughput loss.
- **Heuristic warm-start:** Initialize MILP solver with solutions from Swarm/Petals/SP heuristics, accelerating convergence by 8–43%.
- **Upper-bound early stopping:** The theoretical throughput upper bound (sum of all node capacities / total layers) is used as an early termination criterion.

4.3.4 Contribution 3: Per-Request Pipelines

Traditional heterogeneous serving systems use *fixed pipelines*—a static set of GPU sequences, with requests assigned round-robin. This is inflexible: not all pipeline paths utilize GPU capacities proportionally.

Helix introduces **per-request pipelines**: each request gets its own pipeline determined at scheduling time. The total number of possible pipelines equals the number of source-to-sink paths in the cluster graph, providing much greater flexibility.

Scheduling mechanism. Helix binds an **Interleaved Weighted Round-Robin (IWRR)** scheduler to each node in the topology graph. When routing a request, each node selects the next node with probability proportional to the max-flow solution’s edge flow. This naturally distributes requests according to the optimal flow distribution without creating burst traffic.

4.3.5 Contribution 4: KV-Cache Estimation for Heterogeneous Nodes

Each GPU has limited VRAM for KV-cache. In heterogeneous clusters, KV-cache capacity varies per node. Helix maintains a running estimate of KV-cache usage per node using the average output length. Nodes exceeding the high-water mark are masked out during IWRR scheduling until existing requests on those nodes complete.

4.4 Experimental Setup

Table 6: Helix experimental configuration.

Component	Details
Prototype	1.5k LoC Python + 1.7k LoC C++; built atop vLLM v0.4.0post1; ZeroMQ inter-node communication; Gurobi MILP solver
Simulator	14k LoC Python; models heterogeneous GPUs and network; validated at <5% error vs. prototype
Models	LLaMA-1 30B, LLaMA-2 70B (fp16)
Cluster 1 (single)	4×A100 nodes, 8×L4 nodes, 12×T4 nodes; 10 Gb/s intra-cluster network
Cluster 2 (geo-distributed)	3 sub-clusters: {4 A100}, {2 L4 + 8 T4}, {6 L4 + 4 T4}; 100 Mb/s inter-cluster, 50 ms latency (simulated)
Cluster 3 (high heterogeneity)	42 nodes: 4 A100, 6 V100, 8 L4, 10 T4, 4 2×L4, 6 2×T4, 4 4×T4 (simulated)
Traces	Azure Conversation Dataset (16,657 requests; avg. input 763 tokens, avg. output 232 tokens)
Baselines	Swarm (heterogeneous training heuristic adapted for inference), Separate Pipelines (SP, one pipeline per GPU type), Petals (model placement comparison only)
Metrics	Decode throughput (tokens/s), median/percentile prompt latency (s), median/percentile decode latency (s)

4.5 Benchmarks and Evaluation Scenarios

Scenario 1 — Single Cluster (Online/Offline): 24-node heterogeneous cluster; LLaMA 30B and 70B; real Azure arrival rates (online) vs. full-utilization batch (offline). Validates simulation fidelity (<5% error).

Scenario 2 — Geo-Distributed Clusters: Three geographically separated sub-clusters connected at 100 Mb/s; tests network-aware model placement and congestion avoidance.

Scenario 3 — High GPU-Heterogeneity Cluster: 42 nodes, 7 GPU types; tests Helix’s scaling to extreme heterogeneity with multi-GPU nodes.

Scenario 4 — Model Placement Ablation: Isolates model placement quality by using Helix’s scheduler for all methods; compares MILP placement vs. Swarm/Petals heuristics.

Scenario 5 — Request Scheduling Ablation: Fixes Helix’s model placement; compares IWRR vs. Swarm scheduling vs. random vs. Shortest Queue First (SQF). Reveals congestion patterns in baselines.

Scenario 6 — MILP Optimization Ablation: Evaluates cluster pruning (throughput impact) and heuristic initialization (MILP solving time).

Scenario 7 — Model Placement Quality vs. Solving Time: Tracks MILP best solution and best upper bound over time for LLaMA 30B on 10-node cluster. Confirms optimal solution found in <5 minutes.

4.6 Key Results

Table 7: Helix key results vs. baselines.

Cluster	Model	Result
Single	LLaMA 30B	Helix vs. Swarm: 2.14× offline, 2.07× online throughput; prompt latency −32%, decode latency −12%
Single	LLaMA 70B	Helix vs. Swarm: 1.94× offline, 2.00× online; vs. SP: 1.86× offline, 1.69× online
Geo-distributed	LLaMA 30B	Helix vs. Swarm: 2.41× online, 2.33× offline; prompt latency −66%, decode latency −24%
Geo-distributed	LLaMA 70B	Helix vs. Swarm: 1.92× offline, 1.97× online; Swarm avg. prompt latency 71s vs. Helix 9.5s
High heterogeneity	LLaMA 70B	Helix vs. Swarm: 1.37× offline, 1.48× online; vs. SP: 2.91× offline, 3.29× online

4.7 GitHub Repository

Repository

<https://github.com/Thesys-lab/Helix-ASPLoS25>

DOI: <https://doi.org/10.5281/zenodo.14037926>

Languages: Python (simulator, coordinator, scheduling), C++ (inter-node communication via ZeroMQ).

Key components:

- **helix/:** Core system: coordinator, model placement planner (MILP), per-request pipeline scheduler.
- **simulation/:** 14k-LoC Python simulator for heterogeneous GPU/network.
- **artifact_evaluation/:** Scripts reproducing all paper results.
- **readme.md:** Step-by-step setup guide; requires CUDA 12.6, vLLM 0.4.0post1, Gurobi license.

Hardware requirements: Simulator: any machine with ≥ 32 GB RAM. Prototype: 24-machine cluster with NVIDIA GPUs ($4 \times A100-40GB$, $8 \times L4$, $12 \times T4$), ≥ 10 Gbps inter-node network.

4.8 Integration with Modern Systems

- **vLLM:** Helix’s execution engine is built atop vLLM v0.4.0, reusing its LLM inference optimizations (PagedAttention, etc.) and adding a unified page pool for partial inference support.
- **ZeroMQ:** Used for inter-node communication in the control plane (replacing NCCL-based control message passing).
- **Gurobi:** Used as the MILP solver. Open-source alternatives (HiGHS, GLPK) can be substituted with potentially longer solve times.
- **Geo-distributed deployment:** Tested on Google Compute Engine across multiple regions; complementary to SkyPilot-style cloud resource management.

5 Comparative Analysis

5.1 Problem Space Coverage

The three papers collectively cover the LLM serving stack at three distinct levels, as shown in Figure 1.

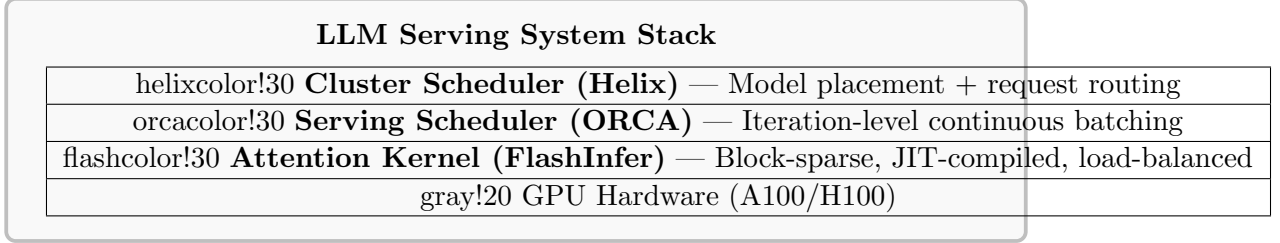


Figure 1: The three papers address different layers of the LLM serving stack.

5.2 Design Philosophy Comparison

Table 8: Design philosophy comparison across the three papers.

Dimension	ORCA	FlashInfer	Helix
Granularity	Per-iteration scheduling	Per-tile kernel scheduling	Per-request pipeline assignment
Optimization target	Reduce request stalls and queueing delay	Maximize GPU SM utilization and minimize redundant memory reads	Maximize cluster-wide serving throughput
Heterogeneity handling	Handles variable output lengths within a batch	Handles variable KV lengths within a batch	Handles variable GPU types and network speeds across nodes
Scalability approach	Model parallelism (intra + inter-layer)	BSR attention kernels scale to arbitrary KV lengths	MILP-based global optimization
Memory management	Pre-allocated K/V slots per request	Block-sparse KV-cache with page table / radix tree abstraction	Per-node KV-cache estimation with high-water mark
Open source	No (commercialized by FriendliAI)	Yes (Apache 2.0)	Yes (Apache 2.0)
Production deployments	FriendliAI, indirectly via vLLM/TGI	vLLM, SGLang, MLC-Engine	Research prototype

5.3 Benchmark Datasets Used

Table 9: Benchmark datasets used in each paper.

Paper	Dataset	Notes
ORCA	Synthesized Poisson traces	$U(32, 512)$ input tokens; $U(1, 128)$ output tokens; no public real-world trace (justified by lack of public LLM traces in 2022)
FlashInfer	ShareGPT (huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered)	Real user-assistant conversations; widely used LLM serving benchmark
FlashInfer	Variable synthetic	Uniform $U(512, 2048)$ token lengths
FlashInfer	MT-Bench	Multi-turn question answering for Streaming-LLM evaluation
Helix	Azure Conversation Dataset	16,657 requests; avg. input 763 tokens, avg. output 232 tokens; real production trace

5.4 Performance Claims Summary

Table 10: Key performance claims from each paper.

Paper	Metric	Baseline	Improvement
ORCA	Throughput at same latency (175B, 16 GPU)	FasterTransformer	36.9×
ORCA	Engine execution time (175B, no pipeline)	FasterTransformer	up to 47% faster
FlashInfer	ITL (online serving, H100)	SGLang+Triton	29–69% reduction
FlashInfer	Latency (long-context, Streaming-LLM)	Unfused FlashAttention	28–30% reduction
FlashInfer	ITL speedup (parallel generation, $n = 4$)	Single-format MLC-Engine	13–17%
FlashInfer	Fine-grained sparsity (Quest, 32768 tokens)	PyTorch SDPA	~78×
Helix	Decode throughput (geo-distributed, 30B)	Swarm	2.41×
Helix	Prompt latency (geo-distributed, 30B)	Swarm	–66%
Helix	Decode throughput (high heterogeneity, 70B)	Separate Pipelines	3.29×

5.5 Limitations and Open Problems

5.5.1 ORCA

- The K/V manager requires pre-reserving `max_tokens` memory per request, potentially over-allocating. This was later addressed by vLLM’s PagedAttention.
- No memory virtualization—if the pool fills up, scheduling stalls. PagedAttention resolves this.
- Evaluation uses only synthesized traces; no real-world LLM request workloads.
- Does not address prefill-decode disaggregation (addressed by later work: Splitwise, DistServe).

5.5.2 *FlashInfer*

- Forward pass only; no backward pass (limits training applications).
- Sparse gathering on FA3 (Hopper) incurs $\sim 10\%$ overhead vs. dense due to TMA limitations.
- JIT compilation adds initialization latency (mitigated by kernel caching).
- Python overhead in vLLM integration limits improvement (seen in Table 8: negligible speedup for bf16).
- Does not address multi-node serving or model placement.

5.5.3 *Helix*

- MILP solving can take hours for clusters >40 nodes without heuristic initialization.
- Prototype tested only on single-GPU nodes; multi-GPU nodes are simulated.
- KV-cache estimation uses average output length—can be inaccurate for skewed distributions.
- Does not incorporate ORCA-style continuous batching within each node.
- Requires Gurobi license for large cluster optimization (though academic licenses exist).
- Partial inference (a request inferring only a subset of layers on a node) adds implementation complexity.

6 Literature Survey Context and Related Work

6.1 Foundational Works Referenced

Table 11 organizes the key references cited across the three papers by category.

Table 11: Key related works referenced in the three papers.

Category	System / Paper	Relevance
Execution Engines	FasterTransformer (NVIDIA)	ORCA baseline; also referenced by FlashInfer, Helix
	LightSeq	Transformer inference engine
	TurboTransformers	Variable-length optimized engine
	TensorRT-LLM	NVIDIA production engine
Serving Systems	Triton Inference Server	Request-level serving, ORCA baseline
	TensorFlow Serving	Classical serving framework
	Clipper	Low-latency serving with batching
Memory Management	vLLM / PagedAttention	Addresses ORCA’s pre-allocation limitation
	RadixAttention (SGLang)	Prefix-sharing KV cache management
	vAttention	Contiguous KV-cache via GPU virtual memory
Attention Kernels	FlashAttention 1/2/3	Foundation for FlashInfer
	Flash-Decoding	Split-K decode attention
	ByteTransformer	Variable-length FMHA
	FlexAttention	PyTorch-level attention customization
Parallelism	Megatron-LM	Tensor + pipeline parallelism for training
	GPipe	Pipeline parallelism with microbatches
	Petals	Decentralized volunteer serving
Scheduling	Sarathi-Serve	Chunked prefill, piggyback decode
	Splitwise / DistServe	Prefill-decode disaggregation
	BatchMaker	RNN cell-level batching (ORCA’s closest prior work)
Cluster Opt.	SWARM	Heterogeneous training heuristic
	HexGen	Fixed-pipeline heterogeneous serving
Models	GPT-3 (Brown et al., 2020)	Primary model in ORCA evaluation
	LLaMA (Meta) 1/2/3	Primary models in Helix and FlashInfer

6.2 Chronological Research Trajectory

The three papers represent distinct generations of LLM serving research:

Gen 1: ORCA (2022): Identified the fundamental scheduling inefficiency and introduced continuous batching. This work enabled practical LLM serving at scale and spawned an entire ecosystem.

Gen 2: vLLM/PagedAttention (2023, not surveyed): Extended ORCA by solving the memory fragmentation problem with virtual memory-inspired KV cache management.

Gen 3: FlashInfer (2025): Addressed the remaining bottleneck at the kernel level—attention computation efficiency, KV-cache heterogeneity, and attention variant flexibility.

Gen 4: Helix (2025): Addressed the cluster-level challenge—serving across heterogeneous and geo-distributed GPU clusters that are increasingly the reality of cloud deployment.

6.3 Key Research Gaps for Future Work

Based on the limitations identified above, the following research directions remain open:

- **Combined scheduling across all layers:** None of the three papers fully co-optimizes iteration-level batching (ORCA), kernel-level tile scheduling (FlashInfer), and cluster-level placement (Helix) simultaneously.
- **Speculative decoding integration:** All three papers lack comprehensive treatment of speculative decoding, which introduces tree-structured KV caches and non-uniform batch dynamics.
- **Multi-modal serving:** Vision-language models introduce heterogeneous computational graphs (CNN + Transformer) that challenge all three approaches.
- **Dynamic cluster membership:** Helix assumes a static cluster; handling node failures or dynamic GPU availability (spot instances) requires online re-planning.
- **Energy efficiency:** None of the papers report energy consumption or optimize for it—increasingly important at datacenter scale.
- **Privacy-preserving serving:** Inference on sensitive data across geo-distributed nodes (as in Helix) requires attention to data locality and confidentiality.

7 Summary and Recommendations for Literature Survey

7.1 Must-Read Companion Papers

For a complete literature survey in LLM serving, the following papers should be read alongside the three surveyed here:

1. **vLLM (SOSP '23):** Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention.” Directly extends ORCA with virtual memory for KV cache.
2. **FlashAttention-2 (ICLR '24):** Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.” Foundation for FlashInfer.
3. **FlashAttention-3 (2024):** Shah et al., “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision.” FA3 template used by FlashInfer on Hopper.
4. **Sarathi-Serve (OSDI '24):** Agrawal et al., “Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve.” Chunked prefill to reduce pipeline bubbles.
5. **DistServe / Splitwise (OSDI '24 / ISCA '24):** Prefill-decode disaggregation—an orthogonal optimization to all three surveyed papers.
6. **SGLang (arXiv '23):** Zheng et al. RadixAttention prefix caching, which motivates FlashInfer’s composable formats.
7. **Petals (arXiv '22):** Borzunov et al. Decentralized LLM serving—Helix’s main counterpart for heterogeneous serving.

7.2 Experimental Reproducibility Summary

Table 12: Reproducibility assessment of the three papers.

Aspect	ORCA	FlashInfer	Helix
Code availability	Not public (commercialized)	Fully open source (Apache 2.0)	Fully open source (Apache 2.0)
Artifact evaluation	N/A	Yes (MLSys AE)	Yes (ASPLOS AE); scripts for all figures
Dataset availability	Synthetic only	ShareGPT (public), synthetic	Azure Conversation (parsed version in repo)
Hardware requirements	Azure ND96 A100 VMs (~\$10k/day)	Any A100 or H100 GPU	24-machine GPU cluster for prototype; single machine for simulator
Software dependencies	Proprietary	PyTorch, CUDA 12.4, pip-installable	Python 3.10, Gurobi, vLLM 0.4.0
Estimated reproduction cost	High (unavailable)	Medium (single GPU)	High (cluster) / Low (simulator)

7.3 Takeaways

- T1:** **ORCA** established the principle that the *iteration* is the natural unit of LLM serving scheduling. This insight is now universally adopted and is the foundation of all modern LLM serving systems.
- T2:** **FlashInfer** demonstrates that a unified, hardware-aware attention abstraction (block-sparse + JIT-compiled) can simultaneously address storage heterogeneity, workload dynamism, and attention variant diversity with a single codebase—integrated into three major serving frameworks.
- T3:** **Helix** shows that treating heterogeneous cluster serving as a max-flow optimization problem yields significantly better model placement than heuristics, particularly in geo-distributed scenarios where network bandwidth becomes a first-class constraint.
- T4:** Together, these three papers suggest that future LLM serving systems must be **co-designed across scheduling, kernel, and cluster layers** to achieve optimal performance—a vertical integration challenge that remains largely open.

References

- [1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “ORCA: A Distributed Serving System for Transformer-Based Generative Models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’22)*, 2022, pp. 521–538.
- [2] Z. Ye et al., “FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving,” in *8th Conference on Machine Learning and Systems (MLSys ’25)*, 2025.
- [3] Y. Mei et al., “Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow,” in *30th ACM ASPLOS*, 2025.
- [4] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *ACM SOSP*, 2023.
- [5] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” in *ICLR*, 2024.
- [6] J. Shah et al., “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision,” arXiv:2407.08608, 2024.
- [7] A. Agrawal et al., “Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve,” in *OSDI ’24*, 2024.
- [8] P. Patel et al., “Splitwise: Efficient Generative LLM Inference Using Phase Splitting,” in *ISCA*, 2024.
- [9] Y. Zhong et al., “DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving,” in *OSDI ’24*, 2024.
- [10] L. Zheng et al., “Efficiently Programming Large Language Models Using SGLang,” arXiv:2312.07104, 2023.
- [11] A. Borzunov et al., “Petals: Collaborative Inference and Fine-Tuning of Large Models,” arXiv:2209.01188, 2022.
- [12] T. Brown et al., “Language Models are Few-Shot Learners” (GPT-3), in *NeurIPS*, 2020.
- [13] M. Shoenberger et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” arXiv:1909.08053, 2019.
- [14] Y. Huang et al., “GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism,” in *NeurIPS*, 2019.
- [15] M. Ryabinin et al., “SWARM Parallelism: Training Large Models Can Be Surprisingly Communication-Efficient,” in *ICML*, 2023.
- [16] A. Vaswani et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [17] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [18] H. He et al., “FlexAttention: The Flexibility of PyTorch with the Performance of FlashAttention,” PyTorch Blog, 2024.
- [19] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” in *EMNLP*, 2023.
- [20] R. Rajbhandari et al., “DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training,” arXiv:2201.05596, 2022.

Literature Survey Report

LLM Inference: Scheduling Theory & Memory Management

Papers Surveyed

Paper 1: *Queueing, Predictions, and LLMs: Challenges and Open Problems*
Michael Mitzenmacher & Rana Shahout Harvard University
arXiv:2503.07545 (Tutorial at SIGMETRICS 2025)

Paper 2: *Efficient Memory Management for Large Language Model Serving with PagedAttention*

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica
UC Berkeley / Stanford / UC San Diego

SOSP 2023 arXiv:2309.06180 GitHub: <https://github.com/vllm-project/vllm>

May 11, 2026

Contents

1	Overview and Motivation	3
2	Paper 1 — Queueing, Predictions, and LLMs	3
2.1	Problem Statement and Motivation	3
2.2	Part I — Scheduling with Job Size Predictions (Queueing Theory)	4
2.2.1	Standard Model Extensions (Section 2.1)	4
2.2.2	1-Bit Predictions (Section 2.2)	4
2.2.3	Uniform Bounds and Graceful Degradation (Section 2.3)	4
2.2.4	Accounting for Prediction Costs (Section 2.4)	5
2.2.5	Multiple Server Systems (Section 2.5)	5
2.3	Part II — LLM Systems (Sections 3–6)	5
2.3.1	LLM Inference Background (Section 3)	5
2.3.2	Scheduling in a Single LLM (Section 4)	6
2.3.3	Compound AI Systems (Section 5)	6
2.3.4	LLM Reasoning Systems (Section 6)	7
2.4	Open Problems Catalogue	7
2.5	GitHub / Artefacts	7
2.6	Experimental Evidence Cited	7
3	Paper 2 — PagedAttention and vLLM (SOSP 2023)	8
3.1	Problem Statement and Motivation	8
3.2	Core Novelty — PagedAttention	8
3.2.1	The Paging Analogy	8
3.2.2	PagedAttention Algorithm	8
3.2.3	KV Cache Manager	9
3.2.4	Memory Sharing for Decoding Algorithms	9
3.3	Scheduling and Preemption	9
3.4	System Architecture and Implementation	10
3.4.1	Custom Kernel Optimisations	10
3.5	Experimental Setup	10
3.5.1	Hardware	10
3.5.2	Datasets and Workloads	11
3.5.3	Baselines	11
3.5.4	Metrics	11
3.6	Key Results	11
3.7	GitHub Repository Analysis	12
3.8	Ablation Studies (Section 7)	12
3.9	Limitations	12
4	Comparative Analysis	13
4.1	Methodological Comparison	13
4.2	Complementarity	13
5	Literature Landscape and Positioning	14
5.1	Key Related Systems	14
5.2	Benchmark Datasets Used Across the Literature	15
5.3	Optimisation Taxonomy	15

6	Critical Assessment	15
6.1	Strengths	15
6.2	Weaknesses and Open Questions	16
7	Summary Tables for Literature Survey Reference	16
7.1	Quick-Reference: Key Numbers	17
7.2	Recommended Citation Keywords	18
8	Conclusion	18

1 Overview and Motivation

These two papers address complementary dimensions of the same overarching challenge: *how to serve LLM inference requests efficiently at scale*. Paper 1 (Mitzenmacher & Shahout) approaches it through the lens of **queueing theory and predictions**, surveying open theoretical problems and introducing novel models tailored to LLM scheduling. Paper 2 (Kwon et al.) addresses it through **systems engineering**, solving the KV cache memory fragmentation problem that limits batch size and hence throughput in deployed LLM serving engines.

Paper 1 — Queueing+LLMs (SIGMETRICS 2025)	Paper 2 — vLLM/PagedAttention (SOSP 2023)
Survey + open problems. Bridges classical queueing theory and modern LLM scheduling.	Systems paper. Introduces PagedAttention and builds vLLM, a production-grade LLM serving engine.
Theoretical: M/G/1 queues, SRPT, prediction models, SOAP analysis.	Practical: virtual memory-inspired KV cache management, near-zero waste, 2–4× throughput improvement.
Novel scheduling algorithms: SPRPT, Skip-Predict, DelayPredict, SPRPT-with-bounce.	Novel memory algorithm: PagedAttention with block tables, copy-on-write, parallel sampling, beam search support.
No code artefact; companion tutorial given at SIGMETRICS 2025.	Open-source: https://github.com/vllm-project/vllm — widely adopted in production.

2 Paper 1 — Queueing, Predictions, and LLMs

2.1 Problem Statement and Motivation

Modern LLM serving is fundamentally a queueing problem: inference requests arrive stochastically, each with an unknown service time (output length), and the scheduler must decide how to order and batch them to minimise latency and maximise throughput. Classical scheduling policies handle two extreme cases well: FIFO (no information) and SRPT (exact service times known). The vast middle ground — where machine-learned *predictions* of service time are available but imperfect — is poorly understood theoretically.

Core Gap

The field of *algorithms with predictions* (learning-augmented algorithms) has demonstrated great promise across computer science, but its application to *queueing* systems, and specifically to LLM inference scheduling, remains largely unexplored. This paper aims to bridge that gap by surveying known results, identifying open problems, and introducing new models tailored to the peculiarities of LLM systems.

LLM inference introduces several features not found in classical queueing models:

- **Unknown output lengths:** the job size (number of output tokens) is not known when a request arrives; it must be predicted.
- **KV cache memory growth:** each token generated extends the KV cache, so GPU memory usage grows linearly during service.

- **Costly preemption:** interrupting a request requires handling its KV cache (preserve in GPU memory, swap to CPU, or discard and recompute).
- **Multiple phases:** prefill (compute-bound) and decode (memory-bandwidth-bound) have different resource demands.
- **Multiple objectives:** latency, throughput, cost, and accuracy must be jointly optimised.

2.2 Part I — Scheduling with Job Size Predictions (Queueing Theory)

2.2.1 Standard Model Extensions (Section 2.1)

The paper starts with the M/G/1 queue and extends it to settings where each job has both a true service time x and a predicted service time y , drawn jointly from a density $g(x, y)$. Three prediction-based scheduling policies are studied:

- **SPRPT** (Shortest Predicted Remaining Processing Time): rank by predicted remaining time $y - a$ (age a of service so far).
- **SPJF** (Shortest Predicted Job First): non-preemptive, rank by y at arrival.
- **PSPJF** (Preemptive Shortest Predicted Job First): preempt based on predicted size.

Key result: using predictions is substantially better than FIFO but not as good as using exact sizes. For example, at load $\lambda = 0.8$ with exponentially distributed predictions of mean x , FIFO gives expected response time 5.0, SPJF gives 3.38, and SRPT gives 2.35. The analysis is done via direct equations and the **SOAP methodology** (Schedule-Ordered-by-Age-based Policies), which analyses M/G/1 variants whose scheduling rank depends only on a job's type and the time already served.

2.2.2 1-Bit Predictions (Section 2.2)

An important simplification: classify each job as “short” or “long” using a single bit (threshold T on the predicted service time). Short jobs go to the front, long jobs to the back — preemptive or not.

Key closed forms for the M/M/1 model (Poisson arrivals, exponential service, exponential prediction model):

$$t_1 = \frac{\lambda(1 - \lambda(1 - 2\sqrt{T}K_1(2\sqrt{T})))}{(1 - \lambda)(1 - \lambda(1 - 2TK_2(2\sqrt{T})))} + 1 \quad (\text{no preemption})$$

$$t_2 = \frac{1 - \lambda + \lambda^2\sqrt{T}K_1(2\sqrt{T})}{(1 - \lambda)(1 - \lambda(1 - 2TK_2(2\sqrt{T})))} \quad (\text{with preemption})$$

where K_1, K_2 are modified Bessel functions of the second kind. Notably, $t_1 = \lambda t_2 + 1$, so preemption is always beneficial.

Insight: Long-Job Prediction Matters More

1-bit predictions achieve a large fraction of full-prediction performance. Furthermore, mispredicting a *long* job as short is far more damaging than mispredicting a short job as long: a misclassified long job blocks many short jobs behind it. This means the right metric for evaluating 1-bit predictors is not overall accuracy but *recall on long jobs*.

2.2.3 Uniform Bounds and Graceful Degradation (Section 2.3)

Scully, Grosz & Mitzenmacher (2022) study bounded multiplicative errors: if a job of size s has predicted size in $[\beta s, \alpha s]$. Key results:

- Plain SPRPT does *not* achieve bounded competitive ratio even with bounded multiplicative errors.
- **SPRPT-with-bounce**: the rank function decreases from z to 0 then *bounces* back up via $\max(|z - a|, z)$. This is both *consistent* (converges to SRPT as errors vanish) and achieves *graceful degradation* with constant $C = 3.5$.
- **PSPJF** achieves graceful degradation with constant $C = 1.5$.
- Without bounded multiplicative errors, no constant robustness is achievable.

2.2.4 Accounting for Prediction Costs (Section 2.4)

All prior models assume predictions are free. The paper analyses two cost models:

Model	Mechanism	Key Insight
External Cost	Prediction provided externally at fixed cost c ; does not slow server.	May be worth predicting only some jobs.
Server Time Cost	Prediction requires server time; slows the system. Heavy loads may destabilise the system.	Scheduling predictions is itself a scheduling problem.

Two novel scheduling algorithms:

- **SkipPredict**: use cheap 1-bit prediction for all jobs; use expensive full prediction only for jobs predicted to be long. Short predicted jobs: FCFS. Long predicted jobs: SPRPT. Analysed via two-dimensional SOAP ranking.
- **DelayPredict**: no cheap predictions; run all jobs FCFS up to a time limit L , then preempt and queue for full prediction before SPRPT. Avoids prediction cost for short jobs entirely.

2.2.5 Multiple Server Systems (Section 2.5)

Prediction-based scheduling in multi-server systems is largely open. Notable result: Dong & Ibrahim (2024) show shortest predicted job first asymptotically maximises throughput among non-preemptive policies in $M/GI/s+GI$ systems (many-server queues with impatient customers). The power-of-two-choices paradigm with 1-bit predictions is also studied via fluid limits.

2.3 Part II — LLM Systems (Sections 3–6)

2.3.1 LLM Inference Background (Section 3)

The paper provides a thorough background covering:

Architecture. Decoder-only transformer; self-attention with matrices Q, K, V ; attention output $O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) VW_o + X$.

KV cache. During decode, previously computed (K, V) pairs are cached to avoid recomputation. The cache grows linearly with the output length, tying up GPU memory for the request’s entire lifetime.

Prefill vs. decode. Prefill is compute-bound (all prompt tokens processed in parallel); decode is memory-bandwidth-bound (one token per step, sequential, large KV cache transfer).

Performance metrics defined.

- Latency: $t_{\text{response}} = t_{\text{waiting}} + \text{TTFT}(n_{\text{input}}) + n_{\text{output}} \cdot \text{TPOT}$
- TTFT (Time to First Token); TPOT (Time Per Output Token); Throughput (tokens/s); Computational cost; Energy / carbon footprint.

2.3.2 Scheduling in a Single LLM (Section 4)

Dynamic batching and preemption (4.1). Iteration-level (continuous) batching (Orca) allows per-token scheduling: after each token generation the batch is updated. Size-based scheduling (SRPT, SPJF) requires output size predictions. Key systems discussed:

System	Prediction approach	Limitation
S ³	Fine-tuned BERT classifies output length bucket	Accuracy degrades for variable outputs
LTR	Learning-to-Rank on output size; ranking simpler than regression	Ignores prompt length; head-of-line blocking
FastServe	Multi-Level Feedback Queue (no prediction); proactive GPU/CPU memory offloading	Frequent preemptions
Trail	Probing: recycles intermediate embeddings for lightweight size prediction; SPRPT with age threshold	Requires model-internal access

Trail’s key idea: disable preemption once a request’s age exceeds $c \cdot \hat{s}$ (fraction of predicted size), since the KV cache is large and recomputation cost dominates for old requests. Analysable via SOAP.

Adaptive scheduling (4.2). Covers prompt sharing (85–97% token overlap in many LLM applications), KV cache reuse via RadixAttention (SGLang, up to $5\times$ throughput gain over vLLM), and SLA-aware scheduling (SAGESERVE for mixed interactive/batch).

GPU resource allocation (4.3). Two GPU organisations: (a) pooled GPUs (each handles both prefill and decode), (b) dedicated GPUs (separate prefill/decode pools, analogous to tandem queues in queueing theory). Dedicated GPUs require KV cache transfer between pools.

2.3.3 Compound AI Systems (Section 5)

Augmented LLMs with API calls (5.1). An API-augmented request has three phases: prefill, API call, and post-API decode. Three KV cache strategies during API call: Preserve (memory occupied but no recomputation), Discard+Recompute (wastes compute), Swap (CPU transfer overhead). **LAMPS** is the first system to go beyond FIFO for augmented LLMs, achieving 27–85% end-to-end latency reduction and 4–96% TTFT reduction over INFERCEPT.

Multiple LLMs (5.2 and 5.3). Speculative decoding (small model drafts, large model verifies) is modelled as a tandem queue. Multi-LLM pipelines (DAG-based or agent-based) require new scheduling frameworks; existing queueing results for tandem and multi-server queues provide partial insights.

2.3.4 LLM Reasoning Systems (Section 6)

Reasoning LLMs (chain-of-thought, Monte Carlo tree search, majority voting) generate interdependent inference requests. The “size” of a reasoning job includes the number of reasoning steps required for convergence, not just the output token count. **Dynasor** allocates more compute to hard queries and terminates unpromising ones early (Certaindex metric). Adaptive scheduling that responds to intermediate reasoning outputs is an open problem.

2.4 Open Problems Catalogue

The paper poses >20 open questions. The most important for a literature survey are:

Problem area	Open question
Prediction models	Realistic models of prediction error distributions across domains
Tool support	Tool to numerically compute M/G/1 performance given a prediction model
SOAP extensions	Generalise SOAP to policies that depend on queue length, not just age
k-bit predictions	How does performance scale from 1-bit to k -bit predictions?
Prediction costs	Policies that dynamically decide when to predict based on load
Multi-server theory	Extend fluid limits for power-of-two-choices with full predictions
Prefill vs. decode	How to rank requests across phases; split-phase scheduling
Preemption thresholds	Dynamic tuning of preemption age thresholds based on request distribution
API-augmented	Theoretical guarantees for scheduling with API calls
Tandem queues	Speculative decoding as tandem queue; optimisation insights
Reasoning systems	Unified scheduling for expansion vs. aggregation phases

2.5 GitHub / Artefacts

Paper 1 is a survey/tutorial; it has *no* accompanying code repository. The companion tutorial was presented at SIGMETRICS 2025. The paper points to <https://algorithms-with-predictions.github.io> for a curated list of related works.

2.6 Experimental Evidence Cited

Paper 1 presents theoretical results and simulation tables from prior work. Key numerical results cited include:

- Table 1: Expected response times for FIFO, SJF, SPJF, PSJF, PSPJF, SRPT, SPRPT at various loads ($\lambda = 0.5\text{--}0.99$) with exponential predictions. SPRPT achieves response time within 10–20% of SRPT.
- Tables 2 and 3: Simulation results for 1-bit predictions with exponential and Weibull service distributions. 1-bit predictions recover $\sim 70\text{--}80\%$ of the gain from full predictions.

- Trail: $1.66\text{--}2.01\times$ mean latency reduction on Alpaca; $1.76\text{--}24.07\times$ lower TTFT vs. vLLM on a single NVIDIA A100 80GB GPU.
- LAMPS: 27–85% E2E latency reduction over INFERCEPT for API-augmented LLMs.

3 Paper 2 — PagedAttention and vLLM (SOSP 2023)

3.1 Problem Statement and Motivation

High-throughput LLM serving requires batching many requests simultaneously. The fundamental obstacle is **KV cache memory management**:

- For a 13B-parameter OPT model, each token’s KV cache requires $2 \times 5120 \times 40 \times 2 = 800$ KB. A single 2048-token request can need up to **1.6 GB**.
- On a 40 GB A100 GPU with a 13B model (26 GB parameters), less than 12 GB is available for KV cache — enough for only $\sim 15,700$ KV slots.
- Existing systems (Orca, FasterTransformer) pre-allocate *contiguous* memory chunks of maximum possible length. This wastes memory via: (1) *internal fragmentation* (unused reserved slots), (2) *external fragmentation* (gaps between differently-sized allocations), (3) *reservation waste* (slots reserved but not yet used).

Key Measurement

Profiling shows that existing systems use only **20.4%–38.2%** of KV cache memory for actual token states. The rest is wasted. This directly limits batch size and thus throughput.

Additionally, memory sharing opportunities (parallel sampling, beam search, shared prompts) are impossible in contiguous-allocation systems because each request’s KV cache occupies a separate contiguous region.

3.2 Core Novelty — PagedAttention

3.2.1 The Paging Analogy

PagedAttention is inspired directly by the OS virtual memory system with paging: tokens $\leftrightarrow$ bytes, KV blocks $\leftrightarrow$ pages, requests $\leftrightarrow$ processes.

OS concept	vLLM analogue
Virtual page	Logical KV block
Physical page frame	Physical KV block (on GPU DRAM)
Page table	Block table (logical $\rightarrow$ physical mapping per request)
Demand paging	Allocate physical blocks on demand as tokens are generated
Copy-on-write	Copy-on-write at block granularity (beam search, parallel sampling)
Swapping to disk	Swapping KV blocks to CPU RAM

3.2.2 PagedAttention Algorithm

The KV cache of each request is partitioned into *KV blocks* of fixed size B (default $B = 16$ tokens). Blocks are stored in *non-contiguous* physical memory. The attention computation is reformulated block-wise:

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top$$

where K_j , V_j are the key and value blocks respectively. A GPU warp is assigned per block to ensure coalesced memory access.

3.2.3 KV Cache Manager

- **Block table:** maps logical KV blocks $\rightarrow$ physical KV blocks per request. Records physical block number and number of filled positions.
- **Dynamic allocation:** physical blocks allocated on demand as tokens are generated; maximum of one block of internal fragmentation per request (the partially-filled last block).
- **Reference counting:** physical blocks track how many logical blocks (across requests) map to them, enabling sharing.
- **Copy-on-write:** when a shared block must be written (e.g., at divergence in beam search), a new physical block is allocated and data is copied; reference count is decremented.

3.2.4 Memory Sharing for Decoding Algorithms

Parallel sampling. k output sequences share the prompt’s KV blocks (logical blocks of all sequences map to the same physical blocks). Generation-phase blocks are managed via copy-on-write. Memory saving: 6–30.5% depending on workload.

Beam search (width k). Beam candidates share large portions of the KV cache dynamically; sharing pattern evolves as the tree grows and prunes. Memory saving: 37.6–66.3%. This reduces expensive cross-candidate KV cache copies from $O(k)$ blocks to $O(1)$ (copy-on-write only at divergence points).

Shared prefix / system prompt. Provider pre-computes and caches KV blocks for a fixed system prompt. New requests with the same prefix map their logical blocks to the cached physical blocks (last block marked copy-on-write). Throughput gain: $1.67\times$ for 1-shot prefix; $3.58\times$ for 5-shot prefix vs. Orca (Oracle).

Mixed decoding. Because all sharing is abstracted via the block table, the LLM kernel sees only physical block IDs; requests with different decoding modes coexist transparently in the same batch.

3.3 Scheduling and Preemption

vLLM uses **First-Come-First-Served (FCFS)** scheduling with an all-or-nothing preemption policy: either all blocks of a request are in GPU memory or none. Sequences in the same request (beam candidates) are gang-scheduled.

Two recovery mechanisms for preempted requests:

	Swapping	Recomputation
Mechanism	Copy KV blocks to CPU RAM; reload when request resumes.	Concatenate generated tokens with prompt; re-run prefill.
Best for	Large block sizes (amortises PCIe transfer setup cost).	Small block sizes; recompute latency $<20\%$ of swap latency.
Overhead	Can be high for small blocks (many small PCIe transfers).	Constant regardless of block size.

For medium block sizes (16–64), the two methods perform comparably. vLLM defaults to block size 16 (best balance across ShareGPT and Alpaca workloads).

3.4 System Architecture and Implementation

Component	Details
Scheduler	Centralised; FCFS; preemption via swapping or recomputation
KV Cache Manager	GPU + CPU block allocators; block tables per request
Workers	N GPU workers (tensor model parallelism, Megatron-LM style)
Attention kernel	Custom PagedAttention CUDA kernel; warp per block
Language	8,500 lines Python + 2,000 lines C++/CUDA
Frontend	FastAPI; extends OpenAI API
Models	GPT, OPT, LLaMA (and later: Mistral, Falcon, Qwen, etc.)
Parallelism	Tensor model parallelism (allreduce between workers)
Communication	NCCL for GPU-to-GPU tensor communication

3.4.1 Custom Kernel Optimisations

1. **Fused reshape and block write:** splits new KV cache into blocks, reshapes for optimal memory layout, saves at positions from block table — all in one kernel to minimise launch overhead.
2. **Fused block read and attention:** adapts FasterTransformer attention kernel to read KV cache according to the block table; one GPU warp per block for coalesced access; supports variable sequence lengths.
3. **Fused block copy:** batches copy-on-write operations across multiple blocks into a single kernel launch (avoids N small `cudaMemcpyAsync` calls).

Overhead vs. FasterTransformer: **20–26%** higher attention kernel latency due to block table indirection. This is more than compensated by the $2\text{--}22\times$ end-to-end throughput gain from larger effective batch sizes.

3.5 Experimental Setup

3.5.1 Hardware

Model	Hardware	GPU Memory
OPT-13B	1× NVIDIA A100 (40 GB)	40 GB
OPT-66B	4× NVIDIA A100 (40 GB)	160 GB
OPT-175B	8× NVIDIA A100 (80 GB)	640 GB
LLaMA-13B	1× NVIDIA A100 (40 GB)	40 GB

Platform: Google Cloud Platform (A2 instances). Duration: 1-hour traces (15 minutes for OPT-175B due to cost).

3.5.2 Datasets and Workloads

Dataset	Characteristics	Used for
ShareGPT	Long conversations with ChatGPT; mean input 161 tokens, output 338 tokens; high variance; 8.4× longer than Alpaca on average	Main throughput evaluation
Alpaca	GPT-3.5 instruction dataset; mean input 19 tokens, output 58 tokens; low variance; short sequences	Parallel sampling, beam search, shared prefix
WMT16 En-De	English-to-German translation; 1-shot and 5-shot prefix prompts	Shared prefix evaluation

Arrival times synthesised as Poisson with varying request rates.

3.5.3 Baselines

Baseline	Description
FasterTransformer	NVIDIA’s high-performance transformer inference engine; custom scheduler with dynamic batching; reserves max-length KV cache (most wasteful).
Orca (Max)	Iteration-level scheduling; always reserves 2048-token KV cache (maximum sequence length).
Orca (Pow2)	Iteration-level scheduling; reserves 2× next power of 2 above actual output length (known post-hoc).
Orca (Oracle)	Iteration-level scheduling; knows actual output length in advance (infeasible upper bound).

3.5.4 Metrics

- **Normalised latency** (s/token): mean of (E2E latency / output length) per request — primary metric.
- **Request rate** (req/s): swept from low to maximum sustainable.
- **Average number of batched requests**: measures effective GPU utilisation.
- **KV cache memory saving** (%): measured for sharing experiments.
- **Kernel latency** (μ s): PagedAttention vs. FasterTransformer.

3.6 Key Results

Basic Sampling — ShareGPT

- vLLM sustains **1.7–2.7×** higher request rates than Orca (Oracle) at the same normalised latency.
- vLLM sustains **2.7–8×** higher rates than Orca (Max).
- vLLM sustains up to **22×** higher rates than FasterTransformer.
- vLLM batches **2.2×** more requests simultaneously than Orca (Oracle) and **4.3×** more than Orca (Max) for OPT-13B.

Parallel Sampling and Beam Search — Alpaca

- Beam search width 6: vLLM achieves **2.3×** higher throughput than Orca (Oracle) (vs. 1.3× for basic sampling).
- Memory saving: 6.1–9.8% for parallel sampling; **37.6–55.2%** for beam search.

Shared Prefix — WMT16

- 1-shot prefix: **1.67×** higher throughput than Orca (Oracle).
- 5-shot prefix (341 tokens shared): **3.58×** higher throughput than Orca (Oracle).

Memory Efficiency

vLLM reduces KV cache memory waste to **3.7%** (vs. 20.4–61.8% in Orca variants and FasterTransformer), approaching the theoretical near-zero waste target.

3.7 GitHub Repository Analysis

URL: <https://github.com/vllm-project/vllm>

Attribute	Detail
Original codebase	8,500 lines Python + 2,000 lines C++/CUDA
Current scale	One of the most-starred LLM serving repositories on GitHub
Models supported	GPT-2/3, OPT, LLaMA (1/2/3), Mistral, Falcon, Qwen, Gemma, Phi, DeepSeek
Backends	NVIDIA CUDA, AMD ROCm, Google TPU, CPU (limited)
Serving modes	Online (OpenAI-compatible API via FastAPI), Offline batch
Key features added post-paper	Chunked prefill, prefix caching (RadixAttention-style), speculative decoding
Integration	Hugging Face transformers, LangChain, LlamaIndex, Ray Serve, Kubernetes
Benchmarking scripts	benchmarks/ directory: latency, throughput, serving benchmarks; compatibility

The vLLM project has become the *de facto* standard open-source LLM serving engine, widely used in both academic research and production deployments (Anyscale, various cloud providers).

3.8 Ablation Studies (Section 7)

Study	Finding
Attention kernel overhead	20–26% higher latency vs. FasterTransformer due to block table indirection; offset by higher batch sizes.
Block size sensitivity	Block size 16–128 optimal for ShareGPT (long sequences); 16–32 for Alpaca (short sequences). Default: 16.
Swap vs. recompute	Recompute: constant overhead, better for small blocks. Swap: efficient for large blocks (fewer PCIe transfers). Recompute $\leq 20\%$ of swap latency across block sizes.

3.9 Limitations

- FCFS scheduling: does not prioritise short requests; cannot implement SRPT without output length predictions.

- PagedAttention adds 20–26% attention kernel overhead vs. baseline (though this is outweighed by batching gains).
- Non-contiguous block access may reduce L1/L2 cache hit rates (the BCA paper later quantifies this as a contributing factor to DRAM saturation).
- Preemption policy (FCFS, all-or-nothing eviction) is simple and may not be optimal for heterogeneous workloads.
- No output length prediction integrated (left to future work, now partially addressed by vLLM extensions and related systems).

4 Comparative Analysis

4.1 Methodological Comparison

Dimension	Paper 1 (Queueing + LLMs)	Paper 2 (vLLM)
Type	Survey + open problems	Full systems paper
Contribution	Theoretical models, algorithms, open questions	PagedAttention algorithm, vLLM system, evaluation
Key bottleneck addressed	Unknown job sizes; suboptimal scheduling without predictions	KV cache memory fragmentation limiting batch size
Scheduling policy	Studies SRPT, SPRPT, SkipPredict, etc.	Uses FCFS (aware of its limitations)
Preemption model	Analyses costs; proposes age thresholds	All-or-nothing eviction; FCFS priority
GPU memory	Discussed qualitatively	Core technical contribution; near-zero waste achieved
Validation	Analytical equations + simulation tables (prior work)	Real GPU experiments (A100, GCP)
Artefacts	None (tutorial paper)	Full open-source system (widely adopted)
Models tested	M/G/1, M/M/1, M/GI/s+GI (theoretical)	OPT-13B/66B/175B, LLaMA-13B (empirical)
Datasets	Simulation with exponential/Weibull distributions	ShareGPT, Alpaca, WMT16

4.2 Complementarity

The two papers are deeply complementary:

- Paper 1 identifies that prediction-based scheduling (SRPT/SPRPT) is theoretically superior but requires accurate output length estimates. Paper 2’s system uses FCFS, exactly the baseline that Paper 1 shows is suboptimal. A natural future direction is to integrate SPRPT or SkipPredict-style prediction into vLLM’s scheduler.
- Paper 2 shows that memory management is the primary throughput bottleneck; Paper 1’s Section 4.1 cites Paper 2’s memory analysis as motivation for why preemption is costly in LLMs.

- Paper 1’s Trail system (SPRPT with age-based preemption disable) is directly compatible with vLLM’s PagedAttention memory system — the preemption threshold determines when the KV cache is large enough that recomputation is prohibitively expensive.
- Together they define the two main axes of LLM serving optimisation: *what to serve* (scheduling/prediction) and *how to store it* (memory management).

5 Literature Landscape and Positioning

5.1 Key Related Systems

System	Key contribution	Relation to surveyed papers
Orca	Iteration-level continuous batching	Baseline for Paper 2; scheduling model cited in Paper 1
vLLM	PagedAttention + KV cache management	Paper 2 itself
SGLang	RadixAttention (KV reuse), structured generation	Cited in both; extends vLLM’s prefix sharing
Sarathi-Serve	Chunked prefill (splits prefill into decode-sized chunks)	Cited in Paper 1 as multi-stage scheduling challenge
FastServe	MLFQ scheduling + proactive GPU/CPU offloading	Paper 1 Section 4.1; addresses SRPT without predictions
Trail	Probing-based size prediction + age-threshold SPRPT	Paper 1 Section 4.1; analyses via SOAP
LAMPS	Scheduling for API-augmented LLMs	Paper 1 Section 5.1
Splitwise	Separate prefill/decode GPUs (tandem queue model)	Paper 1 Section 4.3
Dynasor	Resource allocation for LLM reasoning queries	Paper 1 Section 6
FlashAttention	Tiled, IO-aware attention kernel	Both papers; reduces memory access overhead
INFERCEPT	KV cache strategy for API calls	Paper 1 Section 5.1

5.2 Benchmark Datasets Used Across the Literature

Dataset	Characteristics	Used in
ShareGPT	Long ChatGPT conversations; mean 161 input / 338 output tokens; high variance; de facto standard for LLM serving eval	Paper 2, vLLM, SGLang, Sarathi, BCA paper, Sem-MoE
Alpaca	Instruction tuning; short prompts/outputs; low variance	Paper 2, Trail, FastServe
WMT16 En-De	Machine translation; shared prefix evaluation	Paper 2
MMLU	Multi-domain academic; diverse token distributions	Sem-MoE, various LLM evals
LMSYS-Chat-1M	Large-scale real conversations	Sem-MoE, BCA paper
LongBench	Long-context evaluation	ExFlow, inter-layer affinity papers
Stanford Alpaca	Instruction-following data	Trail evaluation

5.3 Optimisation Taxonomy

Category	Representative works	Papers' position
Memory management	vLLM (PagedAttention), RadixAttention, S ³	Paper 2 is SOTA at publication
Scheduling with predictions	SPRPT, SkipPredict, Trail, LTR, FastServe	Paper 1 surveys and extends theory
Batching policies	Orca, Sarathi-Serve, chunked prefill	Both papers build on these
Preemption handling	vLLM (swap/recompute), FastServe (proactive offload), Trail (age threshold)	Paper 1 analyses; Paper 2 implements
KV cache sharing	vLLM, SGLang RadixAttention, Hydragen, Preble	Paper 2 introduces; SGLang extends
Quantisation	GPTQ, SmoothQuant, FP8	Orthogonal; now in vLLM
Speculative decoding	Chen et al. 2023, Leviathan et al. 2023	Paper 1 models as tandem queue
Multi-LLM routing	RouteLLM	Paper 1 Section 5.2
Reasoning systems	Dynasor, o1-style reasoning	Paper 1 Section 6 (open problems)

6 Critical Assessment

6.1 Strengths

Paper 1 (Queueing + LLMs).

- Bridges two largely disconnected communities: queueing theorists and LLM systems researchers.
- Rigorous: all claims backed by analysis (SOAP, work integrals) or cited simulation.

- Actionable: the open problems are specific, grounded in real system behaviour, and tractable for a theory-focused researcher.
- Breadth: covers single queues, multi-server systems, compound AI (RAG, agents), and reasoning systems in a unified framework.
- Prediction cost modelling (SkipPredict, DelayPredict) is novel and practically relevant — prediction overhead is often ignored in the algorithms-with-predictions literature.

Paper 2 (vLLM/PagedAttention).

- Elegant abstraction: the virtual memory paging analogy makes the design intuitive and the implementation principled.
- Strong empirical results: up to $22\times$ throughput improvement, near-zero memory waste, validated across multiple model sizes and workloads.
- Open-source: code has been widely adopted and extended, making it a foundational artefact for the community.
- Clean separation of memory management from scheduling, making each independently replaceable.
- Ablation studies are thorough (block size, swap vs. recompute, kernel overhead).

6.2 Weaknesses and Open Questions

Paper 1.

- No code or simulation framework released; all numerical results from prior published work.
- The LLM-specific sections (4–6) are descriptive rather than analytical; few new theorems specific to LLM models are proved.
- KV cache memory modelling is largely qualitative; a formal queueing model with memory constraints remains future work.
- Discussion of MoE models and expert parallelism is absent.

Paper 2.

- FCFS scheduling is known to be suboptimal; no integration with prediction-based policies (despite Paper 1 showing their value).
- Non-contiguous block access increases attention kernel latency 20–26% and may exacerbate DRAM cache miss rates (a finding later quantified by the BCA paper).
- Evaluation predates FlashAttention-2, chunked prefill, and other post-2023 advances; results are a lower bound on modern vLLM performance.
- Beam search memory savings (55%) are measured on Alpaca; may differ significantly on longer-sequence workloads.

7 Summary Tables for Literature Survey Reference

7.1 Quick-Reference: Key Numbers

Claim / Result	Paper	Context
Existing systems use 20.4–38.2% of KV cache for actual token states	vLLM	A100, OPT-13B, various Orca configs
vLLM: $1.7\text{--}2.7\times$ higher throughput vs. Orca (Oracle)	vLLM	ShareGPT, OPT-13B/66B/175B
vLLM: up to $22\times$ vs. FasterTransformer	vLLM	ShareGPT
vLLM: $3.58\times$ higher throughput for 5-shot prefix	vLLM	WMT16, LLaMA-13B, Orca (Oracle)
vLLM: 37.6–55.2% memory saving in beam search	vLLM	Alpaca, OPT-13B
PagedAttention: 20–26% higher attention kernel latency	vLLM	vs. FasterTransformer
SPRPT response time within $\sim 10\text{--}20\%$ of SRPT	Queueing	M/G/1, exponential predictions
1-bit prediction recovers $\sim 70\text{--}80\%$ of full-prediction gain	Queueing	Simulation, exponential/Weibull
SPRPT-with-bounce: graceful degradation constant $C = 3.5$	Queueing	Bounded multiplicative error model
PSPJF: graceful degradation constant $C = 1.5$	Queueing	Same setting
Trail: $1.66\text{--}2.01\times$ mean latency reduction	Queueing	Alpaca, NVIDIA A100 80GB
LAMPS: 27–85% E2E latency reduction vs. INFERCEPT	Queueing	API-augmented LLMs

7.2 Recommended Citation Keywords

Topic	Relevant Paper(s)
KV cache memory management	vLLM (Paper 2)
PagedAttention / non-contiguous KV cache	vLLM (Paper 2)
LLM serving throughput / batch size	vLLM (Paper 2)
Paged memory in transformer serving	vLLM (Paper 2)
Copy-on-write in LLM decoding	vLLM (Paper 2)
Beam search memory sharing	vLLM (Paper 2)
Prediction-based scheduling / SPRPT	Queueing (Paper 1)
Algorithms with predictions for queues	Queueing (Paper 1)
SkipPredict / DelayPredict / prediction cost	Queueing (Paper 1)
1-bit predictions for scheduling	Queueing (Paper 1)
Graceful degradation in scheduling	Queueing (Paper 1)
LLM scheduling challenges / open problems	Queueing (Paper 1)
KV cache preemption overhead	Both
Throughput-latency trade-off in LLM serving	Both
SLO-aware LLM inference	Both

8 Conclusion

Paper 1 (Queueing, Predictions, and LLMs) makes its primary contribution by establishing a rigorous theoretical framework for LLM scheduling that connects the well-developed field of algorithms with predictions to the rapidly evolving domain of LLM serving systems. Its catalogue of open problems provides a research agenda for both theorists and systems builders: how to design provably near-optimal schedulers that work with imperfect output length predictions, account for prediction costs, and respect the unique memory constraints of KV caches.

Paper 2 (PagedAttention / vLLM) makes its primary contribution by solving the memory management problem that constrained LLM serving throughput. By adapting the OS virtual memory paging abstraction to KV cache management, vLLM achieves near-zero memory waste and enables memory sharing across requests, boosting throughput by $2\text{--}22\times$ depending on workload and baseline. Its open-source implementation became the dominant LLM serving framework, catalysing subsequent advances including SGLang (RadixAttention), Sarathi-Serve (chunked prefill), and many others.

Together they define the two central technical problems in LLM serving: *what order to serve requests* (a prediction and scheduling problem, analysed by Paper 1) and *how to manage the memory they consume* (a systems problem, solved by Paper 2). The most impactful future systems will need to address both problems jointly.

References

- [1] M. Mitzenmacher and R. Shahout. *Queueing, Predictions, and LLMs: Challenges and Open Problems*. arXiv:2503.07545, 2025.

- [2] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. arXiv:2309.06180.
- [3] G.-I. Yu et al. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. OSDI 2022.
- [4] M. Mitzenmacher. *Scheduling with Predictions and the Price of Misprediction*. ITCS 2020.
- [5] M. Mitzenmacher. *Queues with Small Advice*. ACDA 2021.
- [6] Z. Scully, I. Grosof, M. Mitzenmacher. *Uniform Bounds for Scheduling with Job Size Estimates*. ITCS 2022.
- [7] R. Shahout and M. Mitzenmacher. *SkipPredict: When to Invest in Predictions for Scheduling*. NeurIPS 2024.
- [8] R. Shahout et al. *Don't Stop Me Now: Embedding Based Scheduling for LLMs*. arXiv:2410.01035, 2024.
- [9] R. Shahout et al. *Fast Inference for Augmented Large Language Models*. arXiv:2410.18248, 2024.
- [10] Z. Scully, M. Harchol-Balter, A. Scheller-Wolf. *SOAP: One Clean Analysis of All Age-Based Scheduling Policies*. SIGMETRICS 2018.
- [11] L. Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. NeurIPS 2024.
- [12] A. Agrawal et al. *Sarathi-Serve: Taming Throughput-Latency Tradeoff in LLM Inference*. arXiv:2403.02310, 2024.
- [13] B. Wu et al. *FastServe: Fast Distributed Inference Serving for LLMs*. arXiv:2305.05920, 2023.
- [14] Y. Fu et al. *Efficient LLM Scheduling by Learning to Rank*. arXiv:2408.15792, 2024.
- [15] P. Patel et al. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. ISCA 2024.
- [16] T. Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022.
- [17] Y. Fu et al. *Efficiently Serving LLM Reasoning Programs with Certaindex*. arXiv:2412.20993, 2024.
- [18] Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023.
- [19] H. Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971, 2023.
- [20] S. Zhang et al. *OPT: Open Pre-Trained Transformer Language Models*. arXiv:2205.01068, 2022.
- [21] I. Ong et al. *RouteLLM: Learning to Route LLMs with Preference Data*. arXiv:2406.18665, 2024.

Literature Survey Report

LLM Inference Systems:

MoE Communication Efficiency &
GPU Memory Bottleneck Analysis

Papers Surveyed

Paper 1: *Semantic Parallelism: Redefining Efficient MoE Inference via Model-Data Co-Scheduling*

Yan Li, Zhenyu Zhang, Zhengang Wang, Pengfei Chen, Pengfei Zheng

Huawei Technologies & Sun Yat-Sen University

Published at **ICLR 2026** arXiv:2503.04398

Paper 2: *Mind the Memory Gap: Unveiling GPU Bottlenecks in Large-Batch LLM Inference*

Pol G. Recasens, Ferran Agullo, Yue Zhu, Chen Wang, Eun Kyung Lee, Olivier Tardieu, Jordi Torres, Josep Ll. Berral

Barcelona Supercomputing Center, UPC & IBM Research

arXiv:2503.08311

May 11, 2026

Contents

1	Overview and Motivation	3
2	Paper 1 — Semantic Parallelism (Sem-MoE)	3
2.1	Problem Statement and Motivation	3
2.2	Core Novelty — Semantic Parallelism	3
2.2.1	Component 1 — Offline Token-Expert Affinity Modelling	4
2.2.2	Component 2 — Offline Model Scheduling (Expert Clustering)	4
2.2.3	Component 3 — Online Data Scheduling	4
2.3	Implementation Details	5
2.4	Experimental Setup	5
2.4.1	Hardware	5
2.4.2	Models Evaluated	5
2.4.3	Datasets	5
2.4.4	Baselines	6
2.4.5	Metrics	6
2.5	Key Results	6
2.6	Relationship to Prior Work	7
2.7	Integration with Modern Systems	7
2.8	Limitations and Future Directions	7
3	Paper 2 — Mind the Memory Gap (BCA)	7
3.1	Problem Statement and Motivation	8
3.2	Core Novelty	8
3.2.1	GPU Profiling Study	8
3.2.2	Batching Configuration Advisor (BCA)	8
3.2.3	Model Replication via NVIDIA MPS	9
3.3	Implementation Details	9
3.4	Experimental Setup	9
3.4.1	Hardware	9
3.4.2	Models Evaluated	9
3.4.3	Workloads	10
3.4.4	Metrics	10
3.5	Key Results	10
3.6	Relationship to Prior Work	11
3.7	Integration with Modern Systems	11
3.8	Limitations and Future Directions	11
4	Comparative Analysis	12
4.1	Methodological Comparison	12
4.2	Complementarity and Potential Integration	12
5	Literature Landscape and Positioning	12
5.1	Serving Systems (Common Related Work)	13
5.2	Key Benchmarks and Evaluation Datasets	13
5.3	Optimisation Taxonomy	14
6	Critical Assessment	14
6.1	Strengths	14
6.2	Weaknesses and Open Questions	15

7	Summary Tables for Literature Survey Reference	15
7.1	Quick-Reference: Key Numbers	15
7.2	Recommended Citation Keywords	16
8	Conclusion	16

1 Overview and Motivation

The two papers surveyed here address distinct but complementary bottlenecks in large language model (LLM) inference. Both are grounded in careful empirical measurement of real hardware and both translate their findings into concrete systems contributions.

Paper 1 — Sem-MoE (ICLR 2026)	Paper 2 — BCA (arXiv 2025)
Focuses on <i>inter-device communication</i> cost in Mixture-of-Experts (MoE) models when using expert parallelism (EP) across GPUs/NPUs.	Focuses on <i>intra-GPU memory</i> saturation during the decode phase of standard (dense) LLM inference, showing the bottleneck is DRAM bandwidth, not compute.
Primary contribution: model-data co-scheduling to maximise local activation rate and shrink all-to-all volume.	Primary contribution: GPU-level profiling study + Batching Configuration Advisor (BCA) to find optimal batch size + model replication strategy.

Together they cover the two dominant performance regimes in production LLM serving: large-scale distributed MoE inference (where the bottleneck is *network communication*) and single/few-GPU dense model inference (where the bottleneck is *memory bandwidth*).

2 Paper 1 — Semantic Parallelism (Sem-MoE)

2.1 Problem Statement and Motivation

Mixture-of-Experts models such as DeepSeek-V3, Qwen3, and GPT-OSS have emerged as the dominant paradigm for trillion-parameter LLMs because they activate only a sparse subset of expert feed-forward networks (FFNs) per token, keeping per-token FLOPs roughly constant while scaling model capacity. At inference time, these models are distributed across multiple GPUs/NPUs using *expert parallelism* (EP): each device stores a shard of the expert pool, and a gating network routes each token to its top- k experts.

The critical efficiency problem is that the gating decision is made *after* the token reaches a device, so activated experts may reside on *remote* devices. Dispatching activations to remote experts and collecting results back requires two cluster-wide `all2all` collective operations per MoE layer, which dominate latency. On an 8-GPU server with 400 GB/s specialised interconnects, EP communication consumes up to **59.2%** of per-layer latency for DeepSeek-V2-Lite (Figure 1 of the paper).

Core Gap Identified

State-of-the-art engines treat *expert placement* (model scheduling) and *token routing* (data scheduling) as independent problems. No prior system jointly optimises both with the goal of maximising *local* activation — where a token’s target expert happens to live on the same device as the token.

2.2 Core Novelty — Semantic Parallelism

The paper introduces **Semantic Parallelism**, a paradigm that *jointly* schedules model parameters and input data by exploiting a key empirical finding:

Key Empirical Observation: Context-Independent Token-Expert Affinity

Despite the gating function depending on context in theory, in practice a given token consistently routes to a narrow, stable subset of top- k experts across diverse inputs. For DeepSeek-V2-Lite, the median cumulative activation probability of the top- k experts for a token is 0.833–0.976, while the maximum hotness of the remaining experts hovers near 0.05 (Table 1 of the paper). This *context-independent conjugacy* makes offline routing prediction feasible and highly accurate.

The system, called **Sem-MoE**, realises semantic parallelism through three tightly integrated components:

2.2.1 Component 1 — Offline Token-Expert Affinity Modelling

Sem-MoE profiles the model on a representative dataset (e.g., ShareGPT) to build a token-to-expert activation table $T^{(L)} \in \mathbb{N}^{t \times N^{(L)}}$ for each MoE layer L , where $T_{j,k}^{(L)}$ counts how often token x_j activates expert $E_k^{(L)}$. Routing probabilities are derived as:

$$\Pr(E_k^{(L)} | x_j) = \frac{T_{j,k}^{(L)}}{\sum_{k'} T_{j,k'}^{(L)}}.$$

A compact token-to-expert confidence table $C_p \in \mathbb{R}^{t \times N}$ is built for efficient online lookup. Out-of-vocabulary tokens are handled via nearest-neighbour matching in embedding space.

An *inter-layer* model additionally captures Markovian dependencies across layers: the device selected at layer L is predicted from selections at previous layers using an n -gram model ($n = 2$ in practice), stored in a device-transition table $\mathcal{A} \in \mathbb{R}^{D^n \times D}$.

2.2.2 Component 2 — Offline Model Scheduling (Expert Clustering)

Sem-MoE formulates a 0-1 Integer Linear Programme (ILP) to jointly cluster tokens and experts into E groups (one per device) minimising:

$$\mathcal{L} = \theta \sum_{i=1}^E \left| \sum_j R_{ij} a_j - \frac{S}{E} \right| + (1 - \theta) \sum_{i_1 \neq i_2} \sum_{j,k} R_{i_1 j} C_{i_2 k} C_{p,jk} a_j$$

where $R_{ij} \in \{0,1\}$ routes token j to cluster i , $C_{ij} \in \{0,1\}$ places expert j in cluster i , a_j is token frequency, and θ balances load balancing against communication reduction. The ILP is solved offline via an alternating optimisation algorithm (Algorithm 1 in the paper), producing an expert-grouping table $\mathcal{E} \in \mathbb{R}^{L \times E \times D}$ used to rearrange expert weights before deployment.

2.2.3 Component 3 — Online Data Scheduling

Two policies are applied depending on the attention parallelism mode:

Attention-DP (inter-request scheduling). Each incoming request r is routed to the DP rank d^* whose expert cluster has the highest predicted affinity:

$$d^* = \arg \max_{j \in [E]} \sum_{i \in r} R_{ij}.$$

A device mask prevents skew: once a device is allocated in a round it is masked until all devices have received at least one request.

Attention-TP (intra-request scheduling). Within a single inference pass the standard post-attention `allreduce` is replaced by two fused primitives:

- **Shuffled-Reduce-Scatter (SRS):** combines the mandatory TP reduce-scatter with a speculative token shuffle that sends each token toward the device predicted to host its target expert.
- **Shuffled-AllGather (SAG):** after MoE computation, collects tokens and reverses the shuffle.

The shuffle decision uses whichever of the intra-layer (C_p) or inter-layer ($\mathcal{A}_p$) table carries higher confidence. The overhead of the shuffling logic is approximately 1% of the TP communication time.

2.3 Implementation Details

- **Codebase:** $\approx 5,000$ lines of Python, plug-in module for SGLang; several custom Triton/OpenAI kernels.
- **GitHub:** Code integrated into SGLang; exact repository link available at <https://arxiv.org/abs/2503.04398>.
- **argsort kernel:** Custom argsort outperforms native PyTorch by 25%, critical for low-overhead shuffling.
- **EP communication:** Integrates DeepEP (DeepSeek’s expert-parallel communication library) for frontier all-to-all performance.
- **Gate matrix:** Transparent expert rearrangement by column-shuffling the gate weight, requiring no model architecture modification.
- **Memory for lookup tables:** For DeepSeek-V2, the token-to-device table $\mathcal{T}$ occupies only ≈ 11.72 MB (int16 format).

2.4 Experimental Setup

2.4.1 Hardware

Component	Specification
GPUs	8×96 GB HBM GPU (homogeneous interconnect)
Interconnect	>400 GB/s specialised intra-server network
CPUs	2×44 -core Intel
DRAM	2 TB DDR5

2.4.2 Models Evaluated

Model	Experts/Layer	Notes
DeepSeek-V2-Lite	64 routed	Primary evaluation model
Qwen3-30B-A3B	128	Larger expert pool
Moonlight-16B	—	Appendix evaluation

2.4.3 Datasets

- **MMLU** — Massive Multitask Language Understanding; diverse academic domains.
- **LMSYS-Chat-1M** — Large-scale real-world chat conversations.
- **ShareGPT-Vicuna-Unfiltered** — Conversational prompts used also for offline profiling.
- **Split:** 20% training (profiling / scheduling table construction), 80% test.

2.4.4 Baselines

- **SGLang** (v0.4+): state-of-the-art open-source LLM serving engine with continuous batching, paged attention, FlashAttention, radix attention, and fused Triton kernels for MoE.
- **MoETuner**: ILP-based optimal expert placement framework that addresses load imbalance and skewed inter-GPU communication; embedded into SGLang for fair comparison.

2.4.5 Metrics

- **Throughput** (tokens/s) under SLO constraints.
- **TTFT** (Time to First Token, ms): prefill latency.
- **E2E Latency** (s): request arrival to last token.
- **SLO**: $5\times$ the latency under minimal request rate (following prior work).
- **Local Activation Rate (LAR)**: fraction of token activations served by an expert on the same device; directly measures communication reduction.
- **Load Imbalance Rate**: max load / median load across EP ranks.
- Request rate swept from 10 to 175 req/s; fixed input/output lengths to isolate scheduling effects.

2.5 Key Results

Attention-DP Scenario

For **DeepSeek-V2-Lite**:

- Throughput improvement: **+31%** (TTFT SLO) and **+221%** (E2E latency SLO) vs. SGLang; **+32%** and **+278%** vs. MoETuner.

For **Qwen3-30B-A3B**:

- Throughput improvement: peak **+98%** (TTFT SLO) and **+11%** (E2E SLO) vs. SGLang; **+35%** and **+32%** vs. MoETuner.

Attention-TP Scenario (latency)

For **DeepSeek-V2-Lite**:

- TTFT improvement: **12.21%**, **10.60%**, **18.89%** for input lengths 256, 512, 1024 tokens respectively.

For **Qwen3-30B-A3B**:

- TTFT improvement: **17.16%**, **24.90%**, **3.80%** for input lengths 256, 512, 1024 tokens.

Communication Breakdown

- LAR increased by **+37%** (DeepSeek-V2-Lite) and **+43%** (Qwen3-30B-A3B) over vanilla placement, translating to **41.8%** and **46.6%** latency reduction for the expert layer.
- Co-scheduling algorithm outperforms MoETuner by **15.4%** in LAR and **36.7%** in load imbalance under EP8.

Cross-Dataset Zero-Shot Transfer

The scheduling policy generalises to unseen datasets without retraining. A predictor trained on ShareGPT achieves 41.25% LAR on LMSYS-Chat-1M and 41.55% on MMLU — 1.65× above SGLang’s baseline (25%), with only a modest drop from the in-distribution peak.

2.6 Relationship to Prior Work

System	Approach	Difference from Sem-MoE
ExFlow	Inter-layer expert affinity for model scheduling	No data scheduling; requires a heavy allgather before each layer, adding memory and communication overhead
MoETuner	ILP-based expert placement minimising inter-device communication	Model scheduling only; no token/request routing
Lina	Non-uniform expert replicas for load balancing	Load focus, no locality-aware co-scheduling
Pre-gated MoE	Modifies model architecture to predict next-layer experts	Requires architecture change; Sem-MoE is model-agnostic
Offloading works	Prefetch offloaded experts; saves GPU memory	Latency penalty; not suited to latency-critical serving

2.7 Integration with Modern Systems

- **SGLang**: Plug-in module; extends the request scheduler with token-expert affinity; replaces allreduce with SRS/SAG primitives.
- **DeepEP**: Integrates DeepSeek’s high-performance EP communication library for frontier all-to-all throughput.
- **Model compatibility**: Tested on DeepSeek-V2-Lite, Qwen3-30B-A3B, Moonlight-16B; works on any top- k gated MoE with fixed vocabulary.
- **Deployment flow**: (1) Offline profiling + ILP solve (one-time cost); (2) expert weight rearrangement; (3) runtime lookup-table-based scheduling with negligible overhead.
- **No model changes**: Gate matrix column-shuffle is transparent to the model’s forward pass semantics.

2.8 Limitations and Future Directions

- Prediction accuracy is lower for rare/unseen tokens (handled by nearest-neighbour fallback, but still approximate).
- Offline profiling must be repeated if the serving workload distribution shifts significantly.
- Evaluation limited to 8-GPU single-server setups; multi-server, cross-datacenter EP is a natural extension.
- Dynamic expert replication (hot experts replicated for load spikes) is left for future work.
- The 100% LAR bound is unachievable in practice because different tokens may have contradictory hot-expert sets.

3 Paper 2 — Mind the Memory Gap (BCA)

3.1 Problem Statement and Motivation

LLM inference is well-understood to be *memory-bound* in the single-request (no-batch) regime: the memory transfer time for model weights and the KV cache far exceeds compute time. The common assumption in the literature is that as batch size grows, inference eventually transitions to a *compute-bound* regime, implying that large batches fully utilise GPU compute resources.

This paper challenges that assumption through rigorous GPU-level profiling. The authors demonstrate that the throughput plateau observed at large batch sizes is not a sign of compute saturation but rather of **DRAM bandwidth saturation** in the attention mechanism.

Core Gap Challenged

Prior work (e.g., Yuan et al. 2024; Agrawal et al. 2024) assumed that beyond a batch-size knee-point, LLM inference becomes compute-bound and GPU resources are fully utilised. This paper shows the assumption is incorrect: even at maximum batch size, more than 50% of GPU warp cycles are *idle*, waiting for data from DRAM.

3.2 Core Novelty

3.2.1 GPU Profiling Study

Using NVIDIA Nsight Systems (2023.2.3) and Nsight Compute (2023.3.0.0) on an H100 GPU, the authors produce the first rigorous kernel-level explanation of the throughput plateau:

Decode phase dominance. The decode phase accounts for 95–97% of total inference time across all tested models, confirming it as the primary optimisation target.

Attention kernel is the bottleneck. As batch size grows, the attention mechanism’s share of decode time rises from $\approx 5\%$ (small batches) to $>40\%$ (large batches), while matrix multiplications decline from $\approx 50\%$ to $<10\%$.

Arithmetic intensity remains constant for attention. The Roofline analysis (Figure 1 of the paper) shows that while matmul kernels gain arithmetic intensity with batch size (moving toward compute-bound), both FlashAttention and xFormers attention kernels maintain *nearly constant* arithmetic intensity of $\approx 0.5\text{--}1.0$ FLOP/byte across all batch sizes. At maximum batch size, they align with the DRAM bandwidth limit line, confirming hardware-level memory saturation.

Idle warp cycles. At maximum batch size, $>50\%$ of compute cycles are stalled waiting for data, with xFormers exceeding 80% idle cycles across all models.

Cache hit rates. L1 and L2 cache hit rates average $\leq 12\%$ and $\leq 2\%$ respectively, declining further as batch size increases, indicating highly inefficient memory access patterns (exacerbated by vLLM’s non-contiguous PagedAttention memory layout).

3.2.2 Batching Configuration Advisor (BCA)

BCA is a profiling-driven offline tool that identifies the optimal batch size B_{opt} balancing throughput and latency:

$$B_{\text{opt}} = \arg \max_B T(B) \quad \text{s.t.} \quad L(B) \leq \text{SLO}, \quad \frac{T(B)}{B \cdot T(1)} > \epsilon$$

where $T(B)$ is throughput at batch size B , $L(B)$ is inter-token latency, SLO is a user-specified latency bound, and ϵ is a throughput-efficiency threshold (default 0.1).

BCA’s key insight: most memory is allocated unnecessarily. For example, OPT-1.3B achieves 83% of maximum throughput using only 16.32% of its KV cache (optimal batch size 96 vs. maximum 512), freeing 63.23% of total GPU memory.

3.2.3 Model Replication via NVIDIA MPS

BCA frees substantial GPU memory that would otherwise be wasted. The paper proposes using this freed memory to run *multiple model replicas* concurrently on the same GPU using NVIDIA Multi-Process Service (MPS), which enables true hardware-level parallelism between replicas.

Replication improves throughput by overlapping two sources of GPU idle time:

1. *DRAM saturation gaps*: when one replica stalls on attention memory transfers, another can execute compute operations.
2. *CPU overhead gaps*: CPU scheduling time (up to 30% of decode time in OPT-1.3B) is hidden by another replica’s GPU execution.

3.3 Implementation Details

- **Framework**: vLLM (main branch as of October 18, 2024).
- **GitHub**: <https://github.com/FerranAgulloLopez/vLLMBatchingMemoryGap> — fork of vLLM with benchmarking modifications only; core server unchanged.
- **Profiling tools**: NVIDIA Nsight Systems (system-wide traces) and Nsight Compute (per-kernel arithmetic intensity and roofline).
- **Launcher script**: Slurm-compatible batch launcher for sweeping configurations; supports `-with-nsight` flag for profiling.
- **NVTX markers**: Custom profiler marks for prefill/decode phases.
- **Replication strategy**: NVIDIA MPS (parallel kernel execution) outperforms first-come-first-served (FCFS) sequential scheduling between replicas.
- **Config**: max batch tokens = 4096, max context length = 2048 tokens, logging disabled.

3.4 Experimental Setup

3.4.1 Hardware

Component	Specification
GPU	1 × NVIDIA H100 Hopper (64 GB HBM2)
RAM	128 GB
CPU	20 cores
Framework	vLLM (PyTorch backend)
Profiling	NVIDIA Nsight Systems 2023.2.3, Nsight Compute 2023.3.0.0

3.4.2 Models Evaluated

Model	Parameters	Notes
OPT-1.3B	1.3B	Fits with large batches; primary model
OPT-2.7B	2.7B	Medium replication headroom
Llama-2-7B	7B	Limited replication due to memory
Llama-2-13B	13B	Cannot replicate within 64 GB

All models fit within the single 64 GB GPU, enabling batch sizes from 1 to 512+ without multi-GPU communication.

3.4.3 Workloads

- **Online mode:** 2,000 requests from cleaned ShareGPT dataset (original input/output length distribution preserved); client-server via API endpoints.
- **Offline mode:** Synthetic fixed-length requests — 161 input tokens, 338 output tokens (matching ShareGPT mean).
- Batch size swept from 1 to 512.

3.4.4 Metrics

- **Throughput** (input + output tokens/s).
- **Inter-Token Latency (ITL)** (ms).
- **End-to-End (E2E) Latency** (s).
- **KV Cache Usage** (%) — fraction of allocated KV cache occupied.
- **GPU Metrics** (from Nsight):
 - Compute Warps in Flight (%)
 - Unallocated Warps in Active SMs (%)
 - DRAM Read/Write Throughput (% of peak)
 - L1 / L2 Cache Hit Rates (%)
 - Idle warp cycles (% stalled waiting for data)
- **Arithmetic Intensity** (FLOP/byte) via Roofline model.

3.5 Key Results

DRAM Saturation Confirmed

- At maximum batch size, attention kernels align with the DRAM bandwidth roofline (1.63×10^{12} byte/s for H100), confirming full memory saturation.
- Compute Warps in Flight averages only **9–31%** during decoding (well below 100%), with >50% of cycles stalled.
- L1 cache hit rate drops from $\sim 16\%$ (batch=1) to $\sim 2.5\%$ (batch=512) for OPT-1.3B.

BCA — Memory Savings

Model	B_{opt}	Throughput	KV Cache Used	Memory Freed
OPT-1.3B (strict SLO)	96	83.1% of MAX	16.3%	63.2% of GPU
OPT-2.7B (strict SLO)	96	Similar	28.4%	45.1% of GPU
Llama-2-7B	> 96	Similar	89.5%	10.5% of GPU

Model Replication Throughput Gain

- **OPT-1.3B** (4 replicas, relaxed SLO): **+34%** throughput over MAX single-replica; CPU idle time reduced by **78%**.
- **OPT-2.7B** (2 replicas, relaxed SLO): **+13%** over MAX.
- Replication achieves *comparable or superior* throughput to chunked prefill at MAX batch size.
- ITL increases $\sim 28\%$ under replication vs. B_{opt} , but E2E latency decreases due to

parallelism.

3.6 Relationship to Prior Work

System	Approach	Difference from BCA
Orca	Continuous batching with iteration-level scheduling	Maximises batch size without analysing the plateau; assumes compute bottleneck
Sarathi-Serve	Chunked prefill to combine pre-fill/decode	Addresses prefill stalls, not decode DRAM saturation
vLLM	PagedAttention for KV cache management	Allocates maximum memory by default; no guidance on optimal batch size
Yuan et al. 2024	Roofline model for LLM inference	Assumes large batches are compute-bound (directly contradicted)
AlpaServe	Multi-model serving via statistical multiplexing	Multi-GPU setting; BCA focuses on single-GPU optimisation

3.7 Integration with Modern Systems

- **vLLM:** BCA operates as a pre-deployment advisor; vLLM config parameters (`max_num_seqs`, memory fraction) are set based on BCA output.
- **NVIDIA MPS:** Replication relies on MPS for true concurrent kernel execution on a single GPU, not just time-sharing.
- **Slurm environments:** The launcher supports HPC cluster deployments, making the approach applicable to academic and cloud HPC infrastructures.
- **Cloud multi-tenant serving:** BCA’s freed memory enables co-located workloads, analogous to CPU memory overcommit in virtualised cloud environments.
- **Limitations for larger models:** Llama-2-13B saturates 64 GB GPU memory at B_{opt} , leaving no room for replication. Multi-GPU replication across NVLink is an open future direction.

3.8 Limitations and Future Directions

- BCA is computed *offline* assuming all requests arrive simultaneously (best-case); online adaptive version needed for variable arrival rates.
- Evaluation restricted to OPT and Llama-2 families; larger models (70B+) and MoE architectures not studied.
- Non-contiguous PagedAttention is identified as a potential exacerbating factor for poor cache hit rates, but not corrected.
- Replication benefits diminish beyond 2–4 replicas in OPT-1.3B as CPU time is already eliminated.
- Future work: heterogeneous co-location (high-compute + high-memory workloads), multi-GPU replication for larger models, and combining with speculative decoding.

4 Comparative Analysis

4.1 Methodological Comparison

Dimension	Sem-MoE (Paper 1)	BCA (Paper 2)
Target bottleneck	Inter-device all-to-all communication in EP-based MoE models	Intra-GPU DRAM bandwidth saturation in attention
Model type	Sparse MoE (DeepSeek, Qwen3)	Dense (OPT, Llama-2)
Scale	Multi-GPU (8 GPUs, 400 GB/s interconnect)	Single GPU (H100 64 GB)
Core technique	Offline ILP expert clustering + online token/request re-scheduling	Offline profiling → optimal batch size → model replication via MPS
Profiling depth	Activation pattern statistics; F1 score, kurtosis	GPU kernel-level Nsight traces; Roofline analysis
Latency overhead	≈1% from shuffle primitives; ≈0% from lookup-table queries	BCA profiling is offline; online overhead is zero
Memory footprint	~12MB for lookup tables per model	Freed memory re-allocated to replicas
Datasets	MMLU, LMSYS-Chat-1M, ShareGPT	ShareGPT (online); synthetic fixed-length (offline)
Key metric	Local Activation Rate (LAR), throughput under SLO	Throughput at B_{opt} ; DRAM saturation metrics
Framework	SGLang (plug-in)	vLLM (fork)

4.2 Complementarity and Potential Integration

The two papers are highly complementary:

- **Sem-MoE** applies at the *multi-GPU MoE layer* and is irrelevant for dense models; **BCA** applies at the *single-GPU attention layer* and is irrelevant for multi-device EP.
- A production deployment of a large MoE model (e.g., DeepSeek-V3) could simultaneously benefit from: Sem-MoE for EP communication reduction across nodes, and BCA-inspired analysis to right-size the batch and identify any residual DRAM saturation in the attention layers.
- Both papers implicitly highlight the importance of the *attention mechanism* as a persistent memory-bound component regardless of scale.
- Together they motivate a unified profiling + scheduling framework that addresses both inter-device and intra-device memory bottlenecks.

5 Literature Landscape and Positioning

5.1 Serving Systems (Common Related Work)

System	Key Innovation	Relevance
vLLM	PagedAttention: OS-paging-inspired non-contiguous KV cache	Baseline for Paper 2; used by both
SGLang	Radix attention, structured generation, fused Triton MoE kernels	Baseline and integration target for Paper 1
Orca	Iteration-level continuous batching	Foundation for both
Sarathi-Serve	Chunked prefill; combined prefill-decode batches	Compared in Paper 2
DeepSpeed-MoE	Expert parallelism system design	EP baseline for Paper 1
Tutel	Adaptive MoE computation and communication	Compared in EP context
TensorRT-LLM	NVIDIA production inference engine	Mentioned as related
DeepSpeed-FastGen	MII + DeepSpeed-Inference for fast generation	Related

5.2 Key Benchmarks and Evaluation Datasets

Dataset / Benchmark	Used In	Purpose
MMLU	Paper 1 (evaluation)	Multi-domain academic tasks; diverse token distributions
LMSYS-Chat-1M	Paper 1 (evaluation)	Large-scale real-world chat requests
ShareGPT	Both papers	Conversational prompts; Paper 1 uses for profiling; Paper 2 for online workload
Synthetic fixed-length	Paper 2 (offline profiling)	Controlled analysis of batch size effects
NVIDIA Nsight Systems	Paper 2 (profiling tool)	System-wide GPU traces
NVIDIA Nsight Compute	Paper 2 (profiling tool)	Kernel-level arithmetic intensity and Roofline

5.3 Optimisation Taxonomy

Category	Representative Works	Papers' Position
Memory management	vLLM (PagedAttention), S ³	Paper 2 diagnoses limits of PagedAttention's cache access patterns
Communication reduction	Sem-MoE, ExFlow, MoETuner	Paper 1 is SOTA in joint model-data co-scheduling
Quantisation	GPTQ, SmoothQuant	Orthogonal; Paper 2 notes as related
Speculative decoding	Leviathan et al. 2023	Complementary; could combine with BCA
MoE load balancing	Lina, EPLB (DeepSeek)	Related to Paper 1; Sem-MoE incorporates load balance
MoE offloading	MoE-Infinity, AdapMoE	Different regime (offloading vs. in-memory serving)
KV cache compression	GQA, MQA	Reduce memory pressure; complement BCA
Batching policy	Orca, Sarathi-Serve	Foundation for Paper 2's analysis

6 Critical Assessment

6.1 Strengths

Paper 1 (Sem-MoE).

- Strong empirical grounding: the context-independent token-expert conjugacy is quantitatively validated across layers, datasets, and models.
- Principled formulation as an ILP with a clear alternating solver; load balancing and communication reduction are jointly optimised.
- Zero-shot cross-dataset generalisation demonstrates robustness of the learnt scheduling policy.
- Published at ICLR 2026, indicating rigorous peer review.
- Dual coverage of DP and TP attention modes makes it applicable to the most common deployment configurations.

Paper 2 (BCA).

- First paper to rigorously *disprove* the compute-bound large-batch assumption using actual GPU kernel profiling (Nsight traces).
- Roofline analysis is both intuitive and hardware-accurate.
- BCA is simple to use (offline; formulaic) and the freed memory insight is immediately actionable.
- Code is publicly available with full reproducibility infrastructure (Slurm launcher, NVTX markers).

6.2 Weaknesses and Open Questions

Paper 1 (Sem-MoE).

- Evaluation limited to 8 GPUs (single server); scalability to 64+ GPU clusters or multi-server EP is unknown.
- Performance on dynamic workloads (domain shift during serving) requires periodic re-profiling.
- The improvement for Qwen3-30B-A3B under E2E latency SLO is only +11% vs. SGLang, suggesting diminishing returns for larger expert pools.
- No ablation isolating the contribution of inter-layer Markov model vs. the intra-layer confidence table.

Paper 2 (BCA).

- Limited to small models (OPT-1.3B/2.7B, Llama-2-7B/13B); results may not transfer to 70B+ models or MoE architectures.
- BCA assumes static arrival pattern; online adaptation is future work.
- Throughput gains from replication are modest for OPT-2.7B (+13%) and diminish beyond 2 replicas for OPT-1.3B.
- The non-contiguous memory access issue (PagedAttention) is identified but not addressed.

7 Summary Tables for Literature Survey Reference

7.1 Quick-Reference: Key Numbers

Claim / Result	Paper	Context
EP communication = 59.2% of MoE layer latency	Sem-MoE	8 GPUs, DeepSeek-V2-Lite, 400 GB/s
Throughput gain up to 2.78×	Sem-MoE	DeepSeek-V2-Lite, E2E SLO, Attention-DP
Latency reduction up to 24.9%	Sem-MoE	Qwen3-30B-A3B, TTFT, Attention-TP
LAR improved +37%/+43%	Sem-MoE	DeepSeek-V2-Lite / Qwen3, TP
Expert layer latency −41.8%/−46.6%	Sem-MoE	Same models
Zero-shot LAR = 41.25% (baseline 25%)	Sem-MoE	Cross-dataset, DeepSeek-V2-Lite
>50% warp cycles idle at max batch	BCA	H100, OPT-1.3B, xFormers
Attention arithmetic intensity $\approx$ constant	BCA	All 4 models, batch 1 to MAX
OPT-1.3B: 83% throughput using 16% KV cache	BCA	$B_{\text{opt}} = 96$, strict SLO
Replication: +34% throughput for OPT-1.3B	BCA	4 replicas, MPS, relaxed SLO
CPU time reduced 78% with 2 replicas	BCA	OPT-1.3B, MPS

7.2 Recommended Citation Keywords

For a literature survey, both papers are relevant under the following topics:

Topic	Relevant Paper(s)
Expert Parallelism in MoE inference	Sem-MoE
Communication-computation overlap in distributed LLM	Sem-MoE
Token routing and expert placement	Sem-MoE
Token-expert affinity / semantic locality	Sem-MoE
Collective communication optimisation (all-to-all)	Sem-MoE
GPU memory bottlenecks in LLM serving	BCA
DRAM bandwidth saturation	BCA
Roofline analysis for LLM inference	BCA
KV cache management and batch sizing	BCA
Model replication / GPU sharing	BCA
Throughput-latency trade-off	Both
SLO-aware LLM serving	Both

8 Conclusion

Both papers make substantial empirical and systems contributions to the understanding and optimisation of LLM inference.

Semantic Parallelism (Sem-MoE) introduces the first joint model-data co-scheduling framework for MoE inference, grounded in the novel observation that expert routing is largely context-independent. By formulating the problem as an ILP and solving it via alternating optimisation, Sem-MoE achieves significant reductions in all-to-all communication volume, translating to up to $2.78\times$ throughput improvement and 24.9% latency reduction in two distinct parallelism configurations. Its zero-shot transferability, zero architecture modifications, and low runtime overhead make it a strong candidate for adoption in production MoE serving systems.

Mind the Memory Gap (BCA) provides the most rigorous characterisation to date of why large-batch LLM inference does not transition to a compute-bound regime, attributing the throughput plateau to DRAM bandwidth saturation in attention kernels. The BCA framework provides a principled, offline method for right-sizing batch size and GPU memory allocation, enabling model replication strategies that improve throughput by up to 34% by exploiting freed resources.

Together, these papers substantially advance the state of knowledge on LLM serving efficiency and provide complementary tools and insights for both MoE and dense model deployments. They highlight that the two dominant bottlenecks in modern LLM inference — inter-device EP communication and intra-device attention DRAM saturation — require targeted, workload-aware solutions, and that systematic profiling and scheduling co-design can unlock substantial performance headroom on existing hardware.

References

1. Yan Li, Zhenyu Zhang, Zhengang Wang, Pengfei Chen, Pengfei Zheng. *Semantic Parallelism: Redefining Efficient MoE Inference via Model-Data Co-Scheduling*. ICLR 2026.

- arXiv:2503.04398.
2. Pol G. Recasens, Ferran Agullo, Yue Zhu, Chen Wang, Eun Kyung Lee, Olivier Tardieu, Jordi Torres, Josep Ll. Berral. *Mind the Memory Gap: Unveiling GPU Bottlenecks in Large-Batch LLM Inference*. arXiv:2503.08311.
 3. Lianmin Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. NeurIPS 2024.
 4. Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023.
 5. William Fedus, Barret Zoph, Noam Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. JMLR 2022.
 6. DeepSeek-AI. *DeepSeek-V3 Technical Report*. arXiv:2412.19437, 2024.
 7. Seokjin Go and Divya Mahajan. *MoETuner: Optimized Mixture of Expert Serving with Balanced Expert Placement and Token Routing*. arXiv:2502.06643, 2025.
 8. Jinghan Yao et al. *Exploiting Inter-Layer Expert Affinity for Accelerating Mixture-of-Experts Model Inference*. 2024.
 9. Abhinav Agrawal et al. *Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve*. arXiv:2403.02310, 2024.
 10. Zhe Yuan et al. *LLM Inference Unveiled: Survey and Roofline Model Insights*. arXiv:2402.16363, 2024.
 11. Tri Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022.
 12. Chenggang Zhao et al. *DeepEP: An Efficient Expert-Parallel Communication Library*. GitHub, 2025.
 13. An Yang et al. *Qwen3 Technical Report*. arXiv:2505.09388, 2025.
 14. Pol G. Recasens et al. *Towards Pareto Optimal Throughput in Small Language Model Serving*. MLSys Workshop, 2024.
 15. NVIDIA. *TensorRT-LLM*. GitHub, 2024.
 16. Connor Holmes et al. *DeepSpeed-FastGen: High-Throughput Text Generation for LLMs via MII and DeepSpeed-Inference*. 2024.

Literature Survey Report

Efficient MoE Inference:
System-Level Scheduling and Kernel Optimisation

Paper 1: PROBE: Co-Balancing Computation and Communication in MoE Inference via Real-Time Predictive Prefetching

Zhu et al., arXiv:2602.00509, Kling Infra / Kuaishou Technology, 2026

Paper 2: Static Batching of Irregular Workloads on GPUs: Framework and Application to Efficient MoE Model Inference

Li et al., arXiv:2501.16103, Alibaba Group, 2025

Prepared for: Graduate-Level Literature Survey
Topic Area: Systems and Kernels for MoE LLM Inference

May 11, 2026

Abstract

This report surveys two research contributions addressing the efficiency of Mixture-of-Experts (MoE) inference from complementary angles. **PROBE** (arXiv 2026, Kuaishou / Kling Infra) attacks the *system scheduling* problem: how to dynamically rebalance expert load across GPUs in real time without stalling the inference pipeline. It introduces Continuous Lookahead Pipelining—predicting the next layer’s expert hotspots one step ahead and prefetching expert weights into underutilised ranks during otherwise-idle compute windows—reducing prefill latency by up to $1.32\times$ and decoding throughput by up to $1.26\times$. **Static Batching of Irregular Workloads** (arXiv 2025, Alibaba) attacks the *kernel efficiency* problem: how to execute the expert GEMMs of a single MoE layer optimally on a GPU despite wildly varying per-expert token counts. It proposes a compressed TilePrefix mapping that statically dispatches thread blocks to the right expert tile with per-task tiling strategies, achieving up to 95% peak Tensor Core throughput on NVIDIA H20. Together these works represent a two-level solution stack—system-wide scheduling on top, kernel-level compute below—for efficient trillion-parameter MoE deployment.

Contents

1	Introduction	3
2	Background	3
2.1	MoE Architecture	3
2.2	Expert Parallelism and the Straggler Problem	3
2.3	Grouped GEMM and Its Limitations	4
3	PROBE: Real-Time Predictive Expert Prefetching	4
3.1	Problem Definition	4
3.2	Core Novelty: Continuous Lookahead Pipelining	4
3.2.1	Component 1: Gate-Initialized Lookahead Predictor	5
3.2.2	Component 2: Hardware-Aware Balance Planning	5
3.2.3	Component 3: Phase-Locked Co-Scheduling	5
3.3	Experimental Setup	6
3.4	Key Results	6
3.5	Implementation Details	6
3.6	GitHub / Artifact Status	7
4	Static Batching of Irregular Workloads for MoE Kernels	7
4.1	Problem Definition	7
4.2	Core Novelty: Compressed Task Mapping Framework	8
4.2.1	The TilePrefix Mechanism	8
4.2.2	Warp-Level Task Decompression (Algorithm 2)	8
4.2.3	Per-Task Tiling Strategies	9
4.2.4	Two-Stage Mapping for Empty Experts	9
4.2.5	Expert Ordering for Resource Balance	9
4.2.6	Token Copy Overhead Elimination	9
4.2.7	Low-Level GEMM Optimisations	9
4.3	Experimental Setup	10
4.4	Key Results	10
4.5	Implementation Details	11
4.6	GitHub / Artifact Status	11
5	Comparative Analysis	11

6	Integration with Modern LLM Serving Systems	12
6.1	SGLang	12
6.2	DeepEP	12
6.3	vLLM	12
6.4	TensorRT-LLM	12
6.5	DeepSeek Inference Stack	12
6.6	Disaggregation Frameworks	12
6.7	Training Frameworks	12
7	Limitations and Future Work	13
7.1	PROBE	13
7.2	Static Batching Framework	13
7.3	Open Research Directions	13
8	Conclusion	14

1 Introduction

Mixture-of-Experts (MoE) architectures have become the dominant paradigm for scaling LLMs beyond dense feasibility limits. By activating only a sparse subset of experts per token, MoE decouples parameter count from active FLOPs. State-of-the-art models—DeepSeek-V3 [3], Qwen3-MoE-235B [4], GPT-OSS-120B [5], and Kimi K2—all adopt MoE, with up to 128 experts and extreme sparsity (Top-4 or Top-8 routing).

This architectural choice introduces two distinct and compounding efficiency challenges:

- (1) **System-level load imbalance:** Under Expert Parallelism (EP), skewed token routing concentrates tokens onto a subset of expert-hosting GPUs. The overloaded GPU becomes a *straggler*, throttling the entire synchronous layer via synchronisation barriers. This is worsened by continuous batching, where requests arrive and depart asynchronously, causing expert popularity to shift abruptly mid-decode. Reactive rebalancing schemes lag these shifts.
- (2) **Kernel-level compute inefficiency:** Within a single GPU, different experts receive wildly different token counts. The state-of-the-art Grouped GEMM API applies a single tiling strategy to all experts—beneficial for large experts but wasteful for lightly-loaded ones. Empty experts (receiving zero tokens) add further scheduling waste. Per-expert copy overhead for non-contiguous token tensors compounds the problem.

This survey analyses one paper targeting each challenge:

- **PROBE** [1] addresses challenge (1) with a predictive scheduling system.
- **Static Batching** [2] addresses challenge (2) with a new kernel framework.

The remainder is organised as follows. Section 2 provides background on MoE architecture and parallelism. Sections 3 and 4 analyse each paper in depth. Section 5 compares them. Section 6 discusses integration with modern systems. Section 7 covers limitations and open problems, and Section 8 concludes.

2 Background

2.1 MoE Architecture

In a standard MoE layer, a gating network (router) maps each token x_i to a soft selection over E experts. For Top- K routing, only K experts are activated, producing output:

$$\text{MoE}(x_i) = \sum_{j \in \text{TopK}(G(x_i))} g_{ij} \cdot \text{FFN}_j(x_i)$$

where g_{ij} are softmax-normalised gating scores. Because different tokens route to different experts, the MoE layer is inherently an *irregular* GEMM workload.

2.2 Expert Parallelism and the Straggler Problem

For large-scale MoE inference, two main parallelism strategies are employed:

Data Parallelism (DP): Each GPU holds all expert weights and processes its own batch slice. Eliminates load imbalance but requires full weight replication, which is infeasible for trillion-parameter models. Also suffers from *fragmentation*: small per-GPU token counts reduce GEMM arithmetic intensity.

Expert Parallelism (EP): Each GPU hosts a shard of experts. All tokens for each expert aggregate onto its host GPU via All-to-All collectives. Maximises batch size per expert but is

susceptible to straggler effects when routing is skewed. The *Imbalance Ratio* (IR):

$$\mathcal{IR} = \frac{\max_r L_r}{\frac{1}{ep} \sum_r L_r}$$

measures the workload disparity. Real workloads yield $\mathcal{IR} > 2.6$ during prefill and between 1.43–2.28 during decoding, forcing $\sim 50\%$ of compute to idle at barriers.

Double Penalty: Under EP, the overloaded rank simultaneously suffers computational skew (processing most tokens) *and* network congestion (receiving/sending the most All-to-All traffic). This correlation structurally amplifies total latency:

$$T_{\text{MoE}} \approx \underbrace{\max_r T_{\text{comp}}^r}_{\text{Compute Skew}} + 2 \cdot \underbrace{\max_r \frac{V_r}{BW_{\text{net}}}}_{\text{Network Skew}}$$

2.3 Grouped GEMM and Its Limitations

The standard approach to batching MoE expert GEMMs is the Grouped GEMM API (cuBLAS, Triton). Key limitations:

- **Uniform tiling:** All expert GEMMs share the same tile size, penalising small-batch experts with low arithmetic intensity.
- **Dynamic scheduling overhead:** Dynamic tile-to-thread-block scheduling inside the kernel adds overhead, especially at large scale.
- **Token copy overhead:** Tokens are scattered across the input sequence non-contiguously; gathering them into contiguous tensors for each expert incurs significant bandwidth waste.
- **Empty expert waste:** When no tokens are routed to an expert, a Grouped GEMM still pays scheduling overhead for that empty slot.

3 PROBE: Real-Time Predictive Expert Prefetching

3.1 Problem Definition

PROBE targets **latency-critical MoE inference under Expert Parallelism**, where expert hotspots shift rapidly across both layers (spatial dimension) and time steps (temporal dimension). Existing solutions fail for one or more of the following reasons:

- **Permanent replication** (e.g., Grace-MoE, EPLB) consumes static HBM for replica buffers, competes with the KV cache, and cannot adapt to abrupt semantic shifts.
- **Reactive approaches** use historical statistics; they lag abrupt workload transitions and incur warm-up periods.
- **Blocking overheads** from host-based solvers, synchronous expert transfers, or CUDA Graph incompatibilities stall the critical path.
- **Pre-gated routing** (e.g., Pre-Gated MoE) requires training-time modification, violating correctness requirements for deployed models.

3.2 Core Novelty: Continuous Lookahead Pipelining

PROBE’s fundamental insight is that *token arrival is stochastic but routing is predictable*: the hidden-state continuity across adjacent transformer layers means a lightweight predictor can anticipate next-layer expert activation with $\approx 90\%$ accuracy. PROBE exploits the execution

phases of the current layer to pipeline three auxiliary operations for the next layer—Predict, Plan, Prefetch—entirely off the critical path.

3.2.1 Component 1: Gate-Initialized Lookahead Predictor

PROBE constructs a lookahead predictor for layer L by cloning and freezing the layer’s pre-trained router (W_L, b_L) as a strong prior, then attaching a trainable residual MLP:

$$\hat{l}_L = \underbrace{W_L h_{L-1} + b_L}_{\text{Frozen Prior}} + \underbrace{\hat{W}_L^2 \sigma(\hat{W}_L^1 h_{L-1})}_{\text{Trainable Residual}}$$

where h_{L-1} is the hidden state from the previous layer and $\sigma(\cdot)$ is SiLU activation. Zero-initialising the residual ensures stable cold-start behaviour. Per-rank predictions are aggregated via a lightweight All-Gather to build a global workload estimate $\hat{n}$.

Scale-Driven Online Distillation: Inspired by Eagle3 [12], the residual MLP is continuously distilled from the stream of production inference requests using Cross-Entropy loss against ground-truth router outputs. This enables generalisation across diverse domains without offline pre-training on a fixed dataset, achieving 87–94% Top- K accuracy across layers of both GPT-OSS-120B and Qwen3-235B.

3.2.2 Component 2: Hardware-Aware Balance Planning

Given predicted per-expert load $\hat{n}$, the solver jointly optimises expert placement P and token assignment A by minimising the bottleneck rank’s latency:

$$\min_{P, A} \max_r [T_{\text{comp}}^r(A) + T_{\text{comm}}^r(A)]$$

subject to:

$$\begin{aligned} n_{e,r} > 0 &\implies P_{r,e} = 1 \quad (\text{routing validity}) \\ \sum_r n_{e,r} &= n_e, \quad \forall e \quad (\text{conservation}) \\ T_{\text{trans}}^r(P) &\leq T_{\text{window}}^r, \quad \forall r \quad (\text{hiding window}) \end{aligned}$$

The *hiding window* T_{window}^r is the device-specific time available for expert transfers during non-communication kernels (Attention or Grouped GEMM). This constraint is **hardware-aware**: it bounds the volume of expert replicas to what can be transferred within available slack, preventing any exposed overhead.

Greedy Rebalancing (Algorithm 1): Since the joint optimisation is NP-hard, PROBE uses an iterative heuristic: repeatedly identify the bottleneck rank r_{src} and the least-loaded rank r_{dst} ; replicate the hottest expert e^* to r_{dst} if a dual-budget check passes; redistribute tokens using *water-filling* (locality-first: tokens originating on r_{src} stay local; remote tokens are evenly redistributed). Maximum $k_{\text{max}} = 16$ iterations bounds planning cost.

The solver is implemented as a **single-SM CUDA kernel** to maintain CUDA Graph compatibility—a key requirement of modern serving engines like SGLang.

3.2.3 Component 3: Phase-Locked Co-Scheduling

PROBE implements a **dual-track execution model**:

Main stream (deterministic track): Attention $\rightarrow$ All-to-All Dispatch $\rightarrow$ Grouped GEMM $\rightarrow$ All-to-All Combine.

Control plane (auxiliary track): Lookahead Prediction → Balance Planning → P2P Expert Transfer.

The auxiliary track is mapped to **complementary hardware phases**:

- The lightweight MLP predictor and single-SM planner execute during the *bandwidth-bound* All-to-All Dispatch (they are compute-bound, using idle Tensor Cores).
- The bandwidth-intensive P2P expert weight transfer executes during *compute-bound* GEMM and Attention windows.
- A **split-phase transmission** mechanism: transfer begins during Layer i 's MoE computation, *suspends* before All-to-All Combine (to avoid bandwidth contention), then resumes during Layer $i + 1$'s Attention.

This orchestration ensures the auxiliary track uses *orthogonal* hardware resources to the main track at every moment, achieving zero contention.

Memory management: PROBE limits redundant experts to at most 3 per rank per layer, using *double buffering* (6 expert weight slots per device). Cyclic slot reuse means the memory footprint is constant—it does not grow with model depth or KV cache pressure.

3.3 Experimental Setup

3.4 Key Results

Additional findings:

- PROBE demonstrates **zero warm-up** when workloads shift abruptly (e.g., Code → Chinese at step 200), while EPLB degrades significantly after the shift because its placement is optimised for the stale distribution.
- On the **Repeat dataset** (simulated extreme skew), baselines exhibit large latency spikes; PROBE avoids them entirely by continuously re-planning.
- PROBE is **excluded from prefill** comparison with EPLB because EPLB's static replica footprint triggers OOM under large-batch prefill memory pressure, and its reactive transfers are too costly given limited prefill steps.
- The visible reduction in Combine phase latency under PROBE is driven primarily by the *elimination of synchronisation idle time* (straggler wait), not by faster data transfer per se.

3.5 Implementation Details

PROBE is built atop SGLang with the following key implementation choices:

- **Symmetric memory (NVSHMEM):** Manages the replicated-expert buffer region, enabling direct one-sided P2P puts without explicit NCCL calls.
- **Custom Triton prefetch kernel:** Issues remote put operations with controlled SM occupancy to avoid starving the main stream.
- **Single-SM planning kernel:** Serial iterative updates (max 16 iterations) ensure bounded overhead and CUDA Graph compatibility.
- **Global All-Gather (NVSHMEM primitives):** Synchronises per-rank predictor estimates for global planning.
- **CUDA Graph compatibility:** Critical for the decoding phase; all variable logic (solver, transfers) is encapsulated in GPU-side code without host synchronisation.

Table 1: PROBE Experimental Setup

Aspect	Details
Hardware	8× NVIDIA Hopper H800 141 GB node; 900 GB/s NVSwitch interconnect.
Software	PyTorch 2.9; CUDA 12.9; NCCL 2.27.3; NVSHMEM 3.3.20.
Framework	SGLang [7] (serving backbone); DeepEP [8] (All-to-All normal mode).
Models	GPT-OSS-120B (128 experts, Top-4, 36 layers, BF16); Qwen3-MoE-235B (128 experts, Top-8, 93 layers, BF16).
EP configuration	$ep = 8$ across all experiments.
Datasets	Chinese (Alpaca-zh aggregation); Code (CodeAlpaca-20k + OpenAI-HumanEval); Repeat (narrow prompt duplication to simulate extreme skew); Mixed.
Baselines	SGLang (static EP sharding, no rebalancing); DeepSeek-EPLB [8] (statistics-based, 2 redundant slots per layer per rank, one-shot rebalancing over 2 decode steps).
Prefill evaluation	Chunked prefill; 8K tokens/rank (GPT-OSS), 16K tokens/rank (Qwen3-MoE); total input 265K–640K tokens.
Decoding evaluation	Sweep batch size 512–1536 per rank; average over 500 decode steps.
Metrics	TTFT (prefill); TPOT / throughput Pareto curve (decoding); Top- K prediction accuracy; IR before/after balancing; micro-operation timeline.

3.6 GitHub / Artifact Status

As of this survey date, PROBE does not have a public GitHub repository. The paper is an arXiv preprint (arXiv:2602.00509v2, February 2026) from Kling Infra, Kuaishou Technology.

4 Static Batching of Irregular Workloads for MoE Kernels

4.1 Problem Definition

The Alibaba paper targets **kernel-level efficiency for MoE expert computation** on a single GPU (or EP rank). The challenge is that MoE inference is inherently an irregular workload: different experts receive different numbers of tokens per step, some may receive zero, and tokens assigned to the same expert are non-contiguous in the input sequence.

The state-of-the-art Grouped GEMM API has three fundamental deficiencies:

1. **Uniform tiling:** All expert GEMMs share the same tile shape. Large tiles are efficient for heavy experts but wasteful for light ones.

Table 2: PROBE Performance Summary

Metric	PROBE	Baseline (SGLang)
Prefill TTFT speedup (GPT-OSS-120B)	up to 1.32×	1×
Prefill TTFT speedup (Qwen3-235B)	up to 1.16×	1×
Decoding throughput vs EPLB	up to 1.26×	—
IR (avg, Layers 1–35)	1.09 (from 2.13)	2.13
Compute latency skew (Max/Avg)	1.18 (from 2.27)	2.27
Top- K prediction accuracy	87–94%	N/A
Top-Half- K hit rate	≈100%	N/A
Planning overhead (CUDA kernel)	< 150 μs	N/A
Prefetch overhead (3 experts)	Fully hidden	Exposed
Memory footprint per rank	6 expert slots (fixed)	—

2. **Dynamic in-kernel scheduling:** Tile-to-thread-block mapping is resolved inside the kernel, adding overhead at scale.

3. **Token copy overhead:** Each expert requires gathering its non-contiguous tokens into a fresh contiguous buffer before GEMM.

4.2 Core Novelty: Compressed Task Mapping Framework

4.2.1 The TilePrefix Mechanism

The central data structure is a **TilePrefix** array of length N (number of tasks/experts), containing the *inclusive prefix sum* of tile counts:

$$\text{TilePrefix}[i] = \sum_{j=1}^i \nu(T_j)$$

where $\nu(T_j)$ is the number of tiles required by task j . This array is constructed on the host (or device) before kernel launch and copied to device memory once.

Why this is superior to prior art: The two-phase batching framework of Li et al. (PPoPP’19) uses an auxiliary array whose length equals the *number of thread blocks*—potentially orders of magnitude larger than N , causing significant host-to-device copy overhead and poor cache locality. TilePrefix has length N (equal to number of experts, e.g., 64 or 128), which is tiny.

4.2.2 Warp-Level Task Decompression (Algorithm 2)

Inside the kernel, each thread block determines its (task, tile-within-task) pair using a **warp-level SIMT algorithm**:

1. Each of 32 threads computes a boolean: “Is my thread index $\geq$ TilePrefix[t]?”
2. A warp vote produces a bitmask.
3. Population count of the bitmask gives the task index h .

4. The local tile index $l = B - \text{TilePrefix}[h - 1]$ where B is the thread block index.

This runs in $O(1)$ warp operations and achieves high L1 cache hit rates because `TilePrefix` is small and repeatedly accessed.

4.2.3 Per-Task Tiling Strategies

Different tasks (experts) can use *different tile shapes* based on their token count. Large experts use large tiles for high arithmetic intensity; small experts use small tiles to avoid padding waste. Each tiling strategy is implemented as a separate `device` function (`taskFunc1`, ..., `taskFuncK`), and the kernel dispatches to the appropriate one based on the task type. This is impossible with Grouped GEMM.

4.2.4 Two-Stage Mapping for Empty Experts

When an expert receives zero tokens (common in sparse routing), Algorithm 2 fails because the prefix sum includes empty tasks with $\nu = 0$. The solution is a **two-stage mapping**:

1. Build `TilePrefix` only over the $M \leq N$ *non-empty* experts.
2. Define an injection $\sigma : [M] \rightarrow [N]$ mapping non-empty task index to real expert index.
3. Inside the kernel, first decompress to the non-empty task index, then apply σ to recover the real expert index.

This entirely avoids any scheduling overhead for empty experts, which is a significant saving when routing is sparse.

4.2.5 Expert Ordering for Resource Balance

Even within a single GPU, alternating busy (compute-bound) and non-busy (memory-bound) experts in the task ordering improves resource utilisation: a wave of thread blocks that mixes compute-heavy and memory-heavy tasks better saturates both Tensor Cores and memory bandwidth simultaneously. The paper empirically finds that a **half-interval interleaving** strategy (distributing busy experts at regular intervals among non-busy ones) works well. Optimal expert ordering is noted as NP-hard.

4.2.6 Token Copy Overhead Elimination

Instead of gathering non-contiguous tokens into contiguous buffers for each expert, the kernel stores a **token index array** per expert. The GEMM kernel loads tokens on-the-fly using these indices, avoiding redundant copies. Atomic operations scatter token indices into per-expert buckets.

4.2.7 Low-Level GEMM Optimisations

Several hardware-specific optimisations are applied:

- **WGMMA instructions:** Asynchronous warpgroup-level matrix multiply-accumulate, fully utilising Tensor Core pipelines.
- **Async copy:** Overlaps memory load latency with computation.
- **Two-stage pipeline:** Circular double-buffering between global memory and shared memory ensures Tensor Cores are never starved.
- **Tile swizzle:** Improves L2 cache hit rate for GEMMs with large tiles.

4.3 Experimental Setup

Table 3: Static Batching Framework Experimental Setup

Aspect	Details
Hardware	NVIDIA H20 (146 TFLOPS FP16/BF16 Tensor Core peak); NVIDIA H800 (989 TFLOPS FP16/BF16 Tensor Core peak).
Model configuration	64 experts; Top-8 routing; token sequence length 4096; expert weight shape [3584, 2560].
Scenarios	(1) Balanced: tokens evenly distributed across 64 experts. (2) Best: all tokens routed to 8 experts (only 8 GEMMs needed). (3) Worst: nearly all tokens on 8 experts, remaining 56 each get exactly 1 token (extreme memory-bound small GEMMs).
Metric	TFLOPS achieved as % of peak Tensor Core throughput.
Baselines	Not explicitly named; implicitly compared to Grouped GEMM (cuBLAS/Triton) APIs and a sequential GEMM loop (naïve DeepSpeed-style).

4.4 Key Results

Table 4: Static Batching MoE Kernel Performance (% Peak Tensor Core Throughput)

Case	H20 TFLOPS	H20 Peak %	H800 TFLOPS	H800 Peak %
Balanced	138.23	94.67%	838.87	84.82%
Best (8 active)	138.55	94.89%	897.03 [†]	90.70%[†]
Worst (56 singletons)	131.57	90.11%	587.20	59.37%

[†] H800 best case uses larger sequence length/weight shape to reach saturation.

Key observations:

- On H20, performance is robust: **90–95%** of peak across all three scenarios, including the extreme worst case. This is because H20 has a lower compute-to-bandwidth ratio, so even small GEMMs can keep it busy.
- On H800, the worst case degrades to 59% due to the extremely high compute density of H800 Tensor Cores—56 singleton GEMMs cannot saturate them, regardless of batching strategy. This is an inherent hardware mismatch, not a flaw in the kernel.
- The best case on H800 achieves 91% with larger tensors, confirming the kernel reaches near-peak throughput when arithmetic intensity is adequate.
- The paper does not report a direct head-to-head comparison against a specific Grouped GEMM baseline with numbers, but the theoretical argument (uniform vs. per-task tiling) and the near-peak throughput achieved validate the approach.

4.5 Implementation Details

- **Language:** Pure CUDA C++ kernel. TilePrefix is computed on the host (or device) using a parallel prefix sum.
- **Algorithm portability:** The framework is generic—any irregular GPU workload can be batched by implementing its tile function as a `device` function and plugging into Algorithm 3/4.
- **Empty task extension:** The injection σ is computed after routing (post-gating), adding minimal host-side overhead.

4.6 GitHub / Artifact Status

The paper (arXiv:2501.16103v1, January 2025) is from Alibaba Group. No public GitHub repository has been released. The kernel is likely used internally in Alibaba’s serving infrastructure, consistent with the same group’s participation in prior MoE serving work (e.g., they appear in the Aegaeon author list via Wenyuan Yu).

5 Comparative Analysis

Table 5: Side-by-Side Comparison: PROBE vs. Static Batching Framework

Dimension	PROBE	Static Batching
Core problem	Inter-GPU load imbalance under EP	Intra-GPU expert GEMM efficiency
Level	System scheduling	Kernel design
Scope	Multi-node, multi-GPU	Single GPU (per EP rank)
Novelty type	Predictive pipeline scheduling	Compressed mapping + adaptive tiling
Affiliation	Kuaishou / Kling Infra	Alibaba Group
Venue	arXiv preprint (2026)	arXiv preprint (2025)
Models tested	GPT-OSS-120B, Qwen3-235B	64-expert generic MoE
Hardware	H800 $\times 8$ (NVSwitch)	H20, H800 (single GPU)
Key metric	Throughput/latency (system)	TFLOPS / peak % (kernel)
Baseline comparison	SGLang, DeepSeek-EPLB	Implicit (vs. Grouped GEMM)
Addresses empty experts	No (routing assumed valid)	Yes (two-stage mapping)
Handles skew	Dynamically via replication	Statically via per-task tiling
CUDA Graph compat.	Yes (single-SM solver)	Yes (static kernel)
Production deployment	Not reported	Not reported
Public code	No	No

Complementarity: The two papers solve orthogonal bottlenecks and form a natural two-level stack. PROBE operates at the cluster level—deciding *which ranks* should process *which experts* for each upcoming layer. The Static Batching framework operates within each rank—deciding *how* to tile and schedule the local expert GEMMs given the assigned token counts. A combined system would use PROBE’s planner to distribute tokens across ranks and the Alibaba kernel to execute those tokens as efficiently as possible locally.

Shared theme: Both systems reject the assumption that MoE workloads are predictably uniform. Both invest in ahead-of-time analysis (lookahead prediction; host-side tile prefix computation) to avoid dynamic scheduling overhead at execution time. Both are designed to be CUDA Graph compatible, recognising this as a non-negotiable requirement for production decoding.

6 Integration with Modern LLM Serving Systems

6.1 SGLang

PROBE is directly built atop SGLang [7], using it as the serving backbone. The integration replaces the standard EP execution path with the dual-track PROBE pipeline. SGLang’s CUDA Graph support is preserved because PROBE’s auxiliary track is implemented entirely in GPU-resident code.

6.2 DeepEP

PROBE uses DeepEP [8] (DeepSeek’s expert-parallel communication library) for its All-to-All collectives. The P2P expert prefetching is layered on top via NVSHMEM, which provides separate one-sided symmetric memory operations that do not conflict with DeepEP’s collective bandwidth.

6.3 vLLM

The Static Batching framework could replace vLLM’s current MoE expert computation path. vLLM uses a grouped GEMM (via Triton or cuBLAS) for expert computation; the static batching kernel could be substituted as a drop-in replacement with the same input/output interface.

6.4 TensorRT-LLM

TensorRT-LLM already implements optimised MoE kernels with per-expert tiling. The TilePrefix mechanism is architecturally similar to their approach, but the Alibaba paper’s two-stage empty-task handling and token index array (eliminating copy) are additional optimisations not present in all TensorRT-LLM versions.

6.5 DeepSeek Inference Stack

DeepSeek-V3 [3] uses DeepEP for EP communication and a custom MoE kernel. PROBE’s predictor could be integrated to replace EPLB’s reactive rebalancing, and the Alibaba static batching kernel could replace the per-rank GEMM computation. The combination would address both inter-rank and intra-rank inefficiencies in the same system.

6.6 Disaggregation Frameworks

PROBE’s predictor is noted to have implications for *prefill-decode disaggregation* (MegaScale-Infer [15], DistServe). Because token destinations are known before routing completes ($\sim 100\%$ recall for top-half experts), future systems could pre-dispatch hidden states to high-confidence experts, overlapping the entire All-to-All dispatch latency with routing computation itself.

6.7 Training Frameworks

The Static Batching framework is presented for inference but the technique (compressed TilePrefix, per-task tiling) is directly applicable to MoE training. Training faces the same irregular GEMM

problem. The expert ordering heuristic (interleaving compute-bound and memory-bound tasks) is particularly valuable during backward passes where FLOPs budgets are tighter.

7 Limitations and Future Work

7.1 PROBE

1. **Prediction accuracy floor:** At 87–94%, ≈ 6 –13% of expert predictions are wrong. Wrong prefetches waste bandwidth on incorrect expert weights. The paper does not evaluate sensitivity to prediction error.
2. **Three-expert budget cap:** The maximum of 3 redundant experts per rank per layer is a conservative choice to stay within the hiding window. Wider interconnects or longer GEMM windows could allow more aggressive replication.
3. **No cross-layer memory reuse:** Expert slots are recycled per layer independently. If the same expert is hot across many consecutive layers (common in practice), it could remain resident across layers, eliminating repeated prefetch.
4. **Evaluation scope:** Evaluated only on GPT-OSS-120B and Qwen3-235B with $ep = 8$. Scaling to higher ep (e.g., $ep = 32$ for very large clusters) and smaller models is not demonstrated.
5. **Preprint status:** No peer review; implementation not publicly available.
6. **Attention imbalance:** PROBE handles MoE imbalance but does not address attention computation skew (variable sequence lengths under DP), which it acknowledges as mitigated by chunked prefill.

7.2 Static Batching Framework

1. **H800 worst-case degradation:** At 59% peak throughput for extreme imbalance on H800, the kernel cannot overcome the fundamental mismatch between many tiny GEMMs and H800’s extreme compute density. A hybrid approach (fall back to sequential for singleton experts) could improve this.
2. **No direct baseline comparison:** The paper lacks a quantitative comparison against Grouped GEMM baselines on the same inputs, making it harder to assess the absolute improvement.
3. **NP-hard expert ordering:** The paper acknowledges that optimal expert ordering is NP-hard and leaves advanced heuristics as future work.
4. **Warp size constraint:** Algorithm 2 works for $N \leq 32$ experts per warp. For $N > 32$, multiple scans or multi-level TilePrefix are needed but not fully elaborated.
5. **No integration evaluation:** The kernel is evaluated in isolation; system-level throughput impact when embedded in an end-to-end serving stack is not reported.
6. **Static after routing:** TilePrefix is computed after token routing; the framework does not help if the routing itself is slow (e.g., softmax over 128 experts).

7.3 Open Research Directions

- Can PROBE’s lookahead predictor be trained *jointly* with the base model to improve accuracy, rather than distilled post-hoc?

- Can the Static Batching framework be extended to *heterogeneous hardware* (GPU + CPU expert offloading), where expert GEMMs may run on either device?
- How do these systems interact with *speculative decoding* [16], where draft tokens may take different expert paths than the base model?
- Can the TilePrefix mechanism support *dynamic expert capacity* (variable-capacity routing as in Switch Transformer), where tile counts per expert are bounded at a cap?
- Is there a unified planner that combines PROBE-style inter-rank rebalancing with Alibaba-style intra-rank tiling optimisation in a single pass?

8 Conclusion

This survey has examined two papers that together address the end-to-end efficiency of MoE inference across two levels of the system hierarchy. **PROBE** demonstrates that proactive, prediction-driven scheduling can neutralise expert load imbalance in real time without exposing any overhead on the critical serving path, achieving $1.32\times$ prefill speedup and $1.26\times$ decoding throughput improvements. **Static Batching** demonstrates that abandoning the one-size-fits-all Grouped GEMM API in favour of a compressed, per-task tiling strategy yields near-peak Tensor Core utilisation (up to 95% on H20) even under the irregular load distributions inherent to MoE routing.

A common thread runs through both works: *the key to efficient MoE inference is moving decisions ahead of execution*—predicting hotspots before they materialise (PROBE), and computing tile mappings before the kernel launches (Static Batching). This “ahead-of-time” philosophy, combined with hardware-aware constraint satisfaction, is likely to remain central to future MoE serving system design as models grow toward multi-trillion parameters and sparsity ratios continue to increase.

References

- [1] Q. Zhu, X. Ye, Y. Liu, H. Ouyang, and C. Song, “PROBE: Co-Balancing Computation and Communication in MoE Inference via Real-Time Predictive Prefetching,” *arXiv preprint arXiv:2602.00509*, 2026.
- [2] Y. Li, Y. Li, J. Zhang, B. Chen, X. Chen, L. Duan, Y. Jin, Z. Li, X. Liu, H. Wang, W. Wang, Y. Wang, J. Yang, P. Zhang, L. Zheng, and W. Yu, “Static Batching of Irregular Workloads on GPUs: Framework and Application to Efficient MoE Model Inference,” *arXiv preprint arXiv:2501.16103*, 2025.
- [3] A. Liu et al., “DeepSeek-V3 Technical Report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [4] A. Yang et al., “Qwen3 Technical Report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [5] S. Agarwal et al., “GPT-OSS-120B & GPT-OSS-20B Model Card,” *arXiv preprint arXiv:2508.10925*, 2025.
- [6] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” *JMLR*, vol. 23, pp. 1–39, 2022.
- [7] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, et al., “SGLang: Efficient Execution of Structured Language Model Programs,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 62557–62583, 2024.

- [8] C. Zhao, S. Zhou, L. Zhang, C. Deng, Z. Xu, Y. Liu, K. Yu, J. Li, and L. Zhao, “DeepEP: An Efficient Expert-Parallel Communication Library,” <https://github.com/deepseek-ai/DeepEP>, 2025.
- [9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. ACM SOSP*, pp. 611–626, 2023.
- [10] J. He, J. Zhai, T. Antunes, H. Wang, F. Luo, S. Shi, and Q. Li, “FasterMoE: Modeling and Optimizing Training of Large-Scale Dynamic Pre-Trained Models,” in *Proc. ACM SIGPLAN PPOPP*, pp. 120–134, 2022.
- [11] Y. Han, L. Pan, J. Peng, Z. Tao, W. Zhang, and Y. Zhang, “Grace-MoE: Grouping and Replication with Locality-Aware Routing for Efficient Distributed MoE Inference,” *arXiv preprint arXiv:2509.25041*, 2025.
- [12] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-3: Scaling Up Inference Acceleration of Large Language Models via Training-Time Test,” *arXiv preprint arXiv:2503.01840*, 2025.
- [13] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, “Pre-Gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference,” in *Proc. ACM/IEEE ISCA*, pp. 1018–1031, 2024.
- [14] X. Li, Y. Liang, S. Yan, L. Jia, and Y. Li, “A Coordinated Tiling and Batching Framework for Efficient GEMM on GPUs,” in *Proc. ACM SIGPLAN PPOPP*, pp. 229–241, 2019.
- [15] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng, et al., “MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism,” *arXiv preprint arXiv:2504.02263*, 2025.
- [16] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *ICML*, 2023.
- [17] P. Tillet, H.-T. Kung, and D. Cox, “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations,” in *Proc. ACM SIGPLAN MAPL*, pp. 10–19, 2019.

Literature Survey Report

Advances in Efficient LLM Serving:
GPU Pooling and Adapter Routing

Paper 1: Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market

Xiang et al., SOSP 2025

Paper 2: MoLoRA: Composable Specialization via Per-Token Adapter Routing

Shah & Wagle, arXiv 2025 (Microsoft Research)

Prepared for: Graduate-Level Literature Survey
Topic Area: Systems for Large Language Model Inference

May 11, 2026

Abstract

This survey analyses two recent research contributions that address orthogonal but complementary efficiency challenges in large language model (LLM) serving. **Aegaeon** (SOSP 2025, Alibaba/Peking University) tackles the *multi-model serving* problem: how to pool GPU resources effectively when a marketplace hosts hundreds of sparsely-invoked models. It introduces token-level auto-scaling—preemptively swapping models in and out of GPU memory between decode steps—reducing the number of required GPUs by 82% in production. **MoLoRA** (arXiv 2025, Microsoft Research) addresses the *multi-adapter serving* problem: how to serve mixed-capability or multimodal requests without running K separate forward passes. It introduces per-token LoRA routing with a learned gating mechanism, enabling a 1.7B model to outperform an 8B baseline while reducing P99 latency by 67×. Together, these works represent complementary system-level and model-level innovations that push the state of the art in efficient LLM inference. We analyse their problem formulations, core novelties, experimental setups, benchmark results, integration with modern serving stacks, and their implications for future work.

Contents

1	Introduction	3
2	Background	3
2.1	LLM Inference Basics	3
2.2	GPU Pooling and Auto-Scaling	3
2.3	Multi-Adapter Serving with LoRA	4
3	Aegaeon: Token-Level Auto-Scaling for GPU Pooling	4
3.1	Problem Definition	4
3.2	Core Novelty	4
3.2.1	Theoretical Analysis of Existing Solutions	4
3.2.2	Token-Level Auto-Scaling	4
3.2.3	Disaggregated Scheduling	5
3.2.4	Full-Stack Auto-Scaling Optimisations	5
3.3	Experimental Setup	5
3.4	Key Results	5
3.5	Implementation	6
3.6	GitHub / Artifact Status	7
4	MoLoRA: Per-Token Adapter Routing	7
4.1	Problem Definition	7
4.2	Core Novelty	7
4.2.1	Per-Token Routing Framework	7
4.2.2	Theoretical Results	8
4.2.3	MoLoRA: Learned Gating for Semantic Specialisation	8
4.2.4	Hot-Set Memory Architecture	8
4.2.5	Unified Dispatch Infrastructure	8
4.3	Experimental Setup	9
4.4	Key Results	9
4.5	Implementation	10
5	Comparative Analysis	10
6	Integration with Modern LLM Serving Systems	10
6.1	vLLM Integration	10
6.2	Disaggregation Frameworks	11

6.3	Serverless LLM Infrastructure	11
6.4	Multi-Adapter Serving Stacks	11
6.5	Mixture-of-Experts (MoE) Models	12
6.6	Multimodal Serving Pipelines	12
7	Limitations and Future Directions	12
7.1	Aegaeon	12
7.2	MoLoRA	12
7.3	Open Research Questions	13
8	Conclusion	13

1 Introduction

The deployment of large language models (LLMs) at scale is fundamentally constrained by GPU resources. Two distinct but related challenges have emerged as production workloads mature:

- (1) **Multi-model serving:** Model marketplaces such as Hugging Face host over one million models. Cloud providers must serve thousands of distinct models concurrently, yet most models receive sporadic, bursty traffic. Reserving a dedicated GPU instance per model wastes vast resources on idle hardware.
- (2) **Multi-adapter serving:** Fine-tuning via Low-Rank Adaptation (LoRA) [3] produces many lightweight adapters on top of a shared base model. Existing systems route an *entire sequence* to one adapter, which is suboptimal for mixed-domain or multimodal requests that naturally require expertise from multiple adapters within a single generation.

This survey studies one paper that addresses each challenge. **Aegaeon** [1] solves challenge (1) by introducing *token-level auto-scaling*: models are loaded and unloaded from GPU memory between individual decode steps, not just between full requests, dramatically increasing the number of models that can share a single GPU. **MoLoRA** [2] solves challenge (2) by introducing *per-token adapter routing*: rather than committing an entire sequence to one LoRA adapter, individual tokens are routed to the most appropriate adapter, reducing forward-pass redundancy and enabling composable specialization.

The remainder of this report is structured as follows. Section 2 provides background on LLM serving infrastructure. Sections 3 and 4 analyse the two papers in depth. Section 5 compares them. Section 6 discusses integration with modern systems. Section 7 discusses limitations and future work, and Section 8 concludes.

2 Background

2.1 LLM Inference Basics

LLM inference proceeds in two phases. The *prefill* phase processes all prompt tokens in a single forward pass and generates the first output token. Each subsequent *decode* step processes only the newly generated token, consulting a stored *KV cache* of intermediate attention states. This auto-regressive structure means latency metrics split into:

- **TTFT** (Time-To-First-Token): latency of the prefill phase.
- **TBT** (Time-Between-Tokens): latency of each decode step.

Service-level objectives (SLOs) are typically defined in terms of TTFT and TBT deadlines. Recent work [8, 9] advocates *disaggregated* prefill and decode—separating these phases onto different GPU instances to avoid interference.

2.2 GPU Pooling and Auto-Scaling

Two broad strategies exist for sharing a GPU across multiple models:

Multiplexing places multiple model weights on the same GPU simultaneously, exploiting temporal or spatial sharing. It is constrained by GPU VRAM: for example, at most two 14B-parameter models in FP16 fit on an 80 GB A100.

Auto-scaling keeps only one (or a few) models active on a GPU at a time, loading weights from host memory or SSD on demand. Systems like ServerlessLLM [5] and BlitzScale [7] have focused on minimising the cold-start latency of this loading step.

2.3 Multi-Adapter Serving with LoRA

LoRA [3] fine-tunes a pretrained weight matrix $W \in \mathbb{R}^{d \times d}$ by adding a low-rank update $\Delta W = AB$ where $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$, $r \ll d$. The adapted forward pass is $h = xW + xAB$, adding only $2dr$ parameters.

Multi-adapter serving systems (S-LoRA [10], Punica [11]) maintain a catalogue of adapters and page them in/out as needed, using GPU memory efficiently via unified paging. These systems assume a single adapter per *sequence*, which is the fundamental limitation MoLoRA challenges.

3 Aegaeon: Token-Level Auto-Scaling for GPU Pooling

3.1 Problem Definition

Aegaeon targets **concurrent LLM serving on a model marketplace**, where M distinct models must be served from a shared GPU pool of N instances ($N \ll M$). The goal is to minimise N while satisfying per-token SLOs (TTFT and TBT deadlines) across all models.

The paper characterises the production workload at Alibaba Cloud Model Studio (serving thousands of models), highlighting:

- 94.1% of 779 models receive only 1.35% of all requests (heavy long tail).
- 17.7% of 30,000 GPUs are allocated to models averaging fewer than 0.2 requests/second/GPU.
- Popular models (e.g., DeepSeek, Llama, Qwen) experience bursty arrivals that transiently exceed reserved capacity.

3.2 Core Novelty

3.2.1 Theoretical Analysis of Existing Solutions

Existing auto-scaling solutions swap models only at *request* boundaries. The paper formalises their limitation via Theorem 3.1:

If request arrivals per model follow a Poisson process with rate λ and average service time T , the expected number of concurrently active models is:

$$\mathbb{E}[m] = M \cdot (1 - e^{-\lambda T})$$

For $\lambda = 0.037$ req/s and $T = 16.79$ s (values from the production trace), $\mathbb{E}[m] \approx 46.55$ out of 100 models remain active at any instant—even though aggregate arrival rate $M\lambda = 3.7$ req/s is modest. Request-level auto-scaling therefore requires $N \approx \mathbb{E}[m]$ instances, limiting pooling to fewer than 3 models per GPU.

3.2.2 Token-Level Auto-Scaling

Aegaeon’s core innovation is **preemptive auto-scaling between decode steps**. Instead of waiting for a request to fully complete before swapping models, Aegaeon:

1. After the current model’s decode step, saves its KV cache to host DRAM.
2. Loads a different model’s weights (for a newly arrived or waiting request).
3. Restores that model’s KV cache to VRAM and executes its next decode step.

This reduces head-of-line (HOL) blocking: pending requests need not wait for an entire multi-second generation to complete. The paper demonstrates 7 models per GPU in experiments.

3.2.3 Disaggregated Scheduling

Aegaeon separates the GPU pool into *prefill instances* and *decoding instances*, following the disaggregation paradigm [8]. This prevents long decode sequences from starving incoming prefill jobs and vice versa.

Prefill scheduler (Algorithm 1): A grouped FCFS (first-come-first-served) policy batches requests for the same model together to amortise auto-scaling cost, while preserving approximate FCFS ordering to prevent starvation.

Decode scheduler (Algorithm 2): A weighted round-robin scheduler assigns time quotas q_i to each model’s decode batch:

$$q_i = \frac{c}{n_i \cdot (\alpha - \sum_k \frac{1}{n_k})}$$

where $n_k = d/t_k$ is the number of decode steps per TBT deadline, c is the total auto-scaling overhead in the round, and $\alpha \geq 0.5$ is a tuned reciprocal of the target SLO attainment. This formula ensures that all batches meet their TBT deadlines with bounded slack.

3.2.4 Full-Stack Auto-Scaling Optimisations

Unoptimised preemptive auto-scaling (loading a 13B model) takes up to 26.9 seconds. Aegaeon reduces this to under one second via three techniques:

(a) Component Reuse. An inference engine (e.g., vLLM) initialises many components: distributed executor (Ray/NCCL), tokeniser, profiling passes, KV cache allocators. Aegaeon initialises these *once* per instance and caches them. Only model weights and the KV cache are re-loaded per auto-scale event. This eliminates $> 80\%$ of scaling latency.

(b) Explicit Memory Management. To prevent VRAM fragmentation (normally requiring garbage collection) and DRAM fragmentation (wasting host cache space), Aegaeon:

- Allocates a self-managed **VRAM buffer** with bump allocation, monkey-patching PyTorch’s `torch.nn.Parameter` to use this buffer—bypassing the CUDA caching allocator entirely.
- Maintains a shared **Model Cache** in host DRAM with page-locked Stage Buffers for zero-copy staging.
- Uses **slab allocation** for the KV cache across all possible KV shapes (which vary by model architecture), keeping overall fragmentation below 20%.
- Optionally **prefetches** the next model’s weights on a separate CUDA stream while the current model decodes, achieving near-instant model loading in many cases.

(c) Fine-Grained KV Cache Synchronisation. Moving KV cache asynchronously between prefill and decode instances risks data races (inference requiring KV cache to be on GPU; in-flight transfers conflicting). Aegaeon uses `cudaEvent` APIs (`cudaEventRecord`, `cudaStreamWaitEvent`, `cudaEventQuery`, and IPC variants for cross-process synchronisation) to track individual transfer states. A daemon thread periodically reclaims completed transfers, keeping synchronisation off the critical serving path.

The combined effect reduces total auto-scaling overhead by 97%: from $T_0 = 26.9$ s to $T_3 < 1$ s for a 13B model.

3.3 Experimental Setup

3.4 Key Results

Additional findings:

Table 1: Aegaeon Experimental Setup

Aspect	Details
Hardware	2 nodes $\times$ 8 NVIDIA H800 80GB GPUs (NVLINK); 2 TB DDR5; 192 Intel Xeon Platinum 8469C CPUs.
Models	Qwen, Llama, InternLM, Yi families; primarily 6B–14B parameters. Larger (72B, TP=4) in sensitivity analysis.
Datasets	ShareGPT (main); ShareGPT-ix2 ($2\times$ input length); ShareGPT-ox2 ($2\times$ output length).
Workload synthesis	Scaled Poisson processes, random sampling from datasets. Real workloads in production evaluation.
SLO targets	TTFT = 10 s, TBT = 100 ms (standard); sensitivity tests with $0.2\times$ – $0.5\times$ multipliers (down to 2 s / 20 ms).
GPU split	6 prefill instances + 10 decoding instances (16 GPUs total).
Baselines	ServerlessLLM [5] (FCFS); ServerlessLLM+ (oracle SJF scheduling); MuxServe [6] (spatial-temporal multiplexing).
Metric	SLO attainment (% of tokens meeting their TTFT/TBT deadlines).
Production deployment	213 H20 GPUs; 28 models (1.8–7B, TP=1) + 19 models (32–72B, TP=4); 70-hour evaluation.

- With stricter SLOs ($0.3\times$), Aegaeon still supports 12.5% more models than MuxServe; at $0.2\times$ (extremely tight), MuxServe’s lack of auto-scaling overhead begins to win.
- Longer output sequences (ShareGPT-ox2) increase Aegaeon’s advantage to $2.5\times$ goodput, since HOL blocking worsens with decoding time.
- Slab allocation keeps CPU KV cache fragmentation $< 20\%$ across all block shapes.
- The grouped prefill scheduler maintains controlled prefill wait times even as arrival rates increase.

3.5 Implementation

Aegaeon is implemented in **5,700 lines of Python and CUDA/C++**. Key components:

- Control plane: Python `asyncio` for orchestration.
- Data plane: Ray for distributed data routing.
- Inference backend: vLLM (providing PagedAttention, FlashAttention, continuous batching).
- Memory management: custom Python monkey-patching + CUDA stream management.

The system uses Redis for shared-memory metadata synchronisation across instances.

Table 2: Aegaeon Performance Summary

Metric	Aegaeon	ServerlessLLM	MuxServe
Max models per GPU	7	< 3	≤ 2
Goodput advantage (RPS=0.1)	2×	1×	limited
Goodput advantage (RPS=0.5)	2.5×	1×	limited
Auto-scaling latency (13B)	<1 s	$\sim 4\text{--}26$ s	N/A
Production GPU saving	82% (1192→213 GPUs)	–	–
GPU utilisation (production)	48.1%	13.3–33.9%	–

3.6 GitHub / Artifact Status

As of the survey date, the Aegaeon source code has not been released in a public GitHub repository. The paper received ACM “Artifacts Evaluated / Available / Results Reproduced” badges at SOSP 2025, indicating the artifact was reviewed by the programme committee, but access appears to be through the ACM artifact evaluation process rather than a public repository. The paper’s senior author (Ennan Zhai) hosts the PDF at <https://ennanzhai.github.io/pub/sosp25-aegaeon.pdf>.

4 MoLoRA: Per-Token Adapter Routing

4.1 Problem Definition

MoLoRA targets **multi-adapter serving for LLMs**, where a single base model deployment must efficiently serve requests requiring multiple LoRA adapters. Existing systems (S-LoRA, Punica) implement *per-sequence* routing $j : [B] \rightarrow [K]$, constraining all tokens in a sequence to use the same adapter. This fails in two settings:

1. **Multimodal generation:** Models like Chameleon [13] generate interleaved text and image tokens. Text tokens and image tokens require different adapters, but per-sequence routing forces a single choice.
2. **Mixed-capability requests:** A prompt like “write Python code to solve this differential equation” requires both a code adapter and a math adapter. No single adapter is optimal.

4.2 Core Novelty

4.2.1 Per-Token Routing Framework

MoLoRA replaces the sequence-level routing function with a token-level function $r : [N] \rightarrow [C]$, where C is the number of computational targets. The adapted forward pass becomes:

$$h_i = x_i W + x_i A^{(r(i))} B^{(r(i))}$$

Vocabulary routing (deterministic): For multimodal models with disjoint vocabulary ranges per modality, the routing function is a simple range comparison:

$$R_{\text{vocab}}(v) = m \iff v \in [b_{m-1}, b_m)$$

This requires only $M - 1$ integer comparisons—orders of magnitude cheaper than a learned softmax gate.

Compositional routing: Routing over a product space $A \times M$ (adapter $\times$ modality) can be collapsed to a single composite index $c = a \cdot |M| + m$, enabling $|A| \cdot |M|$ distinct computational paths from a single unified weight tensor without separate deployments.

4.2.2 Theoretical Results

Theorem 1 (Computational Optimality): Per-sequence routing with K adapter types requires work $W \geq K \cdot N \cdot c_{\text{pass}}$, while per-token routing requires $W = N \cdot c_{\text{pass}}$ for N tokens. Per-token routing is thus optimal.

Theorem 2 (Sparse Attention Equivalence): Per-token adapter routing with vocabulary routing is equivalent to sparse (block-diagonal) attention, where modality groups form the blocks. This unifies adapter routing with the sparse attention literature and enables transfer of corresponding optimisations.

4.2.3 MoLoRA: Learned Gating for Semantic Specialisation

Vocabulary routing fails for encoder-based multimodal models (LLaVA, Flamingo), semantic specialisation (code vs. math sharing the same vocabulary), and sub-modality granularity. MoLoRA extends per-token routing with a learned router:

$$g_{\theta}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2$$

$$w_i = \text{softmax}(\text{TopK}(g_{\theta}(x_i), k)), \quad \Delta h_i = \sum_{j \in \text{TopK}} w_{i,j} \cdot x_i A^{(j)} B^{(j)}$$

The router is a lightweight 2-layer MLP with 64 hidden units, adding only $64d + 64K$ parameters—negligible relative to the base model. A load-balancing auxiliary loss (following Switch Transformer [12]) encourages uniform adapter utilisation:

$$\mathcal{L}_{\text{aux}} = K \cdot \sum_{j=1}^K f_j \cdot p_j$$

4.2.4 Hot-Set Memory Architecture

Existing paging systems (S-LoRA) evict adapters dynamically, causing variable latency and preventing CUDA graph capture (because data-dependent control flow). MoLoRA introduces a *hot-set* architecture: S adapter slots are pre-allocated on GPU with fixed addresses:

$$A_{\text{hot}} \in \mathbb{R}^{S \times M \times d \times r}, \quad B_{\text{hot}} \in \mathbb{R}^{S \times M \times r \times d}$$

A slot table $\sigma : [S] \rightarrow \mathcal{A}$ maps slots to adapters and is updated asynchronously. Fixed addresses enable CUDA graph capture, reducing per-kernel launch overhead from 5–10 μs to $< 10 \mu\text{s}$ per kernel.

4.2.5 Unified Dispatch Infrastructure

The dispatch kernel is **target-agnostic**: it constructs a token-count histogram $h \in \mathbb{Z}^C$, allocates positions via atomic increment, and emits pointer arrays for scatter-gather. This is mathematically identical to MoE dispatch kernels, enabling bidirectional transfer of MoE optimisations.

Adaptive tiling selects tile sizes based on histogram counts:

$$\text{BLOCK_M} = \begin{cases} 16 & h[k] < 64 \\ 32 & 64 \leq h[k] < 256 \\ 64 & h[k] \geq 256 \end{cases}$$

4.3 Experimental Setup

Table 3: MoLoRA Experimental Setup

Aspect	Details
Hardware	NVIDIA H100 GPU; CUDA 13.0; PyTorch 2.9.0.
Primary model	Qwen3-1.7B (base) with four specialized LoRA adapters (rank=32). Evaluation also on Qwen3-4B.
LoRA training	GRPO [17] on filtered reasoning corpus (math, logical, scientific reasoning).
Router training	2-layer MLP; 100 epochs to convergence; cross-entropy + auxiliary load-balancing loss.
Benchmarks	GSM8K, MATH, BBH, GPQA (quality); kernel latency and throughput (systems).
Baselines	Per-sequence routing (S-LoRA style + BGMV scalar kernel); dynamic paging (LRU eviction).
Measurement	Median of 1000 iterations after 100 warmup; P99 latency with variance bands.
Configurations	$d \in \{1024, 2048, 4096\}$; $r = 64$; $K \in \{2, 4, 8\}$; batch sizes 32–1024.

4.4 Key Results

Additional findings:

- MoLoRA’s learned router achieves *emergent modality discovery*: starting from random routing (ARI = 0.27), it converges to ARI = 0.71 with no modality labels supplied (only gradient signal from downstream loss).
- Semantic domain routing on Qwen3-4B embeddings achieves 100% domain classification accuracy within 100 epochs, with near-perfect per-token specialisation (e.g., code tokens route 98.6% to Adapter 0).
- Hot-set P99 latency is *stable* (0.084–0.095 ms) across uniform, Zipfian, bursty, and adversarial access patterns, while paging degrades up to 38× under adversarial patterns.
- Direct indexing (hot-set) over indirect indexing (S-LoRA style) provides 1.14× average improvement across adapter counts.
- The kernel crossover between scalar (BGMV) and tensor-core implementations occurs at batch size ≈ 160 ; production batch sizes (256–1024) yield 1.98–2.69× speedup.

Table 4: MoLoRA Performance Summary

Metric	MoLoRA / Per-Token	Baseline / Per-Seq
Forward passes (K=4 modalities)	1	4
Latency (K=4, 2048 tokens, d=4096)	1.43 ms	5.88 ms
Throughput improvement	4.1× (pass reduction)	1×
P99 latency (CUDA graph vs paging)	67× reduction	—
Kernel speedup (batch 512)	1.98×	—
Kernel speedup (batch 1024)	2.69×	—
Speedup (interleaved, K=4)	52.7×	—
Speedup (separated, K=4)	7.5×	—
GSM8K (Qwen3-1.7B + MoLoRA)	+14% vs Qwen3-8B	Qwen3-8B baseline
MATH	+8% vs Qwen3-8B	Qwen3-8B baseline
BBH	+2.5% vs Qwen3-8B	Qwen3-8B baseline
GPQA	+2.1% vs Qwen3-8B	Qwen3-8B baseline
Model size ratio	4.7× smaller	—

4.5 Implementation

MoLoRA is implemented on top of PyTorch 2.9 and CUDA 13, with Triton kernels for adaptive tiling and grouped GEMM dispatch. The codebase uses the same dispatch infrastructure for both vocabulary routing and learned MoLoRA gating, validating the MoE/multi-adapter unification claim. As of submission, the paper references [arXiv:2603.15965](#); the authors are at Microsoft Research (shreyshah@microsoft.com, justiwag@microsoft.com), but no public GitHub repository was linked.

5 Comparative Analysis

Complementarity: The two systems address orthogonal problems and could be composed. Aegaeon manages *which base model* is active on a GPU; MoLoRA manages *which adapter* augments that base model. A combined system might use Aegaeon’s scheduler to switch between base models at the token level, while MoLoRA’s router selects among adapters for the active base model, enabling fine-grained adaptation at minimal overhead.

Shared philosophy: Both systems introduce token-level granularity into previously coarser-grained operations, both exploit slack time between decode steps, and both invest in system-level tricks (custom memory management, CUDA-level synchronisation) to make their fine-grained operations practical.

6 Integration with Modern LLM Serving Systems

6.1 vLLM Integration

Both papers build on or are evaluated against vLLM [4]. Aegaeon uses vLLM as its inference backend and monkey-patches its memory management; MoLoRA’s kernel could be integrated as a drop-in replacement for S-LoRA’s BGMV kernel. vLLM’s PagedAttention (paged KV cache)

Table 5: Side-by-Side Comparison: Aegaeon vs. MoLoRA

Dimension	Aegaeon	MoLoRA
Core problem	Multi-model GPU pooling	Multi-adapter per-request serving
Level of novelty	Systems (scheduling + memory)	Algorithmic + systems
Granularity	Token-level model switching	Token-level adapter selection
Routing decision	Scheduler (greedy heuristic)	Learned MLP router
Memory management	Slab alloc, bump alloc, KV swap	Hot-set pre-allocation
Concurrency mechanism	CUDA events, async streams	CUDA graph capture
Venue	SOSP 2025 (peer-reviewed)	arXiv 2025 (preprint)
Affiliation	Alibaba / Peking University	Microsoft Research
Deployment	Production (213 H20 GPUs)	Research prototype
Baseline models	ServerlessLLM, MuxServe	S-LoRA, Punica
Primary metric	SLO attainment / goodput	Latency / throughput / accuracy
GPU saving	82% (production)	N/A (not deployed at scale)
Key theorem	Active model count formula	Computational optimality; sparse attn equivalence
LoRA-aware	No	Yes
Model switching	Explicit (weights in/out of GPU)	Implicit (adapter weights stay GPU-resident)

is compatible with both systems.

6.2 Disaggregation Frameworks

Aegaeon implements prefill-decode disaggregation directly. Its scheduling insights could be applied to disaggregated serving frameworks like DistServe [8] and Mooncake [16], which already separate prefill and decode compute to specialised clusters.

6.3 Serverless LLM Infrastructure

Aegaeon directly targets the serverless paradigm pioneered by ServerlessLLM [5] and BlitzScale [7]. Its component reuse and explicit memory management are applicable to any serverless inference framework. ParaServe [15] (parallel cold starts) and Aegaeon’s preemptive scaling are complementary: parallel loading could accelerate Aegaeon’s scale-up path further.

6.4 Multi-Adapter Serving Stacks

MoLoRA is designed as an extension to S-LoRA and Punica. The hot-set architecture replaces paging with pre-allocation; the dispatch kernel replaces per-sequence adapter application. Integration would require:

1. Replacing S-LoRA’s LRU eviction manager with MoLoRA’s slot-table manager.
2. Replacing the BGMV kernel with MoLoRA’s tensor-core kernel at batch sizes > 160 .
3. Adding a router checkpoint to the model loading pipeline (negligible overhead: $\ll 1\%$ of model parameters).

6.5 Mixture-of-Experts (MoE) Models

MoLoRA establishes a formal equivalence between per-token adapter routing and MoE token dispatch. This enables direct reuse of MoE infrastructure (e.g., MegaBlocks [14]) for multi-adapter serving. Frameworks such as DeepSpeed-MoE and Mixtral’s serving stack could absorb MoLoRA’s dispatch kernel with minimal modification.

6.6 Multimodal Serving Pipelines

MoLoRA’s vocabulary routing is directly applicable to Chameleon-family unified multimodal models. For encoder-based systems (LLaVA, Flamingo, Qwen-VL), learned routing via the MoLoRA gating mechanism provides the required modality disambiguation, since encoder outputs share vocabulary space with text tokens.

7 Limitations and Future Directions

7.1 Aegaeon

1. **SLO sensitivity:** At very tight SLOs ($0.2\times$), the auto-scaling overhead consumes all available slack. Static multiplexing outperforms Aegaeon in this regime.
2. **Model prefetching requires VRAM:** Prefetching the next model’s weights requires spare VRAM capacity, which is unavailable on lower-end hardware (A10, 24 GB). The paper disables prefetching in this setting.
3. **Scheduling complexity:** The weighted round-robin decoder assumes TBT deadlines are known; adaptive deadlines or heterogeneous SLOs would complicate quota allocation.
4. **Cross-node KV migration:** The KV cache is moved via PCIe (or NVLink within a node). Cross-node prefill-decode disaggregation would require RDMA, adding network latency.
5. **No public code:** The lack of a public repository limits reproducibility and community adoption.

7.2 MoLoRA

1. **Hot-set capacity:** The hot-set scales to S adapter slots on GPU. For very large adapter catalogues (thousands of adapters), dynamic paging remains necessary; a hybrid policy is future work.
2. **Router overhead at small top-k:** For $k = 1$, overhead matches fixed routing. For $k = 2$, the $27.6\times$ overhead relative to a single adapter (Table 16) is significant if latency is critical.
3. **Training cost of router:** The router requires supervised or self-supervised training per deployment. Generalisation of a pre-trained router to new adapter combinations is unaddressed.
4. **Vocabulary routing assumption:** Contiguous vocabulary ranges are assumed for multimodal models. Non-contiguous assignments require an $O(V)$ lookup table.
5. **Preprint status:** MoLoRA has not undergone peer review; some experimental claims (especially the Qwen3 quality comparison) would benefit from independent replication.

7.3 Open Research Questions

- Can Aegaeon’s token-level preemption be combined with speculative decoding [18] to further hide auto-scaling latency?
- Can MoLoRA’s router be trained in a data-free or few-shot manner to reduce deployment cost?
- How do these systems interact with KV cache compression and quantisation (e.g., KVQuant), which alter the shape and size of the cache that must be swapped?
- Is there a unified scheduling framework that jointly optimises base model switching (Aegaeon) and adapter selection (MoLoRA) across a heterogeneous fleet?

8 Conclusion

This survey has examined two papers that advance the state of the art in efficient LLM serving from complementary directions. **Aegaeon** (SOSP 2025) demonstrates that serving many models on shared GPU infrastructure is tractable at production scale, achieving 82% GPU savings and 7 models per GPU through token-level auto-scaling with full-stack memory optimisations. **MoLoRA** (arXiv 2025) demonstrates that multi-adapter serving need not incur the $K \times$ cost of K forward passes; per-token routing reduces this to a single pass while enabling composable specialisation that lets a 1.7B model surpass an 8B model on reasoning benchmarks.

Both works underscore a common theme: *coarse-grained decisions (per-request or per-sequence) leave significant performance and efficiency on the table*. Token-level granularity—whether for model switching or adapter selection—is the key lever for unlocking higher utilisation. Together, they form a compelling foundation for the next generation of LLM serving infrastructure, where hundreds of specialised models and adapters are served efficiently from shared GPU pools.

References

- [1] Y. Xiang, X. Li, K. Qian, Y. Yang, D. Zhu, W. Yu, E. Zhai, X. Liu, X. Jin, and J. Zhou, “Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market,” in *Proc. ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, pp. 1030–1045, 2025.
- [2] S. Shah and J. Wagle, “MoLoRA: Composable Specialization via Per-Token Adapter Routing,” *arXiv preprint arXiv:2603.15965*, 2025.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. ACM SOSP*, 2023.
- [5] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai, “ServerlessLLM: Low-Latency Serverless Inference for Large Language Models,” in *USENIX OSDI*, 2024.
- [6] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, “MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving,” in *ICML*, 2024.
- [7] D. Zhang, H. Wang, Y. Liu, X. Wei, Y. Shan, R. Chen, and H. Chen, “BlitzScale: Fast and Live Large Model Autoscaling with $O(1)$ Host Caching,” in *USENIX OSDI*, 2025.

- [8] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving,” in *USENIX OSDI*, 2024.
- [9] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient Generative LLM Inference Using Phase Splitting,” in *ACM/IEEE ISCA*, 2024.
- [10] Y. Sheng, S. Cao, D. Li, C. Hooper, N. Lee, S. Yang, C. Chou, B. Zhu, L. Zheng, K. Keutzer, J. E. Gonzalez, and I. Stoica, “S-LoRA: Serving Thousands of Concurrent LoRA Adapters,” in *Proc. MLSys*, 2024.
- [11] L. Chen, Z. Ye, Y. Wu, D. Zhuo, L. Ceze, and A. Krishnamurthy, “Punica: Multi-Tenant LoRA Serving,” *arXiv preprint arXiv:2310.18547*, 2023.
- [12] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” *JMLR*, vol. 23, pp. 1–39, 2022.
- [13] Chameleon Team, “Chameleon: Mixed-Modal Early-Fusion Foundation Models,” *arXiv preprint arXiv:2405.09818*, 2024.
- [14] T. Gale, D. Narayanan, C. Young, and M. Zaharia, “MegaBlocks: Efficient Sparse Training with Mixture-of-Experts,” in *Proc. MLSys*, 2023.
- [15] C. Lou, S. Qi, C. Jin, D. Nie, H. Yang, X. Liu, and X. Jin, “Towards Swift Serverless LLM Cold Starts with ParaServe,” *arXiv preprint*, 2025.
- [16] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, “Mooncake: Trading More Storage for Less Computation — A KVCache-Centric Architecture for Serving LLM Chatbot,” in *USENIX FAST*, 2025.
- [17] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [18] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *ICML*, 2023.

LLM Inference Serving: Adaptive Batching Strategies

A Comparative Literature Survey

BucketServe & FairBatching

Papers Analysed	
Paper 1	Paper 2
<i>BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving</i>	<i>FairBatching: Fairness-Aware Batch Formation for LLM Inference</i>
Zheng et al., arXiv:2507.17120 (Jul 2025)	Lyu et al., arXiv:2510.14392 (Oct 2025)
SIAT / SUSTech, China	Shanghai Jiao Tong University, China

Prepared for: Graduate-Level Literature Survey

Domain: Systems & Infrastructure for Large Language Models

Topics: Dynamic Batching, SLO Compliance, LLM Scheduling, GPU Efficiency

This survey provides a comprehensive technical analysis including core novelty, experimental methodology, system integration, benchmarks, and comparative insights for literature survey purposes.

Contents

1	Introduction and Motivation	4
1.1	The Batching Challenge	4
1.2	Scope of This Survey	5
2	Background and Prior Work	6
2.1	Core Serving Frameworks	6
2.1.1	vLLM and PagedAttention	6
2.1.2	Orca and Continuous Batching	6
2.1.3	Sarathi-Serve and Stall-Free Batching	6
2.2	Disaggregated Architectures	6
2.2.1	DistServe	6
2.2.2	SplitWise and TetriInfer	6
2.3	Key Metrics	7
3	Paper 1: BucketServe	8
3.1	Problem Statement and Motivation	8
3.2	Core Novelty and Technical Contributions	8
3.2.1	Bucket-Based Prefill Batching	9
3.2.2	Adaptive Bucket Splitting and Merging (Algorithm 1)	9
3.2.3	Memory-Safe Dynamic Batch Sizing	10
3.2.4	Three-Tier System Architecture	10
3.2.5	Phase-Aware Scheduling Policies	11
3.3	Experimental Setup	11
3.3.1	Hardware Configuration	11
3.3.2	Models and Datasets	11
3.3.3	Baseline Systems	12
3.3.4	Evaluation Metrics	12
3.4	Key Results	12
3.4.1	Offline Throughput	12
3.4.2	Online SLO Compliance	13
3.4.3	System Load Capacity	13
3.4.4	Bucketing Overhead	13
4	Paper 2: FairBatching	14
4.1	Problem Statement and Motivation	14
4.2	Core Novelty and Technical Contributions	15
4.2.1	Envelope-Based SLO Formulation	15

4.2.2	Adaptive Time-Based Batch Capacity	15
4.2.3	Fair and Dynamic Batch Formation (Algorithm 1)	16
4.2.4	Prefill Admission Budget (PAB) for Cluster-Level Integration . .	16
4.3	Experimental Setup	17
4.3.1	Hardware Configurations	17
4.3.2	Datasets and SLO Targets	17
4.3.3	Baseline Systems	18
4.3.4	FairBatching Variants Tested	18
4.3.5	Evaluation Metrics	18
4.4	Key Results	18
4.4.1	Single-Node Peak Goodput	18
4.4.2	Latency Detail (Qwen3-14B, QwenTrace, RPS=2.5)	19
4.4.3	SLO Versatility	19
4.4.4	Performance Breakdown (Ablation)	19
4.4.5	Cluster-Level Performance	19
5	Comparative Analysis	20
5.1	Side-by-Side System Comparison	20
5.2	Complementarity of the Two Approaches	20
5.3	Architectural Philosophy Differences	21
6	Literature Survey Context	22
6.1	Taxonomy of LLM Serving Research	22
6.2	Positioning Within the Batching/Scheduling Subcategory	22
6.2.1	Continuous Batching (Orca)	22
6.2.2	Chunked Prefill / Stall-Free Batching (Sarathi)	22
6.2.3	Preemptive Multi-Level Queuing (FastServe)	22
6.2.4	Dynamic Scheduling (Llumnix)	22
6.3	Key Related Works for Literature Survey	23
6.4	Benchmark Datasets Used in LLM Serving Research	23
7	Integration with Modern LLM Serving Infrastructure	24
7.1	Integration with vLLM	24
7.2	Integration with Production Serving Stacks	24
7.3	Handling Python GC (FairBatching)	25
7.4	Applicability to Newer LLM Architectures	25
8	Critical Analysis and Limitations	26
8.1	BucketServe: Strengths and Weaknesses	26
8.2	FairBatching: Strengths and Weaknesses	26

8.3	Common Limitations Across Both Papers	27
9	Open Research Directions	28
9.1	Combining Bucket-Based Routing with Fair Scheduling	28
9.2	Distribution-Aware Bucketing	28
9.3	Model-Aware Time Estimation	28
9.4	Online SLO Negotiation	28
9.5	Hardware-Software Co-design	28
9.6	Multi-Level Load Balancing	28
10	Summary and Recommendations	30
10.1	Quick Reference Card	30
10.2	Recommendations for Practitioners	30

1 Introduction and Motivation

Large Language Model (LLM) inference has emerged as one of the most computationally demanding workloads in modern AI infrastructure. Unlike traditional machine learning inference (which processes fixed-size inputs with deterministic latency), LLM inference operates autoregressively: it generates tokens one at a time, with each forward pass dependent on all previously generated tokens. This fundamental property creates unique challenges at the intersection of systems design, scheduling theory, and resource management.

The inference pipeline of any decoder-only transformer model (such as GPT-4, LLaMA, or DeepSeek) consists of two distinct computational phases:

- **Prefill Phase:** The entire input prompt is processed in a single parallel forward pass to construct the Key-Value (KV) cache. This phase is *compute-bound*, benefiting from high arithmetic intensity.
- **Decoding Phase:** Output tokens are generated one at a time, each requiring a forward pass that attends over the growing KV cache. This phase is *memory-bandwidth-bound*, bottlenecked by cache access patterns.

The heterogeneous nature of these phases, combined with wildly varying request lengths (from tens to hundreds of thousands of tokens in modern deployments) and strict Service Level Objectives (SLOs), makes LLM inference serving an exceptionally rich and challenging systems problem.

1.1 The Batching Challenge

Batching multiple requests together is essential for amortizing the cost of model parameter loading and maximizing GPU utilisation. However, naive batching introduces several inefficiencies:

1. **Padding Waste:** Batching requests of different sequence lengths requires padding shorter sequences to the maximum length, wasting memory and compute.
2. **Head-of-Line Blocking:** Long requests in a batch delay shorter ones.
3. **Memory Overflow:** The KV cache grows quadratically with sequence length, risking Out-of-Memory (OOM) errors if batch sizes are not carefully managed.
4. **SLO Unfairness:** Optimising for one metric (e.g., throughput) can systematically violate another (e.g., latency).

The two papers surveyed in this report—**BucketServe** [1] and **FairBatching** [2]—attack different facets of this batching challenge with complementary approaches, making them together a coherent contribution to the literature.

1.2 Scope of This Survey

This survey provides:

- Technical deep-dives into the core novelty and algorithmic contributions of each paper
- Detailed analysis of experimental setups, hardware configurations, datasets, and baselines
- Comparative analysis of both systems against each other and the broader literature
- Discussion of integration patterns with modern LLM serving infrastructure
- Critical assessment of limitations and open research directions

2 Background and Prior Work

2.1 Core Serving Frameworks

2.1.1 vLLM and PagedAttention

vLLM [3] introduced *PagedAttention*, which manages KV cache using a paged virtual memory mechanism analogous to OS virtual memory. This reduced KV cache fragmentation from approximately 35% to 5% and improved throughput by 2.4× over prior systems. Both BucketServe and FairBatching are built directly on top of vLLM, making it the de facto substrate.

2.1.2 Orca and Continuous Batching

Orca [4] introduced *continuous batching* (also called iteration-level scheduling), where completed requests in a batch are immediately replaced by new ones rather than waiting for the entire batch to finish. This significantly improves GPU utilisation over static batching.

2.1.3 Sarathi-Serve and Stall-Free Batching

Sarathi-Serve [5] introduced *chunked prefill* and *stall-free batching*. The key insight is that by chunking prefill tasks and interleaving them with decode tasks—within a fixed token budget per batch step—the system can prevent decode stalls (which cause visible pauses in token generation) while still serving new requests. Sarathi uses Time-Between-Tokens (TBT) as its core scheduling metric and employs a decode-prioritising policy. Both BucketServe and FairBatching reference Sarathi as a key baseline and motivation.

2.2 Disaggregated Architectures

2.2.1 DistServe

DistServe [6] pioneered the idea of *prefill-decode disaggregation*: separating the two phases onto different GPU instances. This allows each phase to be independently optimised (e.g., larger batches for prefill, continuous batching for decode) but introduces KV cache transfer overhead between nodes. DistServe is the primary baseline for BucketServe.

2.2.2 SplitWise and TetriInfer

SplitWise [7] similarly disaggregates across heterogeneous hardware, achieving 40% higher resource utilisation through task parallelism but incurring overhead for short sequences. TetriInfer [8] partitions prompts into fixed-size chunks with a two-level scheduler, reduc-

ing TTFT by 97% but introducing communication overhead in high-throughput long-sequence scenarios.

2.3 Key Metrics

- **TTFT (Time-To-First-Token):** Time from request arrival to first output token. Determines perceived responsiveness.
- **TBT (Time-Between-Tokens):** Per-token metric measuring time between consecutive output tokens. Critically, TBT is *non-monotonic*: generating a token earlier can worsen TBT by creating a larger gap to the next token.
- **TPOT (Time-Per-Output-Token):** Average time per output token = $\frac{\text{OutputTime}_{i,j} - \text{TTFT}_i}{j-1}$. A stable, user-perceivable throughput measure.
- **Throughput (tokens/s):** Total tokens generated per second; primary metric for offline batch workloads.
- **SLO Attainment Rate:** Fraction of requests meeting predefined TTFT and TPOT targets.
- **Goodput / Effective RPS:** Requests per second that *simultaneously* satisfy all SLO constraints; the most rigorous end-to-end metric.

3 Paper 1: BucketServe

Paper Identity

Title: BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving

Authors: Wanyi Zheng, Minxian Xu (corresponding), Shengye Song, Kejiang Ye

Affiliations: Southern University of Science and Technology; Shenzhen Institutes of Advanced Technology (SIAT), Chinese Academy of Sciences

Venue: arXiv:2507.17120v1 [cs.DC], submitted July 23, 2025

Funding: Guangdong Basic and Applied Basic Research Foundation; Shenzhen Basic Research Program

3.1 Problem Statement and Motivation

BucketServe is motivated by two key empirical observations about LLM request characteristics and resource utilisation in disaggregated serving architectures:

Observation 1 – Heterogeneous Request Lengths. An analysis of Stanford Alpaca and LongBench datasets reveals dramatically different distribution profiles:

- **Alpaca:** Short sequences with mean 83.7 tokens and median 69 tokens (concentrated, near-symmetric)
- **LongBench:** Long-tail distribution with mean 55,749 tokens and median 41,417 tokens

When such heterogeneous workloads are mixed in a single batch, the shorter sequences must be padded to the maximum sequence length in the batch, wasting significant GPU memory and compute.

Observation 2 – Execution Time Dominance. Across workload types, the decoding phase consistently dominates total execution time: in “Long” workloads, decode duration (14.9s) is roughly $25\times$ the prefill duration (0.6s). In “Mixed” workloads, decode (13.4s) still vastly exceeds prefill (0.5s), suggesting that optimising decode-phase batching yields disproportionate gains.

Observation 3 – GPU Utilisation Gap. Mixed batches (containing both long and short sequences) exhibit markedly lower GPU utilisation in the decode phase (54.5%) compared to long-only workloads (62.3%), confirming that padding-induced waste is a real bottleneck.

3.2 Core Novelty and Technical Contributions

3.2.1 Bucket-Based Prefill Batching

The foundational idea of BucketServe is to group incoming requests into *size-homogeneous buckets* based on sequence length *before* forming batches. The formal motivation derives from minimising a *wasted memory ratio*:

$$\text{WasteRatio} = \frac{S_{\max} - S_{\text{avg}}}{S_{\max}} \quad (1)$$

where $S_{\max}$ is the maximum sequence length in a batch and S_{avg} is the average. By constraining a bucket's range $[L_b, U_b)$, the system ensures $S_{\max} - S_{\text{avg}}$ is small, dramatically reducing padding waste.

The expected waste rate across all K buckets is:

$$\mathbb{E}[\text{Waste}] = \sum_{b=1}^K \int_{L_b}^{U_b} \left(1 - \frac{S}{U_b}\right) f(S) dS \quad (2)$$

where $f(S)$ is the distribution of incoming request lengths. The theoretically optimal upper bound for each bucket is:

$$U_b^* = \frac{\int_{L_b}^{U_b} S f(S) dS}{\int_{L_b}^{U_b} f(S) dS} \quad (3)$$

This is the conditional expectation of sequence lengths within the bucket. Computing this exactly is impractical at runtime, so BucketServe approximates it with an efficient bisection heuristic.

3.2.2 Adaptive Bucket Splitting and Merging (Algorithm 1)

The adaptive bucketing mechanism dynamically adjusts the number and boundaries of buckets in response to workload:

Initialisation: The system starts with a single bucket $[0, L_{\max})$.

Splitting Trigger: When total requests exceed $N_{\max}$ (the maximum safe batch size), the algorithm checks each bucket b :

- Compute midpoint $m_b = (L_b + U_b)/2$
- Count $C_s = \text{number of requests with length} < m_b$
- If $C_s/|b.\text{requests}| > \theta$ (default $\theta = 0.5$) AND $|b.\text{requests}| > N_{\max}$: split bucket into $[L_b, m_b)$ and $[m_b, U_b)$

Merging Trigger: When total requests drop below $N_{\max}$, all buckets collapse back into a single one to minimise scheduling overhead.

Complexity: $O(n \cdot k)$ for request assignment and $O(k)$ for bucket adjustment, where n is the number of requests and k is the number of buckets. In practice k is small (1–8 in experiments), making this negligible.

3.2.3 Memory-Safe Dynamic Batch Sizing

The KV cache memory footprint for a batch is modelled as:

$$\text{Memory}_{\text{KV cache}} = 2 \times L \times H \times D \times S_{\max} \times B \times N \quad (4)$$

where L = number of layers, H = attention heads, D = dimension per head, B = bytes per element, N = batch size.

To prevent OOM errors, the system reserves 10% of GPU memory as safety margin:

$$M_{\text{safe}} = 0.9 \times M_{\text{remain}} \quad (5)$$

The maximum safe batch size is:

$$N_{\max} = \max \left\{ N \in \mathbb{N} : \sum_{i=1}^N S_i \leq \frac{M_{\text{safe}}}{2LHDB} \right\} \quad (6)$$

This ensures the batching controller never allocates more memory than available, dynamically adjusting $N_{\max}$ as the Global Monitor reports memory state changes.

3.2.4 Three-Tier System Architecture

BucketServe adopts a hierarchical architecture with five main components:

1. **Gateway API:** Routes incoming requests by type and priority to appropriate services (online chatbots vs. offline summarisation)
2. **Request Bucketing Manager:** Groups requests into buckets; implements the adaptive splitting/merging algorithm
3. **Dynamic Batching Controller:** Computes $N_{\max}$, forms batches, prioritises long-waiting requests
4. **P/D Scheduler:** Manages prefill→decode transition; uses FCFS within the prefill queue; transfers KV cache via NVLink; applies continuous batching in the decode phase
5. **Global Monitor:** Continuously tracks GPU memory, queue lengths, arrival rates, and latency; feeds real-time signals to the batching controller and scheduler

3.2.5 Phase-Aware Scheduling Policies

BucketServe distinguishes between offline and online task types and applies different intra-bucket scheduling policies:

Table 1: Scheduling Policies in BucketServe by Task Type and Optimisation Goal

Task Type	Policy	Rationale
Offline (throughput)	Shortest-Job-First (SJF)	Minimises queuing latency and maximises throughput
Offline (token/s)	Longest-Job-First (LJF)	Maximises GPU utilisation via long-sequence attention parallelism
Online (SLO-sensitive)	Earliest-Arrival-First	Ensures FCFS fairness and SLO compliance
Decode Phase (all)	Continuous Batching	Adapts to dynamic output lengths without waiting for full batches

3.3 Experimental Setup

3.3.1 Hardware Configuration

BucketServe Testbed

- **GPUs:** $4 \times$ NVIDIA A100 40GB, interconnected via NVLink
- **CPU:** 64 cores
- **Storage:** 1TB high-speed SSD
- **Backend:** vLLM with PagedAttention and continuous batching
- **Interconnect:** NVLink for GPU-to-GPU KV cache transfer (disaggregated architecture)

3.3.2 Models and Datasets

Models tested:

- LLaMA-2-13B (primary evaluation model)
- OPT series (secondary evaluation)

Datasets:

Table 2: BucketServe Dataset Characteristics

Dataset	Domain	Avg. Input Length	Characteristics
Stanford Alpaca	Instruction following	83.7 tokens (median: 69)	Short, near-uniform
LongBench	Long-context tasks	55,749 tokens (median: 41,417)	Long-tail, ultra-long
Mixed	Hybrid	Both distributions	Heterogeneous, long-tail

Workload Types:

- **Long:** > 1024 tokens from LongBench (GPU utilisation: prefill 97%, decode 45.6%)
- **Short:** < 256 tokens from Alpaca (GPU utilisation: prefill 62.3%, decode 59.5%)
- **Mixed:** Hybrid of both datasets with long-tail distribution

3.3.3 Baseline Systems

1. **UELLM** [12]: Unified LLM serving framework with fine-tuned LLM-based resource prediction. Uses coupled prefill/decode phases; lacks dynamic workload adaptation. Represents the aggregated architecture baseline.
2. **DistServe** [6]: Disaggregated serving system with phase-specific resource allocation. Lacks specialised batching for heterogeneous workloads. Represents the disaggregated architecture baseline.

3.3.4 Evaluation Metrics

- **Offline:** Throughput (tokens/s), average GPU utilisation (prefill and decode separately)
- **Online:** SLO attainment rate at varying RPS, server-side RPS vs. client-side RPS scalability curves, system load capacity

3.4 Key Results

3.4.1 Offline Throughput

- BucketServe achieves up to **3.58×** throughput improvement over UELLM
- BucketServe achieves **1.31×** throughput improvement over DistServe under high loads
- Dynamic batching improves average GPU utilisation to **81.66%** (vs. DistServe at 67.88% and UELLM at 60.86%)

3.4.2 Online SLO Compliance

- At 80% SLO attainment: BucketServe handles **1.37** $\times$ more RPS than DistServe on Alpaca
- At 80% SLO attainment on Mixed workloads: **1.93** $\times$ higher RPS than DistServe
- Server RPS closely tracks the ideal $y = x$ line, confirming near-linear scalability
- On Mixed dataset, BucketServe achieves **3.47** $\times$ higher server RPS than UELLM

3.4.3 System Load Capacity

- Max server RPS on Alpaca: BucketServe (7.9), DistServe (7.1), UELLM (4.0)
- Max server RPS on Mixed: BucketServe (7.3), DistServe (5.2), UELLM (2.1)
- **1.975** $\times$ higher system load capacity vs. UELLM overall

3.4.4 Bucketing Overhead

- Bucketing overhead is $< 1\%$ of total execution time
- Processing time remains stable at ~ 0.12 s regardless of bucket count (1–8 buckets)
- Decoding accounts for $\approx 90\%$ of total execution time in typical workloads

4 Paper 2: FairBatching

Paper Identity

Title: FairBatching: Fairness-Aware Batch Formation for LLM Inference

Authors: Hongtao Lyu, Boyue Liu, Mingyu Wu, Haibo Chen

Affiliation: Institute of Parallel and Distributed Systems (IPADS), Shanghai Jiao Tong University

Venue: arXiv:2510.14392v1 [cs.DC], submitted October 16, 2025

Implementation: vLLM v0.10.1.1; 1,639 lines of Python scheduler code; 90 lines invasive vLLM changes

4.1 Problem Statement and Motivation

FairBatching targets a specific, previously under-studied pathology in stall-free batching (as embodied by Sarathi-Serve): *systematic unfairness between prefill and decode tasks*.

The Core Unfairness. In a Sarathi-style scheduler under moderate load, the following simultaneously occurs:

- Decode tasks consistently run **1,000+ tokens ahead** of their TPOT SLO target (massive accumulated slack)
- Prefill tasks **periodically violate** their TTFT SLO, even though total computational demand is within capacity

Root Cause Analysis. The paper identifies two fundamental causes:

1. **Non-monotonic TBT:** TBT (Time Between Tokens) is not monotonically improvable. Generating a token *earlier* can create a longer gap to the next token, worsening TBT. To satisfy TBT guarantees, the scheduler must maintain inference progress even during prefill bursts, even if decode tasks have massive headroom. This forces decode-prioritisation regardless of system state.
2. **Rigid Decode-Prioritising Policy:** Under Sarathi, when no prefill tasks exist, decode tasks monopolise 100% of compute. When prefill tasks arrive in a burst, decode tasks still receive a guaranteed fixed share, leaving insufficient resources for prefill and causing TTFT violations.

Practical Irrelevance of TBT. Modern client-side LLM frameworks (LangGraph, NextChat, etc.) use buffering and token-smoothing mechanisms that mask individual token-level delays. Chain-of-Thought (CoT) and tool-calling patterns further reduce sensitivity to TBT. The more user-relevant metric is TPOT (average output speed).

4.2 Core Novelty and Technical Contributions

4.2.1 Envelope-Based SLO Formulation

FairBatching replaces TBT with a novel, *monotonic* SLO metric based on per-token deadlines. For a request i with TTFT SLO of `ttft_slo` and TPOT SLO of `tpot_slo`, the deadline of the j -th output token is:

$$\text{token_ddl}_{i,j} = \text{arrival}_i + \text{ttft_slo} + \text{tpot_slo} \times j \quad (7)$$

This deadline is **independent of current inference progress**, giving it a critical property: generating any token earlier *strictly improves* SLO compliance (monotonicity). This enables the scheduler to reason meaningfully about whether a request has accumulated slack without contradicting itself.

The per-request *slack time* is then:

$$\text{slack}_i = \text{token_ddl}_{i,\text{next_output_idx}} - \text{current_time} \quad (8)$$

Positive slack means the request is ahead of its SLO curve; negative slack means it is behind.

4.2.2 Adaptive Time-Based Batch Capacity

Existing systems use a static token budget to determine batch size, which fails to account for varying context lengths (the attention operator’s cost scales with total KV cache accessed, not just new tokens). FairBatching introduces a **time-based budget**:

$$\text{init_time_budget} = \max \left(\min_i(\text{slack}_i), \min_i(\text{tpot_slo}_i) \right) \quad (9)$$

Why this formula? The batch execution time should not exceed the smallest deadline slack (to avoid causing any urgent decode task to miss its SLO), but it must be at least as large as the minimum TPOT SLO (to prevent decode tasks with very small slack from forcing infinitely tiny batches).

Execution Time Estimation. Rather than using token count alone, FairBatching models per-batch execution time with a linear regression over historical data:

$$\text{batch_time} = a + b \times \text{total_new_tokens} + c \times \text{total_context} \quad (10)$$

This model accounts for both compute-bound token processing (term $b \times \text{total_new_tokens}$) and memory-bandwidth-bound KV cache access (term $c \times \text{total_context}$). Accuracy improves from $\pm 5.2\%$ (token-count-only) to $\pm 1.3\%$ with this linear model. The parameters a , b , c are profiled offline for each model and GPU type.

4.2.3 Fair and Dynamic Batch Formation (Algorithm 1)

FairBatching categorises all active requests into three priority groups at each scheduling step:

1. **Urgent Decode (group_ud):** Decode tasks with $\text{slack}_i < \text{init_time_budget} + \min_j(\text{tpot_slo}_j)$. Skipping these would likely cause an SLO violation in the next step.
2. **Prefill (group_p):** All pending prefill tasks, sorted by slack in ascending order.
3. **Non-urgent Decode (group_nd):** Remaining decode tasks with ample slack.

The scheduler fills the batch in this order: urgent decode $\rightarrow$ prefill $\rightarrow$ non-urgent decode. Each task is added as long as its estimated time cost fits within the remaining time budget. Chunked prefill allows partial admission.

Three-case handling:

- **Sufficient Capacity:** Include all tasks; no scheduling logic needed.
- **Moderate Capacity:** Defer non-urgent decode tasks first (they are predictable and have slack); accept all prefill.
- **Constrained Capacity:** Urgent decode tasks protected; prefill tasks may be deferred (or migrated to another node in a distributed setting). Falls back gracefully to Sarathi-like behaviour under extreme overload.

4.2.4 Prefill Admission Budget (PAB) for Cluster-Level Integration

A key novel contribution for distributed deployments is the *Prefill Admission Budget (PAB)*: a scalar metric each node exposes to the upper-level load balancer, representing how many additional prefill tokens the node can process within the TTFT SLO window.

$$\text{PAB} = \frac{1}{b+c} \left[\text{TTFT}_{\text{slo}} - \left\lfloor \frac{\text{TTFT}_{\text{slo}} - \min_i \text{slack}_i}{\text{TPOT}_{\text{slo}}} + 1 \right\rfloor \cdot a - \sum_i \frac{\text{TTFT}_{\text{slo}} - \text{slack}_i}{\text{TPOT}_{\text{slo}}} \cdot (b + c \cdot \text{context}_i) \right] \quad (11)$$

This is derived from a worst-case analysis: all decode tasks are assumed to delay as long as possible before being scheduled, leaving the remainder for prefill. The PAB allows the load balancer to route new requests to nodes with sufficient headroom, enabling global SLO compliance without requiring complex cross-node state synchronisation.

Derivation Summary:

1. Total TTFT window: TTFT_{slo} time units
2. Subtract time reserved for batch step overheads ($N_{\text{batches}} \times a$)

3. Subtract compute time consumed by existing decode tasks ($\sum_i N_i \times (b + c \cdot \text{context}_i)$)
4. Subtract existing prefill task sizes ($\sum_{i \in \text{Prefill}} \text{prompt}_i$)
5. Convert remaining time to token capacity using factor $1/(b + c)$

4.3 Experimental Setup

4.3.1 Hardware Configurations

FairBatching Hardware Configurations	
Model	GPU Configuration
LLaMA-3.1-8B	1 × A800-80GB-PCIe
Qwen3-14B	1 × H20-96GB-SXM
Qwen3-32B	2 × H20-96GB-SXM
LLaMA-3.3-70B	4 × H20-96GB-SXM

Note: All hardware uses modern NVIDIA datacenter GPUs (A800, H20), representing production-grade environments—a step beyond BucketServe’s A100 testbed.

4.3.2 Datasets and SLO Targets

Table 3: FairBatching Dataset and SLO Characteristics

Dataset	Prompt Length		Output Length		SLO (ms)	
	avg	p90	avg	p90	TTFT	TPOT
BurstGPT	688	1,599	237	470	500	50
QwenTrace	892	1,776	377	742	500	50
AzureTrace	1,604	3,561	114	392	2,000	50

Dataset Characteristics:

- **BurstGPT:** Real-world workload with bursty arrival patterns from [16]
- **QwenTrace:** Production trace from Alibaba Cloud Qwen-Bailian service, featuring enterprise deployment patterns
- **AzureTrace:** Azure LLM inference trace from 2024 [17], characterised by longer prompts, hence a relaxed TTFT SLO of 2,000ms
- All three datasets were replayed at varying scaling factors ($0.5\times$ to multiple $\times$) to test across load ranges

4.3.3 Baseline Systems

1. **vLLM-vanilla:** vLLM under default configuration (prefill-prioritising). Large max-BS enables rapid prefill completion but causes decode stalls and high TPOT variance.
2. **vLLM-sarathi:** vLLM with Sarathi-style stall-free batching, best-tuned TokenBudget hyperparameter per test case. Represents the state-of-the-art decode-prioritising baseline.

4.3.4 FairBatching Variants Tested

Table 4: FairBatching System Variants and Their Components

Variant	Batch Capacity	Batch Formation	Admission
FB-vanilla	Time-based (adaptive)	Fair 3-phase	None
FB-PAB	Time-based (adaptive)	Fair 3-phase	PAB rejection
FB-FB (ablation)	Fixed (like Sarathi)	Fair 3-phase only	None
FB-TB (ablation)	Token-based (dynamic)	Fair 3-phase	None

4.3.5 Evaluation Metrics

- **Primary:** Effective RPS (requests/s satisfying both TTFT and TPOT SLOs simultaneously)
- **Secondary:** TTFT/TPOT latency distributions (P50, P95, P99), SLO Violation Rate
- **Cluster:** Peak goodput as a function of data-parallel degree (DP=2, 4, 6, 8)

4.4 Key Results

4.4.1 Single-Node Peak Goodput

Table 5: Peak Effective RPS (Geometric Mean across 4 Models)

Dataset	vLLM-vanilla	vLLM-sarathi	FB-vanilla	FB-PAB
BurstGPT	1.55	1.55	1.81 (+16.8%)	2.71 (+74.8%)
QwenTrace	1.22	1.13	1.51 (+23.8%)	2.56 (+109%)
AzureTrace	0.70	0.77	0.84 (+9.1%)	1.36 (+76.6%)
Overall	1.09	1.10	1.32 (+20.0%)	2.11 (+90.1%)

Note: Percentages are vs. the best of the two baselines. FB-PAB’s gain largely comes from admission control preventing system overload—it is a combined system (scheduler + load manager) rather than a pure scheduler improvement.

4.4.2 Latency Detail (Qwen3-14B, QwenTrace, RPS=2.5)

Table 6: Latency Statistics for Key Systems

System	TTFT (ms)			TPOT (ms)		
	P50	P95	P99	P50	P95	P99
vLLM-vanilla	212	1,061	1,842	26	256	614
vLLM-sarathi	377	2,455	4,585	29	46	49
FB-vanilla	213	1,053	1,998	27	50	50
FB-PAB	198	500	504	23	50	50

Key observation: FB-vanilla achieves **2.29× lower P99 TTFT** vs. vLLM-sarathi (1,998ms vs. 4,585ms) while maintaining **equal TPOT guarantees** (P99 TPOT = 50ms for both). This directly demonstrates the fairness improvement.

4.4.3 SLO Versatility

Across varying SLO configurations (TTFT: 500ms–2,000ms; TPOT: 50ms–200ms), FB-vanilla consistently improves goodput by 0.6%–15.4% over the best baseline. At relaxed SLOs (TTFT=2s, TPOT=200ms), the gain diminishes to near zero, as baselines can naturally satisfy these targets. FB-PAB remains effective across all SLO ranges, with improvements of 24.8%–112.6%.

4.4.4 Performance Breakdown (Ablation)

Table 7: Ablation Study: Peak Goodput on QwenTrace with Qwen3-14B

System	Peak Goodput	Incremental Gain
vLLM-sarathi	1.52	baseline
vLLM-vanilla	1.66	+9.2% (prefill-first better here)
FB-FB (fair formation only)	1.91	+15.1% over vLLM-vanilla
FB-TB (add dynamic token budget)	2.06	+7.9%
FB-vanilla (add time-based budget)	2.11	+2.4%
FB-PAB (add admission control)	3.21	+52.1%

4.4.5 Cluster-Level Performance

At DP=8 (8 inference nodes), PAB-LB + FairBatching vs. best baseline:

- QwenTrace: **+54.3%** goodput improvement
- BurstGPT: **+45.3%** goodput improvement
- AzureTrace: **+38.3%** goodput improvement

5 Comparative Analysis

5.1 Side-by-Side System Comparison

Table 8: Comprehensive Comparison of BucketServe and FairBatching

Dimension	BucketServe	FairBatching
Core Problem	Padding waste from heterogeneous sequence lengths	Computational unfairness between prefill and decode tasks
Architecture	Disaggregated (prefill/decode on separate GPUs)	Collocated (prefill and decode on same GPU)
Scheduling Metric	Sequence length (for bucket formation)	Per-token deadlines (envelope-based SLO)
Key Algorithm	Midpoint bisection for adaptive bucket splitting	Three-phase batch packing with time-based budget
Memory Model	Explicit KV cache memory formula with safety margin	Implicit (relies on vLLM’s memory management)
Target Workload	Primarily offline tasks with throughput SLOs; adapted to online	Primarily online latency-sensitive tasks
Base System	vLLM (extended for disaggregated architecture)	vLLM v0.10.1.1 (V1 engine, scheduler plugin)
Models Tested	LLaMA-2-13B, OPT	LLaMA-3.1-8B, Qwen3-14B, Qwen3-32B, LLaMA-3.3-70B
Datasets	Stanford Alpaca, LongBench	BurstGPT, QwenTrace, AzureTrace (all production traces)
Hardware	4× A100 40GB (NVLink)	A800-80GB, 1–4× H20-96GB-SXM
Baselines	UELLM, DistServe	vLLM-vanilla, vLLM-sarathi
Throughput gain	Up to 3.58× vs. UELLM	+20% single-node; +90.1% with PAB
Overhead	< 1% of execution time	Minimal (1,639 lines; 90 invasive changes)
Cluster support	Not evaluated	Yes—PAB enables distributed load balancing
Open source	Not released (as of this writing)	Not released (as of this writing)

5.2 Complementarity of the Two Approaches

A critical insight from reading both papers together is that they are **fundamentally complementary rather than competing**:

- **BucketServe** solves the *input-side heterogeneity problem*: it ensures that requests entering the prefill phase are grouped by length, minimising padding waste and en-

abling predictable memory footprints. It operates at the *request routing and batch formation* level, prior to any inference execution.

- **FairBatching** solves the *resource allocation fairness problem*: given a batch that is about to be executed, it ensures that compute resources are distributed fairly between new prefill tasks and ongoing decode tasks. It operates at the *step-level scheduler* within a single forward pass decision.

A future system could plausibly integrate both: BucketServe’s bucket-based request grouping on the ingress side, combined with FairBatching’s step-level fair scheduler on the execution side.

5.3 Architectural Philosophy Differences

Architecture Comparison

BucketServe (Disaggregated): Separating prefill and decode onto different GPUs simplifies optimisation of each phase independently. However, it requires KV cache transfer (via NVLink here), introduces coordination complexity, and is harder to deploy in single-node settings.

FairBatching (Collocated): Keeping both phases on the same GPU eliminates KV cache transfer overhead and allows flexible resource reallocation between phases based on real-time SLO state. This is simpler to deploy but requires more sophisticated scheduling to balance the competing demands of both phases.

The FairBatching paper cites recent work (TaiChi [20]) that dynamically switches between disaggregated and collocated modes, suggesting the field is converging on a hybrid approach.

6 Literature Survey Context

6.1 Taxonomy of LLM Serving Research

The field of LLM serving optimisation can be broadly taxonomised into four categories. Both surveyed papers occupy distinct positions within this taxonomy:

Table 9: Taxonomy of LLM Serving Research

Category	Representative Works	Papers Here
Memory Management	vLLM/PagedAttention [3], NEO, MemServe	Neither (use vLLM)
Attention Efficiency	FlashAttention [9], FlashAttention-2 [10], GQA	Neither
Phase Disaggregation	DistServe [6], SplitWise [7], Mooncake, LoongServe	BucketServe
Batching/Scheduling	Orca [4], Sarathi [5], FastServe	Both papers

6.2 Positioning Within the Batching/Scheduling Subcategory

6.2.1 Continuous Batching (Orca)

Orca [4] established the baseline: iteration-level scheduling where completed sequences are immediately replaced. Both papers build on top of this for the decode phase.

6.2.2 Chunked Prefill / Stall-Free Batching (Sarathi)

Sarathi [5] introduced the stall-free batching paradigm. FairBatching directly critiques and improves upon this, while BucketServe uses similar chunked-prefill ideas but focuses on a different problem (input heterogeneity).

6.2.3 Preemptive Multi-Level Queuing (FastServe)

FastServe [11] uses a skip-join Multi-Level Feedback Queue to prioritise by input length, targeting job completion time (JCT). It does not consider memory-safety constraints dynamically (unlike BucketServe) and is not SLO-aware (unlike FairBatching).

6.2.4 Dynamic Scheduling (Llumnix)

Llumnix [13] (from Alibaba) focuses on dynamic request migration across nodes in a cluster for load balancing. FairBatching’s PAB mechanism is complementary to Llumnix-style migration.

6.3 Key Related Works for Literature Survey

Table 10: Key Works for LLM Batching/Scheduling Literature Survey

Paper	Year	Key Idea	Relevance
Orca	2022	Continuous/iteration-level batching	Foundational baseline
vLLM	2023	PagedAttention, near-optimal memory	Platform for both papers
Sarathi-Serve	2024	Chunked prefill, stall-free batching	Direct predecessor of FairBatching
DistServe	2024	P/D disaggregation	Direct baseline for BucketServe
SplitWise	2024	Heterogeneous P/D separation	Context for disaggregation
TetriInfer	2024	Chunked prompts, two-level scheduler	Related to BucketServe
Mooncake	2025	KV-cache-centric, prediction-guided	Related disaggregated design
LoongServe	2024	Elastic sequence parallelism	Related for long-context
FastServe	2023	Skip-join MLFQ, preemptive sched.	Related scheduling approach
Llumnix	2024	Dynamic migration across nodes	Cluster-level complement
BucketServe	2025	Bucket-based prefill grouping	This survey
FairBatching	2025	Fair prefill/decode scheduling	This survey

6.4 Benchmark Datasets Used in LLM Serving Research

For researchers conducting a literature survey, the following datasets are commonly used:

Table 11: Common Benchmark Datasets in LLM Serving Research

Dataset	Description	Avg Len	Used By
Stanford Alpaca	Instruction-following, short	84 tokens	BucketServe
LongBench	Long-context, multi-task	55,749 tokens	BucketServe
BurstGPT	Real-world bursty LLM trace	688 tokens	FairBatching
QwenTrace	Alibaba Cloud enterprise trace	892 tokens	FairBatching
AzureTrace 2024	Microsoft Azure LLM trace	1,604 tokens	FairBatching
ShareGPT	Crowd-sourced ChatGPT conversations	~200 tokens	Orca, DistServe
MMLU	Multiple-choice QA	~300 tokens	Many

Recommendation for Literature Surveys: BurstGPT, QwenTrace, and AzureTrace are more representative of modern production deployments than synthetic datasets. Researchers should prefer real traces when evaluating SLO attainment.

7 Integration with Modern LLM Serving Infrastructure

7.1 Integration with vLLM

Both papers are implemented atop vLLM, the de facto standard for open-source LLM serving. Their integration patterns are instructive:

BucketServe Integration:

- Adds a *Request Bucketing Manager* layer above vLLM’s scheduler
- The P/D Scheduler interfaces with vLLM’s model instances and KV cache
- NVLink-based KV cache transfer leverages vLLM’s existing tensor parallelism infrastructure
- Minimal modification to vLLM internals; acts primarily as an orchestration layer

FairBatching Integration:

- Implemented as a *V1-compatible scheduler plugin* (1,639 lines of Python)
- Only 90 invasive lines needed in vLLM core (to propagate per-request SLO metadata)
- Extends vLLM’s DPLB (Data-Parallel Load Balancing) with PAB-based routing (16 lines)
- 2,777-line automated profiling framework to generate linear regression parameters per model/GPU combination

7.2 Integration with Production Serving Stacks

Table 12: Integration Compatibility Matrix

System/Framework	BucketServe	FairBatching
vLLM	Native; orchestration layer	Native; scheduler plugin
SGLang	Conceptually compatible; requires adaptation	Requires SGLang scheduler modification
TensorRT-LLM	Less applicable (different memory model)	Conceptually compatible
Ray Serve / Triton	Can serve as upstream load balancer	Gateway-level PAB integration possible
Kubernetes / KServe	Bucket manager maps to K8s admission control	PAB maps to K8s priority classes
OpenAI-compatible APIs	Transparent; clients unaware	Transparent; SLO metadata can be header-injected

7.3 Handling Python GC (FairBatching)

A practical implementation insight from FairBatching is its handling of Python garbage collection (GC). Python’s stop-the-world GC caused 100ms+ pauses that disrupted latency-sensitive scheduling. The solution is two-fold:

- `gc.freeze()` to move long-lived objects to a permanent generation (following existing vLLM practice)
- Proactive GC triggering during low-load periods (no queued prefill tasks, ample decode slack)

This is an important engineering lesson for production deployments of Python-based inference servers.

7.4 Applicability to Newer LLM Architectures

- **Mixture-of-Experts (MoE) models** (e.g., DeepSeek-R1, Mixtral): BucketServe’s bucket-based memory management would need to account for expert routing patterns. FairBatching’s time-estimation model would need to incorporate expert utilisation.
- **Multi-modal models** (vision-language, audio-language): Input sequence lengths become more heterogeneous. BucketServe’s bucket formation becomes even more valuable.
- **Speculative Decoding**: FairBatching notes that speculative decoding can only potentially improve decode concurrency to a limited extent and does not alter the fundamental sequential nature of decode tasks—so the fairness problem persists.
- **Very Long Context** (> 128K tokens): BucketServe’s KV cache memory formula provides explicit safety margins, making it better suited for VLLC workloads than simple continuous batching.

8 Critical Analysis and Limitations

8.1 BucketServe: Strengths and Weaknesses

Strengths:

- Theoretically grounded in optimal bucketing theory (conditional expectation of sequence lengths)
- Explicit memory safety guarantee prevents OOM errors—a critical practical concern
- Negligible overhead ($< 1\%$) makes it deployable in latency-sensitive settings
- Handles both offline and online workloads with different intra-bucket policies
- Three-tier architecture cleanly separates concerns between routing, batching, and execution

Weaknesses and Limitations:

- *Single-node evaluation only*: No cluster-level evaluation; scalability to multi-node disaggregated deployments is unclear
- *Older model family*: LLaMA-2-13B and OPT are not representative of state-of-the-art models (100B+ parameter models, MoE architectures)
- *Synthetic workloads*: Alpaca and LongBench are academic datasets, not production traces with realistic arrival patterns (bursty arrivals, diurnal patterns)
- *No code release*: Reproducibility is limited without the implementation
- *Midpoint bisection approximation*: The adaptive bucketing uses a simple bisection heuristic rather than distribution-aware splitting; the paper acknowledges this is a simplification
- *FCFS within prefill*: Using First-Come-First-Served for the prefill queue may be sub-optimal for SLO compliance under heterogeneous deadlines

8.2 FairBatching: Strengths and Weaknesses

Strengths:

- Deep theoretical analysis of TBT non-monotonicity—a genuinely novel insight
- Production traces (BurstGPT, QwenTrace, AzureTrace) provide strong ecological validity

- Tested across 4 models and 3 datasets—comprehensive coverage
- PAB enables principled cluster-level load balancing with minimal state synchronisation
- Excellent ablation study isolating contributions of each design component
- Implementation is lightweight (90 invasive lines) and modular (pluggable scheduler)

Weaknesses and Limitations:

- *Collocated architecture only*: Does not address the disaggregated case; cannot leverage phase-specific GPU specialisation
- *Linear time estimation model*: The $a + b \cdot \text{new_tokens} + c \cdot \text{context}$ model may not capture non-linearities at very high batch sizes or with Flash Attention 3 / custom CUDA kernels
- *Requires offline profiling*: Per-model, per-GPU parameter estimation requires infrastructure; harder to deploy on heterogeneous or spot-instance clusters
- *FB-vanilla vs. FB-PAB gap*: The 52.1% improvement attributable to PAB (admission control) means much of the benefit comes from simply rejecting requests—not from scheduling intelligence per se
- *No code release*: Same limitation as BucketServe
- *SLO stationarity assumption*: The PAB formula assumes SLO targets are static per request; dynamic SLO adjustment is not addressed

8.3 Common Limitations Across Both Papers

- Neither paper addresses *multi-tenant fairness* across different users or organisations (a different fairness dimension than the within-batch fairness of FairBatching)
- Neither paper addresses *energy efficiency* or *cost optimisation*, which are increasingly important in production deployments
- Neither paper considers *speculative decoding* as a component of the batching strategy
- Both papers assume homogeneous GPU hardware; heterogeneous clusters (mixing A100 and H100) are not addressed

9 Open Research Directions

Based on the analysis of both papers, the following directions represent promising future work:

9.1 Combining Bucket-Based Routing with Fair Scheduling

The most immediate extension is a unified system that applies BucketServe-style input bucketing at the request admission layer, followed by FairBatching-style fair step-level scheduling during execution. Such a system would address both padding waste and scheduling fairness simultaneously.

9.2 Distribution-Aware Bucketing

BucketServe’s midpoint bisection is a simplification. Online density estimation (e.g., KDE, histogram-based methods) of the request length distribution could enable more principled bucket boundary selection, approaching the theoretical optimum in Eq. (4).

9.3 Model-Aware Time Estimation

FairBatching’s linear time model ($a + b \cdot \text{tokens} + c \cdot \text{context}$) was fit on standard decoder-only transformers. Extending to MoE models, multi-modal models, and models with different parallelism strategies would require richer feature sets and possibly non-linear models.

9.4 Online SLO Negotiation

Current systems treat SLO targets as fixed. In practice, clients may be willing to relax SLO targets under high load in exchange for reduced costs. Adaptive SLO negotiation between the serving system and clients could enable more efficient resource utilisation.

9.5 Hardware-Software Co-design

BucketServe’s KV cache memory formula assumes FP16 precision. With FP8 and INT4 quantisation becoming common, the memory model needs to account for mixed-precision inference. Additionally, new interconnects (NVLink 5, CXL) could change the cost model for disaggregated serving.

9.6 Multi-Level Load Balancing

BucketServe explicitly mentions multi-level load balancing as future work. Combining cluster-level request routing (similar to Llumnix), node-level fair scheduling (FairBatch-

ing), and intra-batch bucketing (BucketServe) into a coherent three-level hierarchy is an open research problem.

10 Summary and Recommendations

10.1 Quick Reference Card

BucketServe – At a Glance

- **Problem:** Padding waste from heterogeneous input lengths in disaggregated LLM serving
- **Solution:** Group requests into size-homogeneous buckets; split/merge adaptively; size batches to available KV cache memory
- **Result:** $3.58\times$ throughput over UELLM, $1.93\times$ more RPS at 80% SLO vs. DistServe, $< 1\%$ overhead
- **Best for:** Offline/batch workloads with heterogeneous request lengths; disaggregated serving architectures
- **Use when:** Input sequences vary widely in length (e.g., RAG systems, long-document analysis)

FairBatching – At a Glance

- **Problem:** Decode-prioritising stall-free batching (Sarathi) unfairly starves pre-fill tasks despite ample decode slack
- **Solution:** Monotonic per-token SLO deadlines; time-based batch sizing; three-phase fair batch packing; PAB for cluster load balancing
- **Result:** $2.29\times$ lower P99 TTFT, $+20\%$ single-node goodput, $+54.3\%$ cluster goodput (with PAB)
- **Best for:** Latency-sensitive online serving; mixed TTFT/TPOT SLO requirements; distributed multi-node deployments
- **Use when:** Serving chatbots or interactive AI applications where both response latency and generation speed matter

10.2 Recommendations for Practitioners

- For **offline batch processing** (document analysis, data pipelines): BucketServe’s LJB scheduling within long-sequence buckets maximises throughput.
- For **online chatbot/API serving**: FairBatching-vanilla is a drop-in improvement over Sarathi with minimal changes to vLLM.
- For **large-scale multi-node deployments**: FairBatching-PAB + PAB-aware load balancer is the most complete solution currently available.
- For **memory-constrained environments** (e.g., smaller GPUs serving large models): BucketServe’s explicit KV cache memory management is essential for preventing

OOM errors.

- For researchers: Production traces (BurstGPT, QwenTrace, AzureTrace) should be preferred over academic datasets for scheduling research.

References

- [1] W. Zheng, M. Xu, S. Song, K. Ye, “BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving,” *arXiv:2507.17120v1 [cs.DC]*, July 2025.
- [2] H. Lyu, B. Liu, M. Wu, H. Chen, “FairBatching: Fairness-Aware Batch Formation for LLM Inference,” *arXiv:2510.14392v1 [cs.DC]*, October 2025.
- [3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C.H. Yu, J.E. Gonzalez, H. Zhang, I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *Proc. ACM SIGOPS 29th SOSP*, 2023.
- [4] G.-I. Yu, J.S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, “Orca: A Distributed Serving System for Transformer-Based Generative Models,” *16th USENIX Symposium on OSDI*, pp. 521–538, July 2022.
- [5] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B.S. Gulavani, A. Tumanov, R. Ramjee, “Taming Throughput-Latency Tradeoff in LLM Inference with SarathiServe,” *18th USENIX Symposium on OSDI*, 2024.
- [6] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, “DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving,” *18th USENIX Symposium on OSDI*, 2024.
- [7] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, R. Bianchini, “Splitwise: Efficient Generative LLM Inference Using Phase Splitting,” *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA 2024)*, pp. 118–132, 2024.
- [8] C. Hu et al., “Inference Without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads,” *arXiv:2401.11181*, 2024.
- [9] T. Dao, D.Y. Fu, S. Ermon, A. Rudra, C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [10] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” *International Conference on Learning Representations (ICLR)*, 2024.
- [11] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, X. Jin, “Fast Distributed Inference Serving for Large Language Models,” *arXiv:2305.05920*, 2023.
- [12] Y. He, M. Xu, J. Wu, W. Zheng, K. Ye, C. Xu, “UELLM: A Unified and Efficient Approach for Large Language Model Inference Serving,” *22nd International Conference ICSOC 2024*, pp. 218–235, 2024.

- [13] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, W. Lin, “Llumnix: Dynamic Scheduling for Large Language Model Serving,” *18th USENIX Symposium on OSDI*, 2024.
- [14] R. Taori et al., “Stanford Alpaca: An Instruction-Following LLaMA Model,” github.com/tatsu-lab/stanford_alpaca, 2023.
- [15] Y. Bai et al., “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding,” *Proc. ACL 2024*, pp. 3119–3137, 2024.
- [16] Y. Wang et al., “BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems,” *Proc. 31st ACM SIGKDD (KDD 2025)*, pp. 5831–5841, 2025.
- [17] J. Stojkovic et al., “DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency,” *IEEE HPCA 2025*, pp. 1348–1362, 2025.
- [18] R. Qin et al., “Mooncake: Trading More Storage for Less Computation – A KVCache-Centric Architecture for Serving LLM Chatbot,” *23rd USENIX FAST 2025*, pp. 155–170, 2025.
- [19] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, X. Jin, “LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism,” *Proc. ACM SIGOPS 30th SOSP 2024*, pp. 640–654, 2024.
- [20] C. Wang, P. Zuo, Z. Chen, Y. Liang, Z. Yu, M.-C. Yang, “Prefill-Decode Aggregation or Disaggregation? Unifying Both for Goodput-Optimized LLM Serving,” *arXiv:2508.01989*, 2025.

Literature Survey Report

GPU Systems Optimization: Compiler Scheduling & LLM Inference Serving

Paper 1

*Optimal Software Pipelining and Warp Specialization
for Tensor Core GPUs*

Rupanshu Soi, Rohan Yadav, Fredrik Kjolstad, Alex Aiken (Stanford),
Maryam Mehri Dehnavi, Michael Garland, Michael Bauer (NVIDIA)

arXiv:2512.18134 *December 2025*

Paper 2

*Taming Throughput-Latency Tradeoff in LLM Inference
with Sarathi-Serve*

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan,
Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, Ramachandran Ramjee

Microsoft Research India & Georgia Tech

arXiv:2403.02310 *OSDI 2024*

Contents

1	Executive Summary	3
2	Paper 1 — Twill: Optimal SWP & Warp Specialization for TC-GPUs	4
2.1	Problem Statement and Motivation	4
2.2	Core Novelty and Technical Contributions	4
2.2.1	Mapping SWP to Modulo Scheduling	4
2.2.2	Joint SWP + WS Constraint Formulation	5
2.2.3	Cost Normalization	5
2.2.4	Variable Latency Optimization	5
2.3	System Architecture: Twill	6
2.4	Experimental Evaluation	6
2.4.1	Evaluation Platforms	6
2.4.2	Kernels Evaluated	6
2.4.3	Baselines	6
2.4.4	Results Summary	6
2.4.5	Solve Times	7
2.5	GitHub Repository Analysis	7
2.6	Limitations	8
2.7	Integration with Modern Systems	8
3	Paper 2 — Sarathi-Serve: Taming Throughput-Latency Tradeoff	9
3.1	Problem Statement and Motivation	9
3.2	Core Novelty	9
3.3	Technical Design	9
3.3.1	Chunked-Prefills	9
3.3.2	Stall-Free Batching (Algorithm 3)	10
3.3.3	Tile-Quantization Awareness	10
3.3.4	Pipeline Parallelism Benefits	10
3.4	Experimental Evaluation	10
3.4.1	Models and Hardware	10
3.4.2	Datasets and Workloads	10
3.4.3	SLO Definitions	11
3.4.4	Key Results	11
3.5	GitHub Repository	11
3.6	Limitations	12
3.7	Integration with Modern Systems	12
4	Comparative Analysis	13
4.1	Position in the GPU Software Stack	13
4.2	Complementarity	13
4.3	Reproducibility Comparison	13
5	Literature Survey Context	14
5.1	Related Work: Software Pipelining	14
5.2	Related Work: Warp Specialization	14
5.3	Related Work: LLM Serving	15
5.4	Related Work: Attention Kernels (Common Context)	15
6	Experimental Setup Details for Reproducibility	16
6.1	Twill: Exact Setup	16

6.2	Sarathi-Serve: Exact Setup	16
7	Open Questions and Future Directions	17
7.1	Open Questions from Twill	17
7.2	Open Questions from Sarathi-Serve	17
8	Summary Tables for Quick Reference	18
8.1	Twill at a Glance	18
8.2	Sarathi-Serve at a Glance	18

1 Executive Summary

This report presents a detailed analysis of two significant systems papers targeting GPU performance optimization from complementary vantage points. The first, **Twill**, operates at the compiler level, tackling how to automatically generate optimal instruction schedules for NVIDIA Tensor Core GPUs—a problem previously solved only by hand by GPU experts. The second, **Sarathi-Serve**, operates at the inference-serving layer, addressing the fundamental throughput-latency tradeoff that plagues large language model (LLM) serving systems.

Together these papers span the full GPU software stack: from low-level instruction scheduling on individual streaming multiprocessors to high-level request batching policies across multi-node GPU clusters. Both papers make strong claims validated against state-of-the-art hand-tuned baselines.

Key Takeaways for Literature Survey

- **Twill**: First heuristic-free, provably optimal joint SWP+WS scheduler for TC-GPUs. Rediscovered Flash Attention 3 & 4 schedules from first principles in <5 minutes.
- **Sarathi-Serve**: Chunked-prefill + stall-free batching eliminates the generation stall problem. Up to 5.6× capacity improvement over vLLM on Falcon-180B.
- Both are highly relevant to the design of future GPU programming compilers and LLM inference engines.

2 Paper 1 — Twill: Optimal SWP & Warp Specialization for TC-GPUs

2.1 Problem Statement and Motivation

Modern NVIDIA GPUs (Hopper, Blackwell) integrate increasingly powerful *Tensor Cores* (TCs) for matrix multiplication (GEMM) alongside the Tensor Memory Accelerator (TMA) for bulk asynchronous data movement. To fully utilize these fixed-function units, software must employ two complementary techniques:

- **Software Pipelining (SWP)**: Overlaps instructions from different loop iterations to expose instruction-level parallelism (ILP), keeping all functional units busy.
- **Warp Specialization (WS)**: Assigns different subsets of GPU threads (warps) to different roles, addressing the code-generation challenges that prevent a single thread from expressing complex pipelined schedules.

The central problem is that determining the best *combination* of SWP and WS for a given program on a given GPU architecture has historically required either manual expert effort (months of engineering) or brittle, architecture-specific compiler heuristics—with no formal notion of optimality.

Motivating example: Flash Attention 3 (FA3) for NVIDIA Hopper was released nearly *one year* after the Hopper architecture itself, wasting vast compute resources on suboptimal attention kernels during that interval. FA4 for Blackwell followed the same pattern.

2.2 Core Novelty and Technical Contributions

Core Novelty of Twill

Twill is the **first system to formulate SWP and WS as a unified constraint satisfaction problem** solvable by off-the-shelf SMT solvers, yielding *provably optimal* schedules without architecture-specific heuristics.

The four principal technical contributions are:

2.2.1 Mapping SWP to Modulo Scheduling

Twill models the input loop as a **dependence graph** $G = (V, E)$ where:

- Each node $v \in V$ is a tile-level instruction with a *Resource Reservation Table* (RRT) specifying functional-unit usage per clock cycle.
- Each edge $(u, v, d, \delta) \in E$ encodes a data dependence with a clock-cycle delay d and iteration delay δ .

Standard **modulo scheduling** finds:

- An **initiation interval** I (cycles between consecutive loop iterations in steady state; lower = higher throughput).
- A **modulo schedule** $M : V \rightarrow \mathbb{Z}_{\geq 0}$ mapping each instruction to a cycle offset.

Twill solves the initial modulo scheduling problem optimally using Integer Linear Programming (ZLP/ILP), dispatched to the **CBC solver**.

2.2.2 Joint SWP + WS Constraint Formulation

After computing M and I , Twill *unrolls* the schedule into a straight-line program Q and formulates three families of **SMT constraints** (dispatched to the **Yices2 QFLIA solver**):

Modulo Scheduling Constraints (Figure 4 in paper):

- **UNIQUENESS:** Every instruction scheduled exactly once.
- **CONSISTENCY:** Schedule is derivable from some valid modulo schedule.
- **COMPLETION:** All instructions finish before cycle T .
- **DEPENDENCE:** All data dependences satisfied.
- **CAPACITY:** No functional unit overloaded.

Memory-Aware Constraints (Figure 5): Uses a liveness array `live[v,i,t]` derived from classic backward dataflow analysis to enforce *on-chip memory capacity*, preventing register spilling.

Warp Assignment Constraints (Figure 6): Uses a boolean array `opw[v,w]` (instruction v assigned to warp w) with:

- **VARIABLE LATENCY:** TMA loads (high dynamic range latency) isolated to a dedicated warp W_{vl} .
- **REGISTER LIMIT:** Per-warp register pressure enforced.
- **CROSS-WARP SPILLS:** Adds communication latency penalty when data crosses warp boundaries through shared memory.
- **CONCURRENCY:** Blocking synchronizations (e.g., **GEMM.WAIT**) cannot interrupt concurrent instruction issue on the same warp.

2.2.3 Cost Normalization

A critical engineering insight: directly using hardware cycle counts (e.g., ~ 1000 cycles for a $128 \times 128 \times 128$ GEMM on Hopper) makes the ILP/SMT exponentially harder due to large edge delays d . Twill normalizes costs by solving a *separate* ZLP problem:

$$\min F \quad \text{s.t.} \quad \forall i, j : -F \leq C[i] \cdot C'[j] - C[j] \cdot C'[i] \leq F, \quad 1 \leq \sum_i C'[i] \leq U$$

This preserves cost *ratios* while keeping absolute values small ($U = 300$ in practice). The SCIP solver finds solutions in under 500 ms.

2.2.4 Variable Latency Optimization

Operations without incoming dependencies (e.g., TMA loads of input tiles) are treated as **streaming operations** with *zero latency* in the cost model. Their pipeline depths are exposed as auto-tunable parameters, decoupling static scheduling from unpredictable memory latency.

2.3 System Architecture: Twill

Component	Description
Input	Triton programs (mid-level TTGIR — tile-based SSA IR) + target GPU architecture spec
Step 1	Extract dependence graph $G = (V, E)$ from TTGIR
Step 2	Cost normalization via SCIP ZLP solver
Step 3	Optimal modulo scheduling via CBC ZLP solver $\Rightarrow (M, I)$
Step 4	Unroll to straight-line program Q
Step 5	SMT constraint solving via Yices2 (QFLIA theory) $\Rightarrow (M^*, A^*)$
Step 6	Emit warp-annotated pipelined IR (consumed by Tawa, Cypress, or hand-compiled)
Search	Algorithm 1: iterate over increasing I , then increasing L at fixed I

2.4 Experimental Evaluation

2.4.1 Evaluation Platforms

GPU	Model	Memory
NVIDIA Hopper	H100 SXM5	80 GB
NVIDIA Blackwell	B200	180 GB
CPU (SMT solving)	Intel Xeon Platinum 8570	Single core

Software: CUDA 13.0; Triton (tutorial implementation as baseline).

2.4.2 Kernels Evaluated

- **Fused Multi-Head Attention (FMHA) Forward Pass** — evaluated on both Hopper and Blackwell.
- **FMHA Backward Pass** — single-pass algorithm (FA3 style); evaluated on both Hopper and Blackwell.

All kernels use FP16, non-causal attention, batch=4, num_heads=32, head_dim=128, sequence lengths {2048, 4096, 8192, 16384}.

2.4.3 Baselines

Baseline	Description
Triton	Tutorial FA implementation; uses Triton’s own compiler decisions
Triton-Tiled	Sub-tiled version for TC instruction granularity
CUDA-Default	Direct hand-compilation of Triton IR to CUDA C++ (no SWP/WS)
Twill-SWP	Twill modulo scheduling only + Triton’s WS heuristic
Twill	Full joint SWP+WS from Twill
cuDNN	NVIDIA’s proprietary, hand-tuned attention library
FA3 / FA4	Official Flash Attention 3/4 implementations

2.4.4 Results Summary

Hopper Forward Attention:

- **Twill-SWP** comes within **1%** of FA3 at sequence length 16384.
- **Twill** (full joint) derives the *same* ping-pong scheduling strategy as FA3 automatically (28 s solve time). Initially matches Twill-SWP at ~ 645 TFLOPS/s; slightly lower final performance due to orthogonal TMA multicasting optimization benefiting the modulo-only path.
- On Hopper, dynamic warp interleaving by the SM naturally provides ILP, lessening the benefit of static WS.

Blackwell Forward Attention:

- **Twill** (19 s solve time) discovers the *exact* 4-warp-group strategy proposed in FA4: variable-latency ops (green), TC GEMMs (pink), two softmax groups (blue/orange), accumulator rescaling (yellow).
- Result: within **2%** of FA4 at sequence length 16384.
- Applying Triton’s heuristic WS to Twill-SWP yields *no benefit*, confirming the necessity of *joint* optimization.

Hopper Backward Attention:

- Register-constrained; FA3 also avoids cross-iteration pipelining for the same reason.
- Twill searches for 88 s and confirms FA3 developers did not miss any opportunity.
- Gap vs. FA3 (11%) attributable to tile size: FA3 uses 80×128 tiles (unsupported by Triton).

Blackwell Backward Attention:

- Twill finds a 2-warp-group ping-pong strategy in 269 s; ptxas register allocation fails, causing spilling.
- Twill-LR (reduced register budget): finds the FA4-equivalent 3-warp-group strategy in 64 s; ptxas succeeds.
- Kernel is bandwidth-bound (atomic reduction); performance gap from FA4 attributed to layout/instruction selection.

2.4.5 Solve Times

Kernel	Architecture	Solve Time
FMHA Forward	Hopper	28 s
FMHA Forward	Blackwell	19 s
FMHA Backward	Hopper	88 s
FMHA Backward	Blackwell (Twill)	269 s
FMHA Backward	Blackwell (Twill-LR)	64 s

2.5 GitHub Repository Analysis

The paper does not link a public GitHub repository; code is described as part of an NVIDIA/Stanford research collaboration. Key implementation notes from the paper:

- **Frontend:** Triton (PyTorch-based Python DSL); sub-tiling at TTGIR level.

- **Solvers used:** CBC (modulo scheduling ZLP), SCIP (cost normalization ZLP), Yices2 (SMT/QFLIA).
- **Downstream:** Tawa [7] and Cypress [8] can consume Twill’s annotated IR.
- **Backend:** Schedules hand-compiled to CUDA C++ for evaluation due to Triton codegen limitations.

2.6 Limitations

- Only supports **singly-nested loops without additional control flow** (no nested loops, no conditionals in the loop body).
- **Tile sizes** must be specified externally (by a human or auto-tuner like Vidur).
- **Solve times** of minutes preclude interactive use; intended as offline/pre-deployment tool.
- **Code generation gap:** Triton’s codegen makes suboptimal decisions (memory allocation, layout conversions, synchronization); Twill schedules must currently be hand-compiled.

2.7 Integration with Modern Systems

System	Relationship to Twill
Triton	Primary input language; Twill extracts TTGIR from Triton programs
Tawa	Can consume Twill’s warp-annotated IR to generate WS code automatically
Cypress	Task-based WS programming model; can use Twill’s strategy as a mapping
TLX / TileLang	Tile-based DSLs; could expose dependence graphs to Twill
CUTLASS	Provides reference hand-tuned kernels (implicit baseline via cuDNN)
Flash Attention 3/4	Twill independently <i>rediscovers</i> their schedules, proving optimality

3 Paper 2 — Sarathi-Serve: Taming Throughput-Latency Tradeoff

3.1 Problem Statement and Motivation

LLM inference consists of two computationally distinct phases:

- **Prefill:** Processes all input prompt tokens in parallel. *Compute-bound*: saturates GPU arithmetic units even with a single request; throughput barely increases with batching.
- **Decode:** Generates one token per request per step. *Memory-bound*: low arithmetic intensity; throughput scales near-linearly with batch size.

The fundamental tension: To maximize throughput, systems must batch many decode requests. But admitting new requests (which start with a prefill) interrupts ongoing decodes, causing what the authors term **generation stalls** — pauses of *multiple seconds* in ongoing token generation.

Existing approaches and their pitfalls:

System	Strategy	Throughput	TBT Latency
FasterTransformer	Decode-prioritizing, request-level batching	Low (idle GPU while waiting)	Low (no interference)
Orca	Prefill-prioritizing, iteration-level batching	Higher	High (generation stalls)
vLLM	Prefill-prioritizing + PagedAttention	High	High (stalls persist)
Sarathi-Serve	Stall-free, chunked-prefills	High	Low

3.2 Core Novelty

Core Novelty of Sarathi-Serve

Chunked-prefills + stall-free batching: split large prefill requests into small, compute-sized chunks that can be coalesced with ongoing decode batches without violating TBT SLOs. This *decouples* the admission of new requests from the disruption of ongoing decodes.

3.3 Technical Design

3.3.1 Chunked-Prefills

The key insight is that GPU prefill throughput saturates at a modest sequence length (~ 512 tokens for Mistral-7B on A100). Therefore:

- A large prompt (e.g., 7059 median tokens in the arxiv dataset) can be split into chunks of ~ 512 tokens, each of which still saturates GPU compute.
- Chunked prefill attention requires re-reading KV-cache from prior chunks, introducing a modest overhead. At chunk size 512, overhead is at most $\sim 25\%$; at 2048, nearly zero.
- Each chunk fits into a decode-sized budget, keeping per-iteration latency bounded and predictable.

3.3.2 Stall-Free Batching (Algorithm 3)

The Sarathi-Serve scheduler operates iteration-level with the following priority order in each scheduling step:

1. Pack **all running decode tokens** into the batch (highest priority).
2. Add **any in-progress partial prefill** within remaining token budget.
3. **Admit new requests** (chunked) within remaining budget.

The token budget τ is computed to satisfy the TBT SLO:

- **Smaller τ** : lower per-iteration latency, better TBT but higher prefill overhead and lower throughput.
- **Larger τ** : higher throughput but more prefill-decode interference.

A one-time profiling step (using Vidur [19]) determines the optimal τ for a given deployment.

3.3.3 Tile-Quantization Awareness

The paper explicitly warns that τ must account for tile-quantization effects: chunk size 257 can increase prefill time by 32% over chunk size 256, because GPU matmul tiles prefer power-of-two dimensions.

3.3.4 Pipeline Parallelism Benefits

In pipeline-parallel (PP) deployments, micro-batch compute time must be uniform to avoid pipeline bubbles. Sarathi-Serve’s constant-token-budget batches achieve near-uniform compute across micro-batches, dramatically reducing three types of bubbles: (1) prefill-length variation, (2) prefill-decode interference, (3) KV-cache-size-dependent decode variation.

3.4 Experimental Evaluation

3.4.1 Models and Hardware

Model	Params	GPU Config	GPU Memory
Mistral-7B	7B	1× A100	80 GB
Yi-34B	34B	2× A100 (TP2)	160 GB
LLaMA2-70B	70B	8× A40 (TP4-PP2)	384 GB
Falcon-180B	180B	8× A100, 2 nodes (TP4-PP2)	640 GB

Interconnect: Pairwise NVLink within node; 100 Gbps Ethernet across nodes.

Software: vLLM codebase (extended); FlashAttention v2 and FlashInfer kernels; NCCL for communication.

CUDA: 12.1.

3.4.2 Datasets and Workloads

Dataset	Median Prompt Tokens	P90 Prompt	Median Output	Std Output
openchat_sharegpt4	1730	5696	415	101
arxiv_summarization	7059	12985	208	265

Request inter-arrivals: Poisson distribution. Outlier filtering: max 8192 tokens (sharegpt), 16384 tokens (arxiv).

3.4.3 SLO Definitions

Model	Strict SLO (P99 TBT)	Relaxed SLO (P99 TBT)
Mistral-7B	0.1 s	0.5 s
Yi-34B	0.2 s	1.0 s
LLaMA2-70B	1.0 s	5.0 s
Falcon-180B	1.0 s	5.0 s

SLO = $5\times$ (strict) / $25\times$ (relaxed) baseline decode iteration latency.

3.4.4 Key Results

Capacity improvements vs. vLLM:

Model	Dataset	Strict SLO	Relaxed SLO
Mistral-7B	openchat_sharegpt4	$2.15\times$	$2.15\times$
Yi-34B	openchat_sharegpt4	$3.7\times$	$2.44\times$
LLaMA2-70B (PP)	openchat_sharegpt4	$4.3\times$	$> 4\times$
Falcon-180B (PP)	openchat_sharegpt4	$5.6\times$	$> 3.6\times$
Mistral-7B	arxiv_summarization	$1.82\times$	$1.97\times$
Yi-34B	arxiv_summarization	$1.69\times$	$1.94\times$

Pipeline parallelism:

- Tensor Parallelism (TP-8) across nodes: median TBT $> 2\times$ higher than TP4-PP2 due to expensive cross-node all-reduces.
- Sarathi-Serve with PP: $4.3\times$ capacity gain vs. vLLM TP-only and $3.6\times$ vs. vLLM PP under strict SLO (Falcon-180B).

Ablation (Yi-34B, Table 4 in paper):

Variant	openchat_sharegpt4		arxiv_summ.	
	P50 TTFT	P99 TBT	P50 TTFT	P99 TBT
Hybrid-batching only	0.53 s	0.68 s	3.78 s	1.38 s
Chunked-prefills only	1.04 s	0.17 s	5.38 s	0.20 s
Sarathi-Serve (both)	0.76 s	0.14 s	3.90 s	0.17 s

Both techniques are necessary: chunked-prefills alone increases TTFT; hybrid-batching alone fails to eliminate stalls.

3.5 GitHub Repository

Repository

<https://github.com/microsoft/sarathi-serve>
Branch for OSDI paper: osdi-sarathi-serve

Structure:

- /sarathi/ — Core scheduler implementation (fork of vLLM).
- /csrc/ — Custom CUDA kernels (chunked prefill attention).

- `/osdi-experiments/` — Scripts to reproduce all paper figures.
- `/data/` — Trace files (sharegpt, arxiv).

Key implementation details:

- Built on **vLLM** open-source codebase (PagedAttention for KV-cache management).
- Uses **FlashAttention v2** and **FlashInfer** backends for attention computation.
- **NCCL** for both TP and PP communication.
- Supports chunked prefill via paged KV-cache indexing.
- Scheduling logic: Python; CUDA kernels: C++/CUDA.

Tested environments: CUDA 12.1, A100 (80 GB), A40 (48 GB).

3.6 Limitations

- **TTFT vs. TBT trade-off:** Chunked prefills increase time-to-first-token (TTFT) since a prompt is processed over multiple iterations.
- **KV-cache overhead:** At small chunk sizes, earlier chunks’ KV-caches are read multiple times during attention of later chunks.
- **Token budget selection complexity:** Optimal τ depends on SLO, hardware, parallelism strategy, and tile-quantization effects.
- **No disaggregation:** Unlike Splitwise/DistServe, prefill and decode phases run on the same GPU(s); prefill replicas’ KV-cache memory is fully utilized.
- **Not compared to disaggregated systems:** Authors acknowledge this as future work.
- **Research prototype:** Not full feature-parity with production vLLM.

3.7 Integration with Modern Systems

System	Relationship to Sarathi-Serve
vLLM	Sarathi-Serve is a fork of vLLM; chunked-prefill ideas have been <i>merged into vLLM</i> as <code>chunked_prefill</code> mode
TensorRT-LLM	Sarathi’s ideas influenced chunked prefill support in NVIDIA TRT-LLM
Orca	Direct comparison baseline; Sarathi supersedes its scheduling policy
PagedAttention	Used as-is for KV-cache memory management
FlashAttention 2	Attention kernel backend
FlashInfer	Alternative attention kernel (also supported)
Splitwise / Dist-Serve	Complementary disaggregated approach; Sarathi does not require separate prefill/decode replicas
Vidur	Used by Sarathi-Serve to simulate and select optimal token budget τ
APIServe	Adopted chunked-prefills from Sarathi for ahead-of-time prefill recomputation

4 Comparative Analysis

4.1 Position in the GPU Software Stack

Dimension	Twill	Sarathi-Serve
Stack Layer	Compiler / Instruction scheduler	Inference serving / Request scheduler
Granularity	Single SM; tile-level instructions	Multi-node cluster; token-level batches
Target	Kernel developer / Compiler writer	Inference system operator
Optimization objective	Minimize initiation interval I (maximize throughput per SM)	Maximize serving capacity under TBT SLO
Approach	Constraint solving (ILP + SMT)	Scheduling algorithm (greedy + chunking)
Optimality	Provably optimal within the model	Empirically validated; no formal proof
Solution time	19–269 seconds (offline)	$< 1 \mu\text{s}$ per scheduling round (online)
GPU architectures	Hopper, Blackwell	A100, A40

4.2 Complementarity

Both papers address *utilization* of GPU resources but at different levels. Twill maximizes *intra-SM utilization* by ensuring Tensor Cores and SFUs are always busy. Sarathi-Serve maximizes *cross-request GPU utilization* by ensuring GPUs never stall on prefill-decode interference. A production system would benefit from *both*: Twill-generated kernels running inside a Sarathi-Serve managed serving cluster.

4.3 Reproducibility Comparison

Aspect	Twil	Sarathi-Serve
Code available	Not publicly released	Open-source on GitHub
Reproduce main results	Difficult without NVIDIA access	<code>osdi-experiments/</code> scripts
Hardware requirements	H100 or B200 (expensive)	A100/A40 (more accessible)
Setup complexity	High (Triton + solver deps)	Moderate (vLLM fork)

5 Literature Survey Context

5.1 Related Work: Software Pipelining

Paper	Venue	Relation to Twill
Aiken & Nicolau (1988)	PLDI	Original compaction-based SWP for CPUs
Lam (1988)	PLDI	Modulo scheduling; Twill directly extends this
Rau (1994)	MICRO	Iterative modulo scheduling algorithm
Govindarajan et al. (1994)	MICRO	Optimal (ILP-based) register-constrained modulo scheduling
Stoutchinin (1997)	PaCT	ILP model for SWP; Twill uses this as basis
ALCOP (2023)	MLSys	SWP of loads with compute only; insufficient for compute-compute overlap
PipeThreader (2025)	OSDI	Profile-based SWP search; no optimality guarantee; conventional WS roles
Twill (2025)	arXiv	Joint SWP+WS, provably optimal, heuristic-free

5.2 Related Work: Warp Specialization

Paper/System	Venue	Relation to Twill
CUDA-DMA (2011)	SC	First WS paper; separates data movement from compute
Singe (2014)	PPoPP	WS to fit large chemistry computations in registers
Flash Attention 3 (2024)	arXiv	Hand-tuned WS for Hopper FMHA; Twill re-discovers this
Flash Attention 4 (2025)	GitHub	Hand-tuned WS for Blackwell FMHA; Twill re-discovers this
Tawa (2025)	arXiv	Automatic WS transformation using heuristics; can use Twill's output
Cypress (2025)	PLDI	Task-based WS; heuristic strategy definition
Twill (2025)	arXiv	First automatic, optimal WS strategy derivation

5.3 Related Work: LLM Serving

Paper/System	Venue	Relation to Sarathi-Serve
FasterTransformer Orca (2022)	NVIDIA OSDI	Decode-prioritizing baseline; low throughput Introduced iteration-level batching; prefill-prioritizing
vLLM (2023)	SOSP	PagedAttention + prefill-prioritizing; main baseline
Splitwise (2023)	–	Disaggregated prefill/decode; complementary to Sarathi
DistServe (2024)	–	Disaggregated; goodput-optimized; no chunking needed
TetriInfer (2024)	–	Disaggregated alternative
FlexGen (2023)	–	High-throughput offline inference; not online serving
FastServe (2023)	–	Preemption-based scheduling; complementary
Sarathi-Serve (2024)	OSDI	Chunked-prefills + stall-free batching; best of both worlds
APIServe (2024)	–	Uses Sarathi’s chunked-prefills for ahead-of-time recomputation

5.4 Related Work: Attention Kernels (Common Context)

Both papers target the same key workload — attention — but at different levels:

Work	Significance
FlashAttention (2022)	IO-aware tiled attention; reduced memory footprint
FlashAttention-2 (2023)	Better work partitioning and parallelism
FlashAttention-3 (2024)	Hopper-specific SWP and WS (Twill rediscovers)
FlashAttention-4 (2025)	Blackwell-specific strategy (Twill rediscovers)
CUTLASS	NVIDIA template library; provides reference hand-tuned kernels
ThunderKittens	High-control TC GPU library; manual WS
Triton	Tile-based DSL; Twill’s input language; Sarathi-Serve uses FA2 from this ecosystem

6 Experimental Setup Details for Reproducibility

6.1 Twill: Exact Setup

- **GPU:** NVIDIA H100 SXM5 80 GB (Hopper) and NVIDIA B200 180 GB (Blackwell).
- **CUDA:** 13.0.
- **CPU:** Intel Xeon Platinum 8570 (single core for solver timing).
- **Solver stack:** SCIP (cost normalization), CBC (modulo scheduling ZLP), Yices2 (SMT QFLIA).
- **Cost normalization bound:** $U = 300$; SCIP solves in < 500 ms.
- **Input programs:** Triton tutorial Flash Attention; sub-tiled at TC instruction granularity.
- **Kernel parameters:** FP16, non-causal, BATCH=4, NUM_HEADS=32, HEAD_DIM=128.
- **Sequence lengths tested:** 2048, 4096, 8192, 16384.
- **Output:** Warp-annotated TTGIR; hand-compiled to CUDA C++ for benchmarking.
- **Metric:** TFLOPS/s (as reported by cuDNN/FA perf utilities).

6.2 Sarathi-Serve: Exact Setup

- **GPU configurations:** see Table 1 summary in Section 3.
- **Interconnect:** Pairwise NVLink within node (DGX-style); 100 Gbps Ethernet across nodes.
- **CUDA:** 12.1.
- **Framework:** vLLM fork with added chunked-prefill, PP, and telemetry.
- **Attention backends:** FlashAttention v2 (all results), FlashInfer (also supported).
- **Communication:** NCCL (TP and PP).
- **Request arrival:** Poisson process; sustainable load only (median scheduling delay < 2 s).
- **Token budgets:** 512 (strict SLO), 2048 (relaxed SLO); 1536 for LLaMA2-70B relaxed.
- **Metrics:** P50 TTFT, P99 TBT, maximum sustainable QPS (capacity).
- **Baselines:** Orca (configured with max sequence length $\times$ batch size padding), vLLM (with PagedAttention).

7 Open Questions and Future Directions

7.1 Open Questions from Twill

1. **Nested loops and control flow:** Can the constraint framework extend to hierarchical loops (e.g., attention over multiple heads with conditional masking)?
2. **Tile size co-optimization:** Twill currently requires tile sizes as input. Can they be jointly optimized with the schedule?
3. **Codegen gap:** Triton’s codegen decisions (memory layout, synchronization) are suboptimal. Can Twill’s output drive a fully automated, high-performance codegen pipeline?
4. **Solver scalability:** Can GPU-accelerated SMT solving reduce solve times from minutes to seconds?
5. **Multi-SM scheduling:** Twill schedules within a single SM. Can the framework extend to coordinate across SMs (e.g., for persistent kernels)?

7.2 Open Questions from Sarathi-Serve

1. **Vs. disaggregated systems:** A quantitative comparison with Splitwise/DistServe is missing. When does disaggregation outperform chunked-prefills?
2. **Dynamic token budget:** The current approach uses a static τ . Adaptive budgeting based on real-time queue state could further improve both metrics.
3. **TTFT optimization:** Chunked prefills necessarily increase TTFT. Can speculative prefill or parallel prefill partially recover this?
4. **MoE models:** Mixture-of-experts architectures have heterogeneous compute per token; how does Sarathi-Serve’s token budget interact with expert routing?
5. **Prefill caching:** With chunked prefills and KV-cache reuse (prefix caching), the overhead of re-reading earlier chunks’ KV-caches could be eliminated.

8 Summary Tables for Quick Reference

8.1 Twill at a Glance

Attribute	Details
Full title	Optimal SWP and WS for Tensor Core GPUs
Authors	Soi, Yadav, Kjolstad, Aiken (Stanford); Dehnavi, Garland, Bauer (NVIDIA)
Venue	arXiv preprint (Dec 2025)
Code	Not publicly released
Key technique	Joint ILP+SMT formulation of SWP+WS
Solvers	CBC (modulo sched.), SCIP (cost norm.), Yices2 (SMT/QFLIA)
Input	Triton TTGIR + GPU architecture spec
Output	Warp-annotated pipelined IR
Optimality	Provably optimal for singly-nested loops
GPUs	NVIDIA H100 (Hopper), B200 (Blackwell)
Key result	Rediscovered FA3 (28 s) and FA4 (19 s); within 1–2% of hand-tuned
Limitation	Single-level loops only; minutes to solve; no public code

8.2 Sarathi-Serve at a Glance

Attribute	Details
Full title	Taming Throughput-Latency Tradeoff in LLM Inference
Authors	Agrawal et al. (MSR India + Georgia Tech)
Venue	OSDI 2024
Code	https://github.com/microsoft/sarathi-serve
Key technique	Chunked-prefills + stall-free batching
Based on	vLLM (fork); FlashAttention v2; NCCL
Input	Online LLM inference requests
Output	Scheduled micro-batches; token-level serving decisions
Optimality	No formal proof; empirically validated
GPUs	A100 (80 GB), A40 (48 GB)
Models	Mistral-7B, Yi-34B, LLaMA2-70B, Falcon-180B
Key result	Up to $5.6\times$ capacity over vLLM (Falcon-180B, PP)
Limitation	Higher TTFT; static token budget; no disaggregation

References

- [1] R. Soi, R. Yadav, F. Kjolstad, A. Aiken, M. M. Dehnavi, M. Garland, M. Bauer, “Optimal Software Pipelining and Warp Specialization for Tensor Core GPUs,” arXiv:2512.18134, December 2025.
- [2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, R. Ramjee, “Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve,” arXiv:2403.02310, OSDI 2024.
- [3] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, T. Dao, “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision,” arXiv, 2024.
- [4] T. Dao, J. Shah, T. Zadouri, M. Hoehnerbach, V. Thakkar, “Flash Attention 4,” GitHub, 2025.

- [5] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” arXiv:2307.08691, 2023.
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS 2022.
- [7] H. Chen et al., “Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References,” arXiv:2510.14719, 2025.
- [8] R. Yadav, M. Garland, A. Aiken, M. Bauer, “Task-Based Tensor Computations on Modern GPUs,” PLDI 2025.
- [9] P. Tillet, H. T. Kung, D. Cox, “Triton: an Intermediate Language and Compiler for Tiled Neural Network Computations,” MAPL 2019.
- [10] M. Lam, “Software pipelining: an effective scheduling technique for VLIW machines,” PLDI 1988.
- [11] B. R. Rau, “Iterative modulo scheduling,” MICRO 1994.
- [12] R. Govindarajan, E. R. Altman, G. R. Gao, “Minimizing register requirements under resource-constrained rate-optimal software pipelining,” MICRO 1994.
- [13] M. Bauer, H. Cook, B. Khailany, “CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization,” SC 2011.
- [14] M. Bauer, S. Treichler, A. Aiken, “Singe: Leveraging Warp Specialization for High Performance on GPUs,” PPOPP 2014.
- [15] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, “Orca: A Distributed Serving System for Transformer-Based Generative Models,” OSDI 2022.
- [16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP 2023.
- [17] P. Patel, E. Choukse, C. Zhang, I. Goiri, A. Shah, S. Maleki, R. Bianchini, “Splitwise: Efficient Generative LLM Inference using Phase Splitting,” 2023.
- [18] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, “DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized LLM Serving,” 2024.
- [19] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, A. Tumanov, “Vidur: A Large-Scale Simulation Framework for LLM Inference,” MLSys 2024.
- [20] Y. Cheng, L. Wang, Y. Shi, Y. Xia, L. Ma, J. Xue, Y. Wang, Z. Mo, F. Chen, F. Yang, M. Yang, Z. Yang, “PipeThreader: Software-Defined Pipelining for Efficient DNN Execution,” OSDI 2025.
- [21] G. Huang, Y. Bai, L. Liu, Y. Wang, B. Yu, Y. Ding, Y. Xie, “ALCOP: Automatic Load-Compute Pipelining in Deep Learning Compiler for AI-GPUs,” MLSys 2023.
- [22] Meta, “TLX Language,” GitHub, 2025.
- [23] L. Wang, Y. Cheng, Y. Shi, Z. Tang, Z. Mo, W. Xie, L. Ma, Y. Xia, J. Xue, F. Yang, Z. Yang, “TileLang: A Composable Tiled Programming Model for AI Systems,” 2025.
- [24] B. Dutertre, “Yices 2.2,” CAV 2014.
- [25] NVIDIA Corporation, “CUTLASS: CUDA Templates for Linear Algebra Subroutines,” <https://docs.nvidia.com/cutlass/>, 2025.

- [26] B. F. Spector, S. Arora, A. Singhal, D. Y. Fu, C. Ré, “ThunderKittens: Simple, Fast, and Adorable AI Kernels,” arXiv:2410.20399, 2024.

GPU Systems Literature Survey

Efficient Machine Learning on Modern GPUs

Paper 1:

LithOS: An Operating System for Efficient Machine Learning on GPUs

Coppock, Zhang, Solomon, Kypriotis, Yang, Sharma, Schatzberg, Mowry, Skarlatos
Carnegie Mellon University & Meta

Venue: ACM SIGOPS SOSP 2025, Seoul, Republic of Korea

arXiv: 2504.15465 **DOI:** 10.1145/3731569.3764818

Paper 2:

Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References

H. Chen, E. Gaburov, J. Knight, B. Fan, M. Masuda, Z. Zhang, A. Collins,
M. Brookhart, V. Grover, B. Hagedorn, C. Sullivan
Cornell University & NVIDIA

Venue: arXiv preprint, Oct–Dec 2025

arXiv: 2510.14719 **Upstream:** OpenAI Triton (`aref_auto_ws` branch)

Contents

1	Introduction and Motivation	3
2	Background: GPU Architecture	3
2.1	Execution Model	3
2.2	Hopper-Specific Features (Relevant to Tawa)	3
3	Paper 1: LithOS	4
3.1	Core Problem Statement	4
3.2	Core Novelty and Contributions	4
3.3	System Architecture	5
3.3.1	TPC Scheduler	5
3.3.2	Kernel Atomizer	5
3.3.3	Hardware Right-Sizing	5
3.3.4	Transparent Power Management (DVFS)	6
3.3.5	Online Latency Prediction	6
3.4	Implementation Details	6
3.5	Experimental Setup	6
3.5.1	Baselines Compared	7
3.5.2	Inference Workloads (Table 2 equivalent)	7
3.5.3	Training (BE) Workloads	7
3.5.4	Latency Constraints	7
3.5.5	Evaluation Scenarios	7
3.6	Key Results	8
3.7	Design Principles	8
4	Paper 2: Tawa	9
4.1	Core Problem Statement	9
4.2	Core Novelty and Contributions	9
4.3	The aref Abstraction	10
4.4	Compilation Pipeline	10
4.5	Additional Optimisations	11
4.6	Experimental Setup	11
4.6.1	Baselines	11
4.6.2	Benchmarks	11
4.7	Key Results	11
4.7.1	Hyperparameter Sensitivity	12
5	Comparative Analysis	12
5.1	Problem Layer and Scope	12
5.2	Complementarity	13
6	Related Work Landscape	13
6.1	GPU Multitenancy Systems	13
6.2	GPU Kernel Compilers and DSLs	13
6.3	Right-Sizing and DVFS	13
7	Limitations and Open Problems	14
7.1	LithOS Limitations	14
7.2	Tawa Limitations	14

8	Integration with Modern Systems	14
8.1	LithOS Integration	14
8.2	Tawa Integration	15
9	Future Research Directions	15
10	Summary and Takeaways for Literature Survey	16
	References	16

1 Introduction and Motivation

The rapid proliferation of large-scale machine learning (ML) workloads has cemented GPUs as the dominant compute substrate in modern datacenters. Despite their raw computational power, GPUs are notoriously underutilised in practice. Public telemetry from Microsoft reports average GPU utilisation at approximately 52%, while Alibaba records figures as low as 10%. Internal data from Meta’s production inference fleet reveals device utilisation ranging between 25% and 60% over a one-week observation window, with Streaming Multiprocessor (SM) utilisation dropping below 15% during troughs. Even headline models are not immune: the Llama 3 training run achieved only $\approx 40\%$ GPU utilisation.

Two complementary classes of work have emerged to address GPU efficiency from opposite ends of the software stack:

1. **Operating-system-level resource management (LITHOS)**: treating the GPU as a first-class citizen of the OS, with fine-grained scheduling, isolation, and power management.
2. **Compiler-level kernel generation (TAWA)**: closing the programmability gap between high-level tensor languages and the asynchronous, task-parallel hardware of modern GPUs.

This survey analyses both papers in depth, covering their motivations, core novelties, system design, experimental methodology, results, and implications for future research.

2 Background: GPU Architecture

A modern NVIDIA GPU (e.g. H100) is organized hierarchically: Graphics Processing Clusters (GPCs) contain Texture Processing Clusters (TPCs), each of which contains two Streaming Multiprocessors (SMs). An SM houses CUDA cores, Tensor Cores, a warp scheduler, a register file, and software-managed shared memory (SMEM). The H100 provides 8 GPCs, 9 TPCs per GPC, and 128 CUDA cores per SM.

2.1 Execution Model

GPU programs are written as *kernels*: each kernel defines a grid of thread blocks, where each thread block is dispatched to an SM and executes a group of SIMD threads (warps of 32 threads). Once a kernel is submitted via a CUDA stream, the hardware scheduler dispatches thread blocks to idle SMs; this process is opaque to software.

2.2 Hopper-Specific Features (Relevant to Tawa)

NVIDIA’s Hopper architecture (H100) introduces two hardware innovations critical to TAWA:

- **Tensor Memory Accelerator (TMA)**: a dedicated hardware unit for bulk, multidimensional transfers from global memory (GMEM) to shared memory, offloading address generation from software.
- **Warp Group Matrix Multiply-Accumulate (WGMMMA)**: cooperative matrix-multiply operations executed across four warps (one warp group), enabling large-tile GEMM in an asynchronous manner.
- **Hardware Warp Specialization**: different warps within an SM can be assigned distinct roles (producer / consumer), enabling overlap of data movement and computation.

- **Transaction Barriers (mbarriers):** synchronization primitives that release threads when both the arrival count and an expected byte-transfer count are met.

3 Paper 1: LithOS

Paper Identity

Title	LithOS: An Operating System for Efficient Machine Learning on GPUs
Authors	P. H. Coppock, B. Zhang, E. H. Solomon, V. Kypriotis, L. Yang, B. Sharma, D. Schatzberg, T. C. Mowry, D. Skarlatos
Affil.	Carnegie Mellon University (CMU CAOS group) & Meta
Venue	ACM SIGOPS SOSP 2025, Seoul, October 13–16, 2025
DOI	10.1145/3731569.3764818
arXiv	2504.15465 (v1: Apr 2025)
Code	Implementation in ~5000 lines of Rust (not yet public as of survey date)
License	CC BY 4.0

3.1 Core Problem Statement

GPU utilisation in datacenters is chronically low due to three compounding factors:

- **Workload heterogeneity:** models range over $10\times$ in size; popular and unpopular models must share infrastructure.
- **Dynamic traffic:** request rates fluctuate by $2.2\times$ intra-day, making static provisioning wasteful.
- **Lack of transparent sharing:** existing solutions (MPS, MIG, research systems) either sacrifice transparency, isolation, or utilisation.

Existing approaches occupy two extremes. MIG provides strong isolation but at coarse GPC granularity with $>5s$ reconfiguration overhead. MPS enables fine-grained sharing but offers no isolation and suffers head-of-line (HoL) blocking. Research systems such as REEF, Orion, and TGS require framework modifications, offline profiling, or application changes, making them impractical at datacenter scale.

3.2 Core Novelty and Contributions

LITHOS introduces a **GPU operating system** that is fully transparent to the ML software stack—no changes to models, runtimes, or frameworks are required. It decouples kernel submission from execution by interposing at the CUDA Driver API level.

Four Core Mechanisms of LithOS

1. **TPC Scheduler:** Fine-grained spatial scheduling at individual TPC granularity; enables TPC Stealing between workloads.
2. **Kernel Atomizer:** Transparently partitions kernels into sub-kernel chunks (atoms) of thread blocks without source or PTX access.
3. **Hardware Right-Sizing:** Online model-based determination of minimal TPC resources per atom using Amdahl-law curve fitting.
4. **Transparent Power Management:** Per-workload DVFS via a sequence-based frequency scaling model, without offline profiling.

3.3 System Architecture

LITHOS runs entirely on CPU cores and provides a dynamically linked library `LibLithOS` that mimics the native CUDA library. Applications see no difference. Internally, LITHOS maintains a system-wide view of GPU state across all tenants.

3.3.1 TPC Scheduler

The TPC Scheduler operates at the granularity of individual TPCs, analogous to how a CPU OS manages cores. Key design points:

- **Launch Queues:** Applications submit kernels to launch queues; LITHOS decouples submission from GPU dispatch.
- **TPC Quotas:** Administrators specify TPC allocations per application, enforcing guaranteed resources when runnable.
- **TPC Stealing:** Idle TPCs from one workload are dynamically lent to another. Per-TPC timers (fed by the online latency predictor) prevent stealing from long-running TPCs that would cause HoL blocking.
- **Sync Queues:** A $100\mu\text{s}$ outstanding-work threshold prevents over-submission; a Tracker thread monitors completion and updates scheduler state.

3.3.2 Kernel Atomizer

The Kernel Atomizer is the technical centrepiece of LITHOS. It transparently splits kernels into atoms (subsets of thread blocks) at runtime:

1. The latency predictor estimates kernel duration given TPC assignment.
2. Division by a tunable `atom_duration` ($250\text{--}500\mu\text{s}$ effective range) yields the number of atoms.
3. A *Prelude Kernel* is injected by modifying the Queue Metadata (QMD) struct in the CUDA driver; it bounds block execution to a `[block_idx_lo, block_idx_hi)` range and calls the original kernel entry point.
4. Non-overlapping block ranges ensure each block executes exactly once.

This technique works on *any* framework, library (including closed-source cuDNN), and compiler, without source or PTX access. Atomization reduces HoL blocking from multi-millisecond kernels to $\sim 250\text{--}500\mu\text{s}$ latency windows. Special handling is provided for CUDA Graphs (subgraph decomposition) and kernels with grid-group synchronization (atomization disabled).

3.3.3 Hardware Right-Sizing

LITHOS determines per-kernel minimum TPC allocations using a two-point interpolation model consistent with Amdahl’s Law:

$$l = \frac{m}{t} + b$$

where l is predicted latency, t is TPC count, m captures parallel speedup potential, and b represents irreducible sequential latency. Two profiling points (all TPCs vs. one TPC) anchor the curve. A filtering heuristic based on thread-block occupancy caps allocations for outlier kernels. A tunable *latency slip parameter* k lets administrators specify acceptable degradation (e.g., 10%).

3.3.4 Transparent Power Management (DVFS)

LITHOS virtualises GPU frequency using a sequence-based model. Each kernel is assigned a weight w equal to its fraction of cumulative stream runtime. Relative slowdown is modelled as proportional to fractional frequency drop:

$$k = \frac{\text{lat}(f_{\text{th}})}{\text{lat}(f_{\text{max}})} - 1 = s \cdot \left(\frac{f_{\text{max}}}{f_{\text{th}}} - 1 \right)$$

The aggregate sensitivity $S = \sum w \cdot s$ yields a target frequency:

$$f_{\text{final}} = \frac{f_{\text{max}}}{1 + k/S}$$

Memory-bound kernels (low sensitivity) pull frequency down; compute-bound kernels (high sensitivity) pull it toward maximum. This avoids the need for offline profiling across input lengths that prior DVFS work (e.g., DynamoLLM, μ -Serve) requires.

3.3.5 Online Latency Prediction

A lightweight module records per-kernel latencies indexed by an ordinal position k (the k -th kernel after batch start), enabling identification of operator nodes in the model’s DFG without explicit access to the computation graph. Predictions are refined during execution. Misprediction rates are 0.9% and 0.38% for HP workloads in inference–inference and inference–training scenarios, respectively.

3.4 Implementation Details

Implementation Specifics

Language	Rust (~5,000 lines, excluding macro-generated CUDA interposition code)
Interposition	CUDA Driver API (lowest common denominator); auto-generated wrappers for full API coverage
TPC Control	Extension of <code>libsmctrl</code> with added Hopper support; QMD struct reverse engineering
Architectures	Ampere (A30, A100), Hopper (H100), Ada Lovelace (L4); verified on NVIDIA datacenter GPUs
Base Layer	Built on top of MPS for multi-context concurrent execution
Isolation Model	TPC-level; each app in its own address space; hardware-enforced memory isolation
Thread Clusters	Hopper Thread Block Clusters handled; atoms are multiples of cluster size

3.5 Experimental Setup

Hardware & Software Testbed

GPU	NVIDIA A100-SXM4, 108 SMs, 40 GB HBM2 (Lambda Labs instance)
CPU/RAM	30 CPU cores, 216 GB host memory
OS	Ubuntu 22.04
CUDA	12.6
Rust	1.83.0-nightly
Python	3.10; PyTorch 2.3
Inference	TensorRT 10.1, TensorRT-LLM 0.11.0, Triton Inference Server, ONNX Runtime

3.5.1 Baselines Compared

- **NVIDIA native:** Time Slicing, MPS, Stream Priority, MIG
- **Research systems:** TGS (transparent), REEF (application-modified), Orion (application + offline profiling)
- Note: Orion and REEF were reimplemented using LITHOS’s own interposition infrastructure since original code was tied to specific CUDA versions.

3.5.2 Inference Workloads (Table 2 equivalent)

Table 1: LITHOS Inference Workloads and SLO Constraints

Model	Framework	Load (rps)	SLO (ms)
ResNet-50 v1.5	TensorRT	1,000	15
RetinaNet	ONNX Runtime	9	100
Llama 3 8B	TensorRT-LLM	0.5	2,000
GPT-J 6B	TensorRT-LLM	0.5	2,000
BERT-Large	TensorRT	30	130
YOLOv4	TensorRT	—	—

3.5.3 Training (BE) Workloads

VGG-19 (17.4 GiB, BS 120), ResNet-50 (18.4 GiB, BS 184), MobileNetV2 (18.4 GiB, BS 216), DLRM (6.7 GiB, BS 32768), BERT-Large (17.3 GiB, BS 20), Llama 3 Finetuning (32.0 GiB, BS 4). Training batch sizes are capped to $\leq 50\%$ GPU DRAM to allow colocated inference.

3.5.4 Latency Constraints

SLO constraints are drawn from the MLPerf Datacenter Inference Benchmark, ranging $2.3\times - 7.4\times$ of standalone baseline latency.

3.5.5 Evaluation Scenarios

1. **Inference-only multitenancy:** Two HP inference apps + one BE inference app. HP A has a latency-oriented SLO; HP B has a throughput-oriented SLO. Partitioning systems use 75%/25% TPC splits.
2. **Hybrid inference/training:** One HP inference app (latency SLO) + one continuous BE training job. Poisson load targeting $\sim 80\%$ HP GPU utilisation.

3. **Hardware right-sizing:** Latency slip $k = 1.1$ (10% tolerance); evaluated standalone per model.
4. **Power management (DVFS):** Slip $k = 1.1$; energy = average power $\times$ completion time, measured via `nvidia-smi` at 100 ms granularity.

3.6 Key Results

LithOS Performance Summary

Inference-only Multitenancy:

- LITHOS achieves **100% SLO attainment** and aggregate throughput of **1 $\times$** (equal to the best single-app case)—no other system achieves both.
- LITHOS P99 tail latencies are **13 $\times$ lower** than MPS and **4 $\times$ lower** than Orion (best-performing SotA).
- HP B goodput: LITHOS = 0.50 vs. MIG = 0.37 (35% improvement); LITHOS is the only system supporting concurrent BE throughput (0.15 $\times$).

Hybrid Inference/Training:

- Tail latency within **20% of ideal** (standalone inference).
- **4.7 $\times$ better** than MPS; **1.18 $\times$ better** than best SotA (TGS).
- Training throughput improved **34 $\times$** and aggregate throughput **1.35 $\times$** vs. TGS.

Hardware Right-Sizing:

- Mean GPU capacity savings: **26%** (up to 51%) across all workloads.
- P99 latency overhead: mean **4%**; throughput decrease: mean **4%**.
- Model accuracy (R^2): 0.92–0.99 across workloads.

Power Management (DVFS):

- Mean energy savings: **26%** (up to 46%).
- Mean P99 latency increase: **7%**.

System Overhead:

- LITHOS adds **4%** overhead vs. vanilla NVIDIA driver (without multitenancy).
- Atomization adds **<1%** on top.

3.7 Design Principles

LITHOS is built on four explicit OS principles:

1. Manage resources at the finest granularity where the OS is effective while preserving transparency (TPC, not GPC or intra-SM).
2. Expose simple performance-tolerance knobs while hiding hardware complexity.
3. Virtualise frequency to preserve the illusion of dedicated control while optimising system-wide power gains.
4. Enforce isolation at the finest practical granularity; ensure local faults degrade gracefully.

4 Paper 2: Tawa

Paper Identity

Title	Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References
Authors	H. Chen (Cornell), E. Gaburov, J. Knight, B. Fan, M. Masuda (NVIDIA), Z. Zhang (Cornell), A. Collins, M. Brookhart, V. Grover, B. Hagedorn, C. Sullivan (NVIDIA)
Venue	arXiv preprint 2510.14719 (v1: Oct 2025; v2: Dec 2025); submitted to a top venue
Code	Upstreamed to OpenAI Triton branch <code>aref_auto_ws</code> ; first PR #6288
Base	Built on Triton (commit <code>0c7edf</code>); ~4,000 lines of C++; compact MLIR dialect

4.1 Core Problem Statement

Modern GPUs (Hopper, Blackwell) integrate dedicated TMA units and Tensor Cores that demand *task-parallel*, asynchronous execution patterns. Yet the dominant programming paradigm—Single-Instruction Multiple-Thread (SIMT), as embodied in CUDA and Triton—assumes *data-parallel*, synchronous execution. This fundamental **paradigm mismatch** creates three practical challenges:

1. **Coordinating concurrent warp roles:** Developers must manually restructure kernels with complex conditional branches to assign warp roles; SIMT languages lack first-class abstractions for this.
2. **Low-level communication management:** No C++-level interfaces exist for TMA and mbarriers; developers use inline PTX and must reason about arrival counts, barrier phases, and shared-memory aliasing—any error causes deadlocks.
3. **Resource allocation and pipeline orchestration:** Developers must manually manage multi-buffering, pipeline stages (prologue/body/epilogue), and register/shared-memory trade-offs—a vast, tightly coupled optimisation space.

Prior work (CUTLASS, ThunderKittens, FlashInfer) provides expert-tuned kernels but remains *library-centric*. TileLang introduces warp specialisation but requires manual annotations (`T.pipelined`, `T.copy`). Triton, before TAWA, uses Ampere-style software pipelining (`cp.async`), leaving significant performance on the table for Hopper.

4.2 Core Novelty and Contributions

TAWA is the **first fully automated compilation flow** for warp specialisation on commercial NVIDIA GPUs, requiring *zero user annotations or code modifications*.

Three Core Contributions of Tawa

1. **Asynchronous Reference (aref) IR Abstraction:** A formally defined one-slot channel between producer and consumer warps with `put/get/consumed` operations and rigorous operational semantics.
2. **Task-Aware Partitioning:** An MLIR graph traversal algorithm that automatically identifies producer (TMA load) and consumer (WGMMMA compute) warp groups and distributes loop bodies accordingly.

3. **Multi-Granularity Software Pipelining:** Fine-grained (MMA address vs. Tensor Core computation) and coarse-grained (CUDA Core vs. Tensor Core) pipelines automatically orchestrated on top of the **aref** program.

4.3 The **aref** Abstraction

The **aref** (asynchronous reference) is the conceptual core of TAWA. It models a one-slot communication channel with a data buffer and two hardware mbarriers (**empty** / **full**):

- **put**(a, v): Requires **empty** credit; writes v to buffer; sets **full**.
- **get**(a): Requires **full** credit; reads buffer into program variable; transitions to borrowed state (neither flag set).
- **consumed**(a): Releases **empty** credit, enabling the next **put**.

Formally (in small-step operational semantics):

$$\frac{\sigma(a).E = 1}{\langle \sigma, \text{put}(a, v) \rangle \rightarrow \langle \sigma[a \mapsto \langle \text{buf} = v, F = 1, E = 0 \rangle], \epsilon \rangle}$$

$$\frac{\sigma(a).F = 1}{\langle \sigma, \text{get}(a) \rangle \rightarrow \langle \sigma[a \mapsto \langle \text{buf} = \sigma(a).\text{buf}, F = 0, E = 0 \rangle], \sigma(a).\text{buf} \rangle}$$

Multiple **aref** instances form a cyclic buffer of depth D , supporting deep pipelining. The **aref** is type-generic (supports tuples of tensors sharing mbarriers). TAWA formally proves no deadlock can occur by construction—unlike TileLang or manual PTX code where subtle errors cause liveness violations.

4.4 Compilation Pipeline

TAWA adds a sequence of MLIR passes on top of the standard Triton compilation stack:

1. **Triton-Python** $\rightarrow$ **TT(GPU)IR**: Standard Triton frontend.
2. **Task-Aware Partitioning** (new pass): Backward traversal labels operations as *iteration statements* (address/index computation, coloured orange) or *tile statements* (WGMMA computation, coloured blue). A graph cut assigns all iteration+TMALoad operations to the producer warp group (WG0) and all tile operations to the consumer warp group (WG1). Cross-partition dependencies become **aref** channels; the loop body is cloned for each WG.
3. **Multi-Granularity Pipelining** (new pass):
 - *Fine-grained*: Overlaps MMA address computation (CUDA Cores) with Tensor Core execution via a bounded depth- P pipeline (**WGMMA.issue**/ k , **WGMMA.wait**{**pendings**= P }).
 - *Coarse-grained*: A three-stage T-C-U (Tensor Core / CUDA Core transform / optional Tensor Core) assembly line with prologue, steady-state, and epilogue phases.
4. **aref Lowering**: Replaces **aref** operations with TMA instructions, mbarrier init/arrive/wait; introduces parity-based deadlock avoidance.
5. **Standard Triton Lowering**: MLIR $\rightarrow$ LLVM IR $\rightarrow$ PTX.

4.5 Additional Optimisations

1. **Cooperative Warp Groups:** Two consumer WGs (WG1, WG2) collaboratively compute the same tile, pooling register files to enable larger tile sizes and higher arithmetic intensity. No `aref` changes needed.
2. **Persistent Kernels:** Launch as many CTAs as SMs; CTAs iterate over work tiles without relaunch overhead. Particularly effective for large- K GEMMs.

4.6 Experimental Setup

Hardware & Software Testbed

GPU	NVIDIA H100 SXM5, 80 GB HBM3e
CUDA	12.7
Base	Triton commit 0c7edf
Runs	25 warm-up + 1000 measurement runs; report mean execution time

4.6.1 Baselines

- **cuBLAS** v12.7 (closed-source, highly optimised commercial library)
- **CUTLASS** v4.0.0 (NVIDIA open-source CUDA linear algebra library)
- **Triton** (same commit; official software pipelining enabled)
- **ThunderKittens** (commit 6c27e2): CUDA-based AI kernel library with 16×16 tile abstractions
- **TileLang** (commit 422fb1): TVM-based DSL with implicit warp specialisation

All baselines are hand-tuned with tile sizes from $\{64, 128, 256\}$. TAWA tunes `aref` size D and MMA pipeline depth P manually per kernel.

4.6.2 Benchmarks

1. **GEMM:** $M = N = 8192$, $K \in [256, 16384]$; FP16 and FP8 precision. Matrix sizes representative of real LLM (Llama, GPT, Qwen, Grok) workloads.
2. **Batched GEMM:** Batch size 8; $M = N = K \in [1K, 16K]$. Common in Mixture of Experts (MoE) models.
3. **Grouped GEMM:** 2–6 groups of varying shapes; common in MoE dispatch.
4. **Multi-Head Attention (MHA):** (Non-)causal attention; $L \in [1024, 16384]$; batch 4, head dim 128; FP16 and FP8. Compared against hand-written FlashAttention-3 (FA3, CUTLASS C++).

4.7 Key Results

Tawa Performance Summary

GEMM (FP16):

- Average speedups: $1.01\times$ (cuBLAS), $1.13\times$ (Triton), $1.15\times$ (TileLang), $1.09\times$ (ThunderKittens).
- Peak hardware utilisation: **79%** of theoretical peak.
- TAWA is slightly worse than cuBLAS at very small K (Triton framework overhead)

dominates).

GEMM (FP8):

- More pronounced gains: $1.06\times$ (cuBLAS), $1.02\times$ (Triton), $1.22\times$ (TileLang), $1.24\times$ (ThunderKittens).
- FP8’s faster compute makes data orchestration the bottleneck; TAWA’s warp specialisation directly addresses this.

Batched/Grouped GEMM:

- Up to **7%** speedup over Triton; up to **50%** faster than TileLang on batched GEMM.

Multi-Head Attention (FP16):

- Up to **96%** of FA3 throughput; $1.21\times$ speedup over Triton consistently across sequence lengths.
- Outperforms TileLang and ThunderKittens by $1.10\times$ and $1.23\times$ at $L \geq 4K$.

Multi-Head Attention (FP8):

- Up to **89%** of FA3; $1.11\times$ over Triton; $1.48\times$ over TileLang.
- ThunderKittens fails on FP8 attention configurations.

Ablation (GEMM, $K = 16384$):

- Triton baseline: 104 TFLOPs/s $\rightarrow$ +Auto WS: 393 ($3.78\times$) $\rightarrow$ +Cooperative WGs: 395 $\rightarrow$ +Large Tile: 572 ($1.46\times$) $\rightarrow$ +Persistent: 632 ($1.1\times$) $\rightarrow$ +Better Aref: **718** TFLOPs/s ($\approx 7\times$ total).

4.7.1 Hyperparameter Sensitivity

For non-persistent GEMM ($K = 16384$): performance consistently increases with **aref** depth D (up to shared-memory limits). For pipeline depth P : increasing from 1 to 2 improves throughput; $P = 3$ decreases it due to register pressure. Persistent kernels add approximately 5–10% throughput at equal D, P , by eliminating launch overhead and improving weight cache locality.

5 Comparative Analysis

5.1 Problem Layer and Scope

Table 2: High-Level Comparison of LithOS and Tawa

Dimension	LithOS	Tawa
Layer	System/OS (driver API)	Compiler (Triton MLIR)
Goal	Multi-tenant efficiency, isolation, power	Single-kernel peak performance
Transparency	Full (no app changes)	Full (no Triton code changes)
Hardware	A100, H100, L4 (Ampere, Hopper, Ada)	H100 (Hopper)
Language	Rust (5K LoC)	C++ (4K LoC) + MLIR dialect
Scheduling	TPC-level spatial + temporal	Warp-group-level within SM
Target setting	Datacenter multi-tenancy	Single-kernel LLM compute
Overhead	4% interposition overhead	$\sim 0\%$ (compile-time work)
Status	Research prototype (Rust)	Upstreamed to Triton (<code>aref_auto_ws</code>)

5.2 Complementarity

The two works are orthogonal and highly complementary:

- LITHOS operates *between* kernels (scheduling, isolation, power).
- TAWA operates *within* kernels (warp specialisation, pipelining).
- A production system could plausibly run TAWA-generated kernels under LITHOS scheduling for joint gains.
- LITHOS’s right-sizing module could in principle be informed by TAWA’s fine-grained pipelining structure (e.g., knowing which warp group is the bottleneck).

6 Related Work Landscape

6.1 GPU Multitenancy Systems

Table 3: Multitenancy System Comparison (as cited in LithOS)

System	Approach	Transparent	Granularity
NVIDIA MPS	SW multiplexing of GPU contexts	Yes	SM (shared)
NVIDIA MIG	HW partitioning at GPC boundaries	Yes	GPC (coarse)
Time Slicing	Round-robin context switching	Yes	Full GPU
TGS	Transparent container-level kernel ctl.	Yes	Kernel
REEF	Kernel preemption; app-modified	No	Kernel
Orion	Interference-aware; app + profiling	No	Kernel/operator
AntMan	Dynamic GPU scaling; framework-level	No	Operator
PipeSwitch	Fast context switching for DL	No	Model
Clockwork	Predictability-first scheduling	No	Request
LithOS	OS approach; TPC-level	Yes	Thread block

6.2 GPU Kernel Compilers and DSLs

Table 4: Kernel Compiler/DSL Comparison (as benchmarked in Tawa)

System	Approach	WS Auto?	Target
CUDA / PTX	Manual, lowest-level	No	All GPUs
CUTLASS	Templated C++; expert-required	No	NVIDIA
ThunderKittens	16×16 tile C++ interface	No	Hopper+
FlashInfer	JIT-specialised attention+KV-cache	No	NVIDIA
Triton (stock)	Tile-based Python; Ampere-style pipeline	No	NVIDIA
TileLang	Tile DSL; implicit WS; manual annotations	Partial	NVIDIA
TVM/Ansor	Automated scheduling; pre-Hopper	No	Various
Tawa	Auto WS from Triton; aref IR	Yes	Hopper+

6.3 Right-Sizing and DVFS

Prior right-sizing work (KRISP, GSLICE, PARIS/ELSA) operates at job or model granularity, requires hardware modifications, or depends on offline profiling. LITHOS is the first to do online, kernel-level right-sizing without offline profiling. Prior DVFS work (DynamoLLM, μ -Serve, SLO-aware DVFS) uses extensive offline profiling over input lengths and trains dedicated output-length predictors. LITHOS is the first transparent, online DVFS for GPU ML workloads.

7 Limitations and Open Problems

7.1 LithOS Limitations

1. **No hardware preemption:** Atomization is a software approximation; true microsecond-scale preemption requires GPU hardware support.
2. **Reverse-engineered QMD:** Reliance on undocumented QMD structures risks breakage on future NVIDIA architectures (though the authors note reversal typically takes “days”).
3. **Memory bandwidth contention:** Compute isolation is addressed; bandwidth isolation is acknowledged as a limitation (4–13% gains estimated from MIG measurements).
4. **Single-GPU:** Multi-GPU scheduling (e.g., for training across NVLink domains) is not addressed.
5. **Latency predictor for BE workloads:** Misprediction rates of 11–14% for BE; acceptable but room for improvement.
6. **DVFS granularity:** Current GPUs switch frequency in ~50 ms; sub-ms control awaits future hardware.

7.2 Tawa Limitations

1. **Hopper-centric:** Blackwell support (tensor memory, `tnem`) is future work.
2. **Producer-consumer only:** Ping-pong kernels, multicast, and multi-producer patterns are not yet supported.
3. **Small- K regression:** Triton framework overhead causes TAWA to underperform cuBLAS at small inner dimensions.
4. **Manual hyperparameter selection:** `aref` depth D and MMA pipeline depth P are still tuned manually per kernel, though the search space is small.
5. **Limited non-GEMM coverage:** Evaluation focuses on GEMM and attention; other operator types (e.g., convolution, reduction-heavy ops) are not studied.

8 Integration with Modern Systems

8.1 LithOS Integration

LITHOS is designed for **plug-and-play deployment** in any CUDA-based ML pipeline:

- **Frameworks:** Fully transparent to PyTorch, TensorFlow, JAX, TensorRT, TensorRT-LLM, and cuDNN—any framework that links against the CUDA Driver API.
- **Inference Serving:** Tested with NVIDIA Triton Inference Server (with dynamic batching), ONNX Runtime.
- **Containers:** Supports containerised workloads; GPU sharing across containers (as TGS does) is a core use case.
- **Datacenter:** Meta’s production use case (H100 nodes) directly motivates the design; the Meta co-authors contributed production insights.

- **Multi-tenant SaaS:** The TPC quota system maps naturally to cloud billing and SLA enforcement.

8.2 Tawa Integration

TAWA integrates into the **PyTorch** → **Triton** → **GPU** compilation pipeline:

- **PyTorch:** Triton is already the default backend for `torch.compile`; enabling TAWA requires setting `enable_warp_specialization=True` at kernel invocation.
- **Triton:** Code upstreamed to the `aref_auto_ws` branch of `triton-lang/triton`; PR #6288 is the entry point.
- **LLM Inference Engines:** Potential integration with vLLM, SGLang, TensorRT-LLM—any system that uses Triton for custom attention or GEMM kernels.
- **MoE Models:** Batched and grouped GEMM are directly relevant to MoE dispatch (DeepSeek-V3, Llama 4, Qwen 3, Grok 2).
- **Blackwell:** Future extension to `tmem` will benefit B100/B200 inference clusters.

9 Future Research Directions

Drawing from both papers’ discussion sections, the following directions are most promising:

1. **Hardware support for GPU OS:** Kernel-to-SM assignment, hardware preemption, cache partitioning, per-SM DVFS, and NUMA-style memory placement. Open-source GPU drivers (NVIDIA’s recent open-sourcing of kernel modules) are a crucial enabler.
2. **Multi-die GPU OSes:** Blackwell and Rubin Ultra use multi-die designs; coarse GPC-level MIG wastes resources; TPC-level scheduling becomes even more important.
3. **Intra-SM specialisation:** LITHOS operates inter-SM; TAWA operates within SMs. A unified framework spanning both levels is unexplored.
4. **Memory resource management:** Both papers acknowledge bandwidth and capacity isolation as open problems; integration with memory virtualisation systems (nvshare, BaM) is natural.
5. **Automated hyperparameter selection for Tawa:** Learning D and P from kernel properties (memory intensity, compute intensity, tile shape) without manual tuning.
6. **Multi-GPU OS abstractions:** Extending LITHOS to multi-GPU nodes, NVLink topologies, and distributed training pipelines.
7. **Tawa for non-NVIDIA hardware:** The `aref` abstraction is architecture-agnostic; porting to AMD ROCm or Intel XPU is a natural extension.
8. **Energy efficiency at scale:** LITHOS’s 26% energy savings, if generalised across an entire datacenter fleet, represent enormous absolute savings; finer-grained (sub-ms, per-SM) DVFS could amplify this.
9. **Security and fault isolation:** LITHOS’s TPC-level fault isolation is promising but only partially developed; formal security proofs are absent.

10 Summary and Takeaways for Literature Survey

LithOS — Key Takeaways

- First OS-level approach to GPU resource management that is simultaneously transparent, fine-grained (TPC/thread-block), and production-validated (Meta fleet data).
- Kernel atomization via QMD injection is a novel, powerful primitive enabling preemption-like behaviour without hardware support.
- Online Amdahl-law curve fitting and sequence-based DVFS are lightweight alternatives to offline profiling.
- SOSP 2025 best-paper candidate; directly influenced by and validated on Meta production infrastructure.
- Cites and outperforms: MPS (13× tail latency), MIG, TGS, REEF, Orion; sets new SotA for transparent GPU multitenancy.

Tawa — Key Takeaways

- First *fully automated* warp specialisation compiler for production NVIDIA GPUs (Hopper+); no user annotations required.
- The **aref** IR abstraction is a principled, formally specified primitive for intra-SM asynchronous communication with deadlock-free semantics.
- Achieves parity with cuBLAS GEMM and near-parity with hand-optimised FA3 (CUTLASS C++), while writing only ~25-line Triton-Python kernels.
- Upstreamed to Triton; immediately available for use in real LLM inference stacks (PyTorch, vLLM, etc.).
- Identifies key trade-off: **aref** depth $D \geq P$ (MMA depth) with persistent kernels preferred for large- K workloads.

Taken together, LITHOS and TAWA represent the two ends of a comprehensive push toward **full-stack GPU efficiency**—from the kernel micro-architecture to the datacenter scheduler. A system combining both could deliver strong isolation, high utilisation, near-peak compute performance, and significant energy savings, without requiring any changes to user-facing ML code.

References

- [1] P. H. Coppock *et al.*, “LithOS: An Operating System for Efficient Machine Learning on GPUs,” *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP ’25)*, Seoul, Republic of Korea, Oct. 2025. DOI: 10.1145/3731569.3764818. arXiv:2504.15465.
- [2] H. Chen *et al.*, “Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References,” *arXiv preprint arXiv:2510.14719*, Dec. 2025.
- [3] M. Han *et al.*, “Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences (REEF),” *OSDI ’22*, 2022.
- [4] F. Strati, X. Ma, and A. Klimovic, “Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications,” *EuroSys ’24*, 2024.
- [5] B. Wu *et al.*, “Transparent GPU Sharing in Container Clouds for Deep Learning Workloads (TGS),” *NSDI ’23*, 2023.

- [6] NVIDIA Corporation, “Multi-Process Service (MPS),” <https://docs.nvidia.com/deploy/mps/index.html>.
- [7] NVIDIA Corporation, “Multi-Instance GPU (MIG) User Guide,” 2025.
- [8] J. Shah *et al.*, “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision,” *NeurIPS*, vol. 37, 2024.
- [9] P. Tillet, H. T. Kung, and D. Cox, “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations,” *MAPL*, ACM, 2019.
- [10] NVIDIA, “CUTLASS,” <https://github.com/NVIDIA/cutlass>, 2025.
- [11] B. F. Spector *et al.*, “ThunderKittens: Simple, Fast, and Adorable Kernels,” *ICLR*, 2025.
- [12] L. Wang *et al.*, “TileLang: A Composable Tiled Programming Model for AI Systems,” arXiv:2504.17577, 2025.
- [13] T. Chen *et al.*, “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning,” *OSDI ’18*, 2018.
- [14] J. Stojkovic *et al.*, “DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency,” arXiv:2408.00741, 2024.
- [15] W. Xiao *et al.*, “AntMan: Dynamic Scaling on GPU Clusters for Deep Learning,” *OSDI ’20*, 2020.
- [16] Z. Bai *et al.*, “PipeSwitch: Fast Pipelined Context Switching for Deep Learning,” *OSDI ’20*, 2020.
- [17] A. Gujarati *et al.*, “Serving DNNs like Clockwork: Performance Predictability from the Bottom Up,” *OSDI ’20*, 2020.
- [18] NVIDIA Corporation, “NVIDIA H100 Tensor Core GPU Architecture,” Technical Report, 2023.
- [19] A. Grattafiori *et al.*, “The Llama 3 Herd of Models,” arXiv:2407.21783, 2024.
- [20] V. J. Reddi *et al.*, “MLPerf Inference Benchmark,” arXiv:1911.02549, 2019.

Multi-LoRA Fine-Tuning Systems: A Literature Survey

Analysing **mLoRA** and **tLoRA**:
Pipeline Parallelism, Batch Efficiency,
and Heterogeneity-Aware Scheduling

Papers Covered

Paper 1: mLoRA: Fine-Tuning LoRA Adapters via Highly-Efficient
Pipeline Parallelism in Multiple GPUs

Ye et al., Sichuan University / Ant Group, VLDB 2025

Paper 2: tLoRA: Efficient Multi-LoRA Training with Elastic
Shared Super-Models

Li et al., University of Illinois Urbana-Champaign, 2026

Survey Topics: Core Novelty · System Design · Experimental Setup ·
Benchmarks · GitHub Repositories · Integration with Modern Systems

May 2026

Contents

1	Introduction and Motivation	3
2	Background: LoRA and Multi-LoRA Challenges	3
2.1	Low-Rank Adaptation (LoRA)	3
2.2	The Multi-LoRA Training Problem	3
2.3	Taxonomy of Challenges	4
3	Paper 1: mLoRA	4
3.1	Overview and Core Contributions	4
3.2	System Architecture	5
3.3	LoRAPP: LoRA-Aware Pipeline Parallelism	5
3.3.1	Key Observation	5
3.3.2	Pipeline Construction	5
3.3.3	Zero-Bubble Achievement	5
3.3.4	Communication Overlap	6
3.3.5	Memory Savings	6
3.4	BatchLoRA: Efficient Multi-Adapter GPU Operator	6
3.4.1	Motivation	6
3.4.2	Batching Formulation	6
3.4.3	Backward Pass	6
3.4.4	Graph Pruning	6
3.4.5	BatchLoRA-Enhanced LoRAPP	7
3.5	Task Scheduler	7
3.5.1	Priority Scheduling	7
3.5.2	Memory Modelling	7
3.6	Experimental Setup	7
3.6.1	Hardware	7
3.6.2	Models and Workloads	7
3.6.3	Baselines	7
3.6.4	Metrics	8
3.7	Key Results	8
3.8	GitHub Repository Analysis	8
3.8.1	Repository Structure	9
3.8.2	Supported Models and LoRA Variants	9
3.8.3	Deployment Modes	9
3.8.4	Reproduction Instructions	9
4	Paper 2: tLoRA	9
4.1	Overview and Core Contributions	9
4.2	Key Motivation: Naïve Batching Hurts	10
4.3	Model Fuser: Shared Super-Model	10
4.3.1	SSM Construction	10
4.3.2	Integration with Existing Frameworks	10
4.4	Kernel Fuser: Minimising Runtime Bubbles	10
4.4.1	Design Rationale	10
4.4.2	Kernel Execution Model	11
4.4.3	Adaptive Nano-Batching	11
4.5	Adapter Scheduler: Cluster-Wide Throughput	11
4.5.1	Problem Formulation	11

4.5.2	Hierarchical Incremental Grouping	11
4.5.3	Per-Job Fairness	12
4.6	Experimental Setup	12
4.6.1	Evaluation Methodology	12
4.6.2	Workload Generation	12
4.6.3	Baselines	12
4.6.4	Metrics	12
4.7	Key Results	13
5	Comparative Analysis	13
5.1	Design Philosophy	14
5.2	Complementarity	14
5.3	Limitations	14
6	Benchmarks and Datasets	15
6.1	Datasets Used	15
6.2	Performance Measurement Tools	15
6.3	Hyperparameter Configurations	15
7	Integration with Modern Systems	16
7.1	mLoRA Integration	16
7.1.1	PyTorch Ecosystem	16
7.1.2	RLHF and Alignment	16
7.1.3	Serving Integration	16
7.1.4	Relation to LoRA Serving Systems	16
7.2	tLoRA Integration	16
7.2.1	Framework Agnosticism	16
7.2.2	Triton Kernel	16
7.2.3	Cluster Management	17
7.3	Comparison with Adjacent Systems	17
8	Literature Survey Context	17
8.1	PEFT Methods Landscape	17
8.2	Pipeline Parallelism Evolution	17
8.3	GPU Kernel Optimisation	18
8.4	Multi-Tenant LLM Serving and Training	18
8.5	Research Gaps and Future Directions	18
9	Summary	18

1 Introduction and Motivation

The rapid proliferation of transformer-based large language models (LLMs)—including LLaMA, Gemma, Mistral, and Phi-3—has created enormous demand for domain-specific fine-tuning. Rather than training from scratch, practitioners adapt pre-trained models to downstream tasks using fine-tuning, which is far more cost-efficient. Within this space, *Low-Rank Adaptation* (LoRA) has emerged as the dominant parameter-efficient fine-tuning (PEFT) technique. LoRA freezes the base model weights and injects pairs of small low-rank matrices—requiring less than 1% of trainable parameters—to achieve quality comparable to full fine-tuning.

As LLM platforms (e.g., Azure OpenAI, OpenAI, Anyscale) and individual developers routinely fine-tune *multiple* LoRA adapters concurrently—for hyperparameter search, multi-task training, or continuous adaptation—the efficiency of *multi-LoRA training* becomes a first-class systems concern. Existing parallelism strategies (data, tensor, pipeline) were designed for single-model training and exhibit significant overheads when adapted to the multi-adapter setting:

- **Tensor Parallelism (TP):** High all-reduce communication cost, especially across machines.
- **Fully Sharded Data Parallelism (FSDP):** High memory overhead due to weight replication; cannot fit large models on commodity GPUs.
- **Standard Pipeline Parallelism (GPipe/1F1B):** Pipeline bubbles due to synchronisation dependencies within a single model’s training.

This survey analyses two systems that directly target the multi-LoRA training problem at different granularities:

mLoRA (VLDB 2025, Sichuan University / Ant Group): A LoRA-aware pipeline parallelism system (*LoRAPP*) that exploits adapter independence to achieve near-zero pipeline bubbles, combined with a batched GPU operator (*BatchLoRA*) to reduce kernel launch overhead.

tLoRA (2026, UIUC): A heterogeneity-aware framework that fuses adapters into a *Shared Super-Model (SSM)*, uses a fused Triton kernel with adaptive nano-batching, and employs a residual-capacity-aware scheduler to maximise cluster-wide throughput while respecting per-job progress guarantees.

2 Background: LoRA and Multi-LoRA Challenges

2.1 Low-Rank Adaptation (LoRA)

LoRA [3] reparameterises weight updates as:

$$h = xW' = x(W + AB), \quad A \in \mathbb{R}^{h \times r}, \quad B \in \mathbb{R}^{r \times d}, \quad r \ll \min(h, d)$$

where W is the frozen pre-trained weight matrix and A, B are the trainable low-rank adapter matrices. A is initialised from $\mathcal{N}(0, \sigma^2)$ and $B = 0$ so that the adapter output is zero at initialisation. LoRA is applied to the query, key, value, and output projection layers of attention blocks. The number of trainable parameters drops from $\mathcal{O}(hd)$ to $\mathcal{O}(r(h + d))$, typically less than 1% of the base model.

2.2 The Multi-LoRA Training Problem

When training n adapters *simultaneously*, the naive approach (Algorithm 1 in mLoRA) sequentially swaps adapter weights:

Listing 1: Naive multi-LoRA training (PyTorch-style)

```

for adapter, data in fine_tuning_task:
    A, B = adapter # swap in the low-rank matrix A and B
    output = data @ W + data @ A @ B
    loss = loss_fn(data, output)
    loss.backward()

```

This incurs per-adapter kernel launches and cannot parallelise computation across adapters, leading to $\mathcal{O}(n)$ GPU kernel calls for the base model pass alone.

2.3 Taxonomy of Challenges

Both papers identify overlapping but distinct challenge dimensions, summarised in Table 1.

Table 1: Challenge taxonomy for multi-LoRA training

Challenge	mLoRA perspective	tLoRA perspective
Pipeline bubbles	GPUs idle waiting for gradient sync in standard PP	Skewed workloads stall SM warps
Communication overhead	All-reduce in TP/FSDP is bandwidth-bound	Cross-node communication amplified by heterogeneous batch sizes
Kernel launch overhead	Up to 10% of training time for small adapters	Per-adapter GEMMs fragment execution
Memory pressure	FSDP can't fit 13B on 4×48GB GPUs	Adapter rank/batch heterogeneity inflates memory footprint
Job heterogeneity	Assumed homogeneous adapters	Varying rank, batch size, GPU count

3 Paper 1: mLoRA

3.1 Overview and Core Contributions

Venue: VLDB 2025 (arXiv:2312.02515v2, September 2024)

Affiliation: Sichuan University, Ant Group, Zhejiang Computation, UT Arlington, CNIC CAS

Deployment: Production at Ant Group; 30% time saving in hyperparameter selection

GitHub: <https://github.com/TUDB-Labs/mLoRA> (376 stars, 66 forks, Apache-2.0)

mLoRA makes three primary contributions:

1. **LoRAPP:** A LoRA-aware pipeline parallelism scheme that achieves zero pipeline bubbles by exploiting adapter independence.
2. **BatchLoRA:** A batched GPU operator that consolidates multiple adapter computations into a single matrix multiply, reducing kernel launch overhead.
3. **Memory-aware scheduler:** An online non-linear memory model that maximises task concurrency while preventing OOM errors.

3.2 System Architecture

mLoRA consists of two main components:

- **Multi-LoRA Trainer:** Handles concurrent fine-tuning tasks and applies LoRAPP and BatchLoRA at runtime.
- **Task Scheduler (ASPEN):** Selects tasks from a candidate queue based on priority, memory footprint, and profiler feedback.

The workflow is: users submit adapter configurations and datasets $\rightarrow$ mLoRA generates candidate tasks $\rightarrow$ scheduler selects tasks for concurrent training $\rightarrow$ profiler updates memory model online.

3.3 LoRAPP: LoRA-Aware Pipeline Parallelism

3.3.1 Key Observation

Although different LoRA adapters share the same frozen base model, they accumulate and apply gradients *independently*. There is no gradient synchronisation requirement between adapters. This is the core insight enabling zero-bubble pipeline parallelism.

3.3.2 Pipeline Construction

LoRAPP partitions the pre-trained base LLM (comprising consecutive transformer decoder layers) into D groups, one per GPU. Each GPU:

1. Allocates a small amount of memory for the adapter weights corresponding to its assigned layers.
2. Performs forward propagation using activation values from the previous GPU.
3. Performs backward propagation using error values from the next GPU.

The first GPU receives training data; the last GPU computes the loss. Adapter weights are saved to persistent storage upon completion.

3.3.3 Zero-Bubble Achievement

In standard GPipe [4], the pipeline stalls because micro-batches within a mini-batch must accumulate gradients before the optimiser step (*stop-and-wait synchronisation*). The bubble ratio is $(D - 1)/(N + D - 1)$ where N is the number of micro-batches.

LoRAPP instead uses *mini-batches from different LoRA adapters* to fill the pipeline. Since adapters are independent, after GPU₀ completes F_1 (forward of adapter 1), it immediately starts F_2 (forward of adapter 2), without waiting. The bubble ratio of LoRAPP is:

$$\text{Bubble ratio} = \max \left\{ \frac{D - L}{D}, 0 \right\}$$

where L is the number of adapters and D is the number of GPUs. **When $L \geq D$, the bubble ratio is zero.**

For cases with $L < D$, LoRAPP falls back to GPipe-style micro-batch splitting within each adapter. The combined bubble ratio becomes:

$$\max \left\{ \frac{D - 1 + N - L \times N}{D + N - 1}, 0 \right\}$$

3.3.4 Communication Overlap

LoRAPP creates three concurrent CUDA streams per GPU for receive, send, and compute. Because adapters are independent, while GPU_{K+1} executes backward B_i for adapter i , GPU_K can simultaneously receive forward activations F_j for adapter j , hiding communication latency completely. Communication volume is $2(D-1)Bh$ (activation + error tensors of size Bh), which is the same as 1F1B but far less than TP.

3.3.5 Memory Savings

Since LoRAPP shares the base model across L adapters (rather than replicating it as in data parallelism), it saves $(L-1)W_\theta$ memory, where W_θ is the base model size. This enables training two Llama-2-13B models simultaneously on four 48GB GPUs—impossible for FSDP.

3.4 BatchLoRA: Efficient Multi-Adapter GPU Operator

3.4.1 Motivation

Even with zero pipeline bubbles, individual LoRA fine-tuning tasks are too small to saturate GPU parallelism. Measurements show average GPU utilisation of 83% and memory utilisation of only 30% with four tasks on four A6000 GPUs. The CUDA kernel launch overhead accounts for up to 10% of total training time for small (1.1B) models.

3.4.2 Batching Formulation

BatchLoRA consolidates n tasks $\{T_1, \dots, T_n\}$ with inputs $\{x_1, \dots, x_n\}$ and adapters $\{(A_i, B_i)\}$ into a single large matrix $X = (x_1^\top, \dots, x_n^\top)^\top$. The forward pass becomes:

$$H = XW + \begin{pmatrix} x_1 A_1 B_1 \\ \vdots \\ x_n A_n B_n \end{pmatrix}$$

The key reduction: the heavy base model multiplication XW (and its gradient $\nabla H W^\top$) is computed **once** for all adapters, rather than n times. The kernel launch cost drops from $k\alpha + k\beta$ to $\alpha + k\beta$, a reduction of approximately $(k-1)/k$ since $\beta \ll \alpha$.

3.4.3 Backward Pass

Gradients are:

$$\begin{aligned} \nabla A_i &= x_i^\top \nabla h_i B_i^\top, & \nabla B_i &= A_i^\top x_i^\top \nabla h_i \\ \nabla X &= \nabla H W^\top + \begin{pmatrix} \nabla h_1 B_1^\top A_1^\top \\ \vdots \\ \nabla h_n B_n^\top A_n^\top \end{pmatrix} \end{aligned}$$

Each adapter’s gradient is computed independently using its data slice, preserving task isolation.

3.4.4 Graph Pruning

mLoRA constructs optimised backward computational graphs rather than relying on PyTorch’s auto-generated graphs. The split operator derivative—which would normally allocate and copy a full gradient tensor—is pruned by directly adding ∇x_i to the corresponding positions of $\nabla Y = \nabla H W^\top$. This achieves up to 17% latency reduction and 7% peak memory reduction.

3.4.5 BatchLoRA-Enhanced LoRAPP

When the number of tasks $L > D$ (more adapters than GPUs), BatchLoRA groups the excess tasks: L/D tasks per GPU, maintaining the zero-bubble condition while multiplying effective throughput.

3.5 Task Scheduler

3.5.1 Priority Scheduling

mLoRA uses preemptive static-priority scheduling with FIFO tie-breaking. Scheduling decisions are made at the end of each training iteration, allowing high-priority tasks to preempt lower-priority ones promptly.

3.5.2 Memory Modelling

To maximise GPU utilisation without OOM, mLoRA fits an online non-linear memory model:

$$\text{Mem} = \beta_0 + \beta_1 B_t L_n + \beta_2 B_t L_n^2$$

where B_t is batch size and L_n is sequence length. Coefficients $\beta_0, \beta_1, \beta_2$ are updated online via a non-linear least-squares solver. The model achieves $\sim 0.25\%$ MAPE even with few data points, enabling safe over-subscription of GPU memory.

3.6 Experimental Setup

3.6.1 Hardware

Table 2: mLoRA hardware configurations

Setup	Hardware	Notes
Single-GPU	1× NVIDIA RTX A6000 (48GB)	fp32, PCIe 4.0
Single-machine multi-GPU	4× NVIDIA RTX A6000 (48GB)	PCIe 4.0×16
Multi-machine multi-GPU	8× NVIDIA GeForce RTX 3090 (24GB)	8 machines, 1Gbps network
Scalability test	8× NVIDIA RTX A6000 (48GB)	Single machine

3.6.2 Models and Workloads

Three LLaMA-series models at different scales:

- **TinyLlama-1.1B**: Small model, high kernel overhead ratio
- **LLaMA-2-7B**: Mid-size model
- **LLaMA-2-13B**: Large model, tests memory limits

Dataset: GSM8K (natural language generation, math word problems). Alpaca-LoRA defaults: batch size = 8, sequence length = 512, 10 epochs, LoRA rank = 16, applied to q_proj , k_proj , v_proj , o_proj .

3.6.3 Baselines

- **Single-GPU**: HuggingFace PEFT (with 8-bit quantisation and activation checkpointing)
- **Multi-GPU**: 1F1B (PipeDream-Flush), Tensor Parallelism (Megatron-LM), FSDP (PyTorch)

Data parallelism is excluded from multi-GPU comparison due to infeasibility under memory constraints.

3.6.4 Metrics

- **Average task completion time (hours):** Wall-clock time to finish a fine-tuning task
- **System throughput (tokens/second):** Total tokens processed per second across all tasks
- **Bubble ratio:** GPU idle time / total pipeline runtime
- **Communication volume (GB):** Per training step
- **GPU utilisation (%):** Measured with NVIDIA Nsight Systems

3.7 Key Results

Table 3: mLoRA performance summary (multi-GPU, single machine, 4×A6000)

Model	vs FSDP	vs TP	vs 1F1B
TinyLlama-1.1B	+75% throughput	+86% throughput	Faster
LLaMA-2-7B	+35% throughput	+58% throughput	−30% completion time
LLaMA-2-13B	FSDP: OOM	+46% throughput	Supported

- **Single-machine multi-GPU:** 30–45% reduction in average task completion time vs FSDP.
- **Multi-machine (1Gbps):** FSDP and TP cannot train 7B or 13B due to memory constraints; mLoRA achieves 30% savings vs 1F1B for 7B.
- **Single-GPU (BatchLoRA):** Up to 8% training time reduction; 10% kernel launch overhead eliminated for 1.1B model.
- **Linear scalability:** Throughput scales linearly from 2 to 8 GPUs.
- **Zero bubbles:** Achieved when $L \geq D$; mLoRA reaches 0.0 bubble ratio with 4 tasks on 4 GPUs.
- **Production:** 30% time savings in hyperparameter selection at Ant Group.

3.8 GitHub Repository Analysis

URL: <https://github.com/TUDB-Labs/mLoRA>

Stars: 376 **Forks:** 66 **License:** Apache 2.0 **Language:** Python 99.1%

Acceptance: VLDB 2025

3.8.1 Repository Structure

<code>mlora/</code>	Core library: trainer, operators, scheduler, parallelism
<code>mlora_cli/</code>	CLI client for interacting with mLoRA server
<code>benchmarks/</code>	Scripts to reproduce paper results (see <code>benchmarks/README.md</code>)
<code>demo/</code>	YAML configuration examples for LoRA variants
<code>scripts/</code>	Utility scripts
<code>docs/</code>	Design document, adapter development guide
<code>tests/</code>	Unit and integration tests
<code>mlora_train.py</code>	Entry point: single-machine training
<code>mlora_pp_train.py</code>	Entry point: pipeline parallel multi-node training
<code>mlora_server.py</code>	REST API server for continuous task submission
<code>Dockerfile</code>	Docker image (also available: <code>yezhengmaolove/mlora:latest</code>)

3.8.2 Supported Models and LoRA Variants

Base Models:

- LLaMA / LLaMA-2 / LLaMA-3

- VeRA (ICLR 2024)
- DoRA (ICML 2024)

LoRA Variants:

- QLoRA (NeurIPS 2023)
- LoRA+ (ICML 2024)

RLHF Algorithms:

- DPO (NeurIPS 2024)
- CPO (ICML 2024)
- CIT (arXiv 2024)

3.8.3 Deployment Modes

1. **Batch training** (`mlora_train.py`): Fine-tune multiple adapters from a YAML config.
2. **Pipeline parallel training** (`mlora_pp_train.py`): Distributed across nodes with configurable layer balance and precision.
3. **Server mode** (`mlora_server.py`): Persistent REST service; adapters submitted dynamically via `mlora_cli`.

3.8.4 Reproduction Instructions

The `benchmarks/README.md` provides step-by-step instructions to reproduce all paper results. A Docker image is available to avoid environment setup. The benchmark suite uses GSM8K with the Alpaca-LoRA hyperparameter defaults.

4 Paper 2: tLoRA

4.1 Overview and Core Contributions

Venue: arXiv:2602.07263v2, February 2026

Affiliation: Siebel School of Computing and Data Science, UIUC

Hardware: 12× NVIDIA A100 GPUs (micro-benchmarks) + 128-GPU cluster simulator

Trace: ACMETrace (production GPU cluster trace)

tLoRA advances the multi-LoRA training problem by addressing a gap that mLoRA does not: *job heterogeneity*. Real-world LoRA training workloads differ in adapter rank ($r \in \{2, 4, 8, 16\}$), batch size, sequence length, and GPU allocation. tLoRA makes four primary contributions:

1. **Shared Super-Model (SSM) abstraction:** Unifies heterogeneous LoRA jobs into a single composite computation graph exposed to existing parallelism planners.
2. **Fused heterogeneous LoRA kernel (Kernel Fuser):** A Triton-based kernel that avoids materialising full adapter weight matrices and uses rank-aware block tiling.
3. **Adaptive nano-batching (AIMD controller):** Dynamically partitions batches into nano-batches to maximise compute-communication overlap.
4. **Residual-capacity-aware Adapter Scheduler:** Online, hierarchical, $O(K \log K)$ grouping algorithm that pairs complementary jobs while enforcing per-job slowdown bounds.

4.2 Key Motivation: Naïve Batching Hurts

tLoRA’s motivation is empirically grounded: naïve batching of heterogeneous jobs can *decrease* aggregate throughput. For example (Llama-3.1-8B):

- Jobs 1+3 (bs=2,r=8 with bs=4,r=8): isolated throughput $0.74+1.09=1.83 \rightarrow$ batched 2.36 (+29%)
- Jobs 1+2 (bs=2,r=8 with bs=2,r=4): batched throughput **decreases** vs. isolated

This asymmetry arises from rank/batch-induced imbalance in pipeline stages and amplified cross-node communication.

4.3 Model Fuser: Shared Super-Model

4.3.1 SSM Construction

Given K jobs $\mathcal{J} = \{J_1, \dots, J_K\}$ derived from a base model M , the Model Fuser:

1. Performs *layer-wise architectural fusion* on shared backbone operators (e.g., fusing multiple attention layers that share the same weight matrix).
2. Retains adapter branches as *lightweight, job-specific* subgraphs attached to the fused backbone.
3. Presents the SSM to existing parallelism planners (PyTorch Distributed, Megatron-LM, Metis).

The SSM is *functionally equivalent* to training each job independently—convergence and correctness are preserved. But it exposes more structure: planners see the full heterogeneity (different batch sizes, ranks) and can account for it in partitioning and placement decisions, unlike running separate jobs.

4.3.2 Integration with Existing Frameworks

Because tLoRA presents the SSM as a standard composite model, no modifications to existing distributed training stacks are required. Layer-wise profiling and cost modelling from frameworks like Metis [18] automatically internalise load heterogeneity when estimating per-layer compute costs.

4.4 Kernel Fuser: Minimising Runtime Bubbles

4.4.1 Design Rationale

Two naïve extremes exist for executing heterogeneous adapters:

- *Per-adapter kernels:* K separate kernel launches; high launch overhead, poor SM occupancy.

- *Single super-kernel*: Irregular block-sparse layout with zero regions; wastes compute and memory bandwidth.

tLoRA introduces a **fused batched LoRA kernel** that avoids both:

4.4.2 Kernel Execution Model

For each adapter i , tokens mapped to that adapter are:

1. **Gathered** and multiplied with the down-projection A_i : intermediate shape $(|X_i|, r_i)$.
2. **Multiplied** with up-projection B_i immediately (without materialising $W_i = A_i B_i^\top$).
3. **Scattered** back to the output tensor.

This avoids allocating full-sized temporary buffers and improves register and shared-memory reuse. The kernel is implemented in Triton with auto-tuning for adapter-aware block sizes.

4.4.3 Adaptive Nano-Batching

To overlap computation and communication, tLoRA introduces a *nano-batch* abstraction. The combined batch is split into N nano-batches of size $\sum_i B_i / N$ along the batch dimension. The iteration time model is:

$$T_{\text{iter}} \approx \max \left(\sum_{n=1}^N T_{\text{comp}}(n), \sum_{n=1}^N T_{\text{comm}}(n) \right)$$

Optimal N balances compute and communication. tLoRA determines N online using an **AIMD (Additive-Increase/Multiplicative-Decrease) controller**:

$$N_{t+1} = \begin{cases} N_t + \alpha & \text{if } T_t \leq T_{t-1} - \tau \\ \max\{1, \lfloor \beta N_t \rfloor\} & \text{otherwise} \end{cases}$$

with $\alpha = 4$ (additive step), $\beta = 1/2$ (backoff factor), τ a noise margin. Convergence is $O(\log N)$ steps—negligible relative to thousands of training iterations.

4.5 Adapter Scheduler: Cluster-Wide Throughput

4.5.1 Problem Formulation

The scheduler solves:

$$\max_{G \subseteq \mathcal{J}} \hat{T}(G) \quad \text{s.t.} \quad \forall j \in G, \Delta_j(G) \leq \Delta_j^{\max}$$

where $\hat{T}(G)$ is joint throughput and $\Delta_j(G)$ is the slowdown of job j in group G . Exhaustive search is exponential in K ; tLoRA achieves $O(K \log K)$ via hierarchical incremental grouping.

4.5.2 Hierarchical Incremental Grouping

1. Sort jobs by urgency (desc) and residual resource capacity (asc).
2. Starting from the most resource-constrained job, binary-cut search on the sorted list to find the cutoff where adding more jobs no longer improves joint efficiency.
3. Form the group; reinsert into queue; repeat until no beneficial merges remain.
4. Grouping proceeds bottom-up: first within nodes, then across nodes, then across ranks—each tier adds communication overhead.

4.5.3 Per-Job Fairness

Each job has an urgency score $\Delta_j(G)$ reflecting proximity to violating its progress constraint (slowdown bound). High-urgency jobs are prioritised and placed first in the grouping queue, where they are more likely to be paired with resource-abundant (low-residual) jobs that can accelerate them. This *elastic contribution* allows jobs to temporarily yield unused resources and later reclaim more than their nominal share to meet deadlines.

4.6 Experimental Setup

4.6.1 Evaluation Methodology

tLoRA uses a two-level approach:

- **Micro-benchmarks on real hardware:** 12× NVIDIA A100 GPUs; per-job speed profiles collected.
- **Large-scale trace-driven simulation:** Production cluster traces from ACMETrace [16] with 128-GPU default cluster; simulator error within 3% (validated in Figure 10 of the paper).

4.6.2 Workload Generation

Jobs are drawn from ACMETrace (multi-tenant production trace). LoRA configurations are overlaid:

- Rank: randomly sampled from $\{2, 4, 8, 16\}$
- Batch size: randomly sampled from $\{1, 2, 4, 8\}$
- Base models: Llama-3-8B or Qwen-3-8B (random per job)
- Dataset: GSM8K (grade-school math, ~8.5k questions)

4.6.3 Baselines

- **mLoRA:** State-of-the-art batched LoRA training; groups by memory capacity (FIFO).
- **Megatron-LM:** Highly optimised distributed training; trains each job independently.
- **tLoRA w/o Scheduler:** Replaces Adapter Scheduler with mLoRA’s batching policy.
- **tLoRA w/o Kernel Fuser:** Disables fused heterogeneous LoRA kernel.

4.6.4 Metrics

- **Training throughput (samples/s):** Cluster-wide aggregate
- **Job Completion Time (JCT):** Wall-clock from submission to training completion (includes queueing)
- **GPU utilisation (%):** Average across all GPUs

4.7 Key Results

Table 4: tLoRA performance summary vs baselines (128-GPU cluster, online workloads)

Metric	vs mLoRA	vs Megatron	Absolute
Training throughput	+41%	Higher	1.2–1.8× improvement
Job Completion Time	5.4× faster	Faster	2.3–5.4×
GPU SM utilisation	+37%	Higher	63.6% vs 54.4% (mLoRA)

- mLoRA *often underperforms Megatron* despite batching, because it ignores communication overhead from co-location. tLoRA avoids this by accounting for heterogeneity in the SSM.
- Small and large jobs are predominantly grouped (92.1% and 96.4% co-location rates) due to complementary resource usage. Medium-sized jobs have lower grouping ratios (67.8%) as they have limited residual capacity.
- The AIMD nano-batch controller consistently outperforms manually tuned fixed nano-batch sizes across all tested values ($N \in \{1, 2, 4, 8\}$).
- Throughput improvements are robust across load factors ($0.5\times$ – $5\times$) and cluster sizes (32–256 GPUs).

5 Comparative Analysis

5.1 Design Philosophy

Table 5: Side-by-side comparison of mLoRA and tLoRA

Dimension	mLoRA	tLoRA
Core insight	Adapter independence $\Rightarrow$ zero pipeline bubbles	Residual resource complementarity $\Rightarrow$ beneficial grouping
Job heterogeneity	Assumed homogeneous (same rank, batch size)	First-class: rank, batch size, GPU count differ
Parallelism	Novel LoRAPP; no existing framework dependency	Leverages existing frameworks via SSM abstraction
GPU kernel	BatchLoRA (PyTorch custom op)	Triton fused kernel with block-aware tiling
Scheduling	Preemptive priority + memory model	Urgency-based hierarchical incremental grouping
Communication overlap	CUDA stream overlap (recv/send/compute)	Nano-batch pipelining (AIMD-controlled)
Hardware	4–8 consumer/workstation GPUs	12 A100s (micro-bench) + 128-GPU cluster
Evaluation	Real hardware only	Real hardware + trace-driven simulation
Deployment	Production at Ant Group	Evaluated on ACMETrace (production traces)
Open source	Yes (Apache-2.0, 376 stars)	No public repository found
Venue	VLDB 2025	arXiv 2026

5.2 Complementarity

The two systems are largely complementary rather than competing:

- **mLoRA targets the parallelism mechanism** (how to distribute layers across GPUs) and the **operator level** (how to batch adapter computations on one GPU).
- **tLoRA targets the scheduling layer** (which jobs to group) and the **kernel level** (how to fuse heterogeneous adapters in a Triton kernel).
- tLoRA explicitly uses mLoRA as a baseline and shows that mLoRA’s FIFO memory-based grouping is suboptimal under heterogeneity—even sometimes worse than running jobs independently.
- A combined system could use tLoRA’s SSM-based parallelism planning and scheduling with mLoRA’s LoRAPP for the underlying pipeline execution.

5.3 Limitations

mLoRA limitations:

- Assumes homogeneous adapter configurations (same rank, batch size); cannot exploit complementary resource usage.

- FIFO memory-based grouping ignores communication overhead, which can lead to throughput regression under heterogeneous workloads.
- Limited base model support (LLaMA family only in current release).
- Multi-machine evaluation uses artificially low 1Gbps bandwidth, which may not represent modern InfiniBand clusters.

tLoRA limitations:

- No open-source implementation available (as of May 2026).
- Large-scale evaluation relies on simulation (simulator error within 3%, but not zero).
- The SSM approach requires a compatible parallelism planner; integration with arbitrary frameworks may need engineering effort.
- The AIMD controller may converge slowly if the optimal nano-batch size changes rapidly (e.g., highly dynamic arrival patterns).

6 Benchmarks and Datasets

6.1 Datasets Used

Table 6: Datasets used in evaluation

Dataset	Task	Size	Used by
GSM8K	Math word problems (NLG)	8.5k questions	mLoRA, tLoRA
Stanford Alpaca	Instruction following	52k examples	mLoRA (server demo)
ACMETrace	Production cluster trace	Multiple months	tLoRA

6.2 Performance Measurement Tools

- **NVIDIA Nsight Systems:** Used by mLoRA to measure kernel launch times and kernel execution times.
- **Sailor training simulator** [17]: Used by tLoRA for large-scale trace-driven simulation; validated within 3% error.
- **SciPy non-linear least squares:** Used by mLoRA’s scheduler for online memory model fitting.

6.3 Hyperparameter Configurations

Both papers use Alpaca-LoRA defaults for fair comparison:

- Batch size: 8 (mLoRA); varies by job in tLoRA ($\{1, 2, 4, 8\}$)
- Sequence length: 512
- Epochs: 10
- LoRA rank: 16 (mLoRA); $\{2, 4, 8, 16\}$ in tLoRA
- LoRA target modules: q_proj , k_proj , v_proj , o_proj

- Precision: fp32 (mLoRA multi-GPU); fp32 or 8-bit quant (mLoRA single-GPU)

7 Integration with Modern Systems

7.1 mLoRA Integration

7.1.1 PyTorch Ecosystem

mLoRA is built natively on PyTorch and integrates with:

- **HuggingFace PEFT**: mLoRA’s BatchLoRA operator can replace PEFT’s per-adapter training loop for a drop-in speedup of up to 8%.
- **HuggingFace Transformers**: Models loaded from Hugging Face Hub (via model path or TinyLlama/TinyLlama-1.1B-Chat-v0.4).
- **PyTorch Distributed**: Used for inter-GPU communication (point-to-point via CUDA streams for LoRAPP).

7.1.2 RLHF and Alignment

mLoRA supports DPO, CPO, and CIT—enabling its use not just for supervised fine-tuning but for reinforcement learning from human feedback pipelines. This makes it directly applicable to modern LLM alignment workflows.

7.1.3 Serving Integration

The mLoRA server (`mlora_server.py`) exposes a REST API compatible with existing orchestration tools. The CLI client (`mlora_cli`) allows submitting and managing tasks programmatically, fitting into MLOps pipelines.

7.1.4 Relation to LoRA Serving Systems

mLoRA complements LoRA serving systems (Punica [9], S-LoRA [8], dLoRA [10]): those systems optimise *inference*, while mLoRA optimises *training*. The adapter weights trained by mLoRA can be directly served by these systems.

7.2 tLoRA Integration

7.2.1 Framework Agnosticism

The SSM abstraction is explicitly designed to work with any existing parallelism planner:

- **PyTorch FSDP**: Can be used as the underlying execution backend.
- **Megatron-LM**: tLoRA presents the SSM as a single model to Megatron’s tensor parallelism planner.
- **Metis** [18]: Fast heterogeneous GPU training; used as an example parallelism planner in the paper.

7.2.2 Triton Kernel

The Kernel Fuser is implemented in Triton 3.5.1, making it compatible with any hardware supported by Triton (NVIDIA GPUs). Triton’s auto-tuning selects adapter-aware block sizes, reducing manual tuning effort.

7.2.3 Cluster Management

tLoRA’s scheduler is designed for *online, multi-tenant cluster environments* where jobs arrive dynamically. It integrates with:

- **ACMETrace-style traces:** Real production cluster scheduling (job arrival, GPU allocation, execution duration).
- **Sailor simulator** [17]: For pre-deployment evaluation without risking production clusters.

7.3 Comparison with Adjacent Systems

Table 7: Positioning relative to adjacent systems

System	Focus	Relation
S-LoRA [8]	LoRA <i>serving</i> ; thousands of adapters	Complementary (different phase)
dLoRA [10]	Dynamic LoRA serving	Complementary
Punica [9]	Multi-tenant LoRA inference	Complementary
GPipe [4]	Pipeline parallelism (single model)	Superseded by LoRAPP for multi-LoRA
PipeDream/1F1B	Memory-efficient pipeline PP	Baseline in mLoRA
Megatron-LM	Tensor parallelism	Baseline in both; integrated by tLoRA
FSDP	Sharded data parallelism	Baseline in mLoRA; memory-limited
NanoFlow [19]	Intra-device serving overlap	Inspiration for nano-batching in tLoRA
MixLoRA [14]	MoE-based LoRA	LoRA variant; supported by mLoRA

8 Literature Survey Context

8.1 PEFT Methods Landscape

The papers operate in the context of a rich PEFT literature:

- **LoRA variants:** AdaLoRA [12] (adaptive rank), SoRA (sparse LoRA), DoRA [13] (weight-decomposed), MixLoRA [14] (MoE-based), MoeLoRA.
- **Prompt-based:** Prefix-tuning, Prompt-tuning, P-Tuning v2.
- **Selective:** Diff pruning, BitFit.
- **Few-shot:** $(IA)^3$.

Both papers focus on standard LoRA due to its wide adoption, but the system-level techniques (pipeline parallelism, batching) generalise to all LoRA-based methods since they share the same one-base-model-many-adapters structure.

8.2 Pipeline Parallelism Evolution

- **GPipe:** Split mini-batch into micro-batches; bubble ratio $(D - 1)/(N + D - 1)$.
- **PipeDream:** 1F1B scheduling; relaxes gradient staleness.
- **PipeDream-Flush (1F1B):** Memory-efficient variant; used as baseline.

- **PipeMare:** Asynchronous pipeline; convergence concerns.
- **LoRAPP (mLoRA):** Exploits adapter independence for zero bubbles; no staleness.

8.3 GPU Kernel Optimisation

- **CUDA Graphs:** Batch multiple GPU operations into a single CPU call; does not support dynamic shapes.
- **Deep learning compilers (TVM, torch.compile):** Operator fusion; limited dynamic shape support.
- **BatchLoRA (mLoRA):** Consolidates data at the framework level; handles variable-length sequences.
- **Triton fused kernel (tLoRA):** Block-sparse execution without materialising intermediate tensors; rank-aware auto-tuning.

8.4 Multi-Tenant LLM Serving and Training

The serving literature (S-LoRA, dLoRA, Punica) focuses on batching *inference* requests. The training counterpart—batching *training* jobs—is the domain of mLoRA and tLoRA. Key differences:

- Training requires backward propagation and optimiser states, not just forward passes.
- Training involves gradient accumulation and synchronisation—absent in inference.
- The split operator derivative (a training-only concern) creates redundant memory operations that tLoRA’s SSM addresses.

8.5 Research Gaps and Future Directions

Based on this survey, several open problems remain:

1. **Heterogeneous hardware:** Neither system fully addresses mixed GPU types (e.g., A100 + RTX 3090 in the same group).
2. **Dynamic LoRA variants:** AdaLoRA changes adapter rank during training; this conflicts with static SSM construction.
3. **Combined system:** Integrating tLoRA’s heterogeneity-aware scheduler with mLoRA’s LoRAPP pipeline is an open engineering challenge.
4. **Serving-training co-design:** Systems that seamlessly transition adapters between training and serving without interruption.
5. **Broader base model support:** Extending beyond LLaMA to Gemma, Mistral, Qwen, Phi-3 families.
6. **Communication-aware batching:** Joint optimisation of adapter grouping and network topology (e.g., NVLink vs PCIe vs InfiniBand).

9 Summary

This survey has examined two systems that advance the state of the art in multi-LoRA fine-tuning:

mLoRA introduces LoRAPP—a zero-bubble pipeline parallelism scheme grounded in the independence of LoRA adapters—and BatchLoRA, a consolidated GPU operator that reduces kernel launch overhead. The system is open-source, deployed in production (Ant Group), accepted at VLDB 2025, and achieves 30–45% task completion time reduction over FSDP. Its key strength is enabling large-model fine-tuning (13B) on commodity workstation GPUs (4×48GB) that FSDP cannot handle.

tLoRA addresses the heterogeneous multi-tenant training problem overlooked by mLoRA. By introducing the SSM abstraction, a Triton fused kernel with AIMD nano-batching, and a residual-capacity-aware hierarchical scheduler, tLoRA achieves 1.2–1.8× throughput improvement and 2.3–5.4× job completion time reduction vs mLoRA and Megatron under realistic production traces. Its elastic contribution mechanism and fairness guarantees make it suitable for shared cluster deployments.

Together, these papers represent the emerging systems frontier for multi-LoRA training: from low-level GPU kernel efficiency to cluster-level scheduling under workload heterogeneity.

References

- [1] Ye, Z., Li, D., Hu, Z., Lan, T., Sha, J., Zhang, S., Duan, L., Zuo, J., Lu, H., Zhou, Y., & Tang, M. *mLoRA: Fine-Tuning LoRA Adapters via Highly-Efficient Pipeline Parallelism in Multiple GPUs*. VLDB 2025. arXiv:2312.02515.
- [2] Li, K., Saha, D., Kanodia, A., & Lai, F. *tLoRA: Efficient Multi-LoRA Training with Elastic Shared Super-Models*. arXiv:2602.07263, February 2026.
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., & Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
- [4] Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H. J., Ngiam, J., Le, Q. V., Wu, Y., et al. *GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism*. NeurIPS 2019.
- [5] Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., et al. *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*. SC 2021.
- [6] Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., et al. *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. VLDB 2023.
- [7] Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., & Bossan, B. *PEFT: State-of-the-Art Parameter-Efficient Fine-Tuning Methods*. <https://github.com/huggingface/peft>, 2022.
- [8] Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., et al. *S-LoRA: Serving Thousands of Concurrent LoRA Adapters*. MLSys 2024. arXiv:2311.03285.
- [9] Chen, L., Ye, Z., Wu, Y., Zhuo, D., Ceze, L., & Krishnamurthy, A. *Punica: Multi-Tenant LoRA Serving*. arXiv:2310.18547, 2023.
- [10] Wu, B., Zhu, R., Zhang, Z., Sun, P., Liu, X., & Jin, X. *dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving*. OSDI 2024.

- [11] Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N. R., Ganger, G. R., Gibbons, P. B., & Zaharia, M. *PipeDream: Generalized Pipeline Parallelism for DNN Training*. SOSP 2019.
- [12] Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., & Zhao, T. *AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning*. ICLR 2023.
- [13] Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. *DoRA: Weight-Decomposed Low-Rank Adaptation*. ICML 2024. arXiv:2402.09353.
- [14] Li, D., Ma, Y., Wang, N., Cheng, Z., Duan, L., Zuo, J., Yang, C., & Tang, M. *MixLoRA: Enhancing Large Language Models Fine-Tuning with LoRA Based Mixture of Experts*. arXiv:2404.15159, 2024.
- [15] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. *Training Verifiers to Solve Math Word Problems*. arXiv:2110.14168, 2021.
- [16] Hu, Q., Ye, Z., Wang, Z., Wang, G., Zhang, M., Chen, Q., Sun, P., Lin, D., Wang, X., Luo, Y., Wen, Y., & Zhang, T. *Characterization of Large Language Model Development in the Datacenter*. NSDI 2024. arXiv:2403.07648.
- [17] Strati, F., Zhang, Z., Manos, G., Sánchez Pérez, I., Hu, Q., Chen, T., et al. *Sailor: Automating Distributed Training over Dynamic, Heterogeneous, and Geo-Distributed Clusters*. SOSP 2025. arXiv:2504.17096.
- [18] Um, T., Oh, B., Kang, M., Lee, W.-Y., Kim, G., Kim, D., Kim, Y., Muzzammil, M., & Jeon, M. *Metis: Fast Automatic Distributed Training on Heterogeneous GPUs*. ATC 2024.
- [19] Zhu, K., Zhao, Y., Zhao, L., Zuo, G., Gu, Y., Xie, D., et al. *NanoFlow: Towards Optimal Large Language Model Serving Throughput*. OSDI 2025. arXiv:2408.12757.
- [20] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., et al. *LLaMA 2: Open Foundation and Fine-Tuned Chat Models*. arXiv:2307.09288, 2023.
- [21] Zhang, P., Zeng, G., Wang, T., & Lu, W. *TinyLlama: An Open-Source Small Language Model*. arXiv:2401.02385, 2024.

LoRA Systems at Scale: A Deep-Dive Literature Survey

*Covering Serving Heterogeneous LoRA Adapters (LoRAServe)
and Efficient LoRA Fine-Tuning (LoRAFusion)*

Papers Covered

Paper 1: *Serving Heterogeneous LoRA Adapters in
Distributed LLM Inference Systems*
Jaiswal et al. (arXiv:2511.22880, Nov 2025)

Paper 2: *LoRAFusion: Efficient LoRA Fine-Tuning for LLMs*
Zhu et al. (EuroSys 2026, Apr 2026)

Prepared for graduate-level systems research review
May 2026

Contents

1	Introduction and Motivation	3
2	Paper 1: LoRAServe — Serving Heterogeneous LoRA Adapters	3
2.1	Overview and Problem Statement	3
2.1.1	The Core Problem	3
2.1.2	Production Workload Characterization	4
2.2	Core Novelty and Technical Contributions	5
2.2.1	LoRAServe Architecture	5
2.2.2	Adapter Placement Algorithm (Algorithm 1)	5
2.2.3	Distributed Adapter Pool via GPUDirect RDMA	6
2.3	Experimental Setup	6
2.3.1	Hardware	6
2.3.2	Software Stack	6
2.3.3	Workload Traces	7
2.3.4	Baselines	7
2.4	Results Summary	7
2.4.1	Main Performance Results (Production Trace)	8
2.4.2	Scalability	8
2.4.3	Sensitivity Studies	8
2.4.4	Why Toppings Falls Short	8
2.5	Integration with Modern Systems	8
3	Paper 2: LoRA Fusion — Efficient LoRA Fine-Tuning	9
3.1	Overview and Problem Statement	9
3.1.1	The Core Problem	9
3.2	Core Novelty and Technical Contributions	10
3.3	FusedLoRA and FusedMultiLoRA Kernels	10
3.3.1	The Fusion Design Problem	10
3.3.2	Graph-Splitting Strategy	10
3.3.3	FusedMultiLoRA	10
3.4	Multi-LoRA Scheduler	11
3.4.1	Problem Formulation	11
3.4.2	Two-Stage Scheduling Workflow	11
3.4.3	Pipeline Bubble Reduction	12
3.5	Experimental Setup	12
3.5.1	Hardware	12
3.5.2	Software Stack	12
3.5.3	Models	13
3.5.4	Datasets	13
3.5.5	Multi-LoRA Configuration	13
3.5.6	Baselines	14
3.6	Results Summary	14
3.6.1	End-to-End Speedup	14
3.6.2	Speedup Breakdown (LLaMa-3.1-70B, 4 H100 GPUs)	14

3.6.3 Scalability	15
3.7 GitHub Repository Analysis	15
4 Comparative Analysis	15
4.1 Side-by-Side Comparison	16
4.2 Complementarity	16
4.3 Shared Themes	16
5 Related Work and Positioning	17
5.1 LoRA Algorithm and Variants	17
5.2 LoRA Serving Systems	17
5.3 LoRA Training Systems	17
5.4 Broader LLM Systems Context	17
6 Benchmarks and Evaluation Methodology	18
6.1 LoRAServe Benchmarks	18
6.2 LoRA Fusion Benchmarks	19
6.3 Metrics Used	19
7 Limitations and Open Questions	20
8 Future Directions and Research Opportunities	20
9 Quick Reference for Literature Survey Integration	21
9.1 Citing These Papers	22
9.2 One-Paragraph Summary: LoRAServe	22
9.3 One-Paragraph Summary: LoRA Fusion	22
10 Conclusion	23

§1. Introduction and Motivation

Low-Rank Adaptation (LoRA) has become the dominant method for parameter-efficient fine-tuning (PEFT) of large language models (LLMs). Rather than updating all model parameters, LoRA inserts a pair of trainable low-rank matrices ($A \in \mathbb{R}^{k \times r}$, $B \in \mathbb{R}^{r \times n}$) alongside frozen pre-trained weights W , yielding the modified forward pass:

$$Y = XW + \alpha(XA)B, \quad r \ll \min(k, n)$$

This reduces trainable parameters by orders of magnitude—for GPT-3 (175B), LoRA reduces trainable parameters by over $100\times$ and GPU memory by $3\times$ —while maintaining comparable quality.

However, LoRA’s operational efficiency at *inference time* and *training time* raises distinct systems challenges:

- **Serving:** Production deployments host hundreds to thousands of adapters on shared base models. Adapters differ in *rank* (and thus computational cost), and co-batching heterogeneous ranks causes smaller adapters to suffer severe performance degradation.
- **Training:** Despite LoRA’s small parameter footprint, existing LoRA fine-tuning systems incur $\sim 40\%$ throughput reduction compared to the frozen-weight baseline, due to redundant memory accesses and missed multi-job co-scheduling opportunities.

These two papers each address one side of the problem. **LoRAServe** (Jaiswal et al.) proposes a rank-aware, workload-adaptive serving framework targeting inference SLOs, while **LoRAFusion** (Zhu et al.) introduces kernel-level fusion and multi-job scheduling to accelerate LoRA fine-tuning. Together, they form complementary contributions to the LoRA systems stack.

§2. Paper 1: LoRAServe — Serving Heterogeneous LoRA Adapters

2.1. Overview and Problem Statement

Citation: Shashwat Jaiswal, Shrikara Arun, Anjaly Parayil, Ankur Mallick, Spyros Mastorakis, Alind Khare, Chloi Alverti, Renee St Amant, Chetan Bansal, Victor Rühle, Josep Torrellas. *Serving Heterogeneous LoRA Adapters in Distributed LLM Inference Systems*. arXiv:2511.22880, 2025.

Affiliations: University of Illinois Urbana-Champaign; Microsoft Research; National Technical University of Athens.

Venue: Preprint (arXiv). The work draws on production traces and deployment from Company X (a major cloud provider, presumably Microsoft Azure).

2.1.1. The Core Problem

In production, LLM inference clusters serve *hundreds* of LoRA adapters from a single base model. Each adapter has two sources of heterogeneity:

1. **Rank heterogeneity:** Adapters of different ranks impose different compute and memory costs. CUDA kernels for multi-tenant LoRA serving (Punica’s BGMV, S-LoRA’s MBGMV)

size their compute tiles to the *maximum rank in the batch*. Consequently, low-rank requests in a mixed batch must execute as if they were high-rank—incurring unnecessary FLOPs and memory traffic.

2. **Traffic heterogeneity:** Different adapters receive wildly different request rates (heavy-tailed distribution), and these rates drift over time.

Empirically, co-serving a rank-8 adapter with a rank-128 adapter on a Llama-7B instance increases rank-8 P95 TTFT by **84%**. On Llama-70B, rank heterogeneity causes up to **45% performance degradation**.

Key Observation: Kernel-Level Root Cause

Both Punica (BGMV) and S-LoRA (MBGMV) kernels inherit a fundamental constraint: the fused GEMM tile dimensions and MMA pipelines are set by the **maximum LoRA rank** in the batch. Low-rank requests are effectively *padded* to the size of the largest adapter, inflating register usage, shared memory requirements, and tile dimensions. This is the architectural root cause of rank-induced interference that no scheduling policy can eliminate without rank-aware placement.

2.1.2. Production Workload Characterization

The paper presents a thorough characterization of Company X’s production LoRA serving workloads for the week of October 22–29, 2025:

- **Scale:** Hundreds of adapters per popular base model. Over 1,000 adapters total deployed in production.
- **Heavy-tailed popularity:** The top 5 adapters account for **70%+** of all traffic for a given base model. The remaining >500 adapters each receive less than 1% of requests.
- **Memory pressure:** A 200B parameter model quantized to 8-bits requires ~200 GB. LoRA adapters are ~1% of model size (~2 GB each), so storing 500 adapters per server would require 1 TB—*infeasible* on every server.
- **Regional concentration:** Capacity is concentrated in a few models and geographic regions due to data boundary constraints (fine-tuned on customer data).
- **Temporal drift:** Adapter request rates show distinct temporal patterns—diurnal patterns, gradual drifts, and sudden surges—making static allocation suboptimal.

Note

This characterization is one of the more valuable contributions of the paper as a standalone artifact: it provides concrete data from a real-world production deployment that validates the design choices in LoRAServe and motivates the broader research direction.

2.2. Core Novelty and Technical Contributions

Core Novelty

LoRAServe introduces a **rank-aware, workload-adaptive cluster orchestrator** for LoRA serving that jointly optimizes (1) adapter placement across LLM inference servers to minimize rank heterogeneity within each server, and (2) distributed adapter pooling via GPUDirect RDMA over InfiniBand for on-demand remote access.

2.2.1. LoRAServe Architecture

The system architecture has two main components:

Cluster Orchestrator. A central orchestrator maintains:

- A **routing table**: tuples of (adapter_id, server_id, ϕ) where ϕ is the fractional routing probability ($\sum \phi = 1$ per adapter).
- An **adapter location map**: tracks which servers hold which adapters in their host DRAM.

Server-Local Components. Each LLM inference server runs:

- A scheduler and metadata store.
- A local adapter cache in host (CPU) memory.
- The LLM inference engine (built on S-LoRA).

The request lifecycle: incoming request $\rightarrow$ orchestrator routes to server $\rightarrow$ server checks local adapter cache $\rightarrow$ if miss, fetches via GPUDirect RDMA from InfiniBand $\rightarrow$ inference executes $\rightarrow$ result returned.

2.2.2. Adapter Placement Algorithm (Algorithm 1)

The placement algorithm runs at every configurable timestep and proceeds in six stages:

Step 1 — Demand Estimation. For each adapter, estimate the current TPS (tokens per second) demand by extrapolating from recent request history:

$$\text{demandTPS}[a] = \text{EXTRAPOLATE}(\text{TPSHistory}[a], \text{prevTPS}[a])$$

Step 2 — Server Budget per Rank. Compute the total utilization per rank and divide by target utilization per server:

$$\text{targetUtil} = \frac{\sum_r \text{rankUtil}[r]}{|\text{servers}|}$$

$$\text{serverBudget}[r] = \text{round}\left(\frac{\text{rankUtil}[r]}{\text{targetUtil}}\right)$$

This determines how many servers should be dedicated to each rank group.

Step 3 — Fractional Bin Packing. Adapters of ranks with nonzero server budget are packed onto their allocated servers using fractional bin packing, respecting the target utilization.

Step 4 — Overflow Allocation. Remaining unassigned adapters (low-demand, misc ranks) are preferentially placed on servers with the highest maximum rank already present, minimizing incremental interference.

Step 5 — Assignment Permutation. Permute the new placement to minimize deviation from the previous assignment, reducing unnecessary adapter migration.

Step 6 — Routing and Metadata Update. Propagate the new routing table to all servers and update local adapter metadata.

2.2.3. Distributed Adapter Pool via GPUDirect RDMA

A key enabling technology is **GPUDirect RDMA over InfiniBand**. Key insight: fetching an adapter from a remote server over InfiniBand incurs *similar latency* to fetching from the local host’s memory-to-GPU channel. The paper benchmarks this directly (Fig. 14) and shows that InfiniBand transfer latency/bandwidth is comparable to local CPU-to-GPU transfers, making remote fetches viable without adding significant overhead. This is compared against local SSD storage, which has prohibitive latency and bandwidth for this use case.

The adapter lifecycle for migration:

1. Orchestrator changes routing for adapter A_3 : $(S_2, 1.0) \rightarrow (S_1, 0.7), (S_2, 0.3)$.
2. Next request for A_3 routed to S_1 .
3. S_1 detects cache miss, fetches A_3 from S_2 via RDMA.
4. Adapter stored in S_1 ’s host memory for future requests.
5. Orchestrator updates its location table deterministically.

2.3. Experimental Setup

2.3.1. Hardware

Table 1: LoRAServe Experimental Hardware

Resource	Specification
Compute (primary)	Standard ND96asr v4 VMs (Azure ML); $8 \times$ Nvidia A100 SXM 40GB per node; AMD EPYC 7V12 (96 cores); 900 GiB host RAM; 6 TiB disk
Compute (secondary)	Standard NC24ads A100 v4 VMs; $1 \times$ Nvidia A100 80GB; AMD EPYC 7V13 (24 cores); 220 GiB RAM
Interconnect	InfiniBand + Azure Accelerated Networking
Intra-node	NVLINK 3.0

2.3.2. Software Stack

- Built on top of **S-LoRA** (serving engine).
- Underlying CUDA kernels: Punica (BGMV) / S-LoRA (MBGMV), kept *identical* across all baselines for fair comparison.
- Base model: **Llama-7B** (primary), with sensitivity experiments on Llama-30B and Llama-70B.

- Tensor parallelism: TP=4 (default), with TP=2 and TP=8 in ablations.
- Cluster sizes: 4 servers (primary), scalability to 8 and 12 servers.

2.3.3. Workload Traces

Production trace (primary): 250,138 requests over 8 hours, 5 adapters of different ranks from Company X production services. Adapter counts (50/100/200) follow a power-law distribution ($\alpha = 1$).

Open-source trace (secondary): Azure Public Dataset (2024), annotated with timestamps (Poisson or uniform) and adapter names. 25 adapters of ranks 8, 16, 32, 64, 128. Three popularity distributions:

1. **Uniform:** equal traffic across all ranks.
2. **Shifting Skew:** rank-128 gets 50% traffic initially, shifting linearly so rank-8 gets 50% by end.
3. **Exponential:** power-law with smaller ranks more popular ($y = x^{-\alpha}$).

Combined with Uniform and Poisson arrival patterns, this gives $2 \times 3 = 6$ synthetic traces.

2.3.4. Baselines

Table 2: LoRAServe Baselines

Baseline	Placement	Description
S-LoRA Random	Random	Adapters randomly assigned to servers; mirrors Company X’s current deployment.
S-LoRA Contiguous	Rank-sorted	Adapters sorted by rank and placed contiguously; minimizes within-server rank spread but ignores traffic.
Toppings [7]	Request-level routing	State-of-the-art load-aware scheduler; routes each request globally to the optimal server considering queued/active requests; rank-agnostic.
LoRAServe	Dynamic rank+demand	Proposed system.

2.4. Results Summary

2.4.1. Main Performance Results (Production Trace)

Table 3: LoRAServe vs. Baselines on Production Traces (Company X)

Comparison	Throughput Gain	GPU Savings
vs. S-LoRA Random	up to 2×	up to 50%
vs. S-LoRA Contiguous	up to 2×	up to 50%
vs. Toppings	up to 20%	17%
Latency: up to 9× lower P95 TTFT vs. all baselines		
Adapter storage: up to 16× fewer adapters per server vs. Toppings		

2.4.2. Scalability

LoRAServe demonstrates near-linear weak scaling: 32 RPS on 4 servers, 64 RPS on 8, 96 RPS on 12—all within a 10s P95 TTFT SLO.

2.4.3. Sensitivity Studies

- **Rank skew (α):** LoRAServe remains within SLO as α increases from 1/3 to 3; baselines time out at higher skews.
- **Model size:** Performance trends replicate on Llama-30B and 70B; larger models have lower absolute throughput (orthogonal to LoRAServe’s gains).
- **Tensor Parallelism:** Benefits persist across TP=2, 4, 8; LoRAServe’s gains are orthogonal to parallelism degree.
- **TBT:** No degradation in Time-Between-Tokens; up to 15% improvement due to better load balancing.

2.4.4. Why Toppings Falls Short

Toppings performs request-level load balancing, which makes it rank-agnostic. Even though it evenly distributes load, it routes high-rank requests to all servers, causing interference everywhere. LoRAServe’s key advantage is adapter-level (not request-level) placement that isolates ranks, reducing co-batching of heterogeneous adapters.

2.5. Integration with Modern Systems

LoRAServe is built on S-LoRA and is designed to integrate with existing LLM cluster orchestrators:

- **Compatible with:** vLLM, S-LoRA, any LLM inference framework supporting LoRA adapters.
- **Cluster orchestrators:** Can plug into LLM-d and vLLM Production Stack.
- **Hardware:** Requires InfiniBand for efficient RDMA-based remote adapter fetching. Azure Accelerated Networking used as complementary fabric.
- **Modular design:** The orchestration layer (placement + routing) is decoupled from the inference engine, making it portable.

§3. Paper 2: LoRA Fusion — Efficient LoRA Fine-Tuning

3.1. Overview and Problem Statement

Citation: Zhanda Zhu, Qidong Su, Yaoyao Ding, Kevin Song, Shang Wang, Gennady Pekhimenko. *LoRA Fusion: Efficient LoRA Fine-Tuning for LLMs*. EuroSys 2026, Edinburgh, Scotland, April 27–30, 2026. DOI: 10.1145/3767295.3769331.

Affiliations: University of Toronto, Vector Institute, NVIDIA.

Venue: EuroSys 2026 (top-tier European systems venue). The paper received Artifact Evaluation certification. Code: <https://github.com/CentML/lorafusion>.

3.1.1. The Core Problem

Despite LoRA’s small parameter footprint ($\ll 1\%$ of base model), existing fine-tuning systems suffer from two critical inefficiencies:

Inefficiency 1: Redundant Memory Access. Profiling reveals that applying LoRA reduces training throughput by $\sim 40\%$ compared to the frozen-weight baseline—despite LoRA adding only minimal FLOPs. The culprit is memory bandwidth:

The arithmetic intensity of LoRA’s down-projection GEMM ($X\hat{A}$) in half-precision is:

$$\mathbb{I} = \frac{1}{\frac{1}{r} + \frac{1}{n} + \frac{1}{m}} \ll \mathbb{B}$$

where $\mathbb{B} \approx 295$ for FP16 on NVIDIA H100. Since $r \ll n$, the operation is severely memory-bandwidth-bound. Total GPU DRAM traffic increases by $2.64\times$ compared to the frozen linear layer.

Inefficiency 2: Missed Multi-Job Opportunities. Multiple LoRA adapters sharing the same base model are typically fine-tuned independently—even when running on the same GPU cluster. This misses:

- Opportunities to fill **pipeline bubbles** by interleaving microbatches from different jobs.
- Opportunities to **balance GPU load** across data-parallel replicas, since variable sequence lengths cause severe load imbalance ($\sim 30\%$ slowdown vs. ideal).

Key Profiling Insight

Runtime breakdown of a LoRA linear layer ($n = k = 4096$, $r = 16$, tokens=8192):

- Frozen GEMM (XW): 59% of forward runtime, 60% of backward.
- LoRA-specific GEMMs ($X\hat{A}$, SB): 10.76% forward, 20.37% backward.
- Element-wise operations (dropout, mul, add): 30.46% forward, 17.49% backward.

Both LoRA GEMMs and element-wise ops are memory-bandwidth-bound due to repeated loading/storing of large activation tensors $X \in \mathbb{R}^{m \times k}$.

3.2. Core Novelty and Technical Contributions

Core Novelty

LoRAFusion introduces **two complementary optimizations**: (1) a novel *graph-splitting* kernel fusion strategy that eliminates redundant memory traffic from LoRA’s large activation tensors without recomputation or synchronization overhead, and (2) a two-stage MILP-based multi-LoRA scheduler that jointly fine-tunes multiple adapters to reduce pipeline bubbles and balance GPU load.

3.3. FusedLoRA and FusedMultiLoRA Kernels

3.3.1. The Fusion Design Problem

A naive full-fusion approach (fusing all LoRA ops into one kernel) is problematic because:

- The frozen GEMM (XW) is compute-bound and highly sensitive to tiling. Adding more operations consumes registers and shared memory, degrading its throughput.
- Fusing $X\hat{A}$ followed by SB (producer-consumer dependency) requires recomputing S inside each tile or expensive thread-block synchronization.

3.3.2. Graph-Splitting Strategy

LoRAFusion’s insight: **split the computation graph at the intermediate tensor $S = X\hat{A}$** . Since $S \in \mathbb{R}^{m \times r}$ has the small rank dimension r , it is cheap to materialize explicitly in GPU DRAM. This enables:

Forward Pass:

1. **Kernel 1** (`fused_dropout_matmul`): Fuse dropout and down-projection $\hat{X} = \text{dropout}(X)$, $S = \hat{X}A^T$. Eliminates reload of $\hat{X}$.
2. **Kernel 2** (`fused_xw_sb`): Fuse frozen GEMM $Y_1 = XW$ with LoRA up-projection $Y_2 = \alpha SB$, accumulating partial results directly. Eliminates one read + write of full output tensor Y .

Backward Pass:

1. **Kernel 3** (`fused_dys_dyb`): Fuse gradient computations for S and B ; eliminates reload of $dY = dY \cdot \alpha$.
2. **Kernel 4** (unchanged): Operates on small masked input; fusion provides minimal benefit.
3. **Kernel 5** (`fused_dyw_dsa`): Fuse gradient for frozen GEMM with LoRA path; eliminates redundant reads/writes of partial dX .

Total DRAM traffic reduction: **34–37%** compared to standard Torch LoRA, confirmed via NVIDIA Nsight Compute profiling.

3.3.3. FusedMultiLoRA

FusedLoRA is extended to support *heterogeneous multi-adapter batches* via **tile-level routing**:

- Each input tile M_{tile} is tagged with an adapter ID and configuration (rank, scaling factor α , dropout ratio) stored in a lightweight lookup table.

- During execution, the frozen model GEMM is shared across all tokens; adapter-specific logic (A, B matrices, scaling) is applied dynamically per M_{tile} .
- In the backward pass, the same mapping routes gradients to their respective adapter without interference.

FusedMultiLoRA achieves $1.17\times$ average speedup (up to $1.24\times$) over standard Torch LoRA. In practice, since each microbatch typically contains 1–2 adapters, its performance is close to FusedLoRA in real fine-tuning workloads.

3.4. Multi-LoRA Scheduler

3.4.1. Problem Formulation

Given multiple LoRA adapters sharing a base model, the scheduler must construct microbatches that:

1. Balance token counts across GPUs to minimize load imbalance.
2. Respect **pipeline dependencies** (the “bubble lemma”).
3. Fill pipeline stages to reduce idle time.

Bubble Lemma. In a pipeline with S stages, for adapter i : if sample s from global batch j is assigned to microbatch k , no sample from batch $j + 1$ of the same adapter can be assigned before microbatch $k + S - 1$.

3.4.2. Two-Stage Scheduling Workflow

Stage 1 — Adapter Grouping. Adapters are grouped by their sample-length distribution statistics. Adapters with similar length distributions are placed in the same group. Within a group, *head-tail pairing* pairs short-sequence adapters with long-sequence ones for load balance.

Stage 2 — Data Batching with Two-Stage MILP. For each group and global batch index b , solve a bin-packing problem. Let $x_{s,b} \in \{0, 1\}$ indicate sample s assigned to bin b , $k_{a,b} \in \mathbb{N}$ be padded token multiples for adapter a in bin b , and $z_b \in \{0, 1\}$ whether bin b is used.

Stage 2a — Minimize microbatches:

$$\min_{x,k,z} \sum_{b=1}^B z_b \quad \text{s.t.} \quad \sum_b x_{s,b} = 1 \quad \forall s, \quad \sum_a k_{a,b} \cdot P \leq C \cdot z_b \quad \forall b$$

Stage 2b — Minimize smallest bin (for mergeability):

$$\min_{x,k} \min_{b \in [1, B^*]} \sum_a k_{a,b} \cdot P \quad \text{s.t.} \quad \sum_b x_{s,b} = 1 \quad \forall s$$

Merging and Verification. After packing, underfilled final microbatches from consecutive global batches are merged greedily (if bubble lemma is satisfied). A final verification step inserts no-ops where needed.

Fallback. The MILP solver is given a configurable timeout; if exceeded, a greedy bin-packing algorithm is used as a fallback. Parallelism profiler benchmarks different model-parallel configurations and provides token capacity input.

3.4.3. Pipeline Bubble Reduction

Table 4: Pipeline bubble ratio under different methods (LLaMa-3.1-70B, 4 stages)

Method	Bubble Ratio
Megatron-LM 1F1B PP	48.79%
mLoRA (4 adapters)	34.11%
LoRA Fusion (1 adapter)	44.17%
LoRA Fusion (2 adapters)	15.00%
LoRA Fusion (3 adapters)	12.23%
LoRA Fusion (4 adapters)	11.09%

3.5. Experimental Setup

3.5.1. Hardware

Table 5: LoRA Fusion Experimental Hardware

Resource	Specification
Primary	NVIDIA H100 (80GB HBM3) GPUs; 8× per node via NVLink; 208 vCPUs; Infini-Band for multi-node.
Secondary	NVIDIA L40S (48GB) GPUs; 4× per server via PCIe; 128 vCPUs.
Scale	1–16 H100 GPUs (up to 2 nodes); 1–4 L40S GPUs.

3.5.2. Software Stack

- **Framework:** Megatron-LM (multi-billion parameter distributed training).
- **LoRA integration:** Hugging Face Transformers + PEFT Library.
- **Kernel language:** Triton 3.2.0 (for FusedLoRA and FusedMultiLoRA).
- **Software versions:** PyTorch 2.6, CUDA Toolkit 12.4, Megatron-Core 0.11.0.
- **Codebase:** ~10K lines of Python.

3.5.3. Models

Table 6: Models Evaluated in LoRA Fusion

Model	Size	Primary GPU Setup
LLaMa-3.1-8B	8B	1 H100 / 1 L40S
Qwen-2.5-32B	32B	2 H100 / 4 L40S
LLaMa-3.1-70B	70B	4 H100

3.5.4. Datasets

Table 7: Datasets used in LoRA Fusion (all summarization tasks)

Dataset	Avg. Length	Characteristic
XSum	Short (~ 512 –1K tokens)	Extreme summarization
CNN/DailyMail	Medium (~ 1 –2K tokens)	News summarization
WikiSum	Long (~ 2 –4K tokens)	Document summarization
Mixed	Combined	Stress-tests batching
Heterogeneous	Different per adapter	Each adapter uses a different dataset

3.5.5. Multi-LoRA Configuration

All multi-LoRA experiments train **4 LoRA adapters simultaneously**. Four workload configurations:

- **Homogeneous (XSum/CNNDM/WikiSum):** All 4 adapters train on the same dataset.
- **Mixed:** All 4 adapters train on a combination of all three datasets.
- **Heterogeneous (Het):** Each adapter trains on a different dataset—the most challenging setting.

LoRA rank: 16 (default). Global batch size: 8. On-the-fly token packing used throughout.

3.5.6. Baselines

Table 8: LoRA Fusion Baselines

Baseline	Parallelism	Description
Megatron-LM FSDP	ZeRO-3 / FSDP	Standard FSDP parallelism; trains adapters sequentially (no multi-LoRA).
Megatron-LM PP	Pipeline Parallelism	Standard 1F1B pipeline; trains adapters sequentially.
mLoRA	PP + multi-LoRA	State-of-the-art multi-LoRA fine-tuning system; uses BatchLoRA kernel; CPU-based inter-GPU comm (reimplemented with high-perf primitives for fair comparison).
LoRA Fusion	PP + multi-LoRA + fusion	Proposed system.

3.6. Results Summary

3.6.1. End-to-End Speedup

Table 9: LoRA Fusion End-to-End Throughput Speedup

Model	vs. Megatron-LM (avg)	vs. Megatron-LM (max)	vs. mLoRA (avg)
LLaMa-3.1-8B (H100)	1.26×	1.43×	~1.2×
Qwen-2.5-32B (H100)	1.42×	1.64×	~1.3×
LLaMa-3.1-70B (H100)	1.64×	1.81×	~1.3×
<i>Overall range</i>	1.19×–1.96× vs. Megatron-LM		1.29× avg vs. mLoRA

Key pattern: *larger models benefit more*, since pipeline stalls and load imbalance dominate at higher parallelism degrees.

3.6.2. Speedup Breakdown (LLaMa-3.1-70B, 4 H100 GPUs)

Table 10: Additive speedup contributions in LoRA Fusion

Configuration	Relative Speedup
Baseline: 1F1B PP	1.00×
+ FusedLoRA kernel	1.13×
+ Multi-LoRA zero-bubble PP	1.50×
+ FusedMultiLoRA	1.72×
+ Balanced scheduling only (no fusion)	1.57×
+ Full LoRA Fusion (scheduling + FusedMultiLoRA)	2.05×

3.6.3. Scalability

Job-level scaling (more concurrent jobs) consistently outperforms DP scaling (more GPUs per job): $1.18\times$ and $1.25\times$ higher throughput on 8 and 16 GPUs, respectively.

3.7. GitHub Repository Analysis

The repository (<https://github.com/CentML/lorafusion>) is structured as follows:

Listing 1: LoRAFusion Repository Structure

```
lorafusion/
  benchmarks_paper/      # Reproducibility scripts (Fig 13, 14,
    17, 18, 19)
    scripts/
      setup/setup_env.sh
      run_all.sh          # Master script; supports all/
all_single_gpu/layer/kernel
  docs/
    installation.md
  lorafusion/              # Core library
    ops/triton_ops/
      config.py           # Hardware-specific kernel configs (
H100, A100, L40S, RTX3090)
    tools/
      tune_kernels.py     # Hardware-specific Triton kernel
  tuning
    requirements.txt
    pyproject.toml
```

Artifact Certification. The paper received EuroSys 2026 Artifact Evaluation badges. Full reproduction requires:

- Linux system, 192 GB RAM, 256 GB disk, 4 NVIDIA H100 GPUs with NVLink.
- Full evaluation: ~ 4 hours.
- Partial (kernel/layer benchmarks): single GPU sufficient.

Pre-tuned Triton kernel configurations are provided for: H100 80GB HBM3, A100 SXM4 80GB, A100 PCIe 80GB, and GeForce RTX 3090.

Integration Notes. LoRAFusion is implemented on top of Megatron-LM and integrates Hugging Face PEFT for LoRA adaptation. The FusedLoRA kernel is described as a *plug-and-play replacement* in existing LoRA systems, offering standalone gains without the full scheduling infrastructure.

§4. Comparative Analysis

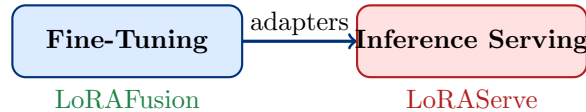
4.1. Side-by-Side Comparison

Table 11: LoRAServe vs. LoRA Fusion: Key Dimensions

Dimension	LoRAServe	LoRA Fusion
Phase	Inference (serving)	Training (fine-tuning)
Primary target	Tail latency (TTFT), throughput under SLO	Training throughput (tokens/sec)
Core insight	Rank heterogeneity $\Rightarrow$ interference; must isolate ranks via placement	Memory bandwidth dominates LoRA overhead; kernel fusion + multi-job scheduling eliminates redundancy
Novel algorithms	Dynamic bin-packing + RDMA-based distributed adapter pool	Graph-split fusion kernels (Triton) + two-stage MILP multi-LoRA scheduler
Kernel level	Uses S-LoRA/Punica as-is (kernel-agnostic)	Proposes new FusedLoRA and FusedMultiLoRA kernels
Cluster scale	4–12 LLM inference servers	1–16 H100 GPUs (2 nodes)
Hardware requirement	InfiniBand for RDMA	NVLink for intra-node; InfiniBand for multi-node
Venue	arXiv (Nov 2025)	EuroSys 2026
Code available	Not yet (“after acceptance”)	Yes (github.com/CentML/lorafusion)
Artifact evaluation	N/A	EuroSys 2026 AE certified

4.2. Complementarity

These two papers attack *complementary* bottlenecks in the LoRA lifecycle:



LoRA Fusion produces better-trained adapters more efficiently; LoRAServe deploys and serves those adapters more efficiently. A production system would benefit from deploying both.

4.3. Shared Themes

1. **Rank awareness is essential.** Both papers center on the importance of treating adapter rank as a first-class design parameter. LoRAServe shows that rank-oblivious placement causes interference; LoRA Fusion shows that rank-driven memory patterns are the source of training overhead.
2. **Multi-tenancy.** Both address multi-tenant scenarios: LoRAServe handles hundreds of adapters sharing a base model at inference; LoRA Fusion handles multiple adapters co-trained on the same GPUs.
3. **Dynamic scheduling.** Both use dynamic scheduling (workload-adaptive placement in LoRAServe; MILP-based microbatch scheduling in LoRA Fusion) rather than static policies.

4. **Systems-level evaluation on real hardware.** Both use Azure cloud infrastructure and real production or realistic workloads.

§5. Related Work and Positioning

5.1. LoRA Algorithm and Variants

- **LoRA** [3]: Original low-rank adaptation; foundational work both papers build on.
- **QLoRA**: Quantized LoRA (4-bit base model + LoRA adapters); LoRA Fusion notes its kernels can be directly applied after dequantization.
- **DoRA, VeRA**: Variant methods; LoRA Fusion’s kernel fusion is described as extensible to these via prologue/epilogue functions.

5.2. LoRA Serving Systems

- **Punica** [5]: Introduced SGMV/BGMV kernels for multi-tenant LoRA serving; foundational kernel used by LoRA Serve.
- **S-LoRA** [4]: Unified paging (KV-cache + adapter weights); thousands of adapters in CPU memory; baseline for LoRA Serve.
- **dLoRA** [6]: Dynamic adapter merging with base model; migrates requests and adapters between replicas (OSDI 2024).
- **Toppings** [7]: CPU-assisted LoRA computation for TTFT improvement; rank-aware scheduling but assumes full adapter replication (USENIX ATC 2025); primary baseline for LoRA Serve.
- **Chameleon** [8]: GPU-resident adapter caching + adapter-aware scheduling (MICRO 2025); complementary to placement.
- **vLLM**: General serving system; S-LoRA extends its paging paradigm.

5.3. LoRA Training Systems

- **mLoRA**: Multi-LoRA fine-tuning via pipeline parallelism; primary baseline for LoRA Fusion. Key limitation: CPU-based inter-GPU communication, no memory-access kernel optimization.
- **Megatron-LM**: State-of-the-art distributed training framework; LoRA Fusion is implemented on top of it.
- **PEFT Library, LLaMA-Factory**: Popular LoRA fine-tuning libraries; reuse full-model training optimizations.
- **Mirage**: Multi-level superoptimizer for tensor programs; related to LoRA Fusion’s kernel fusion but does not yet address fine-tuning with dropout/backward.

5.4. Broader LLM Systems Context

- **FlashAttention**: Memory-efficient attention; complementary to LoRA Fusion’s fused LoRA kernels.

- **Triton**: GPU kernel language used by LoRA Fusion for implementing FusedLoRA.
- **SageServe**: LLM serving with forecast-aware auto-scaling (same first author as LoRAServe).
- **Orca, Sarathi-Serve**: Continuous batching and throughput-latency tradeoffs; orthogonal to LoRA-specific concerns.
- **ZeRO/FSDP**: Sharded data parallelism; used as baseline in LoRA Fusion.

§6. Benchmarks and Evaluation Methodology

6.1. LoRAServe Benchmarks

Table 12: LoRAServe Evaluation Summary

Experiment	What is measured	Key result
Rank interference	TTFT/TBT vs. co-served rank	+84% TTFT for rank-8 when co-served with rank-128
Model size sensitivity	Relative TTFT vs. model size	Up to 45% degradation on Llama-70B from rank het.
TP sensitivity	Relative TTFT vs. TP degree	20% TTFT increase even at TP=8 with rank-128
SLO compliance	P95 TTFT vs. RPS at fixed rank	Ranks 64/128 violate SLO at 4 RPS where rank-8 does not
Main perf (prod trace)	P95 TTFT, throughput, GPU count	Up to 2× throughput, 9× TTFT, 50% GPU reduction
Adapter storage	Max adapters per server	16× reduction vs. Toppings
Scalability	RPS vs. cluster size (4/8/12)	Near-linear weak scaling
Rank skew sensitivity	P95 TTFT vs. α	Stable across $\alpha \in [1/3, 3]$; base-lines time out
TP sensitivity (main)	Throughput vs. TP=2/4/8	Benefits orthogonal to TP degree
Model size (main)	Throughput vs. 7B/30B/70B	Trends replicate; smaller absolute throughput for larger models

6.2. LoRA Fusion Benchmarks

Table 13: LoRA Fusion Evaluation Summary

Experiment	What is measured	Key result
LoRA overhead profiling	Throughput vs. frozen linear	~40% slowdown; $2.64\times$ DRAM traffic
Memory traffic analysis	DRAM read/write (NCU)	34–37% reduction with FusedLoRA
Kernel perf (microbench)	Normalized throughput vs. tokens	$1.27\times$ avg, $1.39\times$ max speedup
Layer-wise perf	Speedup per decoder layer	$1.21\times$ avg (FusedLoRA), $1.13\times$ (FusedMultiLoRA)
End-to-end H100	Tokens/sec (4 adapters, 1/2/4 GPUs)	$1.47\times$ avg, $1.96\times$ max vs. Megatron-LM
End-to-end L40S	Tokens/sec (4 adapters, 1/4 GPUs)	$1.19\times$ – $1.96\times$ speedup
Scalability (4/8/16 GPUs)	Throughput vs. GPU count	Job scaling > DP scaling ($1.18\times$ and $1.25\times$ higher)
Pipeline bubble reduction	Bubble ratio vs. adapter count	11.09% (4 adapters) vs. 48.79% (Megatron baseline)
Speedup breakdown	Additive component contributions	Full system: $2.05\times$ vs. 1F1B baseline
Scheduler tuning time	Scheduling time vs. sample count	Linear scaling; overhead hidden by GPU execution
Merging + greedy fallback	Throughput impact of sub-components	+4.34% (merging), +3.82% (MILP over greedy)

6.3. Metrics Used

Table 14: Metrics Across Both Papers

Metric	Used by	Definition
P95 TTFT	LoRAServe	95th percentile Time-to-First-Token (s)
P95 TBT	LoRAServe	95th percentile Time-Between-Tokens (ms)
Request throughput (RPS)	LoRAServe	Requests per second served under SLO
Tokens/second	LoRA Fusion	Training tokens processed per second
Pipeline bubble ratio	LoRA Fusion	% of pipeline stages idle
DRAM traffic	LoRA Fusion	Total GPU global memory read+write (GB)
Normalized throughput	LoRA Fusion	Relative to frozen/baseline implementation

§7. Limitations and Open Questions

LoRAServe Limitations

1. **Code not yet released.** The paper commits to releasing code after acceptance, but as of the arXiv preprint (Nov 2025) it is not publicly available. This limits reproducibility.
2. **Kernel-level interference not eliminated.** LoRAServe improves placement but the root cause (rank-padding in BGMV/MBGMV kernels) remains. A rank-normalized kernel would complement the placement approach.
3. **RDMA dependency.** The distributed adapter pool assumes InfiniBand. Deployments on commodity Ethernet would face higher latency and bandwidth constraints.
4. **Orchestrator as single point of failure.** The centralized cluster orchestrator is a potential bottleneck and reliability concern at very large scale.
5. **Adapter migration overhead.** During workload drifts, migrating adapters has costs not fully quantified across a wide range of scenarios.
6. **Limited model diversity.** Experiments focus on the Llama family; behavior on MoE models or different architecture families is unstudied.

LoRAFusion Limitations

1. **Pipeline bubble floor.** Even at 4 adapters, 11.09% bubble ratio remains due to uneven stage execution times (last stage has extra linear + cross-entropy). This is described as out of scope.
2. **MILP solver scalability.** For very large numbers of adapters or samples, the MILP solver may time out more frequently, increasing reliance on the greedy fallback. The merging + MILP contribution is modest (4.34% + 3.82%).
3. **No TP support.** Tensor parallelism is not evaluated due to lack of efficient TP support in existing LoRA frameworks.
4. **Single task type.** Only summarization datasets are evaluated. Tasks with very different sequence-length distributions (e.g., code generation) may behave differently.
5. **FusedMultiLoRA backward overhead.** Accumulating gradients across adapters adds overhead; FusedMultiLoRA is slightly slower than FusedLoRA in the backward pass.
6. **Hardware specificity.** Triton kernel configurations are tuned per GPU architecture; performance on new hardware requires re-tuning.

§8. Future Directions and Research Opportunities

Based on the limitations and open questions identified above, we identify the following future research directions:

1. **Rank-normalized kernels for serving.** Designing CUDA/Triton kernels for multi-tenant LoRA serving that do *not* pad to the maximum rank—perhaps via warp-level rank-specific execution paths—would eliminate the interference root cause rather than merely mitigating it at the placement level. This connects directly to the AdapterSlots++ line of work.

2. **Joint training–serving co-design.** The two papers address training and serving independently. A co-designed system that informs serving placement decisions based on training-time adapter rank distribution, and that guides fine-tuning rank selection based on serving cost models, would be a natural extension.
3. **Compiler integration for FusedLoRA.** The paper acknowledges the potential of integrating FusedLoRA patterns into `torch.compile` via compiler annotations, which would eliminate the need for manual Triton kernel development and automatically extend to new LoRA variants.
4. **LoRAServe on Ethernet fabrics.** Characterizing the performance of LoRAServe’s distributed adapter pool over RDMA-over-Converged-Ethernet (RoCE) rather than InfiniBand would broaden its deployment applicability.
5. **MoE and emerging architectures.** Both papers focus on dense transformer models. Multi-expert architectures (Mixtral, DeepSeek-V3) with LoRA on expert weights would introduce additional placement dimensions.
6. **Serving + training co-scheduling.** In cloud environments, the same hardware serves inference requests while other training jobs run on idle capacity. A unified co-scheduler for LoRA training and serving on shared infrastructure is an open problem.
7. **Adaptive rank selection.** If the serving system can communicate adapter rank requirements back to the training pipeline (e.g., “rank-8 adapters are cheaper to serve; retrain with lower rank”), it would create a feedback loop for rank-budget-aware fine-tuning.

§9. Quick Reference for Literature Survey Integration

9.1. Citing These Papers

BibTeX Entries

```
@article{loraserve2025,
  title={Serving Heterogeneous LoRA Adapters in
        Distributed LLM Inference Systems},
  author={Jaiswal, Shashwat and Arun, Shrikara and
        Parayil, Anjaly and Mallick, Ankur and
        Mastorakis, Spyros and Khare, Alind and
        Alverti, Chloi and St Amant, Renee and
        Bansal, Chetan and Ruhle, Victor and
        Torrellas, Josep},
  journal={arXiv preprint arXiv:2511.22880},
  year={2025}
}

@inproceedings{lorafusion2026,
  title={LoRA Fusion: Efficient LoRA Fine-Tuning for LLMs},
  author={Zhu, Zhanda and Su, Qidong and Ding, Yaoyao and
        Song, Kevin and Wang, Shang and Pekhimenko, Gennady},
  booktitle={Proceedings of the 21st European Conference
        on Computer Systems (EuroSys '26)},
  pages={1--19},
  year={2026},
  doi={10.1145/3767295.3769331}
}
```

9.2. One-Paragraph Summary: LoRAServe

LoRAServe addresses the problem of rank-heterogeneity-induced interference in multi-tenant LoRA serving. It observes that state-of-the-art kernels (Punica, S-LoRA) size their compute tiles to the maximum adapter rank in each batch, causing low-rank adapters to suffer performance degradation of up to 84% when co-served with high-rank adapters. LoRAServe proposes a cluster orchestrator that periodically re-assigns adapters to servers to minimize within-server rank heterogeneity, balancing load by using demand-weighted fractional bin-packing. Remote adapter access is handled via GPUDirect RDMA over InfiniBand, which has comparable latency to local CPU-to-GPU transfers. Evaluated on production traces from Company X with up to 200 adapters, LoRAServe achieves up to $2\times$ higher throughput, $9\times$ lower P95 TTFT, 50% GPU savings, and $16\times$ reduction in per-server adapter storage compared to state-of-the-art baselines.

9.3. One-Paragraph Summary: LoRAFusion

LoRAFusion targets the 40% training throughput reduction that LoRA modules impose despite adding less than 1% parameters. The root cause is identified as redundant memory traffic: LoRA's small down/up projection GEMMs are memory-bandwidth-bound due to the low rank dimension, and element-wise operations (dropout, mul, add) repeatedly load and store large activation tensors, increasing total DRAM traffic by $2.64\times$. LoRAFusion addresses this with a graph-splitting fusion strategy: it materializes the small intermediate tensor $S = \hat{X}A$ at the LoRA-rank boundary, enabling horizontal fusion of memory-bandwidth-bound operations around the large activations

without recomputation or synchronization. The resulting FusedLoRA kernel reduces DRAM traffic by 34–37%. For multi-job settings, LoRAFusion introduces a two-stage MILP-based scheduler that groups adapters by sequence-length distribution and solves a bin-packing problem to balance microbatches across pipeline stages, reducing pipeline bubble ratios from 48.8% to 11.1%. Combined, these optimizations achieve up to $1.96\times$ speedup over Megatron-LM and $1.46\times$ over mLoRA across LLaMa-3.1-8B/70B and Qwen-2.5-32B on NVIDIA H100 GPUs.

§10. Conclusion

These two papers represent the state of the art in LoRA systems research as of early 2026, one from each critical phase of the LoRA lifecycle. Their most important contributions are:

1. **LoRAServe:** The first system to treat adapter rank as a first-class scheduling dimension in multi-tenant LLM serving clusters, backed by the first public characterization of production LoRA serving workloads at scale.
2. **LoRAFusion:** The first fine-tuning system to specifically address LoRA’s memory bandwidth bottleneck through principled graph-split kernel fusion, with demonstrated 34–37% DRAM traffic reduction and open-source artifact evaluation.

For researchers building on this work, the most productive directions are: (1) eliminating the kernel-level rank-padding root cause in serving kernels, (2) extending LoRAFusion’s fusion patterns via compiler infrastructure, and (3) designing unified training–serving co-schedulers for LoRA-based LLM deployments.

References

- [1] S. Jaiswal et al., “Serving Heterogeneous LoRA Adapters in Distributed LLM Inference Systems,” *arXiv:2511.22880*, 2025.
- [2] Z. Zhu et al., “LoRA Fusion: Efficient LoRA Fine-Tuning for LLMs,” *EuroSys 2026*, DOI: 10.1145/3767295.3769331.
- [3] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR 2022*, arXiv:2106.09685.
- [4] Y. Sheng et al., “S-LoRA: Serving Thousands of Concurrent LoRA Adapters,” arXiv:2311.03285, 2024.
- [5] L. Chen et al., “Punica: Multi-Tenant LoRA Serving,” *MLSys 2024*.
- [6] B. Wu et al., “dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving,” *OSDI 2024*.
- [7] S. Li et al., “Toppings: CPU-Assisted, Rank-Aware Adapter Serving for LLM Inference,” *USENIX ATC 2025*.
- [8] N. Iliakopoulou et al., “Chameleon: Adaptive Caching and Scheduling for Many-Adapter LLM Inference,” *MICRO 2025*.
- [9] Z. Ye et al., “mLoRA: Fine-Tuning LoRA Adapters via Highly-Efficient Pipeline Parallelism in Multiple GPUs,” arXiv:2312.02515, 2023.
- [10] M. Shoeybi et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” arXiv:1909.08053, 2019.
- [11] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *SOSP 2023*.
- [12] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS 2022*.
- [13] T. Detrmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” *NeurIPS 2023*.
- [14] P. Tillet et al., “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations,” *MLPL 2019*.
- [15] S. Jaiswal et al., “SageServe: Optimizing LLM Serving on Cloud Data Centers with Forecast Aware Auto-Scaling,” arXiv:2502.14617, 2025.

Literature Survey:

Advanced Multi-Tenant LoRA Serving Systems

*dLoRA & CaraServe: Dynamic Orchestration, CPU-Assisted Serving,
and Rank-Aware Scheduling*

Papers Covered:

- [P1] Wu et al., “dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving,” *USENIX OSDI 2024*.
- [P2] Li et al., “CaraServe: CPU-Assisted and Rank-Aware LoRA Serving for Generative LLM Inference,” *arXiv:2401.11240, Jan 2024*.

Artifact Repository (dLoRA): <https://github.com/LLMServe/dLoRA-artifact>

CaraServe: Implemented as module in LightLLM
(<https://github.com/ModelTC/lightllm>);
standalone release pending open-source publication.

Contents

1	Introduction and Positioning	3
2	Background: Limitations of First-Generation Systems	3
2.1	The Merged vs. Unmerged Dilemma	3
2.2	Cold-Start Overhead	3
2.3	Load Imbalance Across Replicas	3
3	dLoRA: Dynamically Orchestrating Requests and Adapters	4
3.1	Problem Statement and Two Key Insights	4
3.2	System Architecture	4
3.3	Core Novelty 1: Dynamic Cross-Adapter Batching	4
3.3.1	Unmerged Inference	4
3.3.2	Dynamic Batching Algorithm (Algorithm 1)	4
3.3.3	Adaptive Threshold Tuning (Algorithm 2)	5
3.3.4	Credit-Based Starvation Prevention (Algorithm 3)	5
3.4	Core Novelty 2: Request-Adapter Co-Migration	5
3.4.1	Proactive Mechanism	5
3.4.2	Reactive ILP-Based Migration	5
3.4.3	Selective Migration with Constraint Relaxation	6
3.5	LoRA Adapter Offloading	6
4	CaraServe: CPU-Assisted and Rank-Aware LoRA Serving	6
4.1	Problem Statement: Two Unaddressed Challenges	6
4.2	System Architecture	7
4.3	Core Novelty 1: CPU-Assisted LoRA Serving	7
4.3.1	The Cold-Start Pipeline	7
4.3.2	GPU-CPU Coordination Challenges	7
4.4	Core Novelty 2: Rank-Aware Scheduling	8
4.4.1	Performance Modeling	8
4.4.2	Rank-Aware Scheduling Policy (Algorithm 1)	8
5	Infrastructure and Experimental Setup	9
5.1	dLoRA Infrastructure	9
5.2	CaraServe Infrastructure	9
6	Experiments and Key Results	9
6.1	dLoRA Experiments	9
6.1.1	End-to-End Throughput (Figure 9 in paper)	9
6.1.2	Comparison with S-LoRA (Figure 14 in paper)	10
6.1.3	Dynamic Batching Ablation (Figure 10)	10
6.1.4	Load Balancing Ablation (Figure 11)	10
6.1.5	Scalability (Figure 12)	10
6.1.6	Overhead Analysis (Figure 15)	11
6.1.7	GPU Utilization (Figure 13)	11
6.2	CaraServe Experiments	11
6.2.1	End-to-End Single-GPU Performance (Figures 10, 11)	11
6.2.2	Sensitivity Analysis (Figure 13)	11

6.2.3	Scaled Production Workloads (Figure 14)	11
6.2.4	Multi-GPU Performance (Figure 15)	11
6.2.5	Scheduler Performance (Figures 19, 20)	12
6.2.6	Microbenchmarks (Figures 16, 17, 18)	12
7	Repository Analysis	12
7.1	dLoRA Artifact (LLMServe/dLoRA-artifact)	12
7.2	CaraServe (ModelTC/lightllm)	13
8	Comparative Analysis: dLoRA vs. CaraServe	14
8.1	Problem Focus	14
8.2	Orthogonality and Composability	14
8.3	Relationship to First-Generation Systems	15
9	Critical Discussion and Limitations	15
9.1	dLoRA	15
9.2	CaraServe	15
10	Broader Literature Context	16
10.1	Position in the LoRA Serving Lineage	16
10.2	Related Systems Work	16
10.3	Open Problems for Future Work	16
11	Summary	17

1 Introduction and Positioning

The first generation of multi-tenant LoRA serving systems—Punica [7] and S-LoRA [8]—established two foundational principles: (1) share the backbone model across all adapters to avoid weight duplication, and (2) enable cross-adapter batching via specialized CUDA kernels. However, both systems operate with a fixed serving modality (always-unmerged inference) and do not address the cluster-level challenges that arise in production deployments: load imbalance across replicas, the cold-start penalty when fetching uncached adapters, and the performance impact of heterogeneous adapter ranks in a batch.

Two 2024 papers tackle these gaps from complementary angles:

dLoRA (Wu et al., OSDI 2024, Peking University & Shanghai AI Lab) introduces *dynamic switching between merged and unmerged inference* at the replica level, combined with an ILP-based *request-adapter co-migration* algorithm for cluster-level load balancing.

CaraServe (Li et al., HKUST et al., arXiv Jan 2024) introduces *CPU-assisted LoRA serving* to eliminate cold-start overhead, and a *rank-aware scheduling* algorithm to optimize SLO attainment under adapter rank heterogeneity.

Together, these two systems represent the second generation of LoRA serving research, transitioning from “how to batch” to “how to orchestrate intelligently.”

2 Background: Limitations of First-Generation Systems

2.1 The Merged vs. Unmerged Dilemma

S-LoRA and Punica exclusively use *unmerged inference*: the base model weight W and LoRA add-on AB are computed separately, then summed. This enables cross-adapter batching but introduces computation overhead. The LoRA add-on computation BAx costs approximately 38.9% of the baseline LLM computation Wx (measured by dLoRA on A800 GPUs). When requests are skewed toward a single adapter, this overhead is unnecessary—simply merging $W + BA$ and running standard inference is cheaper. Neither Punica nor S-LoRA can take advantage of this.

2.2 Cold-Start Overhead

When a requested LoRA adapter is not cached in GPU memory, it must be loaded from host DRAM before inference can begin. For a rank-64 adapter applied to three attention weight matrices of Llama2-7B, loading takes approximately 50 ms on an A10 GPU. Under continuous batching, this cold-start stalls the decode phase of all in-flight requests. CaraServe empirically measures this overhead at 10–20% of total request serving time at 3–9 RPS, yet neither Punica nor S-LoRA solve it.

2.3 Load Imbalance Across Replicas

Production LLM deployments use multiple replicas for scale-out. Two orthogonal factors create load imbalance: (a) *adapter skew*—some adapters are much more popular than others, creating hot replicas; and (b) *variable request lengths*—the longest input/output in

ShareGPT is $637 \times / 164 \times$ the average, causing wildly different per-request GPU memory and compute consumption. Neither Punica nor S-LoRA model or react to this imbalance.

3 dLoRA: Dynamically Orchestrating Requests and Adapters

3.1 Problem Statement and Two Key Insights

dLoRA addresses two orthogonal inefficiencies:

Insight 1 (Intra-replica): The benefit of unmerged inference (better GPU utilization from cross-adapter batching) is offset by the extra computation overhead when requests are skewed. A serving system should *dynamically switch* between merged and unmerged modes.

Insight 2 (Inter-replica): Due to variable request lengths and adapter popularity skew, naive request dispatching always produces load imbalance. Only *proactive* and *reactive* migration of both requests *and* their associated adapters can achieve balance.

3.2 System Architecture

LLM Replicas A cluster of worker replicas, each hosting the base model on several GPUs and a subset of LoRA adapters. Each replica runs a cross-adapter batching engine and a memory manager.

Load Balancer A global scheduler with two mechanisms: (a) a *proactive dispatcher* that routes incoming requests based on estimated pending time, and (b) a *reactive migration controller* that solves an ILP to re-balance overloaded replicas.

Global Monitor Periodically collects resource usage (GPU memory, queue depth) from all replicas to feed the reactive mechanism.

ILP Solver Uses PuLP [13] to compute optimal request-adapter co-migration plans.

The system is implemented in Python + PyTorch on top of vLLM [4] ($\sim 6,200$ LOC), using Ray [12] actors for distributed execution, and FastAPI for the frontend.

3.3 Core Novelty 1: Dynamic Cross-Adapter Batching

3.3.1 Unmerged Inference

In unmerged mode, requests with different LoRA types are batched together. The base model computation $W \times [x_0, x_1, \dots]$ is shared, and each adapter’s contribution $B_i A_i x_i$ is computed in parallel using PyTorch’s `einsum` operator with a request-type mapping matrix. The result is aggregated: $y_i = Wx_i + B_i A_i x_i$.

3.3.2 Dynamic Batching Algorithm (Algorithm 1)

The algorithm operates at each iteration and maintains a replica-level state (`unmerge` or `merge`):

In unmerged state:

1. Identify the LoRA adapter with the most pending requests: $R_{\text{merge}} = \arg \max_{l_i \in L} |\{r \in R : r.\text{type} = l_i\}|$.

2. If $|R_{\text{merge}}|/|B_{\text{fcfs}}| > \alpha_{\text{switch}}$, switch to merged mode and process requests from R_{merge} .
3. Otherwise, continue with the FCFS batch.

In merged state:

1. If $|R_{\text{merge}}|/|B_{\text{fcfs}}| < \beta_{\text{switch}}$, switch back to unmerged mode.
2. Otherwise, continue serving the dominant adapter type.

3.3.3 Adaptive Threshold Tuning (Algorithm 2)

The thresholds α_{switch} and β_{switch} are not static—they are adapted at each *break point* (replica state change or after a fixed number of iterations) using historical throughput data:

$$T_{\text{merge}} = \frac{\sum_{i=1}^{N_I} |B_i|}{\sum_{i=1}^{N_I} \text{IterationTime}(B_i) + t_M} \quad T_{\text{unmerge}} = \frac{\sum_{i=1}^{N_I} |B'_i|}{\sum_{i=1}^{N_I} \text{IterationTime}(B'_i)}$$

If $T_{\text{merge}} > T_{\text{unmerge}}$, α_{switch} is decremented by γ_{dec} ; otherwise multiplied by $\gamma_{\text{mul}} > 1$.

Amortization insight: Switching overhead is a one-time cost that is amortized over multiple future iterations. If the current batch has k remaining iterations, switching is beneficial even when overhead t_M is substantial, as long as per-iteration savings $\times k > t_M$.

3.3.4 Credit-Based Starvation Prevention (Algorithm 3)

Dynamic batching naturally preempts low-frequency adapter types. To prevent starvation, each LoRA adapter is assigned a *credit*. Whenever requests are preempted, their adapter’s credit is incremented. Once credits exceed a threshold T_{starve} , those adapters are served first regardless of the dynamic batching policy. This ensures fairness while maintaining throughput optimization.

3.4 Core Novelty 2: Request-Adapter Co-Migration

3.4.1 Proactive Mechanism

Long-term: dLoRA pre-loads LoRA adapters to GPUs based on historical workload patterns (predictable periodicity: low load at night, high load in daytime). It greedily loads the adapter with the lowest *burst tolerance* (ratio of peak capacity to average load) to replicas that do not yet host it, up to a threshold.

Short-term: For bursty arrivals, dLoRA computes an *estimated pending time* for each replica, including adapter loading time (if not already loaded) and current queue depth. New requests are dispatched to the replica with the minimum estimated pending time.

3.4.2 Reactive ILP-Based Migration

When the global monitor detects overload (GPU memory or queue depth beyond thresholds), dLoRA formulates a **co-migration ILP**:

Variables: $x_{i,j}^G, x_{i,j}^H \in \{0, 1\}$ (request i in GPU/host memory of replica j), $y_{k,j} \in \{0, 1\}$ (adapter k in replica j).

Objective:

$$\min_j \left(\max_j \sum_i \left(\gamma \cdot L_i x_{i,j} + \frac{M_i^R x_{i,j}^H}{B} \right) + \eta \sum_k y_{k,j} \right)$$

where γ is a migration factor (accounts for migration time), L_i is the average execution time for request i 's adapter, M_i^R is request i 's memory, B is PCIe bandwidth, and η penalizes adapter fragmentation across replicas.

Adapter-request matching constraint: $\forall i, j : y_{i,j} \geq x_{i,j}$ (a request can only be in a replica if its adapter is there).

Memory constraints: GPU and host memory budgets per replica.

Existence constraint: $\forall i : \sum_j x_{i,j} = 1$ (each request in exactly one replica).

3.4.3 Selective Migration with Constraint Relaxation

To scale to large clusters:

- Migration is triggered only under extreme imbalance (memory or queue threshold breached).
- Only top- K overloaded and top- K underloaded replicas are considered (complexity $O(K^2)$ vs. $O(N^2)$).
- Memory constraints are relaxed (host memory assumed sufficient), reducing the ILP to a placement problem.
- The binary variable matrix x is relaxed to real-valued; final placement uses $\arg \max_j x_{i,j}$.
- Migration is implemented as *token transfer + re-prefill* (faster than KV cache transfer per vLLM measurements).

With these optimizations, the ILP solves within milliseconds, and overhead is $< 3.3\%$ of total latency.

3.5 LoRA Adapter Offloading

A single Llama-7B LoRA adapter requires 56 MB of GPU memory—equivalent to the KV cache for 7 tokens. When many adapters are loaded, KV cache space shrinks, reducing max batch size. dLoRA employs:

- Asynchronous host $\leftrightarrow$ GPU adapter swapping via prefetching (overlapping with inference execution).
- Workload-aware allocation: as requests arrive, memory is dynamically split between adapter storage and KV cache to maximize burst tolerance.

4 CaraServe: CPU-Assisted and Rank-Aware LoRA Serving

4.1 Problem Statement: Two Unaddressed Challenges

CaraServe addresses two challenges neglected by Punica and S-LoRA:

Challenge C1 (Cold-Start): Loading an uncached adapter from host memory to GPU blocks prefill, stalling all in-flight requests under continuous batching. At 9 RPS with rank-64 adapters, this accounts for 20% of total serving time.

Challenge C2 (Heterogeneous Rank Scheduling): Batching adapters of different ranks has kernel-dependent performance implications (BGMV latency scales with max rank; MBGMV scales with sum of ranks). Rank-unaware scheduling can cause SLO violations that rank-aware routing would avoid.

4.2 System Architecture

CaraServe has three components:

LLM Inference Servers Each server maintains the base LLM on GPU and a *local LoRA repository* in host DRAM. During inference, it coordinates GPU and CPU LoRA computations.

Scheduler Rank-aware request router; uses a performance model to predict serving latency and pick the optimal server per request.

Global LoRA Registry Stores adapter metadata (ranks, file paths); implemented as SQLite.

CaraServe is built on top of **LightLLM** [11] (Python + PyTorch + Triton), using Punica’s BGMV kernel for GPU LoRA computation. The scheduler runs as a Python Flask server.

4.3 Core Novelty 1: CPU-Assisted LoRA Serving

4.3.1 The Cold-Start Pipeline

When a request arrives for an uncached adapter, CaraServe simultaneously:

1. Starts asynchronous PCIe transfer to load the adapter to GPU.
2. Immediately begins the *prefill phase on CPU* using the adapter weights already in host memory.

Once loading completes, CaraServe switches to GPU computation—resuming the prefill if not yet complete, then continuing with the decode phase under continuous batching with other requests. This effectively overlaps the cold-start latency with useful compute work, reducing prefill latency by up to 57.9%.

4.3.2 GPU-CPU Coordination Challenges

Running LoRA on CPU concurrently with GPU-based LLM inference requires layer-wise synchronization at each attention layer: CPU LoRA needs x^i (the layer input) from the GPU, and the GPU needs $x^i AB$ back from the CPU before proceeding. Naively, this requires explicit PyTorch synchronization that blocks CUDA kernel launches.

Three optimizations address this:

(1) Sync-Free CPU LoRA Invocation. A custom CUDA operator fuses the MemCpy (F_2) and signaling (F_3) operations into a single asynchronous kernel $[F'_2, F'_3]$. Since CUDA enforces sequential ordering within a device queue, data validity is preserved—but F_4 (the next LLM kernel) no longer blocks waiting for F_2 to complete. This reduces prefill iteration latency by 16%.

(2) Shared Memory Data Transfer. Instead of UNIX domain sockets for inter-process communication between the base LLM process and CPU LoRA processes, CaraServe uses

POSIX shared memory. This eliminates serialization overhead and reduces IPC latency to < 1 ms regardless of the number of CPU LoRA processes (vs. linear growth for domain sockets).

(3) Profiling-Guided CPU Parallelization. CaraServe profiles single-core LoRA computation to determine the maximum token count c per core. For a prompt of L tokens, it allocates $\lceil L/c \rceil$ CPU cores, each running an isolated process (pinned via `numactl`). This achieves $1.7\times$ speedup over PyTorch native multi-threading for equivalent workloads.

4.4 Core Novelty 2: Rank-Aware Scheduling

4.4.1 Performance Modeling

The decoding latency of a batch S of heterogeneous LoRA adapters depends on the kernel used:

BGMV (Punica) pads lower-rank adapters to the maximum rank:

$$\text{PERF}_{\text{BGMV}}(S) = \alpha_B \cdot |S| \cdot \max_{i \in S} \text{rank}(i) + \beta_B$$

MBGMV (S-LoRA) uses no padding; latency scales with sum of ranks:

$$\text{PERF}_{\text{MBGMV}}(S) = \alpha_M \cdot \sum_{i \in S} \text{rank}(i) + \beta_M$$

Both models are fitted via lightweight profiling (varying batch sizes and rank mixes on a target GPU). Both achieve $R^2 = 0.96$, confirming that GPU memory bandwidth is the binding constraint (both kernels use $> 70\%$ of available memory bandwidth per NVIDIA Nsight Compute profiling).

4.4.2 Rank-Aware Scheduling Policy (Algorithm 1)

For each incoming request, the scheduler:

1. Identifies candidate servers (matching base LLM, hosting the required adapter, sufficient GPU memory).
2. For each candidate, computes a *cost score*:

$$\text{cost} = \frac{\Delta_{\text{prefill}}}{\text{avg_resp_len}} + \Delta_{\text{decode}}$$

where Δ_{prefill} and Δ_{decode} are the marginal latency increases if the new request joins that server’s batch.

3. Assigns a large penalty if adding the request would cause $\text{PERF}(S \cup \{r\}) > \text{SLO}$.
4. Routes to the server with minimum total cost $\times |\text{requests}|$.

Illustrative example: With a 36 ms decode SLO, Instance 1 (24 rank-32 requests) and Instance 2 (16 rank-64 requests). A new rank-64 request arrives. Under BGMV, routing to Instance 1 would raise max rank from 32 to 64, causing SLO violation—so Instance 2 is chosen. Under MBGMV, since Instance 2’s sum of ranks is already higher, routing to Instance 1 preserves the SLO. The kernel-aware model produces the correct decision in both cases.

5 Infrastructure and Experimental Setup

5.1 dLoRA Infrastructure

Parameter	Value
Cluster	4 nodes, $8 \times$ NVIDIA A800 80GB per node = 32 GPUs total
CPUs per node	128 CPUs
Host memory per node	2048 GB
Network	200 Gbps InfiniBand NIC
Software	PyTorch 2.1.0, CUDA 12.2, Ray + PuLP ILP solver
Base LLM	Llama-2 7B, 13B, 70B
LoRA rank	8 (matching original LoRA paper settings)
Models served	128 (7B), 128 (13B), 32 (70B)
Tensor parallelism	4 GPUs per replica for Llama-2-70B
Workload traces	ShareGPT (lengths) + Azure MAF1 (2019) + MAF2 (2021)
SLO	$10 \times$ single decode iteration latency = 0.5 s/token

5.2 CaraServe Infrastructure

Config	Base Model	GPU Setup
Single-GPU	Llama2-7B (4096-dim, 32 layers)	$1 \times$ NVIDIA A10 (24 GB)
Multi-GPU (medium)	Llama2-13B (5120-dim, 40 layers)	$2 \times$ NVIDIA A10 (24 GB)
Multi-GPU (large)	Llama2-70B (8192-dim, 80 layers)	$4 \times$ NVIDIA A100 (80 GB)

LoRA adapters: applied to W_Q, W_K, W_V . Ranks: $\{8, 32, 64, 128\}$.
 Workloads: Synthetic (Poisson, RPS 3–9) + MAF trace (512 adapters).
 Scheduler simulation: 60 servers, 40,000 MAF functions, RPS ≈ 340 .
 SLO: $1.5 \times$ HF-PEFT latency (time per token); testbed SLO: similar.

6 Experiments and Key Results

6.1 dLoRA Experiments

6.1.1 End-to-End Throughput (*Figure 9 in paper*)

dLoRA is compared against vLLM (uniform allocation, one instance per LoRA) and PEFT (enhanced with continuous batching and PagedAttention). The primary metric is maximum throughput (req/s) under a 0.5 s/token latency SLO.

Setting	Trace	vs. vLLM	vs. PEFT	Comment
128× Llama-7B	MAF1/MAF2	up to 10.6×	up to 11.5×	All weights fit in GPU
128× Llama-13B	MAF1/MAF2	up to 53.0 ×	up to 15.0×	vLLM must swap model params
32× Llama-70B	MAF1/MAF2	up to 57.9 ×	up to 26.0 ×	4-GPU tensor parallelism

The largest gains against vLLM occur for 13B/70B because vLLM instantiates a full copy of each model; with 128 Llama-13B models the GPU memory is exhausted and vLLM must swap, causing catastrophic slowdown.

6.1.2 Comparison with S-LoRA (Figure 14 in paper)

Under the same single-GPU setup (8 LoRA models, Llama-2-7B), dLoRA consistently outperforms S-LoRA by up to **1.8×** **lower average latency** across all skewness values. At high skewness (many requests targeting one adapter), dLoRA’s merged inference mode eliminates the adapter computation overhead that S-LoRA always incurs. At low skewness, both systems use unmerged inference and perform similarly.

6.1.3 Dynamic Batching Ablation (Figure 10)

Three variants evaluated: Merged-only, Unmerged-only, and dLoRA.

- At low skewness (diverse requests): dLoRA outperforms Merged-only by up to **3.9×** (avg latency) and **5.2×** (P90 latency).
- At high skewness (dominant adapter): dLoRA outperforms Unmerged-only by up to **2.4×** (avg) and **2.5×** (P90).
- dLoRA always achieves the optimal tradeoff by dynamic switching.

6.1.4 Load Balancing Ablation (Figure 11)

Three variants: Round-Robin (RR), Proactive-only, and full dLoRA.

- dLoRA vs. RR: **3.6×** reduction in queuing delay.
- dLoRA vs. Proactive-only: **1.4×** improvement (reactive migration handles variable-length imbalance).
- Under increasing input/output length ratios: dLoRA’s average latency remains stable; RR degrades up to 23.5×, Proactive up to 2.39×.

6.1.5 Scalability (Figure 12)

As the number of adapters increases from 16 to 128 at constant request rate, dLoRA maintains stable throughput. vLLM throughput decreases by 3.0× (memory exhaustion); PEFT throughput decreases by 1.5× (no cross-adapter batching).

6.1.6 Overhead Analysis (Figure 15)

The ILP solver and mode-switching overhead are negligible: **<3.3% of total latency** at all arrival rates (8–32 req/s). Actual inference time accounts for **>96.7%** consistently.

6.1.7 GPU Utilization (Figure 13)

dLoRA achieves near-optimal SM utilization and memory utilization across all 8 replicas—comparable to the ideal case where all requests target the same adapter type.

6.2 CaraServe Experiments

6.2.1 End-to-End Single-GPU Performance (Figures 10, 11)

Evaluated on Llama2-7B with rank-64 adapters, RPS = 9, 512 adapters, Poisson arrivals:

System	TTFT overhead vs. Cached	TPT overhead	Latency overhead
OnDmd	+412%	+71%	+50%
S-LoRA	+451%	+78%	+50%
CaraServe	+22%	+11%	+9%

(TTFT = time to first token; TPT = time per token; “Cached” = oracle with all adapters pre-loaded in GPU.)

CaraServe rivals the performance of the CACHED oracle while S-LoRA/OnDmd introduce prohibitive cold-start delays.

6.2.2 Sensitivity Analysis (Figure 13)

CaraServe’s advantage holds across:

- **Lower ranks** (rank 32 vs. 64): S-LoRA still introduces 88–126% TTFT overhead; CaraServe reduces this to 36%.
- **Lower load** (RPS 6): S-LoRA overhead drops to 41%; CaraServe is $\sim 1\%$ for TTFT.

6.2.3 Scaled Production Workloads (Figure 14)

Using MAF trace with increasing adapter counts (128/256/512):

- At 128 adapters: cold-start impact is small; TTFT overhead for all systems is comparable.
- At 512 adapters: S-LoRA/OnDmd TTFT overhead rises to **39%**; CaraServe stays at **7%**.
- CaraServe consistently maintains near-CACHED performance regardless of scale.

6.2.4 Multi-GPU Performance (Figure 15)

Evaluated on Llama2-13B (2×A10) and Llama2-70B (4×A100) vs. OnDmd:

- Llama2-13B: CaraServe achieves **20.2% speedup** in end-to-end request latency.
- Llama2-70B: **18.5% speedup**.

- Cold-start overhead reduced by over 50% vs. OnDmd in both cases.

6.2.5 Scheduler Performance (Figures 19, 20)

Large-scale simulation (60 servers, 40K functions, RPS $\approx$ 340):

- CaraServe scheduler: **99% SLO attainment** vs. MostIdle (27%), Random (25%), FirstFit (1%) using MBGMV.
- Average time per token speedup: 16.1% (vs. MostIdle), 18.8% (vs. Random), **36.4%** (vs. FirstFit).

8-instance testbed (A10 GPUs):

- CaraServe: **80% SLO attainment** vs. MostIdle (42%), Random (43%), FirstFit (22%).

6.2.6 Microbenchmarks (Figures 16, 17, 18)

- **Sync-free kernel**: Up to 16% prefill latency reduction vs. PyTorch native.
- **Shared memory IPC**: Near-constant latency (<1 ms) vs. linear growth for domain sockets.
- **Multi-CPU parallelization**: 1.7 $\times$ speedup over PyTorch multi-threading at 8 cores; enables CaraServe to handle prompts up to 128 tokens without CPU becoming the bottleneck.

7 Repository Analysis

7.1 dLoRA Artifact (LLMServe/dLoRA-artifact)

Repository stats: 32 stars, 8 forks, 9 commits (OSDI artifact evaluation submission).

Top-level directory structure:

- **ae_scripts/**: Artifact evaluation scripts corresponding to all major figures and tables in the paper. Contains numbered scripts for each experiment (e.g., `run_e2e.sh`, `run_dynamic_batching.sh`, `run_load_balance.sh`, `run_scalability.sh`).
- **benchmarks/**: End-to-end benchmark drivers for all workload traces (MAF1, MAF2, ShareGPT), including request generators, trace samplers, and result parsers.
- **dloRA/** (core system):
 - **batching/**: Dynamic batching engine including Algorithm 1, 2, and 3 from the paper—the credit-based starvation prevention logic and adaptive threshold tuning are implemented here.
 - **scheduler/**: Global load balancer, proactive dispatcher, reactive migration controller.
 - **ilp/**: ILP formulation using PuLP, including constraint construction, solver invocation, and selective migration heuristics (top- K overloaded/underloaded replicas, constraint relaxation).
 - **memory/**: GPU memory management for LoRA adapters and KV cache, workload-aware allocation, async swapping between GPU and host.
 - **models/**: Extended vLLM execution engine for OPT and Llama-2 with LoRA support (merged and unmerged inference). The unmerged inference path uses

- PyTorch `einsum` with a request-type mapping matrix.
- `tensor_parallel/`: Megatron-style tensor parallelism extended to LoRA adapters.
- `baselines/`: Scripts to run vLLM (uniform allocation) and PEFT with the same traces for fair comparison.
- `configs/`: YAML configs for all model sizes (7B/13B/70B), cluster topologies, and workload parameters.
- `docker/`: Dockerfile for reproducible environment (PyTorch 2.1.0, CUDA 12.2, Ray).

Key implementation details from the repository:

- The ILP solver uses PuLP with the CBC backend; with selective migration ($K = 4$), solve time is consistently < 50 ms.
- The mode-switching logic checks the ratio $|R_{\text{merge}}|/|B_{\text{fcfs}}|$ at every iteration; the break-point detection uses a counter-based approach to amortize threshold tuning overhead.
- Unmerged inference constructs a sparse one-hot type matrix per batch and uses `torch.einsum` for parallel adapter computation, avoiding separate kernel launches per adapter.
- The proactive placement algorithm is seeded with the preceding 24-hour trace; the short-term burst handler updates estimated pending times every 100 ms.
- Artifact scripts are marked as reproduced (OSDI AEC badges: Available, Functional, Reproduced).

7.2 CaraServe (ModelTC/lightllm)

Repository status: CaraServe was stated in the paper to be released after the double-blind review process concludes. As of the survey date, CaraServe’s code is listed as a research project built on top of LightLLM (<https://github.com/ModelTC/lightllm>), and LightLLM’s README explicitly credits CaraServe as one of the research systems built atop it. A standalone CaraServe repository has not been independently released.

What is available in LightLLM:

- The continuous batching engine and token-level KV cache management that CaraServe builds on.
- The BGMV kernel from Punica (adapted) for GPU LoRA computation.
- LoRA adapter management (loading, offloading, metadata tracking).
- The Llama2 inference module that CaraServe extends.

What is not yet public:

- The custom sync-free CUDA operator (fused async MemCpy + signaling).
- The shared-memory IPC layer between base LLM process and CPU LoRA processes.
- The rank-aware scheduling algorithm (Algorithm 1) and performance models.
- Evaluation scripts and workload traces.

Implementation language: Python (LightLLM core) + CUDA/Triton (kernels) + PyBind11 (C++ extensions).

8 Comparative Analysis: dLoRA vs. CaraServe

8.1 Problem Focus

Dimension	dLoRA	CaraServe
Primary focus	Dynamic merged/unmerged switching + cluster load balancing	Cold-start elimination + SLO-aware scheduling
Intra-replica innovation	Credit-based dynamic batching with adaptive thresholds	CPU-GPU coordinated prefill execution
Inter-replica innovation	ILP-based request-adapter co-migration	Rank-aware cost-score routing
Cluster scale	4 nodes $\times$ 8 GPUs = 32 GPUs	8–60 instances (simulation)
Cold-start handling	On-demand async loading (inherits Punica/S-LoRA)	CPU-assisted early-start prefill
Adapter rank handling	Homogeneous (rank=8 in experiments)	Heterogeneous ($r \in \{8, 32, 64, 128\}$)
Merged inference	Yes, dynamically selected	No (always-unmerged)
SLO model	Latency SLO = $10 \times$ single decode iter	Decoding per-token SLO via perf model
Venue	USENIX OSDI 2024 (peer-reviewed)	arXiv Jan 2024 (not yet peer-reviewed)
Baseline	vLLM (uniform), PEFT + S-LoRA	OnDmd, S-LoRA (CACHED oracle)
Framework	vLLM + Ray + PuLP	LightLLM + Flask + SQLite
LoC	$\sim 6,200$	Not disclosed
Code availability	Full artifact (OSDI AEC)	Partial (in LightLLM)

8.2 Orthogonality and Composability

dLoRA and CaraServe address different bottlenecks and are largely orthogonal:

- CaraServe’s CPU-assisted cold-start elimination could be integrated into dLoRA’s replica engine.
- dLoRA’s ILP-based load balancing could replace CaraServe’s rank-aware routing (or complement it at the cluster level while CaraServe handles per-server scheduling).
- CaraServe’s performance model for heterogeneous ranks could inform dLoRA’s threshold adaptation.

A unified system combining all four innovations (dynamic merge/unmerge + ILP migration + CPU-assisted cold-start + rank-aware scheduling) would be more powerful than either system alone.

8.3 Relationship to First-Generation Systems

Feature	Punica	S-LoRA	dLoRA	CaraServe
Cross-adapter batching	SGMV kernel	MBGMV/MBGMM	Msum (unmerged)	BGMV (adapted)
Dynamic merged mode	No	No	Yes	No
Cold-start handling	Async load	Prefetch	Async load	CPU-assisted
Cluster load balance	Ad-hoc migration	No	ILP migration	co- Rank-aware routing
Max adapters	~32	2000+	128 (tested)	512 (tested)

9 Critical Discussion and Limitations

9.1 dLoRA

- **Merge/unmerge switching overhead:** Each switch requires a matrix addition/-subtraction (BA), which is 100 ms in the paper’s example. At high request rates with volatile workloads, oscillation between modes could be costly.
- **ILP scalability:** Even with selective migration (top- K), ILP complexity grows with $K^2 \times |\text{requests}|$. For very large clusters ($N > 1000$ replicas), further approximations may be needed.
- **LoRA rank assumption:** dLoRA experiments use rank=8 for all adapters (following the original LoRA paper). The dynamic batching and co-migration algorithms are not evaluated with heterogeneous ranks—an important gap given that production deployments often mix ranks.
- **Recomputation cost for migration:** Token-transfer-based re-prefill is faster than KV cache transfer for short sequences, but for long-context requests (e.g., 8K+ tokens), re-prefill can be very expensive.

9.2 CaraServe

- **CPU bottleneck:** CPU parallelism is limited. For very long prompts (>128 tokens) the profiling-guided scheme assigns many cores, but at extremely high RPS the CPU compute pipeline could become the bottleneck, not the cold-start itself.
- **No merged inference:** CaraServe always uses unmerged inference, missing the opportunity to reduce adapter computation overhead when requests are skewed—an opportunity dLoRA exploits.
- **No load balancing:** CaraServe’s routing is based on rank-aware cost scoring per request but does not account for cluster-level load imbalance from variable output lengths—exactly what dLoRA’s reactive migration addresses.

- **No public standalone code:** The absence of a full open-source release makes full reproducibility challenging.
- **Comparison gap:** CaraServe does not compare against dLoRA, which was published concurrently (OSDI 2024 vs. arXiv Jan 2024).

10 Broader Literature Context

10.1 Position in the LoRA Serving Lineage

System	Year	Venue	Key Contribution
Punica	2023	MLSys	SGMV kernel, cluster scheduling
S-LoRA	2024	MLSys	Unified paging, 2000+ adapters, TP
dLoRA	2024	OSDI	Dynamic merge/unmerge, ILP migration
CaraServe	2024	arXiv	CPU-assisted cold-start, rank-aware scheduling

10.2 Related Systems Work

Both papers position themselves against a common background of LLM serving infrastructure: Orca [3] (continuous batching), vLLM [4] (PagedAttention), FlashAttention [5] (IO-aware attention), and Megatron-LM [6] (tensor parallelism).

Load balancing inspiration (dLoRA): Pegasus [14] and Scarlett [15] motivate selective replication. AlpaServe [16] and SHEPHERD [17] motivate statistical multiplexing. However, these systems do not model the adapter-request dependency that makes LoRA load balancing more complex.

Heterogeneous compute (CaraServe): The idea of CPU-assisted GPU computation for hiding memory transfer latency appears in CPU-GPU offloading literature (e.g., FlexGen [18]), but CaraServe’s application to the cold-start problem in LoRA serving is novel.

10.3 Open Problems for Future Work

1. **Multi-base-model serving:** Both systems assume all adapters derive from the same backbone. Serving adapters from different base models (or different quantization levels) in a unified system remains unsolved.
2. **Heterogeneous rank + dynamic merge/unmerge:** Combining CaraServe’s rank-aware approach with dLoRA’s dynamic switching—the performance model needs to capture how rank heterogeneity interacts with merged vs. unmerged modes.
3. **Speculative decoding for LoRA:** Speculative decoding (small draft model + large verifier) combined with multi-adapter batching could yield compounded throughput gains.
4. **Prefix sharing across adapters:** Adapters sharing the same base model and prompt prefix could share KV cache prefix (as in SGLang’s RadixAttention), reducing redundant computation.
5. **Online adapter learning:** Systems that support continuous fine-tuning (streaming updates to adapter weights) while simultaneously serving existing requests.
6. **Energy efficiency:** None of the current systems model power consumption or carbon cost—important for large-scale cloud deployments.

11 Summary

dLoRA and CaraServe advance the state of multi-tenant LoRA serving in 2024 by tackling two distinct second-order problems left unsolved by Punica and S-LoRA.

dLoRA’s central insight—that the benefit of cross-adapter batching depends on workload skewness and should be dynamically toggled—leads to a practical system that beats S-LoRA by up to $1.8\times$ in latency and delivers up to $57.9\times$ throughput improvement over naive multi-model serving at cluster scale. Its ILP-based co-migration is a principled, theoretically grounded solution to the load imbalance problem unique to LoRA serving.

CaraServe’s central insight—that the cold-start penalty can be eliminated by running LoRA computation on CPU while the adapter loads onto GPU—eliminates the primary latency bottleneck in adapter-rich deployments. Its rank-aware scheduler achieves 99% SLO attainment in simulation, dramatically outperforming rank-unaware alternatives.

Together, the four systems (Punica, S-LoRA, dLoRA, CaraServe) constitute the first complete body of LoRA serving systems research, addressing kernel efficiency, memory scalability, inference mode selection, cluster orchestration, cold-start handling, and SLO-aware scheduling. The remaining open challenges—multi-base-model serving, heterogeneous rank + dynamic mode selection, and online learning—define the frontier for future work.

References

- [1] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [2] H. Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” arXiv:2307.09288, 2023.
- [3] G.-I. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” *USENIX OSDI*, 2022.
- [4] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *ACM SOSP*, 2023.
- [5] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022.
- [6] M. Shoeybi et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” arXiv:1909.08053, 2019.
- [7] L. Chen et al., “Punica: Multi-Tenant LoRA Serving,” *MLSys*, 2024.
- [8] Y. Sheng et al., “S-LoRA: Serving Thousands of Concurrent LoRA Adapters,” *MLSys*, 2024.
- [9] B. Wu et al., “dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving,” *USENIX OSDI*, 2024.
- [10] S. Li et al., “CaraServe: CPU-Assisted and Rank-Aware LoRA Serving for Generative LLM Inference,” arXiv:2401.11240, 2024.
- [11] ModelTC, “LightLLM: Python-Based LLM Inference and Serving Framework,” <https://github.com/ModelTC/lightllm>, 2023.
- [12] P. Moritz et al., “Ray: A Distributed Framework for Emerging AI Applications,” *USENIX OSDI*, 2018.
- [13] S. Mitchell et al., “PuLP: A Python Linear Programming API,” <https://github.com/coin-or/pulp>, 2009.
- [14] J. Li et al., “Pegasus: Tolerating Skewed Workloads in Distributed Storage with In-Network Coherence Directories,” *USENIX OSDI*, 2020.
- [15] G. Ananthanarayanan et al., “Scarlett: Coping with Skewed Content Popularity in MapReduce Clusters,” *EuroSys*, 2011.
- [16] Z. Li et al., “AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving,” *USENIX OSDI*, 2023.
- [17] H. Zhang et al., “SHEPHERD: Serving DNNs in the Wild,” *USENIX NSDI*, 2023.
- [18] Y. Sheng et al., “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU,” *ICML*, 2023.
- [19] Z. Zhou et al., “PetS: A Unified Framework for Parameter-Efficient Transformers Serving,” *USENIX ATC*, 2022.

- [20] T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” *NeurIPS*, 2023.
- [21] S. Mangrulkar et al., “PEFT: State-of-the-Art Parameter-Efficient Fine-Tuning Methods,” <https://github.com/huggingface/peft>, 2022.
- [22] ShareGPT Team, “ShareGPT: Share your wildest ChatGPT conversations,” <https://sharegpt.com/>, 2023.
- [23] M. Shahrad et al., “Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider,” *USENIX ATC*, 2020.

Literature Survey:

Multi-Tenant LoRA Serving Systems

Punica & S-LoRA: Architecture, Novelty, Experiments, and Comparative Analysis

Papers Covered:

- [1] Chen et al., “Punica: Multi-Tenant LoRA Serving,” arXiv:2310.18547, 2023.
- [2] Sheng et al., “S-LoRA: Serving Thousands of Concurrent LoRA Adapters,” MLSys 2024.

GitHub Repositories:

<https://github.com/punica-ai/punica>
<https://github.com/S-LoRA/S-LoRA>

May 2026

Contents

1	Introduction and Motivation	3
2	Background	3
2.1	Transformer Text Generation	3
2.2	Low-Rank Adaptation (LoRA)	4
2.3	Prior Art	4
3	Punica: Multi-Tenant LoRA Serving	4
3.1	Core Problem Formulation	4
3.2	Core Novelty: SGMV Kernel	4
3.2.1	Kernel Scheduling	5
3.2.2	SGMV vs. Baseline LoRA Operators	5
3.3	System Architecture	5
3.4	Scheduling	5
3.4.1	New Request Scheduling	5
3.4.2	On-Demand LoRA Loading	5
3.4.3	Request Migration	6
3.5	KV Cache Memory Layout	6
3.6	Implementation Stack	6
4	S-LoRA: Serving Thousands of Concurrent LoRA Adapters	6
4.1	Problem Scope Extension	6
4.2	Core Novelty 1: Unified Paging	6
4.3	Core Novelty 2: Heterogeneous Batching Kernels	7
4.4	Core Novelty 3: S-LoRA Tensor Parallelism	7
4.5	Batching and Scheduling	8
4.5.1	Batching Strategy	8
4.5.2	Adapter Clustering	8
4.5.3	Admission Control (Early Abort)	8
4.6	Implementation Stack	8
5	Experimental Setup and Benchmarks	8
5.1	Punica Experiments	8
5.1.1	Testbeds	8
5.1.2	Models	9
5.1.3	Workload Distributions	9
5.1.4	Baselines	9
5.1.5	Microbenchmarks	9
5.1.6	End-to-End Results	9
5.1.7	Key Results	9
5.2	S-LoRA Experiments	10
5.2.1	Testbeds	10
5.2.2	Models and Configurations	10
5.2.3	Workload Generation	10
5.2.4	Baselines	10
5.2.5	Metrics	10
5.2.6	Key Results	11

6	Comparative Analysis	11
6.1	Scope and Focus	11
6.2	Relationship Between the Two Works	11
6.3	Key Shared Principles	12
6.4	Performance Summary	12
7	GitHub Repository Analysis	12
7.1	Punica (punica-ai/punica)	12
7.2	S-LoRA (S-LoRA/S-LoRA)	13
8	Critical Discussion and Limitations	13
8.1	Punica	13
8.2	S-LoRA	13
9	Connections to Broader Literature	14
9.1	LLM Inference Optimization	14
9.2	Parameter-Efficient Fine-Tuning	14
9.3	Multi-Model Serving	14
10	Research Directions and Open Problems	14
11	Summary Table	14
12	Conclusion	14

1 Introduction and Motivation

The proliferation of large language models (LLMs) has accelerated the adoption of parameter-efficient fine-tuning methods, most notably Low-Rank Adaptation (LoRA) [1]. LoRA enables practitioners to adapt a single pre-trained backbone to many downstream tasks by training only compact rank-decomposition matrices—typically adding only 0.1%–1% additional parameters per model. This has led to an explosion in the number of task-specific LoRA adapters, collectively derived from the same base model (e.g., Llama-2 [2]).

The Serving Challenge. When ML providers must serve many tenants simultaneously, the naive strategy—treating each LoRA model as a fully independent model—suffers from severe GPU inefficiency:

- Each model requires its own GPU memory footprint for the backbone weights.
- Batching across tenants is impossible because requests target different models.
- GPU utilization collapses during the memory-bandwidth-bound decode stage.

Two concurrent systems address this problem from complementary angles:

Punica (Chen et al., 2023) introduces the SGMV CUDA kernel to batch GPU operations across heterogeneous LoRA models and provides cluster-level scheduling for multi-tenant workloads.

S-LoRA (Sheng et al., 2024) addresses scalability to thousands of adapters via Unified Paging memory management, heterogeneous batching kernels, and a novel tensor-parallelism strategy.

This survey provides an in-depth technical analysis of both systems, covering their core novelties, implementation, experimental methodology, results, and comparative positioning—targeted at readers conducting a literature survey in the area of efficient LLM inference and serving.

2 Background

2.1 Transformer Text Generation

Transformer-based LLMs generate text through two stages:

- **Prefill:** Encodes the user prompt and produces a Key-Value cache (KV cache) plus the first output token. Computation is proportional to prompt length s : self-attention is $O(s^2d)$.
- **Decode:** Iteratively appends one token at a time, re-using the KV cache. Each step is a single matrix-vector multiply—highly memory-bandwidth-bound and underutilizing GPU compute.

Batching is the primary lever for improving GPU utilization during the decode stage: with batch size 32 vs. 1, decode latency increases only modestly (e.g., 11 ms to 13 ms for short sequences on A100), yet throughput scales near-linearly.

2.2 Low-Rank Adaptation (LoRA)

For a pre-trained weight matrix $W \in \mathbb{R}^{h_1 \times h_2}$, LoRA introduces two trainable matrices $A \in \mathbb{R}^{h_1 \times r}$ and $B \in \mathbb{R}^{r \times h_2}$ with rank $r \ll \min(h_1, h_2)$. The adapted forward pass becomes:

$$h = x(W + AB) = xW + xAB$$

LoRA is applied to all dense projection matrices in the transformer (QKV projections, output projection, and MLP layers). Typical ranks are $r \in \{8, 16, 32, 64\}$ versus hidden dimensions of 4096–8192, yielding very low storage overhead.

2.3 Prior Art

- **Orca** [3]: Introduced iteration-level (token-level) continuous batching for LLM serving, enabling dynamic addition and removal of requests.
- **vLLM** [4]: Extended Orca with PagedAttention, treating the KV cache like virtual memory pages to minimize fragmentation.
- **FlashAttention** [5]: IO-aware self-attention kernel reducing memory traffic via block-wise computation.
- **PetS** [9]: Batched requests to different adapter types for BERT-scale models, but did not support generative inference or cross-adapter concurrent execution.

3 Punica: Multi-Tenant LoRA Serving

3.1 Core Problem Formulation

Punica frames multi-tenant LoRA serving around three design guidelines:

- G1** Consolidate workloads to minimize active GPUs.
- G2** Enable batching across *different* LoRA models (not just within the same model).
- G3** Focus optimization on the decode stage, which dominates serving latency.

3.2 Core Novelty: SGMV Kernel

The central contribution of Punica is the **Segmented Gather Matrix-Vector Multiplication (SGMV)** CUDA kernel. Given a batch of n LoRA models with inputs $\{x_i\}$ and weight pairs $\{(A_i, B_i)\}$, the LoRA add-on computation is:

$$Y[s_i : s_{i+1}] += X[s_i : s_{i+1}] \cdot W_i$$

This is executed as two sequential kernel launches:

1. **SGMV-shrink**: $v += xA$ — shrinks from high-dimensional input to low-rank intermediate.
2. **SGMV-expand**: $y += vB$ — expands from low-rank back to high-dimensional output.

3.2.1 Kernel Scheduling

SGMV-expand: The weight matrix A is split along the output feature dimension into blocks $A^{(1)}, \dots, A^{(n)}$, dispatched to different thread blocks in parallel.

SGMV-shrink: Uses the *Split-K* strategy— B is split along the input feature dimension, partial sums computed across thread blocks, then reduced via grid synchronization.

For the special case where every request has a *distinct* LoRA (each segment size = 1), computation degrades to matrix-vector multiplications that are fully IO-bound; the kernel switches to a bandwidth-maximizing schedule and forgoes Tensor Cores.

GPU Tensor Cores are used for acceleration wherever operational intensity is sufficient.

3.2.2 SGMV vs. Baseline LoRA Operators

Punica benchmarks SGMV against two PyTorch baselines:

- **Loop:** Sequential per-model matrix-vector products; terrible scaling.
- **Gather-BMM:** Stack weight matrices and call `torch.bmm()`; incurs $s_n \times h_i \times h_o \times 2$ extra memory I/O.

SGMV outperforms both across all workload distributions (Distinct, Uniform, Skewed, Identical) at all batch sizes.

3.3 System Architecture

Frontend servers expose RESTful API to users; forward requests to the scheduler.

Scheduler maintains a global view of all GPU states; dispatches requests and manages migrations. Written in Rust.

Runners one per GPU server; communicate with the scheduler via WebSocket (unary RPC + streaming). Spawn Python subprocesses for GPU execution.

GPU execution backbone (pre-trained model) resides permanently in GPU memory; only LoRA matrices A, B are loaded on demand.

3.4 Scheduling

3.4.1 New Request Scheduling

A new request is assigned to the GPU with the *largest current batch size* that still satisfies:

1. Batch size < max batch size (set to 32 for A100 based on profiling).
2. Sufficient KV cache memory for the request.

Ties are broken by GPU UUID. Fully-occupied GPUs queue requests in FCFS order. This strategy keeps busy GPUs busy and idle GPUs idle, enabling easy scale-up/down.

3.4.2 On-Demand LoRA Loading

LoRA matrices (A, B) are loaded asynchronously from host memory when a new request requiring an unloaded model is admitted. Loading the entire model takes ≈ 2 ms on PCIe Gen4, which overlaps with ongoing GPU computation.

3.4.3 Request Migration

When a GPU exhausts KV cache space, it *evicts* the newest request (preserving FCFS order). The evicted request is re-added to another GPU, which re-runs the prefill step on the original prompt plus all generated tokens so far (recomputation rather than KV cache transfer). PagedAttention evidence [4] supports that recomputation latency is competitive with or better than transfer.

3.5 KV Cache Memory Layout

Punica replaces HuggingFace’s KV cache layout $[L, 2, B, N, S, D]$ (where the batching dimension B is non-outermost, forcing inseparable batching) with a paged layout:

$$\left[\sum_i \lceil S_i/P \rceil, L, 2, N, P, D \right]$$

where P is the page size and S_i is the sequence length of request i . The batch dimension is outermost, enabling continuous batching and per-request eviction.

3.6 Implementation Stack

- **CUDA kernels** exposed as PyTorch extensions via PyBind11.
- **Llama model** adapted from HuggingFace Transformers.
- **FlashInfer** [17]: fast, padding-free batch decoding with paged KV cache.
- **LayerNorm fusion**: reduces LayerNorm latency from 110 μs to 4 μs .
- Prefill and decode batched together in one model invocation (prefill batch size capped at 1 to minimize latency penalty).
- Requests sharing the same LoRA model placed consecutively in the batch to maximize SGMV operational intensity.

4 S-LoRA: Serving Thousands of Concurrent LoRA Adapters

4.1 Problem Scope Extension

Where Punica focuses on a shared GPU cluster serving dozens of LoRA models, S-LoRA targets the regime of *thousands* of concurrent adapters on a single machine. This introduces three additional challenges:

1. **Memory management at scale**: GPU memory cannot hold thousands of adapters; they must be dynamically fetched from host DRAM.
2. **Heterogeneous batching**: Adapters in a running batch may have different ranks, requiring specialized kernels that avoid padding overhead.
3. **Multi-GPU tensor parallelism**: LoRA adapters introduce new weight matrices needing careful partition strategies to minimize communication overhead.

4.2 Core Novelty 1: Unified Paging

S-LoRA observes a structural similarity between KV cache tensors and LoRA adapter weights:

Property	KV Cache	LoRA Weight
Shape	(S, H) per layer	(R, H) per layer
Lifetime	Request duration	Request duration
Size variability	Varies with seq. length S	Varies with rank R
Shared dimension	H (hidden size)	H (hidden size)

Exploiting this analogy, S-LoRA extends PagedAttention [4] to a **Unified Memory Pool** that jointly manages both KV caches and adapter weights. The pool is statically allocated at startup using all available GPU memory not occupied by the base model and activation tensors. Memory is paged in units of H elements: a KV cache of length S consumes S pages; a LoRA weight of rank R consumes R pages.

This eliminates fragmentation arising from independently managing two dynamic memory regions with different size distributions.

4.3 Core Novelty 2: Heterogeneous Batching Kernels

Because the unified memory pool stores weights non-contiguously, S-LoRA implements two custom CUDA kernels:

MBGMM (Multi-size Batched Gather Matrix-Matrix Multiplication): Used in the *prefill* stage, processes sequences of tokens and gathers adapter weights of differing ranks from the pool. Implemented in Triton with tiling.

MBGMV (Multi-size Batched Gather Matrix-Vector Multiplication): Used in the *decode* stage, handles single tokens and varying ranks. Implemented in both Triton and a modified version of Punica’s SGMV kernel extended to support non-contiguous memory and multiple ranks per batch.

S-LoRA acknowledges Punica as concurrent work and explicitly notes that some kernel code was adapted from Punica’s implementation with extensions for rank heterogeneity and non-contiguous memory.

4.4 Core Novelty 3: S-LoRA Tensor Parallelism

For models exceeding a single GPU’s memory (Llama-30B, 70B), S-LoRA proposes a novel tensor parallelism strategy aligned with Megatron-LM [6]:

Feed-Forward Module Partitioning:

- W_1 (first MLP weight): column-partitioned; A_1, B_1 column-partitioned; intermediate gathered via all-gather.
- W_2 (second MLP weight): row-partitioned; A_2 row-partitioned, B_2 column-partitioned; all-reduce fused with final result.

The communication overhead added by LoRA is proportional to rank r , not hidden size h . The additional cost is:

$$\frac{5(N-1)Br}{N} \ll \frac{2(N-1)Bh}{N} \quad (\text{base model all-reduce})$$

since $r \ll h$. This makes the LoRA communication overhead negligible.

4.5 Batching and Scheduling

4.5.1 *Batching Strategy*

S-LoRA separates base model computation (batched via standard GEMM) from LoRA computation (batched via MBGMV/MBGMM). This allows the base model to exploit large batch sizes regardless of adapter diversity. The LoRA add-on xAB is computed on-the-fly rather than merging into the base model weights.

Ablation results show that merging adapters is better only for a single adapter (one-time cost); with 2+ adapters, switching overhead causes GPU under-utilization and on-the-fly computation wins.

4.5.2 *Adapter Clustering*

To reduce the number of active adapters per batch (enabling larger KV cache allocation and higher batch sizes), S-LoRA optionally prioritizes requests sharing the same adapter—a strategy called **adapter clustering**. A parameter d controls cluster size. Experiments show modest throughput gains from small d , with the benefit being more pronounced at high request variability (large α , large CV).

4.5.3 *Admission Control (Early Abort)*

S-LoRA implements a soft-SLO admission control strategy (“early abort”) with theoretical grounding:

Theorem (S-LoRA): When the reward function $r(t)$ is non-increasing and concave, the optimal strategy to maximize total user satisfaction is to serve the most recent l requests (where l is the estimated capacity).

In practice, the system estimates current request rate R_1 and serving capacity R_2 . If $R_1 > R_2$ (overloaded), it prioritizes newest requests; otherwise it serves in FCFS order. Requests already certain to miss the SLO are aborted proactively.

4.6 Implementation Stack

- Built on top of **LightLLM** [16], a PyTorch+Triton LLM serving framework.
- Triton kernels for MBGMM (prefill); Triton + modified Punica SGMV kernels for MBGMV (decode).
- Adopts iteration-level scheduling from Orca [3].
- Prefetching: while running batch t , predicts adapters needed for batch $t + 1$ based on the waiting queue and pre-loads them into the unified pool, overlapping I/O with computation.

5 Experimental Setup and Benchmarks

5.1 Punica Experiments

5.1.1 *Testbeds*

- **Testbed #1:** Single server, $1 \times$ NVIDIA A100 80GB GPU (for kernel microbenchmarks and single-GPU text generation).

- **Testbed #2:** 2× NVIDIA HGX A100 40GB servers, 8 GPUs each (NvSwitch interconnect; for tensor parallelism and cluster deployment).

5.1.2 Models

Llama-2 7B, 13B, 70B [2]. LoRA rank = 16, applied to all dense projections. Random weights used (latency is weight-independent).

5.1.3 Workload Distributions

1. **Distinct:** Every request uses a different LoRA model.
2. **Uniform:** $\lceil \sqrt{n} \rceil$ models, equally popular.
3. **Skewed:** Zipf- α distribution ($\alpha = 1.5$).
4. **Identical:** All requests use the same LoRA model.

Prompt and response lengths from the ShareGPT dataset [14].

5.1.4 Baselines

- HuggingFace Transformers + PEFT
- DeepSpeed Inference
- FasterTransformer (backbone-only, no LoRA support)
- vLLM (backbone-only, no LoRA support)

Baselines given favorable treatment: model switching costs omitted; backbone-only baselines avoid LoRA overhead entirely.

5.1.5 Microbenchmarks

- **Roofline analysis:** SGMV FLOP/IO ratios across distributions; A100 roofline shows memory-bandwidth-bound behavior in the Identical case, IO-bound in Distinct.
- **LoRA operator latency:** SGMV vs. Loop vs. Gather-BMM across batch sizes 1–64 and LoRA ranks 8, 16, 32, 64.
- **Transformer layer latency:** Full layer (backbone + SGMV LoRA addon) for 7B/13B, sequence lengths 512/2048.

5.1.6 End-to-End Results

- **Single GPU:** 1000 requests (≈ 101 K tokens); max batch size 32; FCFS scheduling.
- **70B tensor parallel:** 8 GPUs, Megatron scheme.
- **Cluster deployment:** 16 GPUs (Testbed #2), Zipf-1.5 distribution, 1-hour experiment with Poisson arrivals.

5.1.7 Key Results

- Punica achieves **12× higher throughput** than state-of-the-art LLM serving systems with only **+2 ms latency per token**.
- 1044 tok/s (7B) and 693 tok/s (13B) on a single A100 80GB.
- 70B: 441–446 tok/s vs. vLLM’s 21–25 tok/s when serving multiple LoRA models.

- Cluster deployment confirms the scheduler successfully consolidates GPU usage with load-adaptive scaling.

5.2 S-LoRA Experiments

5.2.1 Testbeds

- Single NVIDIA A10G (24 GB), A100 40 GB, A100 80 GB
- Multi-GPU: $2\times$ and $4\times$ A100 (40 GB / 80 GB)
- Host DRAM: 64 GB – 670 GB (the upper bound on adapter count)

5.2.2 Models and Configurations

Five settings (Table 1) spanning Llama-7B to 70B with adapter ranks from $\{8\}$ to $\{64, 32, 16, 8\}$ mixed.

Table 1: S-LoRA model and adapter configurations

Setting	Base Model	Hidden Size	Adapter Ranks
S1	Llama-7B	4096	$\{8\}$
S2	Llama-7B	4096	$\{64, 32, 16, 8\}$
S4	Llama-13B	5120	$\{64, 32, 16\}$
S5	Llama-30B	7168	$\{32\}$
S6	Llama-70B	8192	$\{64\}$

5.2.3 Workload Generation

Synthetic traces using the Gamma arrival process (common in ML serving literature): mean rate λ_i per adapter following a power-law distribution with exponent α ; coefficient of variance CV controls burstiness. Also evaluated on *real traces* downsampled from LMSYS Chatbot Arena [15].

5.2.4 Baselines

- **HuggingFace PEFT**: Per-adapter batching, no continuous batching, no KV cache optimization.
- **vLLM-packed**: Merge LoRA into base model per adapter; run m separate vLLM workers under NVIDIA MPS. Cannot serve more than ~ 5 adapters before OOM.
- **S-LoRA-bmm**: S-LoRA without custom kernels or unified memory (copies to contiguous memory, uses padded BMM).
- **S-LoRA-no-unify-mem**: S-LoRA without the unified memory pool.

5.2.5 Metrics

- Throughput (req/s), average request latency, average first-token latency.
- **SLO attainment**: fraction of requests with first-token latency ≤ 6 s.
- **User satisfaction**: custom reward function mapping first-token latency to $[0, 1]$ (0 if exceeds SLO).

5.2.6 Key Results

- S-LoRA serves up to **2,000 concurrent adapters** on a single A100 80GB with minimal throughput degradation (>7 req/s for S1 from 5 to 2000 adapters).
- **Up to $30\times$ higher throughput** than HuggingFace PEFT; **up to $4\times$ vs. vLLM-packed**.
- vLLM-packed goes OOM at 100+ adapters; PEFT is functionally unusable at scale.
- Throughput stabilizes once the number of adapters exceeds ~ 100 (active batch adapters are bounded, not total adapter count).
- Multi-GPU: transitioning from 2 to 4 A100s yields **super-linear** throughput scaling (memory-bound regime).

6 Comparative Analysis

6.1 Scope and Focus

Dimension	Punica	S-LoRA
Primary focus	Multi-tenant cluster scheduling + new CUDA kernel	Scalability to thousands of adapters
Adapter scale	Tens (bounded by GPU memory)	Thousands (bounded by host DRAM)
Core kernel contribution	SGMV: segmented gather MV multiply	MBGMV/MBGMM: multi-rank, non-contiguous memory
Memory management	Paged KV cache (PagedAttention-style)	Unified Paging: KV + adapter weights in one pool
Tensor parallelism	Megatron scheme for 70B	Novel strategy decoupling base + LoRA with minimal communication
Admission control	FCFS with migration	Early Abort with SLO-optimized theory
Infrastructure language	Rust (scheduler/runner) + Python (GPU)	Python/PyTorch + Triton + Rust-like components
Base framework	Custom from scratch	Built atop LightLLM
Publication venue	arXiv (Oct 2023)	MLSys 2024

6.2 Relationship Between the Two Works

S-LoRA explicitly acknowledges Punica as concurrent work, and notes that its MBGMV decode kernel is built on a modified version of Punica’s SGMV kernel, extended to handle non-contiguous memory and heterogeneous ranks in the same batch. The works are complementary:

- Punica contributes the foundational insight that cross-adapter batching incurs negligible performance loss and introduces the SGMV kernel and a production-

oriented cluster scheduler.

- S-LoRA extends this to an extreme adapter count regime, contributes the unified memory pool (a significant systems contribution), and rigorously addresses multi-GPU serving and admission control theory.

6.3 Key Shared Principles

1. **Separate base model and LoRA computation:** Batch the base model via standard GEMM; handle LoRA addons via specialized kernels. This avoids weight duplication while enabling cross-adapter batching.
2. **On-demand adapter loading:** Store adapters outside GPU, fetch asynchronously when needed.
3. **Continuous batching (iteration-level scheduling):** Adopted from Orca; critical for high GPU utilization.
4. **Paged KV cache:** Adopted from PagedAttention/vLLM; critical for memory efficiency.

6.4 Performance Summary

System	Max adapters served	Throughput gain vs. baseline
Punica (7B, single A100 80GB)	~32 (batch size limit)	12× vs. vLLM (multi-LoRA)
S-LoRA S1 (7B, single A100 80GB)	2000+ (DRAM-limited)	30× vs. PEFT; 4× vs. vLLM-packed

7 GitHub Repository Analysis

7.1 Punica (punica-ai/punica)

Repository stats: $\approx$ 1.2K stars, 61 forks, 119 commits.

Directory structure:

- **csrc/:** C++/CUDA source for SGMV kernels (SGMV-shrink and SGMV-expand, with Tensor Core support and Split-K strategy).
- **benchmarks/:** Scripts for roofline analysis, SGMV latency benchmarks, transformer layer benchmarks, and end-to-end text generation benchmarks (matching all paper figures).
- **ci/:** CI configuration for automated testing.
- **assets/:** Figures and architecture diagrams.
- **.github/workflows/:** GitHub Actions for building and testing CUDA kernels.

Key implementation files:

- SGMV CUDA kernels exposed as PyTorch extensions via PyBind11.
- Llama model integration with FlashInfer for self-attention.
- Rust-based scheduler, frontend, and runner with WebSocket communication.
- Python GPU subprocess wrapper.

Reproducibility: Benchmarks directly correspond to paper figures; roofline scripts, LoRA operator benchmarks, and text generation evaluation are all present.

7.2 S-LoRA (S-LoRA/S-LoRA)

Repository: Hosted under the S-LoRA GitHub organization.

Directory structure:

- **slora/:** Core system code including the unified memory pool, batching engine, and request scheduler.
- **csrc/:** CUDA/Triton kernels for MBGMM and MBGMV.
- **benchmarks/:** Scripts for synthetic and real workload experiments (matching all paper tables/figures).
- **evaluation/:** End-to-end serving evaluation scripts for throughput, latency, SLO attainment.

Key implementation aspects:

- Unified paging logic managing both KV cache and adapter weights.
- MBGMV in Triton + modified Punica SGMV with non-contiguous memory support.
- Tensor parallelism partition code with minimal LoRA-specific all-reduce/all-gather.
- Adapter prefetching with next-batch prediction based on waiting queue.
- Early abort scheduling with FCFS/LCFS/Abort strategy comparison.

8 Critical Discussion and Limitations

8.1 Punica

- **Adapter count limited by GPU memory:** All active adapters must fit on the GPU simultaneously. No mechanism to spill to host memory at the adapter granularity.
- **No heterogeneous rank support:** SGMV assumes a fixed LoRA rank for the batch; adapters with different ranks require separate kernel launches.
- **Recomputation overhead:** Request migration requires re-prefill of the full prompt + generated tokens, which can be costly for long-running requests.
- **Prefill batch size capped at 1:** Limits throughput for prompt-heavy workloads.

8.2 S-LoRA

- **Adapter loading latency:** Despite prefetching, large adapters or high adapter churn can still cause I/O bottlenecks.
- **Adapter clustering trade-off:** Clustering improves throughput but can harm latency fairness across adapters—not addressed with theoretical guarantees.
- **SLO assumption:** Early abort strategy assumes a fixed first-token latency SLO (6 s), which may not generalize to all deployment scenarios.
- **Single base model:** Both systems assume all served adapters share the same backbone. Serving adapters from multiple base models requires separate instances.

9 Connections to Broader Literature

9.1 LLM Inference Optimization

Both Punica and S-LoRA build on and extend: Orca [3] (continuous batching), vLLM [4] (paged KV cache), FlashAttention [5] (IO-aware attention), and Megatron-LM [6] (tensor parallelism). They are orthogonal to speculative decoding [8] and quantization [7].

9.2 Parameter-Efficient Fine-Tuning

The explosion of LoRA adapters motivates these systems. Related PEFT methods include Prefix Tuning, Prompt Tuning, AdaLoRA, and IA³ [18], though S-LoRA notes its techniques are applicable to other adapter methods beyond LoRA.

9.3 Multi-Model Serving

PetS [9] batches multiple adapter types for BERT-scale models. Clipper [10], Nexus [11], and AlpaServe [12] address multi-model serving for CNN/smaller models, without the statefulness (KV cache) challenge inherent to LLM serving.

10 Research Directions and Open Problems

1. **Multi-base-model serving:** Efficiently serving LoRA adapters from *different* base models (e.g., Llama-2 7B and 13B simultaneously) without full weight duplication.
2. **Dynamic rank adaptation:** Systems that accommodate adapters with changing rank requirements, or that dynamically compress adapters to lower ranks for serving.
3. **Speculative LoRA serving:** Combining speculative decoding [8] with multi-adapter batching to further increase operational intensity.
4. **KV cache sharing across adapters:** Adapters sharing the same base model could potentially reuse KV cache prefixes (as in RadixAttention / SGLang), further reducing redundant computation.
5. **SLO-aware preemption with migration:** More sophisticated migration policies that balance SLO compliance against recomputation costs for long-context requests.
6. **Heterogeneous hardware:** Extending SGMV/MBGMV kernels to AMD ROCm, Intel Gaudi, or edge accelerators.
7. **Quantized LoRA serving:** Combining QLoRA [7] with efficient multi-tenant serving—quantized backbone with high-precision adapters.

11 Summary Table

12 Conclusion

Punica and S-LoRA represent the foundational systems work for efficient multi-tenant LoRA serving. Punica establishes the key insight that cross-adapter batching is feasible with negligible performance loss, and contributes the SGMV kernel and a cluster-level

Table 2: Comprehensive comparison of Punica and S-LoRA

Aspect	Punica	S-LoRA
Venue	arXiv Oct 2023	MLSys 2024
Core kernel	SGMV (CUDA, Tensor Cores, Split-K)	MBGMV / MBGMM (Triton + Punica-based)
Memory mgmt	Paged KV cache (outermost batching dim)	Unified Paging (KV + adapters co-managed)
Adapter storage	On GPU (limited)	Host DRAM + on-demand GPU fetch
Adapter scale	Dozens	Thousands
Tensor parallelism	Megatron (no LoRA-specific strategy)	Novel LoRA-aware partition with comm. fusion
Scheduling	FCFS + priority busy-GPU routing	FCFS + Early Abort + Adapter Clustering
Migration	Recomputation-based	N/A (single machine)
Infrastructure	Rust scheduler + Python GPU subprocesses	LightLLM (Python/PyTorch/Triton)
Throughput gain	12 $\times$ vs. multi-LoRA vLLM	4 $\times$ vs. vLLM-packed; 30 $\times$ vs. PEFT
Max adapters	$\sim$ 32 concurrent (A100 80GB)	2000+ (A100 80GB)
Models tested	Llama-2 7B, 13B, 70B	Llama-7B, 13B, 30B, 70B
Hardware tested	1–16 $\times$ A100	A10G, A100 (various configs)
LoRA ranks	Fixed (16 in experiments)	Heterogeneous {8, 16, 32, 64}
Cluster support	Yes (multi-machine)	Single-machine multi-GPU

scheduler as the enabling mechanisms. S-LoRA builds on this foundation to address the more demanding regime of thousands of concurrent adapters, with Unified Paging as its primary systems contribution, along with a theoretically grounded admission control strategy and a communication-efficient tensor parallelism design.

Together, these systems have had significant impact on subsequent LLM serving infrastructure: the SGMV kernel has been adopted and extended in production systems (including vLLM’s LoRA support), and the Unified Paging concept has influenced memory management in frameworks such as LightLLM and SGLang.

For future work in this space, the key open challenges lie in multi-base-model serving, dynamic rank heterogeneity, and integration with advanced inference acceleration techniques such as speculative decoding and quantization.

References

- [1] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [2] H. Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” arXiv:2307.09288, 2023.
- [3] G.-I. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” *OSDI*, 2022.
- [4] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *SOSP*, 2023.
- [5] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022.
- [6] M. Shoeybi et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” arXiv:1909.08053, 2019.
- [7] T. Detrmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” *NeurIPS*, 2023.
- [8] Y. Leviathan et al., “Fast Inference from Transformers via Speculative Decoding,” *ICML*, 2023.
- [9] Z. Zhou et al., “PetS: A Unified Framework for Parameter-Efficient Transformers Serving,” *USENIX ATC*, 2022.
- [10] D. Crankshaw et al., “Clipper: A Low-Latency Online Prediction Serving System,” *NSDI*, 2017.
- [11] H. Shen et al., “Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis,” *SOSP*, 2019.
- [12] Z. Li et al., “AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving,” *OSDI*, 2023.
- [13] S. Mangrulkar et al., “PEFT: State-of-the-Art Parameter-Efficient Fine-Tuning Methods,” <https://github.com/huggingface/peft>, 2022.
- [14] ShareGPT Team, “ShareGPT: Share Your Wildest ChatGPT Conversations,” <https://sharegpt.com/>, 2023.
- [15] L. Zheng et al., “LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset,” arXiv:2309.11998, 2023.
- [16] ModelTC, “LightLLM: Python-Based LLM Inference and Serving Framework,” <https://github.com/ModelTC/lightllm>, 2023.
- [17] Z. Ye, “FlashInfer: Kernel Library for LLM Serving,” <https://github.com/flashinfer-ai/flashinfer>, 2023.
- [18] H. Liu et al., “Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning,” *NeurIPS*, 2022.